

残酷共学 · 增量笔记

自 2026-08-12 23:01 BJT 后新增
122 篇

本次增量 · 带索引

点击目录条目可跳转

数据来源：intensivecolearn.ing 活动公开打卡

生成时间：2026-08-13 23:01 (BJT)

目录

主题章节 → 笔记索引 (点击可跳转)

01 AMM机制 (17 篇)	7
笔记 001: D06 DEX 内与三角套利	7
笔记 002: 昨天记的 GUA (mexc现货 → gate永续, 开仓 3.25%、开清之差 0.84%、日成本 2	7
笔记 003: 跟随昨天的进度, 期权IV和LP APR可以进行横向对比, 很多时候defi lp由于竞争没那么激烈反而	8
笔记 004: 8月12日 从 Pending State 推演套利机会	11
笔记 005: 主题: 建立链上套利的基础认知与全局地图	12
笔记 006: Day 9 链上套利残酷共学	19
笔记 007: 看了下所谓“无风险闪电贷套利”是怎么回事,	19
笔记 008: 一、交付了什么	26
笔记 009: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践, Agent 仅负责	27
笔记 010: 学习笔记: 双 DEX 价差监控脚本 v1 + v2 (Day 5-6)	28
笔记 011: 今日共学打卡 AI 量化 × 数据墙: 为什么顶级模型懂原理, 却不一定会赚钱?	29
笔记 012: 研究了下群主發的μToken套利, 不過官推目前還沒有甚麼資訊, 我用claude分析了下。	30
笔记 013: Day 9 笔记: DEX 间套利 —— 同一链上不同 DEX 的价差	30
笔记 014: 主题: 闪电贷套利交易流程 · 来源: 每日推送学习	33
笔记 015: Day 7 跨 CEX/DEX 比價工具完工: 8 個 venue、81 個測試、一天攢下 1.5 萬	34
笔记 016: 链上 State Engine 设计	35
笔记 017: Day 3 + Day 4 学习笔记: 从流动性池到跨链路线, 我开始看懂套利前的“现场”	35
02 LI.FI与跨链 (36 篇)	38
笔记 018: 1、针对新币DOS上市, 使用AI找到BSC、ERC链的地址, 验证并找到跨链桥的方案, 最后形成报告。结 . . .	38
笔记 019: 今日打卡	39
笔记 020: 链上套利学习笔记 Day 8: LI.FI 路由——多目标权衡	41
笔记 021: 【Day 12 打卡 · 2026-08-12】真实报价数据积累	42
笔记 022: 今日完成了一个可复现的 LI.FI 路由实验并记录限制。	42
笔记 023: 2026-08-12 — Day 8(Week 2 开篇):成本模型函数化 + 真实 Quote 验	43

笔记 024: markdown	43
笔记 025: 第五天 LI.FI 跨链聚合与路由比较	44
笔记 026: 我发现自己的“跨链套利闭环”，其实穿越了时间	46
笔记 027: Day 8 LI.FI 线上报价工具与“无路由”故障验证	49
笔记 028: 	49
笔记 029: 【Day 8 打卡】2026/8/12	54
笔记 030: Day 9 : 全自動套利系統架構 v0.8	55
笔记 031: https://github.com/ZeroNullNone/onchain_arbitrage_	61
笔记 032: LI.FI 的切面合约 (Facet Contracts) 是基於 EIP-2535 多切面代理標準建立	62
笔记 033: 学习心得：从套利概念学习到 USDT 跨链观察框架	63
笔记 034: LayerZero 多链资产价差扫描：从资产发现到可执行报价	63
笔记 035: Day 9 (08-13) : 阶段复盘收官 + 执行层全面升级	64
笔记 036: D7 学习进展：第一周混合检索与知识地图 v1	65
笔记 037: 今天查阅了 LI.FI 官方 Status Tracking 相关说明。跨链交易提交后，不能只依据源	66
笔记 038: 建立跨链监控	66
笔记 039: 2026-08-13	68
笔记 040: 链上套利残酷共学 · Day 8 心得	68
笔记 041: 2026年8月13日 链上套利学习笔记	68
笔记 042: 2026-08-13 : Day08 打卡内容	69
笔记 043: Day 9 : 固定 1,000 USDC 获取一份 LI.FI 单向 Quote。	69
笔记 044: 一、今天建立了什么	70
笔记 045: Day 9 · 2026-08-13 · 把第一周的工具压一压,再用 24 小时真实市场数据测它一遍	72
笔记 046: 链上套利学习笔记 Day 9 : 信号 vs 策略	77
笔记 047: 今日学习 :	78
笔记 048: 一、今日主题	78
笔记 049: August 13, 2026 · Day 9 / 21 · 时间分配：先动手 60min → 后概	79
笔记 050: 2026-08-13 — Day 9(Week 2 第 2 天):监控雏形落地 — 跨链路径轮询脚本	81

笔记 051: 1、使用跨链系统进行套利实践，24小时内，只发现了2次净利润为正的套利机会，一次成功，一次失败。成功	
笔记 052: \	82
笔记 053: Day 9 · 从可行动告警推进到执行前决策闸门	83
03 MEV与抢跑 (4 篇)	84
笔记 054: 1. BuilderNet refund 机制	84
笔记 055: \	86
笔记 056: 2026.08.13	86
笔记 057: Day 8 (8/13) 打卡	87
04 做市与LP (3 篇)	88
笔记 058: 最后还是要学rust!	88
笔记 059: 链上套利残酷共学 2026-08-13 打卡	88
笔记 060: 链上套利残酷共学 · Day 9 打卡草稿：Curve LLAMMA 的软清算为什么依赖套利者	89
05 其他 (1 篇)	92
笔记 061: 学习记录	92
06 套利框架 (43 篇)	92
笔记 062: Day 8 · 把散落的策略证据归进一张表,然后剪枝	92
笔记 063: 11:30上新anthropic盘前合约，开局基本没有价差，大概过几分钟才有币安多-ok空的一点价差	93
笔记 064: 链上套利学习 Day 7 笔记	94
笔记 065: 今日打卡 从单次机会 PnL 走向完整库存循环 PnL	104
笔记 066: 链上套利 Day 5 学习笔记	106
笔记 067: 8月11日（周二）——、	108
笔记 068: Day 5 : Hermes与套利研究 workflow	108
笔记 069: Day07 纸面模拟系统	110
笔记 070: 今天继续学习 DEX-CEX 套利，主要思考了一个问题：拿到的两个价格是不是同一时刻的价格。	111
笔记 071: 8 月 13 日：先把新渠道放进观察层	111
笔记 072: 稳定币与锚定资产套利 (Stablecoin / Peg Arbitrage)	112

笔记 073: 今天学到的东西（方法论层面）	112
笔记 074: Day 9（08/13）学习总结，素材来自今日同学精华：	113
笔记 075: 加密市场里的“期货/合约套利”，本质上是：	113
笔记 076: 两次都读取 latest，不保证两个池来自同一个区块；应先固定 chain ID、区块号和区块哈希。	117
笔记 077: 今天把套利机会评估流程固化成可复用的 Hermes 技能「arbitrage-opportunity	117
笔记 078: D11（8/13）案例对比分析 + 清算监控实操	117
笔记 079: Day 8 打卡 链上套利残酷共學	118
笔记 080: 今天研究期权波动率曲面的静态套利。重点不是判断隐含波动率看起来高或低，而是检查同一批期权价格是否违反	118
笔记 081: markdown	119
笔记 082: 原子套利:一笔交易里买完就卖	120
笔记 083: 2026-08-13 學習打卡	122
笔记 084: 期现套利要算完的几笔账	122
笔记 085: Day 9 非线性滑点下的最大套利规模	124
笔记 086: Day 9 闪电贷、盈亏比与跨链套利	125
笔记 087: 【Day 13 打卡 · 2026-08-13】数据分析与候选验证	125
笔记 088: 今天继续围绕链上套利策略做了一些整理，重点思考了一个问题：	125
笔记 089: 先说个我得出的结论：链上套利这行，看着是捡钱，其实全是坑，而且坑几乎都长在一个地方——你看到价差的那	126
笔记 090: 链上套利共學 · Day9 打卡笔记（08/13）	127
笔记 091: Day 9（2026-08-13）· 阶段二开启：案例 + 工具 + Hermes Agent（8/	129
笔记 092: Day 9 DEX 间价差：从「看到」到「敢做」的可执行报价	129
笔记 093: 2026-08-13 打卡	129
笔记 094: 参考	130
笔记 095: 今日实践：给 LayerZero 跨链套利监控程序加上「实时询价」能力	130
笔记 096: 套利基础学习打卡④ 看到价差≠能赚钱：从毛价差到真实套利利润	131
笔记 097: 【8月13日 学习打卡】链上套利 Day 9：基础设施运维与套利赛道认知	141
笔记 098: 2026-08-13 Day 9 四平台永续资金费率监控：费率差不等于套利利润	142

笔记 099: 早上先处理了一个装错工具的插曲。群里提到的 Astro 搬砖工具，我先做安全审计，仓库里只有安装脚本	142
笔记 100: 今日打卡 从“发现价差”到“建立证伪机器”	142
笔记 101: Day 9 2026.08.13 免费 RPC 降级陷阱: 一个进程活着, 但搜索已经死了	144
笔记 102: 一、Delta-Neutral Funding Arb 的两个方向	145
笔记 103: 今天没有开始新版 Day 10, 先完成课程与研究方向的复审。	146
笔记 104: 本地跑 Reth 节点后, 用 REVM + CacheDB 做模拟最快。通过本地 RPC 拉状态即可	146
07 学习计划 (5 篇)	146
笔记 105: 统计了热钱包 0xd712...9b96 最近一批调执行器且链上成功的交易。计划 25 笔, receipt	146
笔记 106: Day 9 — DeFi 协议间套利	146
笔记 107: Day 09 · 2026-08-13 · 用 AI Agent 搭建案例研究 workflow	147
笔记 108: Day 9 - 比较不同路由	148
笔记 109: 如果你那么有钱, 为什么还那么不聪明?	149
08 实盘与数据 (9 篇)	155
笔记 110: 今天学习了 Polymarket 的结构化套利: 核心不是预测事件, 而是利用市场之间严格成立的逻辑关系	155
笔记 111: 今天学习帖子: 套利实操笔记 03 1200%年化, 但最终我却亏损出场 如果你准备做资金费率套利, 开	155
笔记 112: 今天做了什么: D6 数据收集日: ①链上直查 Aave V3 以太坊主网 7 天清算事件——仅 3 笔	155
笔记 113: 今天做了dune的base链的视图。能够实时监控和某个DEX合约地址交互的钱包的交易量。以bn聚合器	156
笔记 114: Day 9 两阶段预筛: 用 28 条报价腿替代 48 条完整路径	159
笔记 115: Day 9 · 不要把“看得见”误当成“能预知”	160
笔记 116: 笔记 1: Polymarket 套利: 真正的壁垒不是发现价差, 而是数学与执行基础设施\	161
笔记 117: 第九天学习记录: 今天把 Robinhood Chain 的 Local-Lite 观察链路和自有执行	164
笔记 118: 根据自身能力选择验证层级——套利研究学习笔记	164
09 风险与安全 (4 篇)	171
笔记 119: 链上套利残酷共学打卡 - 2026-08-12 (Day 8)	171
笔记 120: 01.8 原始证据、聚合工具与交叉验证	171
笔记 121: 今天完成了 P0 阶段的最后一个缺口: 两跳 V3 → V4 路径的保守 PnL 评估。	181
笔记 122: 链上套利共学 Day 9 LI.FI 跨链交易执行细节: 五步调用法 + 授权 + 签名发送 + 状态	181

笔记 001: D06 | DEX 内与三角套利

作者：averywexler | 时间：2026-08-12 15:03 UTC | 字数：545 | 评分：63

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D06 | DEX 内与三角套利

套利首先是一条路径，而不是两张价格截图。三角描述路径形状，DEX 内或 DEX 间描述交易场所。

今日术语

路径：资产从起点经过若干市场回到终点的兑换顺序；

Hop：路径中的一次兑换；

DEX 内套利：同一协议不同池、费率档或路线之间的不一致；

三角套利：A→B→C→A 的闭环；

原子执行：所有步骤在一笔交易中全部成功，否则全部撤销。

参考材料

Uniswap Swap Routing (<https://developers.uniswap.org/docs/trading/swapping-api/concepts/swap-routing>)

跟着算

假设汇率，尚未计成本：

1. 10,000 USDC → 5 WETH；
2. 5 WETH → 10,250 DAI；
3. 10,250 DAI → 10,239.75 USDC；
4. 屏幕毛价差为 239.75 USDC。

若三跳各收 0.3%，只考虑三次费用后约剩 10,147.87 USDC；Gas、价格冲击、竞争和失败风险还没有扣。

笔记 002: 昨天记的 GUA (mexc现货 → gate永续，开仓 3.25%、开清之差 0.84%、日成本 2

作者：haze | 时间：2026-08-12 15:17 UTC | 字数：2490 | 评分：77

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

昨天记的 GUA (mexc现货 → gate永续，开仓 3.25%、开清之差 0.84%、日成本 2.562%) 今天在看板上找不到了。

这是「行消失」这个取值第一次出现，它既不是收敛也不是不收敛，收官算准确率时必须单独归一类，不能混进 Y/N/部分（否则污染分母）。

可能原因，尚未验证：

1. 某条腿下架 / 停止交易 (haze 的初判)
2. 价差收敛到看板筛选阈值以下，不再显示
3. 24h 量跌破看板收录门槛（昨天这行的两腿量就没记到，本来就可能很薄）
4. 看板数据源或筛选条件变动

已验证（同日，看板搜索框搜 GUA）——不是下架，是现货腿换了：

开清差价 腿A → 腿B 开清之差 费率原始 → 日成本% 指数差价 量A / 量B
:----- :----- :----- :----- :----- :-----
0.95 / 0.97 okxdex / gate 0.02 0.118/4h → +0.708 0.368 5.8M / 1.1M
0.11 / 0.19 okxdex / binance 0.08 0.005/4h → +0.03 0.061 5.8M / 20.8M
0.11 / 0.15 okxdex / mexc 0.04 0.005/4h → +0.03 0.022 5.8M / 2.2M

三行全部排除，硬筛第 5 条：现货腿 okxdex 带代币图标 = DEX，不在本阶段范围。昨天那行的 mexc 现货腿已从看板消失（今日 GUA 的现货腿只剩 okxdex 一个），mexc现货 → gate永续 这个组合因此不再存在——消失的是组合，不是币种。

但这三行本身是四条法则的教科书样本，全部留档：

1. 共同腿法则的「正确用法」第一次跑通 —— 之前理解成「换腿差价不变就排除」，这次是反过来的用法：固定现货腿 okxdex（量恒为 5.8M），换三个合约腿，差价 0.95 / 0.11 / 0.11。两个所一致、gate 独高 8.6 倍 → 异常在 gate 那条腿，不在 okxdex。共同腿法则真正在做的事是定位异常腿，不只是给出排除信号。
2. 昨天 GUA 的 3.25% 由此得到解释 —— 昨天记的是 mexc现货 → gate永续，含 gate 这条异常腿。今天从另一个现货腿看过去，gate

依然是三者中的异常值。昨天判它「结构性堵死候选」用的理由是盘口宽（开清之差 0.84%），理由对，但归因错了：不是这个币盘口宽，是 gate 那条腿有问题。

3. 指数差价第一次当「异常腿定位器」用——gate +0.368 vs binance 0.061 / mexc 0.022。指数差价独立于开清差价，却指向同一条腿。此前只把它当资金费率的领先指标（还刚出过反例），这个新用途比领先指标那个用途可靠：它是同一时刻的横截面比较，不涉及对未来的预测。

4. 「排序即陷阱」拿到干净实证——差价最大的那行（gate 0.95%）合约腿量 1.1M，是三者最薄；差价最小的两行量 20.8M / 2.2M。差价与深度在这个横截面上完全负相关。看板默认按开清差价降序，顶部那行大概率就是最薄的那行——《原理框架》法则二从今天起是实证不是转述。

回填结论：GUA 的 次日收敛 = 行消失，原因已查明为组合消失（mexc 现货腿退出看板），非币种下架。此行保留作排除模式样本，归类：异常腿型（区别于 CASHCAT 的结构性宽盘口型）。

这件事的分量比它看起来大——它是「价差持续时长」字段的一个极端数据点：一个 3.25% 开仓差价的机会，寿命不到 24 小时。而且要问的不是「机会没了可惜」，是反过来的那一问：

如果昨天真的开了 GUA 这一单，今天这条腿消失，我此刻持有什么？

验证后这个答案要分两种情形，不能像最初记的那样一口咬定是「裸敞口」——「从看板消失」只等于这个组合不再被收录，不等于「不能交易」：

| 情形 | 实际处境 | 后果 |

| :----- | :----- | :----- |

| mexc 现货真的停牌 / 退市 | 一条腿无法平 | 单腿裸敞口，方向性风险全额暴露 |

| 仅量跌破看板收录门槛 | 两条腿都还能平 | 流动性枯竭，平得掉但滑点可能吃掉全部利润 |

从今天的截图只能确认「组合退出看板」，区分这两种情形需要去 mexc 站内查挂牌状态——尚未做。但两种情形指向同一条操作结论：决定敞口大小的不是开仓时的差价，是平仓时那条腿还在不在、还有多深。

今天检测题 3(b) 问的是「一腿成交、一腿没成交」（开仓时），GUA 给的是时间轴另一端的版本：开仓时两条腿都好好的，风险发生在持仓期间。昨天判它「结构性堵死候选」的理由是盘口宽（0.84%），今天归因修正为异常腿 + 现货腿本身就脆——宽盘口是表症不是病因。

笔记 003: 跟随昨天的进度，期权IV和LP APR可以进行横向对比，很多时候defi lp由于竞争没那么激烈反而

作者：allen123wzh | 时间：2026-08-12 15:48 UTC | 字数：4930 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

跟随昨天的进度，期权IV和LP APR可以进行横向对比，很多时候defi lp由于竞争没那么激烈反而能提供比买卖期权更高的收益。

计划从Robinhood chain上uniswap池的NVDA AAPL TSLA SPCX开始先做一个LP pool的volume/fee统计表，然后跟期权市场NVDA的短期期权（1month以内）对比，再进一步筛选。中间还要学习很多期权相关知识，先列了初步方法。

Robinhood Chain 链上美股 LP 研究方法

目标是比较 NVDA、AAPL、TSLA、SPCX 在 Robinhood Chain 上的 LP 机会。第一阶段只研究 Uniswap ; Up、Ekubo 等 ve(3,3) / incentive pool 单独处理，因为其 LP 收益很大程度取决于 weekly rewards / emissions，不能直接用 Volume × Fee 衡量。

一、先建立正确的 Pool Universe

以 Robinhood 官方 Stock Token 合约为准，避免同 ticker 假币或非官方资产。

当前先研究：

NVDA

AAPL

TSLA

SPCX

对每个 Stock Token 搜索 全部 Uniswap pools，主要关注：

Stock / USDG

Stock / WETH（如果有实际成交量）

Stock / 其他主流资产（仅在交易量明显时纳入）

排除 meme token 等无关交易对。

每个 pool 记录：

字段	用途
Stock	NVDA / AAPL / TSLA / SPCX
Pair	NVDA/USDG 等
Version	Uniswap v3 / v4
Pool Address / PoolId	链上查询
Fee / Fee model	固定或 dynamic
24h Volume	交易活跃程度
7d Volume	降低单日噪声
TVL / Liquidity	粗略资本规模
Tx Count	判断成交结构

DEX Screener 主要用于发现 pool 和快速初筛，最终计算以 Robinhood Chain RPC 的链上数据为准。

二、第一层筛选：Volume / TVL

先计算：

24h Volume / TVL

以及更稳定的：

7d Volume / Average TVL

它衡量一个池子的资金周转效率。

例如：

\$1M TVL、\$100K daily volume → 0.1x

\$200K TVL、\$1M daily volume → 5x

第二个池虽然 TVL 小，但单位资本对应的交易活动高得多。

不过 Volume/TVL 只适合初筛，不能直接代表 LP 收益。

原因是 Uniswap v3/v4 是 concentrated liquidity，网页显示的总 TVL 并不是当前价格附近真正与你竞争的资本。

三、核心：计算 Active Liquidity

对于每个候选 pool，从链上读取：

current tick

sqrtPriceX96

liquidity

tickSpacing

initialized ticks

每个 tick 的 liquidityNet

然后重建当前价格附近的 liquidity distribution。

统一统计：

±2% liquidity

±5% liquidity

±10% liquidity

例如：

| Stock | Pool | TVL | ±2% Liq | ±5% Liq | ±10% Liq |

| :--- | :--- | :--- | :--- | :--- | :--- |

| NVDA | Pool A | \$1.0M | \$150K | \$300K | \$500K |

| SPCX | Pool B | \$500K | \$80K | \$130K | \$260K |

这样才能知道：

如果我自己做一个 ±5% LP，实际是在和多少资本竞争？

因此后续应逐渐用 Active Liquidity 替代 TVL 作为主要分母。

四、计算真实 LP Fees

Uniswap v3

v3 是固定 fee tier。

读取 pool 的：

fee()

然后可以先近似：

24h gross LP fees $\approx$ 24h volume $\times$ fee tier

例如：

\$1M volume $\times$ 0.30% = \$3,000 gross fees

之后如果存在 protocol fee 等情况，再进一步修正。

Uniswap v4

v4 不能默认固定手续费。

部分 pool 可以采用 dynamic fee，所以最好直接读取 PoolManager 的 Swap events，取得每笔 swap 实际 fee。

对过去 24h / 7d 所有 swaps：

LP Fees = Σ Swap Notional $\times$ Actual Fee

这样得到真正的：

24h LP fees

7d LP fees

volume-weighted average fee

而不是简单用 DEX Screener volume 乘一个假定 fee。

五、真正应该比较的指标

最终不要只做：

Volume 排名

而应该做：

1. Volume / TVL

最简单的资金周转率，用于初筛。

2. Fees / TVL

考虑不同池的手续费差异。

3. Fees / Active Liquidity

最重要。

例如：

7d LP Fees / Average $\pm$ 5% Active Liquidity

这更接近一个 $\pm$ 5% LP 的潜在手续费效率。

4. Slippage / Market Depth

模拟：

\$1K swap

\$10K swap

\$50K swap

\$100K swap

观察 price impact。

这能直接验证 liquidity 是否真正集中在 spot 附近。

六、最终建议的主表

最终每个 pool 做成：

Stock	Version	Fee	24h Vol	7d Vol	TVL	$\pm$ 2% Liq	$\pm$ 5% Liq	$\pm$ 10% Liq	24h Fees	7d Fees	7d Fees / $\pm$ 5% Liq
NVDA	v3	 	 	 	 	 	 	 	 	 	
NVDA	v4	 	 	 	 	 	 	 	 	 	
AAPL	v4	 	 	 	 	 	 	 	 	 	
TSLA	v4	 	 	 	 	 	 	 	 	 	
SPCX	v3	 	 	 	 	 	 	 	 	 	
SPCX	v4	 	 	 	 	 	 	 	 	 	

这样最终可以回答三个不同问题：

交易最多的是谁？

→ 看 Volume。

市场深度最好的是谁？

→ 看 $\pm 2\%$ / $\pm 5\%$ liquidity 和 slippage。

单位 LP 资本手续费收入最高的是谁？

→ 看 Fees / Active Liquidity。

这三者不一定是同一个 Stock Token。

执行顺序

第一阶段先把 NVDA / AAPL / TSLA / SPCX 的全部 Uniswap pools 找齐，确认 v3/v4、Pool Address/PoolId、交易对、fee model、24h/7d volume 和 TVL。

第二阶段对其中有意义的池读取链上 tick 数据，计算 $\pm 2\%$ / $\pm 5\%$ / $\pm 10\%$ Active Liquidity。

第三阶段扫描 swap 数据，计算真实 24h / 7d LP fees，尤其正确处理 v4 dynamic fee。

第四阶段计算：

7d Fees / $\pm 5\%$ Active Liquidity

并结合 slippage 做最终排名。

最后再单独开第二套模型研究 Up / Ekubo 等激励池，把 weekly rewards / active LP capital、token emissions、voting/bribe 等因素加入，而不是和 Uniswap 的纯手续费收益混算。

因此现在第一步非常明确：先做一张 NVDA、AAPL、TSLA、SPCX 的 Uniswap 全池清单，然后再开始链上计算。

笔记 004: 8月12日 | 从 Pending State 推演套利机会

作者：Lier Mi | 时间：2026-08-12 15:56 UTC | 字数：1387 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月12日 | 从 Pending State 推演套利机会

今天继续深入链上套利的预测层：真正有竞争力的系统，不只计算已经存在的价差，还要推演待处理交易执行后可能产生的价差。

基于最新区块计算得到的是 Latest State。但下一笔套利交易真正执行时，面对的往往不是这个状态，而是：

Projected State = Latest State + 排在套利交易之前的状态变化

假设当前两个池子的价格一致，待处理交易池中出现一笔大额 Swap。它执行后会改变某个池子的储备、价格和 Tick，进而制造新的跨池价差。

搜索者需要完成以下推演：

解析目标交易 → 还原调用路径 → 模拟状态变化 → 更新受影响的池子 → 重新搜索套利路径 → 计算最优投入 → 构造条件化交易包
这类机会常被理解为 Backrun：套利交易被安排在目标交易之后，利用目标交易造成的价格偏移完成价格修复。

但 Pending State 并不是唯一确定的状态。公开交易池只能说明“有哪些交易正在传播”，不能保证：

交易一定会被打包；

交易会进入下一个区块；

多笔交易会按照当前 Gas 报价排序；

私有订单不会插入；

构建者不会选择另一组交易；

目标交易不会因为 nonce、余额或滑点而失败。

因此，不能简单地把所有待处理交易依次执行，再把结果当成下一个区块。更合理的做法，是维护多个候选世界：

目标交易未被打包；

目标交易单独执行；

高优先级交易先于目标交易执行；

多笔相关交易共同改变同一个池子；

私有流量或竞争者改变最终排序。

换句话说，Pending State 不是一个状态，而是一组带概率的状态分支。

机会的价值也不应只看模拟利润，而应计算风险调整后的期望收益：

$EV = P(\text{目标交易成交且排序正确}) \times \text{净利润} - P(\text{失败}) \times \text{失败成本}$ 机会成本

其中，净利润还要扣除基础费、优先费或支付给构建者的越接近套利产生的毛利润，最终留给搜索者的收益就越薄。

Bid。竞争越激烈，可支付的

Bid

今天还理解了“原子性”与“条件性”的区别。

原子性保证一组操作要么全部成功，要么全部回滚；条件性则要求交易只有在关键前提仍成立时才允许执行。套利合约除了完成多跳兑换，还应验证：

当前区块没有超过有效范围；

实际余额增量高于 minProfit；

池子价格没有越过允许边界；

借入资产和闪电贷费用能够完整偿还；

最终利润足以覆盖链外竞价成本。

对于 Backrun，理想结构通常是将目标交易与套利交易按照指定顺序组成交易包，并在目标区块状态下整体模拟。需要验证的不只是“套利交易能否成功”，而是：

目标交易成功 → 状态按预期变化 → 套利交易成功 → 净利润为正

只模拟最后一笔套利交易是不够的，因为它依赖前置交易创造的状态。如果前置交易失败、被替换或顺序改变，后面的利润计算就失去了基础。

今天最大的收获是：高级套利不是预测价格，而是预测状态转换和交易顺序。

看到 Pending 交易只是信息优势的起点。真正的能力，是为不确定的排序建立多个状态假设，只在收益能够覆盖预测误差时出手。

笔记 005: 主题：建立链上套利的基础认知与全局地图

作者：Qy | 时间：2026-08-12 15:59 UTC | 字数：6016 | 评分：60

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

主题：建立链上套利的基础认知与全局地图

\

一、我对链上套利的初步理解

链上套利，是利用不同

DEX、流动性池、区块链或交易路径之间暂时存在的价格差异，通过买入、兑换、跨链和卖出等操作，形成完整的交易闭环。

一笔套利是否成立，不能只看页面显示的价差，还必须扣除：

1\、Gas；

2\、DEX 和协议手续费；

3\、跨链桥费用；

4\、滑点与价格冲击；

5\、执行延迟带来的价格变化；

6\、失败交易成本；

7\、资金占用成本。

只有扣除全部成本后，净收益仍然为正，才可能是真正的套利机会。

我今天最重要的认识是：

看到价差，不等于能够成交；

能够成交，也不等于最终能够盈利。

二、今天已经大致理解的概念

1\、Gas

Gas 是在区块链上执行交易时需要支付的网络费用。

即使交易失败，也可能产生 Gas 成本。

2\、脚本

脚本是按照预先设定的规则，自动获取数据、进行计算、监控信号或执行任务的程序。

3\ 钱包

钱包是管理链上账户并对交易进行签名的工具。

钱包并不是简单地“存放资产”，而是帮助用户管理账户，以及控制资产所需的密钥。

4\ Token

Token 是可以在区块链上持有、转移和使用的数字资产。

名称相同的 Token 不一定是同一种资产，还需要核验：

- 所在区块链；
- Token 合约地址；
- 是否为原生、封装或跨链版本。

5\ DEX

DEX 是去中心化交易所。

用户通常通过智能合约和流动性池完成资产兑换，而不是把资产托管给传统的中心化交易所。

三、今天重点学习的概念

1\ AMM：自动做市商

AMM 通过智能合约和流动性池，按照一定的公式自动计算资产的兑换价格，不需要传统交易所中的买卖双方挂单撮合。

当用户进行交易时，流动性池中两种资产的数量和比例会发生变化，AMM 会根据变化后的池子状态重新计算价格。

例如，一个池子里有 ETH 和 USDC。

当用户使用 USDC 购买 ETH 时：

- 池中的 ETH 减少；
- 池中的 USDC 增加；
- ETH 在这个池子中变得更加稀缺；
- 继续购买 ETH 的价格会提高。

因此，每次交易都可能影响后续交易者面对的价格。

2\ 滑点

滑点，是预计成交价格与实际成交价格之间的差异。

交易金额越大，从有限流动性池中买走或放入的资产比例通常越高，池内两种资产的比例变化也越明显。

自己的交易会推动成交价格向不利方向变化，因此交易金额越大，滑点通常也可能越大。

滑点还可能来自：

- 市场价格变化；
- 其他交易先执行；
- 网络拥堵；
- 交易等待时间过长；
- 流动性不足；
- 报价过期。

3\ 跨链桥

跨链桥，是帮助资产或信息从一条区块链转移到另一条区块链的系统。

跨链并不是简单地多支付一条链的 Gas，还可能涉及：

- 源链和目标链的 Gas；
- 跨链桥费用；
- 跨链等待时间；
- 等待期间的价格变化；
- 桥接失败或资金卡住；
- 目标链上没有 Gas；
- 同名资产版本不同；
- 跨链桥和智能合约的安全风险。

因此，跨链套利通常比同一条链上的 DEX 间套利，多出更多成本、步骤和不确定性。

4\ 区块确认

用户提交链上交易后，交易需要被打包并写入区块。

后续继续产生新区块，通常意味着这笔交易获得了更多确认，交易结果也更加稳定。

套利机会可能只存在很短时间。

如果等待交易打包、区块确认或跨链的时间过长，原本存在的价差可能已经消失。

5\ MEV

MEV 可以初步理解为：

通过选择、调整或利用链上交易的执行顺序而获得的额外价值。

链上交易不是提交后立即完成的。交易需要等待被纳入区块，而交易的排列顺序可能改变最终价格和收益。

即使我先在页面上看到一个套利机会，也可能有其他机器人：

- 更快提交交易；
- 支付更高的优先费；
- 使用更专业的交易通道；
- 获得更有利的交易排序。

其他机器人完成套利后，市场价差可能已经消失。此时，我的交易可能失去利润，甚至出现亏损。

常见的 MEV 场景包括：

- DEX 套利竞争；
- 清算竞争；
- 抢跑；
- 回跑；
- 夹子交易。

现阶段，我只需要知道：

交易执行顺序和机器人竞争，会影响我能否真正捕获一个机会。

暂时不需要学习 MEV Bot 开发。

6\ 数据源

数据源，是用来获取价格、报价、流动性、交易、Gas 和链上状态等信息的来源。

常见的数据源包括：

- DEX 页面；
- LI.FI 等聚合器；
- 区块浏览器；
- 区块链 RPC；
- 项目 API；
- 链上数据分析平台；
- 订单簿或做市商报价。

不同数据源可能存在时间延迟、字段差异和计算口径差异。

因此，数据源显示存在价差后，不能直接认为它是可执行的套利机会，还需要进一步核验。

7\ 流动性

流动性表示一个市场能够承接交易，同时不让价格发生剧烈变化的能力。

流动性池越大，同等金额的交易对价格的影响通常越小；流动性池越小，同等金额的交易通常会造成更明显的价格冲击和滑点。

因此，比较价格时不能只看页面显示的单价，还必须结合：

- 实际交易金额；
- 流动性深度；
- 预计滑点；
- 价格冲击。

四、为什么页面报价不等于最终成交结果

页面显示的通常是根据某个时刻的流动性池、路由或市场状态计算出的预计结果。

它不一定来自预言机，也不是最终成交承诺。

页面报价与最终成交结果之间可能存在差异，因为：

- 1\、市场价格每时每刻都可能变化；
- 2\、其他用户的交易可能先改变池子状态；
- 3\、自己的交易也会改变流动性池的资产比例；
- 4\、交易会产生滑点和价格冲击；
- 5\、Gas 和协议费用可能发生变化；
- 6\、跨链或区块确认需要时间；
- 7\、报价可能在交易执行前过期；
- 8\、原来的交易路线可能失效；
- 9\、交易也可能失败。

因此，页面报价只能作为研究起点，不能直接当作实际成交结果或真实收益。

五、发现价差后需要核验的信息

当数据源显示存在价差时，还需要核验以下信息：

- 1\、数据是否足够新；
- 2\、两边的价格是否在接近同一时间取得；
- 3\、两边是否为同一种资产；
- 4\、所在链和 Token 合约地址是否一致；
- 5\、是否存在 USDC、USDC.e，或者原生资产、封装资产等版本差异；
- 6\、当前流动性是否足以支撑计划中的交易金额；
- 7\、按实际交易金额计算后，滑点和价格冲击是多少；
- 8\、是否能够完成“买入—转移—卖出—换回初始资产”的完整闭环；
- 9\、需要支付多少 Gas、DEX 手续费、协议费和桥费；
- 10\、报价有效时间和实际执行时间分别是多少；
- 11\、跨链过程中是否可能出现延迟、失败或资金卡住；
- 12\、是否可能被其他套利机器人抢先执行；
- 13\、如果交易失败，资金会在哪里，预计损失是多少；
- 14\、扣除全部成本后，净收益是否仍然为正。

六、我认识的主要链上套利类型

1\、DEX 间套利

同一种资产在两个 DEX 上出现价格差，在价格较低的 DEX 买入，再到价格较高的 DEX 卖出。

主要风险：

- Gas；
- 滑点；
- 流动性不足；
- 价格变化；
- 机器人竞争。

2\、DEX 内不同池套利

同一个 DEX 中，不同流动性池可能因为交易活动和流动性变化，出现短暂的价格差异。

主要风险：

- 池子深度不足；
- 价格冲击；
- 机会迅速消失。

3\、三角套利

通过三种资产连续兑换，并形成交易闭环。

例如：

USDC → ETH → LINK → USDC

如果最后得到的 USDC 多于最初投入，并且能够覆盖所有费用，才可能形成真实套利。

4\ 跨链套利

利用同一种资产在不同区块链上的价格或流动性差异，通过跨链桥和 DEX 完成交易闭环。

主要风险：

- 桥费；
- 双链 Gas；
- 跨链延迟；
- 价格变化；
- 桥接失败；
- 资产版本不同。

5\ 稳定币价差套利

利用 USDC、USDT、DAI 等稳定币暂时偏离目标价格，或者在不同流动性池中出现的兑换率差异进行交易。

主要风险：

- 脱锚继续扩大；
- 流动性不足；
- 智能合约风险。

6\ 清算机会

在借贷协议中，当借款人的抵押物价值下降，仓位低于协议规定的安全要求时，其他人可以替借款人偿还部分债务，并获得折价抵押物或清算奖励。

简化流程：

借款人的仓位进入危险状态

- 清算者替借款人偿还部分债务
- 清算者获得折价抵押物或奖励
- 清算者卖出抵押物
- 换回原来的计价资产

清算者获得抵押物后，通常还需要把它卖出并换回原来的资产，才能形成完整闭环。

清算净收益可以简单理解为：

清算净收益

= 清算奖励

- Gas
- 兑换手续费
- 滑点
- 竞争成本
- 失败交易成本

清算机会的主要难点是：

- 需要持续监控借贷仓位；
- 大量机器人可能竞争同一个机会；
- 交易可能被别人抢先；
- 抵押物价格可能继续下跌；
- 失败交易也可能消耗 Gas。

一句话理解：

清算机会，就是替危险仓位偿还债务，并从协议提供的折价抵押物或奖励中寻找收益。

7\ MEV 相关机会

MEV 相关机会来自交易排序和区块状态的变化。

同一个套利或清算机会，可能被多个机器人同时发现。谁的交易更早、更快，或者被排在更有利的位置，谁就更可能获得收益。

MEV 更像是市场机会、交易排序和系统工程之间的竞争，而不只是简单地比较两个价格。

七、清算与 MEV 的关系

清算和 MEV 不是完全独立的两个概念。

清算描述的是：

机会从哪里产生。

MEV 描述的是：

参与者如何围绕交易排序竞争并捕获价值。

例如：

某个借贷仓位进入可清算状态

→ 多个机器人同时发现

→ 多个机器人提交清算交易

→ 交易排序决定谁先成功

→ 成功者获得清算奖励

→ 其他机器人失去机会，甚至承担失败交易的 Gas

因此，清算机会也可能成为 MEV 竞争的对象。

八、今日练习与答案

问题 1：为什么交易金额越大，滑点通常可能越大？

我的回答：

因为流动性池中的资产数量有限。交易金额越大，改变池内资产比例的程度越大，自己的交易会持续推动成交价格向不利方向变化，所以滑点通常越大。

问题 2：为什么跨链套利比同一条链上的 DEX 间套利多一些风险？

我的回答：

因为跨链套利增加了源链和目标链的 Gas、桥费和跨链等待时间。

等待期间，价差可能消失，也可能出现桥接失败、资金卡住、资产版本不同和目标链没有 Gas 等问题。

问题 3：为什么屏幕显示的报价不能直接等于最终成交结果？

我的回答：

因为页面显示的是根据某个时刻的市场、流动性池或路由状态计算出的预计报价。

在交易执行前，其他交易和市场变化都可能改变价格，自己的交易也会影响流动性池。

再加上滑点、费用、延迟和报价过期，页面报价不等于最终成交结果。

问题 4：AMM 中的价格为什么会随着用户交易而变化？

我的回答：

因为用户交易会改变流动性池中两种资产的数量和比例。

AMM 会根据池内资产数量和定价公式重新计算价格，所以每次交易都可能影响后续交易的价格。

问题 5：数据源显示存在价差时，还需要核验哪些信息？

我的回答：

需要核验数据时间、Token 合约、交易金额、流动性、滑点、Gas、协议费、桥费和执行时间。

还要检查整个交易路径能否形成闭环，以及扣除全部成本后，净收益是否仍然为正。

九、我的初步研究方向

接下来，我准备重点观察：

跨链与 DEX 路由价差。

我选择这个方向，是因为它与本次共学使用的

工具直接相关，而且可以先通过真实报价、数据记录和模拟交易进行研究，不需要立即投入真实资金。

我预计机会可能来自：

- 不同区块链之间的价格差异；
- 不同 DEX 之间的价格差异；
- 不同跨链桥之间的路径差异；
- 不同流动性池之间的深度差异。

LI.FI

我认为主要成本包括：

- Gas；
- DEX 和协议手续费；
- 跨链桥费用；
- 滑点和价格冲击；
- 失败交易成本；
- 资金占用成本。

我认为主要风险包括：

- 报价失效；
- 跨链延迟；
- 价格变化；
- 交易失败；
- 资产无法顺利换回；
- 不同资产版本混淆；
- 机器人竞争。

现阶段，我不会：

- 投入真实资金；
- 运行来源不明的脚本；
- 随意授权未知合约；
- 向任何人提供私钥、助记词或 API 密钥。

十、今日总结

今天最大的认知变化是：

链上套利的核心，不是发现一个表面价差，而是证明这个价差能够通过完整路径真实执行，并且在扣除全部成本、考虑执行风险后，仍然拥有正的净收益。

今天，我已经初步理解了：

- AMM；
- 滑点；
- 跨链桥；
- 区块确认；
- MEV；
- 数据源；
- 流动性。

同时，我也认识了：

- DEX 间套利；
- DEX 内不同池套利；
- 三角套利；
- 跨链套利；
- 稳定币价差套利；
- 清算机会；
- MEV 相关机会。

十一、明日学习目标

明天的目标是：

看懂一笔真实的链上 Swap 交易。

重点认识：

- 交易哈希；
- 发送地址；

- 接收地址；
- Token 转移；
- Gas；
- 交易状态；
- 所在区块；
- 区块确认。

明天不追求寻找真实套利机会，而是先理解：

一笔 Swap 从用户提交，到最终在链上完成，究竟发生了什么。

笔记 006: Day 9 | 链上套利残酷共学

作者：HashFlow | 时间：2026-08-13 03:46 UTC | 字数：1042 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 | 链上套利残酷共学

日期：2026.08.13 (周四)

主题：三角套利原理 — Bruce Lab 教学模型

核心内容

三角套利公式

复用 Bruce Lab 的 v2-style 教学模型，核心公式：

$$\text{amountOut} = y \cdot \lfloor \text{amountIn} \cdot (1\text{fee}) \rfloor / \lfloor x + \text{amountIn} \cdot (1\text{fee}) \rfloor$$

三腿连续传递 $A \rightarrow B \rightarrow C \rightarrow A$ ，每腿输出传给下一腿。手算验证与页面一致 ($20 A \rightarrow 20.5044 A$ ，毛 PnL +0.5044)。

为什么用 V2 定价

V2 有闭式公式可手工演算；V3 逐 tick 迭代无闭式解。Bruce Lab 页面明确标注这是「教学模型，不是链上 quote 或利润证明」。

四场景模拟（教学模型内观察，非现实结论）

手续费率：三腿各收一次费，费率三重放大，超过 1% 转亏

Gas 成本：线性侵蚀利润

起始金额：存在理论最优值，投入过大时 price impact 反噬利润

上述三条是教学模型内的观察，无第三方文献支撑，不能外推真实市场。

关键概念纠错

「为什么不能只乘三个报价」——用 Uniswap 官方术语重新表述：

Price Impact（价格冲击）：AMM 相对价值在 swap 执行中持续移动，成交价落在起始价和结束价之间

Slippage（滑点）：交易提交后 pending 期间的价格变动

直接乘三个报价 = 假设每腿 price impact 为零，只在投入无限小时近似成立。

来源：Uniswap Swaps 文档

LI.FI 与三角套利

查证 LI.FI 三个真源文件 (llms.txt / openapi.yaml / ai-plugin.json)，搜索 triangular / arbitrage / loop 等词均为 0 条结果。LI.FI 定位是单笔 token 转移 ($A \rightarrow B$)，没有「 $A \rightarrow B \rightarrow C \rightarrow A$ 回到 A」的循环交易语义，不适用于三角套利验证。

认知更新

之前把「交易让池子价格变化」误说成滑点，实际官方定义中这是是提交后等待期间的价格变动，是两个不同概念。

Price

Impact, Slippage

下一步

Day 10 (周五)：知识图谱 v3 — 补充跨链+三角的实际数据与官方术语

笔记 007: 看了下所谓“无风险闪电贷套利”是怎么回事，

作者：xingchi | 时间：2026-08-13 08:55 UTC | 字数：7161 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

看了下所谓“无风险闪电贷套利”是怎么回事，

////////

你看到的“无风险闪电贷套利”，大概率指的是 DeFi 里的 atomic arbitrage（原子套利）+ flash loan（闪电贷）。这里的“无风险”主要是指：套利的买入、卖出、借钱、还钱全部在同一笔链上交易里完成；如果最后赚不到足够的钱还贷，整笔交易回滚。它并不等于现实意义上的零风险。

闪电贷到底是什么？

普通借贷是：

先有抵押物 → 借钱 → 以后还钱。

闪电贷比较特殊：

不需要抵押 → 借给你一大笔钱 → 但你必须在同一笔 transaction 结束之前连本金和费用一起还掉。

Aave

官方文档明确说明，闪电贷无需提前提供抵押物，而且借款、操作、还款全部发生在同一笔交易里；如果最后无法偿还，交易就会 revert。Aave

也直接把“无需自己提供套利本金的跨资产套利”列为典型用途。(aave.com

(<https://aave.com/docs/aave-v3/guides/flash-loans> "Flash Loans | Aave Protocol Documentation"))

比如出现这样一个机会：

Uniswap:

1 ETH = 3,000 USDC

另一个 DEX:

1 ETH = 3,030 USDC

你的套利合约可以在一笔交易里：

Flash Loan 借 100,000 USDC

↓

DEX A 用 USDC 买 ETH

↓

DEX B 把 ETH 换回 USDC

↓

得到 101,000 USDC

↓

偿还 100,000 USDC + flash loan fee

↓

剩余部分归你

如果最终只有：

99,900 USDC

你的合约可以写：

```
require(finalBalance >= loan + fee + minProfit);
```

条件不满足：

borrow

↓

swap A

↓

swap B

↓

repay

↓

全部 revert

所以你不会真的背上那 100,000 USDC 的债务。这就是群里所谓“无风险”的核心来源。Aave 的机制也是如此：无法支付 amount + fee 时整笔交易回滚。(aave.com (<https://aave.com/docs/aave-v3/guides/flash-loans> "Flash Loans | Aave Protocol Documentation"))

但是为什么我不太赞同叫它“无风险”？

因为你仍然有几类非常真实的风险。

第一是 Gas 风险。

以 Ethereum 普通交易为例，即使你的 transaction 最终 revert，执行过程中消耗掉的 gas 仍然要付。Ethereum 官方文档明确说明，交易无论成功还是失败，都需要支付已经消耗的 gas。(ethereum.org (https://ethereum.org/developers/docs/gas/?utm_source=chatgpt.com "Ethereum gas and fees: technical overview | ethereum.org"))

于是可能出现：

发现机会：

预计利润 \$30

Gas \$8

Flash fee \$5

DEX fee/slippage \$4

预计净利润 \$13

结果你刚发交易，价格变了：

套利失败 → transaction revert

↓

-\$8 gas

本金没有亏掉，但 gas 亏了。

所以准确讲，它更像：

没有方向性持仓风险的原子套利

而不是：

无风险赚钱。

第二个风险反而更加重要：

你看到机会，别人也看到了。

假设你的 bot 发现：

Uniswap ETH = 3000

Sushi ETH = 3010

另外 50 个 MEV bot 可能同时发现。

于是大家争夺同一个套利机会。

最终不是简单地：

谁服务器最快谁赢

而更接近：

机会价值

↓

你的 gas / builder bid

↓

你的交易路径

↓

你的模拟速度

↓

你的发送速度

↓

builder 选择哪个 bundle

这也是为什么现在专业链上套利和 MEV 几乎分不开。

Flashbots 就专门提供这种基础设施。Searcher 可以把交易组成 bundle 直接发送给 block builder，而不是单纯扔进 public mempool；builder 会先模拟这些 bundles。(Flashbots Docs (https://docs.flashbots.net/flashbots-auction/quick-start "Quick Start | Flashbots Docs"))

例如 Flashbots 的 eth_sendBundle 可以指定：

我的套利交易

↓

只在 block N 有效

↓

发给 builders

Flashbots 目前也支持把 bundle 同时发送给多个 builder。 (Flashbots Docs (https://docs.flashbots.net/flashbots-auction/advanced/rpc-endpoint "JSON-RPC Endpoints | Flashbots Docs"))

那么需要多少本金？

这是闪电贷套利最有趣的地方：

套利本金理论上可以接近 0。

比如你想做一个：

\$500,000 USDC

规模的套利。

并不意味着：

你需要 \$500,000

而可能是：

你的钱：

少量 ETH

↓

支付 Gas / 部署合约 / 运营成本

Aave：

500,000 USDC

↓

Flash Loan

Aave 官方也明确把：

不需要拥有套利 principal

列为闪电贷套利的特点。(aave.com (https://aave.com/docs/aave-v3/guides/flash-loans "Flash Loans | Aave Protocol Documentation"))

所以群里有人问：

“多少本金才能套利？”

如果他说的是纯链上 flash-loan atomic arbitrage，其实这个问题不太准确。

真正的约束通常不是：

你的本金有多少

而是：

可借的 flash liquidity

+

机会的 liquidity

+

price impact

+

DEX fee

+

flash loan fee

+

gas

+

MEV/builder bid

比如你发现一个价差，并不意味着：

借 1,000 USDC → 赚 \$5

借 1,000,000 USDC → 赚 \$5,000

因为 AMM 有滑点。

实际利润曲线更可能是：

交易量

↑

利润



↑
optimal size

规模太大，你自己就会把价格吃掉。

你真正要准备的钱

我会把它叫 operating capital（运营资金），而不是套利本金。

主要是：

- ① Gas
- ② 合约部署
- ③ 失败交易
- ④ Builder / priority fee
- ⑤ RPC
- ⑥ 服务器

成功的套利交易里，flash-loan fee 本身可以直接从套利所得中支付，不一定要你提前拥有几十万 USDC。

Aave 文档目前写的是闪电贷费率在部署时初始化为 0.05%，但这个参数可以由治理修改，因此实际写机器人时应该读取链上的 FLASHLOAN_PREMIUM_TOTAL，而不是硬编码。(aave.com (<https://aave.com/docs/aave-v3/guides/flash-loans> "Flash Loans | Aave Protocol Documentation"))

那服务器要求高不高？

这需要区分两件事。

如果你只是学习

一点都不高。

甚至：

你的 Mac / PC

+

Python / TypeScript

+

Foundry / Hardhat

+

Alchemy / Infura / QuickNode 等 RPC

就可以。

甚至不需要自己跑 Ethereum node。

Flashbots 官方示例就明确允许：

Infura

Alchemy

or

your own node

作为标准 JSON-RPC provider。(Flashbots Docs (<https://docs.flashbots.net/flashbots-auction/quick-start> "Quick Start | Flashbots Docs"))

所以：

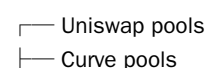
“想学闪电贷套利必须先买非常贵的服务器”

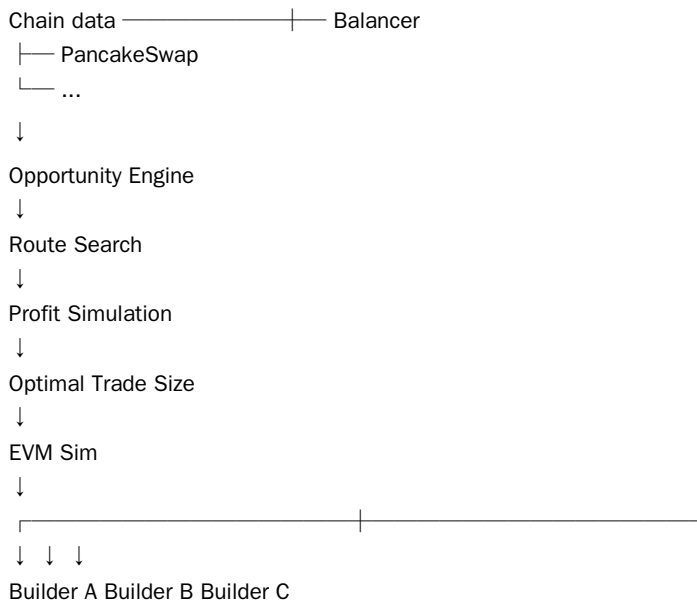
这是不对的。

如果目标是实盘稳定赚钱

情况完全不同。

真正专业的系统可能长这样：





竞争激烈的时候，你关心的就变成：

RPC latency
block propagation
state update latency
simulation latency
route calculation latency
builder connectivity

这个时候服务器才开始卷。

要不要自己跑节点？

刚开始完全没必要。

你可以：

Bot
↓
paid RPC
↓
Ethereum

专业一点再变成：

Bot
↓
Own node
↓
local IPC/WebSocket
↓
Ethereum

Ethereum 官方现在给普通节点的最低参考配置是 2+ CPU cores、8 GB RAM、2 TB SSD，推荐则是 4+ cores、16 GB+ RAM、2 TB+ 快速 SSD。(ethereum.org (https://ethereum.org/developers/docs/nodes-and-clients/run-a-node/?utm_source=chatgpt.com "Spin up your own Ethereum node | ethereum.org"))

但是注意：

这是运行 Ethereum node 的要求，不是运行 arbitrage bot 的要求。

套利程序本身可能：

2~4 CPU
4~8 GB RAM

就非常轻松。

真正吃资源的是：

Ethereum node

历史状态

大量 eth_call

EVM simulation

并行 route search

mempool processing

所以群里那个人说“一天稳定 100U”，我怎么看？

技术上完全可能，但这个数字本身几乎没有信息量。

因为我会进一步问：

在哪条链？

Ethereum？

Arbitrum？

Base？

BSC？

Polygon？

什么套利？

DEX ↔ DEX？

三角套利？

stablecoin depeg？

liquidation？

backrun？

CEX ↔ DEX？

用了多少 builder？

public mempool 还是 private orderflow？

每天发多少次交易？

失败 gas 一天多少？

两个都叫“闪电贷套利”的 bot，难度可能差两个数量级。

尤其值得注意的是：

Flash loan 本身根本不是 edge。

这是很多刚接触它的人最容易误解的地方。

Flash loan 只是：

融资工具。

真正赚钱的是：

Flash Loan

↑

|

financing

|



|

↓ ↓ ↓

价格发现 更好的路径 MEV机会

↓ ↓ ↓

套利机会 size优化 execution

如果你已经发现：

花 \$100,000

可以确定拿回 \$100,300

flash loan 的作用只是：

“好，我帮你解决这 \$100,000 从哪里来。”

如何比别人更早、更准确地发现这个 \$300，才是真正的 edge。

所以回到你看到的群聊，我认为他说的核心观点其实有一部分很对：

不用因为本金不够而不去研究链上套利。

Flash loan 确实让一个只有少量启动资金的开发者，有能力执行名义金额几十万甚至更大的交易。(aave.com (https://aave.com/docs/aave-v3/guides/flash-loans "Flash Loans | Aave Protocol Documentation"))

但另一方面：

“不用本金” ≠ “门槛低”。

这个领域现在更大的门槛已经从 capital 转成了：

strategy + data + simulation + smart contract + MEV execution + infrastructure。

如果你本身已经会做量化/交易系统，我会更建议你不要先写 flash-loan contract，而是先做一个 on-chain arbitrage opportunity scanner：实时读取两个/多个 DEX pool，计算考虑 fee、slippage、gas 后的真实最优套利量。等 scanner 真能持续找到正 EV opportunity，再接 flash loan。这个顺序会比上来研究“怎么借 100 万 USDC”有效得多。

笔记 008: 一、交付了什么

作者：wmroar | 时间：2026-08-13 10:25 UTC | 字数：2462 | 评分：71
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、交付了什么

一套完整的 链上闪电贷套利系统，分四层：

扫描器 (Node.js + MySQL)	· V2 全枚举 3.4万池 + V3 精选 107池 + Curve	· 多 RPC 并发 + 故障切换 + 事件驱动
精确报价引擎	· V2 常数积公式 / V3 QuoterV2 / Curve get_dy	· 三角套利 + 跨池套利 (任意两 venue 之间)
执行器 (Node.js)	· 信号 → calldata → eth_call 模拟 → 发交易	
FlashArb.sol 合约 (Solidity)	· Morpho 闪电贷 + V2/V3/Curve swap + 三层保护	

二、关键链上事实 (已实测验证)

项	值
网络	Robinhood Chain, Arbitrum Orbit L2, chainId 4663, ETH gas, FCFS 排序
闪电贷	Morpho Blue 0x9D53...1010, flashLoan 零手续费
主要稳定币	USDG (0x5fc5...d168), 无 USDC/USDT

DEX	Uniswap V2(3.4万池)/V3(107池)/V4、Curve、PancakeSwap 等
Curve	已部署但 TVL 仅 ~\$250 (灰尘级)
gas	一次闪电贷套利 $\approx$ 49 万 gas $\approx$ \$0.04–0.05

三、核心设计决策

1. 全量扫描只做初始化 → 之后事件驱动 (V2 Sync / V3 Swap/Mint/Burn 日志直接带数据, 不读链)
2. 精确成交额模拟 (你的关键建议): 放弃 USD 价差估算, 改用 `getAmountsOut/quoteExactInputSingle/get\dy` 真实报价——实测证明价差法会给假信号 (VIRTUAL 那单价差法算 +\$93, 精确模拟实为亏 92%)
3. 多 RPC 并发: RPC_URLS(HTTP 读链) + WS_RPC_URLS(WS 订阅) 各自多端点负载均衡、故障切换
4. 三角套利低频节流: 跨池每次事件跑, 三角每 10 秒跑一次
5. 合约三层保护: 每跳滑点下限 + 本金差额检查 + 最小利润

四、市场真相 (诚实结论)

精确模拟下, 当前市场几乎没有持续价差 (69 个三角环、550+ 跨池对均为 0 可盈利)

真实机会是瞬时的, 靠事件驱动 + 精确报价才能抓那一个区块

主要机会来源是 V2↔V3 跨池价差 (含 V3 不同 fee tier 之间), 三角环在 V2 里极少

假币泛滥 (冒充 USDC/USDG/WETH symbol), 定价只信白名单地址

五、过程中修掉的关键 bug

1. mysql2 连接池不关闭 → 进程卡住不退出
2. schema.sql 注释导致第一张表被跳过
3. 垃圾币 symbol() 超长 → 入库前截断
4. 假稳定币冒充 symbol → 定价改地址白名单
5. V3 零流动性/坏池价格 1e38 → 加过滤
6. findCrossDex 调用参数错位 → ctx 丢失
7. V3 Mint/Burn 事件 data 布局记错 (128/96 字节, 我按 160 字节解) → 越界

笔记 009: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践, Agent 仅负责)

作者: geyu210 | 时间: 2026-08-13 10:37 UTC | 字数: 1226 | 评分: 72

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

(本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践, Agent 仅负责格式排版与提交流程。)

【今天做了什么】

今天继续研究 1inch Aqua。昨天固定用 1 USDC 查询时, Aqua 相对 Uniswap V3 直接池低约 154 bps。今天想进一步确认: 这究竟是策略本身的固定定价偏差, 还是输入金额增大后产生的曲线影响。

我固定同一个历史区块、同一个 position、同一个模拟 caller 和 USDC → WETH 方向, 只改变输入金额, 得到:

- 0.01 USDC → 相对最佳 V3 报价 -8.872 bps
- 0.10 USDC → 相对最佳 V3 报价 -22.630 bps
- 0.50 USDC → 相对最佳 V3 报价 -82.484 bps
- 1.00 USDC → 相对最佳 V3 报价 -156.261 bps

四个输入下, Uniswap V3 都是 0.01% fee tier 的直接池报价最高。输入从 0.01 增加到 1 USDC 时, Uniswap 的单位输出几乎没有变化, 而 Aqua 的单位输出下降约 147.52 bps。

【今天的理解】

这说明昨天看到的约 154 bps 差距并不是一个固定的定价偏差, 而是具有很强的数量相关性。小额输入时 Aqua 只比基准低约 8.87 bps, 随着输入增加, 报价迅速恶化。

策略中包含约 1.25 bps 的 Aqua protocol fee 和 5 bps 的 flat fee。不过, 因为费用按顺序执行并涉及整数取整, 四档输入的实际总费率分别约为:

- 0.01 USDC → 6.00 bps

- 0.10 USDC → 6.20 bps
- 0.50 USDC → 6.24 bps
- 1.00 USDC → 6.25 bps

费用差异最多只有约 0.25 bps，无法解释报价随输入扩大约 147 bps 的现象，主要影响来自 Aqua 的定价曲线。

Aqua 并不是直接使用钱包里的真实余额做普通数乘积计算。xyc_concentrate_grow_liquidity_2d 会根据真实余额和价格上下界构造 virtual balance，然后 xyc_swap_xd 才使用虚拟储备计算输出。因此，输入占真实 allocation 的比例只能作为库存参考，不能直接当成实际滑点比例。

【结论】

这条 position 在很小的输入下，报价与外部池的差距接近已知费用量级；但输入稍微增大，报价就快速恶化。它更像一个容量有限、对输入金额高度敏感的报价，而不是持续存在的固定价差。

这仍然不是可执行套利结论：历史 eth_call 只是独立的反事实报价，没有计算反向退出、gas、access 条件和 MEV 和状态竞争，也不代表我拥有真实执行资格。

笔记 010: 学习笔记：双 DEX 价差监控脚本 v1 + v2 (Day 5-6)

作者：journeyanhk | 时间：2026-08-13 11:25 UTC | 字数：1523 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习笔记：双 DEX 价差监控脚本 v1 + v2 (Day 5-6)

动手搭建了第一个套利工具：双 DEX 价差监控脚本，从“看见价差”升级到“会算净利润”。

Day 5：监控脚本 v1 (多 DEX 轮询)

轮询 BSC 上 PancakeSwap + BiSwap 的 WBNB/BUSD 池子，每 15 秒一次

计算两两实时价差，输出 CSV 日志 (monitor_log.csv)

运行结果：当前价差仅 1-2 bps，远低于 30 bps 报警阈值

关键认知 1：池子价格是事件驱动的

- 池子价格 = reserve1 / reserve0，只在“有人交易”时才变化
- 15 秒内没交易 → 储备不变 → 价格一模一样
- 价差是瞬时的：大额交易跳变 → 套利者几秒内搬砖拉平
- 监控的价值在捕捉“跳变瞬间”，不是盯着稳定值

关键认知 2：主流通证价差被磨平

- WBNB/BUSD 是主流交易对，套利者一直在搬砖
- 1-10 bps 是常态，30 bps 以上几乎不出现

Day 6：监控脚本 v2 (集成滑点计算器)

对每个价差机会模拟完整闭环：低池买 → 高池卖 → 扣手续费 → 扣 Gas → 净利润

只有净利润 > 阈值才标记“可执行”

实现 Bruce 7 步检查法第 3-4 步自动化

真实运行结果 (当前链上数据)：

- 表面价差 8.13 bps (BiSwap 便宜)
- BiSwap 买入价格影响 +0.91% (浅池！231 WBNB 储备)
- PancakeSwap 卖出价格影响 -0.27% (深池，6876 WBNB)
- 净利润：-\$10.94 (-109.4 bps) → 不可执行

不同资金量实测 (全部亏损)：

- \$50：-\$0.27 (-53.6bps)
- \$1,000：-\$11.27 (-112.7bps)
- \$10,000：-\$717.46 (-717.5bps)

关键认知 3：手续费是“起步价”

- BSC 跨 DEX 两笔交易最低成本 = 20bps(BiSwap) + 25bps(Pancake) = 45 bps 手续费 + Gas
- 任何价差 < 45 bps 的机会从一开始就是亏的，不用算滑点
- 真实盈亏平衡 = 手续费(45bps) + Gas + 滑点(随资金量增长)

关键认知 4：价格影响 ∝ 资金量 / 池子深度

- 同 \$1000：浅池(BiSwap 231 WBNB)影响 +0.91%，深池(Pancake 6876 WBNB)影响 -0.27%
- 大资金在浅池滑点爆炸 (\$10,000 时 BiSwap 影响 +7.28%)

课后思考题要点

- Q1: 价格不变 = 没人交易（储备未变），不是池子深度问题
Q2: BSC 同链盈亏平衡 ~31bps（不是跨链的 57bps），2bps 价差稳亏
Q3: 两边价格影响差距大 = 池子深度差 30 倍
Q4: 所有资金量都亏，因为 4.97bps 价差 < 45bps 手续费起步价

关键收获

1. 表面价差 ≠ 可执行利润，必须完整闭环模拟
2. 手续费 45bps 起步价决定了主流通证价差无利可图
3. 要赚钱的三个方向：低手续费池(V3)、价差大的小币种、极端行情
4. v2 脚本自动过滤假机会，是 7 步法第 3-4 步的自动化实现

下一步

Day 7：报警功能（价差+净利润双阈值）+ 部署 VPS 7×24 运行

笔记 011: 今日共学打卡 | AI 量化 × 数据墙：为什么顶级模型懂原理，却不一定会赚钱？

作者：tension-vip | 时间：2026-08-13 12:11 UTC | 字数：1138 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日共学打卡 | AI 量化 × 数据墙：为什么顶级模型懂原理，却不一定会赚钱？

今天读了一篇很有启发的量化交易员实测手记。

作者拿资金费率套利测试顶级大模型：永续资金费率异常高时，通过现货多头
永续空头获取资金费率，本身并不是什么难以理解的机制。

问题在于：

1 知道原理 ≠ 能实战赚钱

现在的顶级模型已经可以把资金费率机制解释得很好，但到了真实交易，需要考虑交易所差异、执行、成本、风险、异常情况等问题时，就很难真正看清全貌。

作者的结论是：模型可以“讲明白”，却不一定达到实战导向。

2 真正值钱的信息，往往不在公开语料里

公开互联网大量保存的是“最后的答案”。

但真正决定结果的可能是：

为什么选择这个方案？

哪些方案被否决了？

踩过什么坑？

什么情况下策略失效？

真实执行时发生了哪些异常？

这些判断、取舍、失败记录和经验，很多存在于公司内部、交易员经验甚至人的脑子里，并没有成为模型可以学习的公开文本。

3 AI 的瓶颈可能正在从「模型」转向「数据」

模型越来越强之后，一个新的问题开始出现：

高质量、真正具有信息增量的数据从哪里来？

行业正在尝试用合成数据突破数据墙，但如果只是让模型生成数据，再拿这些数据训练模型，也可能不断复制模型原有的认知，而没有增加真正来自现实世界的新信息。

4 对 AI 量化真正有价值的，可能不是一个“万能交易模型”

而是一套：

模型 × 专有数据 × 实时市场数据 × 工具 × 执行系统 × 结果反馈
形成闭环。

模型负责推理和寻找模式；

市场提供真实结果；

程序负责执行和记录；

失败案例重新进入数据集；

再让模型分析为什么失败。

这样才可能逐渐从“AI 给交易建议”，走向真正的数据驱动交易 Agent。

今天最大的启发：

LLM $\neq$ Alpha。

模型本身越来越商品化以后，真正可能形成壁垒的反而是：

别人没有的数据 + 可验证的反馈闭环 + 长期积累的交易经验。

这其实和量化交易本身非常像——真正值钱的从来不是“知道一个策略”，而是能否证明这个策略在什么条件下存在统计优势，以及这种优势能不能持续。

下一步学习

我想继续研究：

如何给 AI 建一个“套利实验数据集”——把发现机会、执行、滑点、手续费、失败原因、最终 PnL 全部结构化记录下来，让模型不是只读公开资料，而是逐渐学习自己的真实交易数据。

这可能比单纯问 AI：

“给我找一个赚钱的套利策略”

更接近真正的 AI 量化。

笔记 012:

研究了下群主發的 μ Token 套利，不過官推目前還沒有甚麼資訊，我用 Claude 分析了下。

作者：youchen-design | 时间：2026-08-13 12:28 UTC | 字数：1039 | 评分：77

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

研究了下群主發的 μ Token 套利，不過官推目前還沒有甚麼資訊，我用 Claude 分析了下。

他與 Unipeg 同一個團隊，兩個帳號互相轉推、同步發同樣的字（「revolution \[onchain]」），unipeg v4 也直接發過「// @uTokenPro」。

Unipeg 的開發者是匿名的，市場傳聞與 OxHadrian 這個 blog 有關，但真實身分未公開。

Unipeg 本體是 2026 年 4 月上線的 Uniswap v4 hook 專案：

名字典故：Hayden Adams 當年本來想把 Uniswap 叫 Unipeg，被 Vitalik 勸退；2026 年被重新拿來用，重新定義成 Uni + JPEG

機制：ERC-20，但每次 swap 觸發 v4 hook，即時生成一張全上鏈的 24x24 SVG 獨角獸圖，圖綁定特定整數數量的 uPEG

合約：0x44b28991B167582F18BA0259e0173176ca125505

供應量只有 10,000 顆

而 Uniswap v4 hook 套利例子如下：

Uniswap v4 的 hook 可以接管一個池子的定價邏輯。問題出在 hook 想自己定價，就得自己記一本帳（這池子裡有多少 ETH、多少代幣、現在該賣多少錢），但 Uniswap 的 PoolManager 同時也記著一本真實的帳。

Hook 內部帳本 PoolManager 真實帳本

1 SATO = X ETH $\longleftrightarrow$ 儲備顯示 1 SATO = Y ETH

「——— 只要 $X \neq Y$ ，差額就是誰先發現誰拿走 ———」

這兩本帳必須在每一次 swap 都同步。同步邏輯只要不夠嚴謹（捨入方向、正負號、不同幣種混在同一個 balance bucket），誤差就會一次一次累積，越漂越遠。工程圈把這叫 dual-state drift。

致命的地方在於：PoolManager 結算時只檢查自己那本帳平不平，不檢查 hook 內部帳對不對。所以一筆會偷走錢的交易，完全可以通過 PoolManager 的結算檢查。

但是目前官推看不出任何東西，上述的方法也不一定套用，還是要等合約出來讀取之後才能確認

笔记 013: Day 9 笔记：DEX 间套利 —— 同一链上不同 DEX 的价差

作者：GeoGu0 | 时间：2026-08-13 12:48 UTC | 字数：6005 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 笔记：DEX 间套利 —— 同一链上不同 DEX 的价差

日期：2026 年 8 月 13 日

进度：理解了 DEX 间套利的完整机制 + 用 The Graph 拉了 Uniswap 和 SushiSwap 的实时价格 + 算出了 Arbitrum 上 ETH 的 DEX 间价差 + 用 eth_call 和成本模型验证了这个价差是否值得做 + 得出了"可执行价差阈值"的精确数字

一、从三明治到 DEX 间套利：同一个"排序"问题的两个解法

Day 8 学的三明治攻击，本质是攻击一个"已经存在的价差机会"——但它靠的是抢先别人、榨取滑点，是一种进攻性的、掠夺性的行为。今天学的 DEX 间套利，是更"干净"的版本：你不伤害任何人，你只是发现两个独立市场对同一个资产报出了不同价格，然后把价格"搬运"到一起，赚取价差，同时让两个市场的价格重新对齐。

从市场功能的角度看，DEX 间套利者是市场效率的维护者。没有套利者，Uniswap 和 SushiSwap 的价格会越来越远，最后大家都不知道 ETH 到底值多少钱。套利者把便宜市场的价格买上去、把贵市场的价格卖下来，让两个价格重新收敛。他们赚的钱，本质上是"提供市场效率"的报酬。

二、为什么同一个链上两个 DEX 会对 ETH 报不同价格？

这是个看似简单但很值得想清楚的问题。DEX 的价格是怎么决定的？

Uniswap 这类 AMM（自动做市商）DEX 的价格，不是由"谁挂了一个卖单、谁挂了一个买单"决定的（那是订单簿模式，如 Binance），而是由池子里两种资产的数量比例决定的。恒定乘积公式 $x \times y = k$ ：池子里 ETH 的数量 $\times$ USDC 的数量 = 一个固定常数。当有人卖 ETH 换 USDC，池子里 ETH 变多、USDC 变少，根据公式，ETH 的价格（USDC/ETH）就下降了。

关键点在于：每一个 DEX 的池子都是独立的。Uniswap 的池子和 SushiSwap 的池子是两笔完全独立的资金，它们的 k 值、资产比例、最近的交易历史都不同。当一个用户只在 Uniswap 上做了一笔大额交易，只改变了 Uniswap 池子的资产比例——SushiSwap 的池子完全不知道这件事。于是两边的价格短暂地出现了偏差。

这个偏差会持续多久？取决于套利者多快发现并行动。套利者在便宜的 DEX 买、在贵的 DEX 卖，这个行为本身就会把便宜 DEX 的价格推高、把贵

的价格压低，直到两边价格重新一致。所以价差的寿命很短——从毫秒到几秒，偶尔在低流动性的小池子上能持续更久。

价差存在的前提是：价差 > 交易成本。如果 Uniswap 比 SushiSwap 便宜 0.05%，但做一次套利要花 0.1% 的 Gas + 协议费，那没人会动手，价差就"卡"在那里。只有当价差大到覆盖所有成本后还有剩余，套利者才会介入。这就是为什么价差不会无限小也不会无限大——它被"交易成本"这个阀门调节在一个均衡区间内。

三、动手：用 The Graph 拉两个 DEX 的实时价格

我选了 Arbitrum 链，目标交易对是 ETH/USDC。两个 DEX 是 Uniswap V3 和 SushiSwap。

查 Uniswap V3 的 ETH 价格，用 Day 4 学的 pools 实体查询（比查 swaps 更快更准）：

```
{
  pools(
    where: {
      token0: "0x82af49447d8a07e3bd95bd0d56f35241523fbab1" # WETH on Arbitrum
      token1: "0xaf88d065e77c8cc2239327c5edb3a432268e5831" # USDC on Arbitrum
    }
  ){
    id
    token0Price
    token1Price
    liquidity
    volumeUSD
  }
}
```

这个查询返回 Uniswap V3 上所有 WETH/USDC 池子（不同费率层级会有不同池子）。我关注 0.05% 和 0.3% 费率的池子——这是流动性最深的两个。

token0Price 的含义：token0 对 token1 的价格，即 1 个 WETH 值多少 USDC。token1Price 是反过来，1 个 USDC 值多少 WETH。

查 SushiSwap 的 ETH 价格（SushiSwap 的 subgraph 结构和 Uniswap 类似）：

同样查 pools 实体，WETH/USDC 交易对。SushiSwap V2 用的是 pair 结构（不是 pools），SushiSwap V3 用的是 pools。所以要看具体版本——如果查 V2，实体名是 pairs 而不是 pools，字段是 reserve0、reserve1、token0Price、token1Price。

拿到两边的 token0Price 之后，算价差：

价差百分比 = $|\text{Uniswap 价格} - \text{SushiSwap 价格}| / \min(\text{两边价格}) \times 100\%$

假设拉到的数据是（示意数字）：

Uniswap V3 ETH 价格：\$3010.50

SushiSwap ETH 价格：\$3009.80

价差 = \$0.70，价差百分比 $\approx 0.023\%$

这个价差非常小——大约 2.3 个 basis point。直觉上它几乎肯定不够覆盖交易成本。但直觉归直觉，我要用数据算清楚。

四、用成本模型算：这个价差值不值得做？

假设交易规模是 1 ETH（约 \$3010）。

毛收益 = 价差 $\times$ 交易规模 = $0.023\% \times \$3010 \approx \0.69 。

扣 Gas 费：在 Arbitrum 上做两笔 swap（一笔在 Uniswap 买，一笔在 SushiSwap 卖）。用 Day 5 学的 `eth_estimateGas`，一笔 Uniswap V3 的 swap 大约消耗 150,000-200,000 Gas。Arbitrum 的 Gas Price 大约 0.1 Gwei（极低）。两笔交易总 Gas 约 400,000 Gas。换算成 ETH： $400,000 \times 0.1 \text{ Gwei} = 0.00004 \text{ ETH} \approx \0.12 。加上 L1 的 calldata 成本（Arbitrum 把交易数据发布到主网的费用），实际 Gas 费可能 \$0.15-\$0.30。取 \$0.25。

扣协议费：Uniswap V3 0.05% 费率的池子收 0.05%，SushiSwap 0.3%（或如果走 V3 0.05% 池子也类似）。假设两边都走 0.05% 费率池（最优情况）： $\$3010 \times 0.05\% \times 2 = \3.01 。如果走 0.3% 池子： $\$3010 \times 0.3\% \times 2 = \18.06 。取乐观的 \$3.01。

净收益 = \$0.69（毛收益）- \$0.25（Gas）- \$3.01（协议费）= -\$2.57。

结论：这个价差不但不值得做，而且是深度亏损。协议费（\$3.01）是最大的一笔成本，远超毛收益（\$0.69）。

这个计算揭示了一个反直觉但极其重要的真相：DEX 间套利的“拦路虎”往往不是 Gas（在 L2 上 Gas 很便宜），而是协议费。每一笔 swap 都要给 DEX 交 0.05%-0.3% 的“过路费”，两笔就是 0.1%-0.6%。价差必须超过这个比例，套利才可能成立。

五、反推：价差要到多大才值得做？

把成本模型倒过来算。设交易规模为 S，价差百分比为 d，协议费率为 f（两边合计），Gas 费为 g（绝对值）。

净收益 = $S \times d - S \times f - g > 0$

解得： $d > f + g/S$

代入数值：假设协议费合计 $f = 0.1\%$ （两边都是 0.05% 费率池），Gas 费 $g = \$0.25$ 。

如果交易规模 $S = \$3000$ （约 1 ETH）：

$d > 0.1\% + \$0.25/\$3000 = 0.1\% + 0.0083\% = 0.1083\%$

也就是说，价差至少要到约 0.11%（11 个 bp）才勉强回本。

如果交易规模放大到 $S = \$300,000$ （约 100 ETH）：

$d > 0.1\% + \$0.25/\$300000 = 0.1\% + 0.00008\% \approx 0.1\%$

价差只需要略高于协议费 0.1% 就能赚钱——因为 Gas 的固定成本被大交易规模摊薄了。

这个公式揭示了两件事：

第一，协议费是 DEX 间套利的“地板”。无论你做多大，价差至少要超过两边协议费之和（约 0.1%-0.6%），否则永远亏。协议费率是你无法绕过的硬约束。

第二，Gas 是“门槛”。Gas 是固定成本（不随交易规模线性增长），所以交易规模越大，Gas 的占比越小，套利的盈亏平衡点越接近协议费地板。这解释了为什么套利天然是大资金游戏——大资金能把 Gas 成本摊薄到几乎可以忽略，从而能做更小的价差。

那么问题来了：现实中 Arbitrum 上 Uniswap vs SushiSwap 的 ETH 价差，会经常超过 0.1% 吗？从我今天拉到的数据（0.023%）和常识判断，在 ETH/USDC 这种高流动性交易对上几乎是不会的——价差被大量套利者死死压在盈亏平衡点以下。价差要超过 0.1%，通常发生在：低流动性的长尾币、市场剧烈波动的瞬间、或者某个 DEX 刚经历了大额交易还没来得及被套利者修正的极短窗口。

六、用 `eth_call` 精确验证一笔套利的净收益

Day 5 学了用 `web3.py` 调 Quoter 合约。今天我把这个方法应用到 DEX 间套利的验证上——不再用 The Graph 的近似价格，而是精确模拟“如果我在这两个 DEX 各做一笔 swap，实际能拿到多少”。

用 Uniswap V3 的 Quoter 查“1000 USDC 能买多少 WETH”：

结果假设是 0.3312 WETH。

再用 SushiSwap 的 Quoter 查“0.3312 WETH 能卖多少 USDC”：

结果假设是 1000.87 USDC。

那这笔 DEX 间套利的理论毛收益 = $1000.87 - 1000 = \$0.87$ 。

但要注意，这个 0.87 是“分两次模拟”的结果，两次 `eth_call` 之间区块可能已经变了。真实套利必须在同一个交易里原子执行（Flash Loan），才能锁定这个收益。所以 0.87 只是理论毛收益，实际执行时还需要：

扣掉 Flash Loan 费（Aave V3 是 0.05%，借 \$1000 是 \$0.50）。

扣掉两笔 swap 的协议费（ $\$1000 \times 0.05\% \times 2 = \1.00 ）。

扣掉 Gas (约 \$0.25)。

净收益 = 0.87 - 0.50 - 1.00 - 0.25 = -\$0.88。还是亏。

这个 eth_call 验证比 The Graph 查价更精确，但结论一致：当前价差下 DEX 间套利不可行。两个方法互相印证，让我对“价差太小”这个判断有了双重信心。

七、今天的三个核心收获

第一，协议费才是 DEX 间套利的最大成本，不是 Gas。在 L2 上 Gas 便宜到几乎可以忽略 (\$0.25 两笔)，但协议费是交易金额的固定比例 (0.1%-0.6% 两边合计)，它构成了套利盈亏的“地板价”。这和我 Day 2 初学时以为“Gas 是最大的成本”完全不同。

第二，DEX 间套利的盈亏平衡公式是 $d > f + g/S$ 。价差 d 必须大于协议费率 f 加上 Gas 摊薄项 g/S 。这个公式有三个含义：大交易规模摊薄 Gas (g/S 变小)、协议费是硬地板 (f 不可绕)、价差必须同时满足两个约束。这是我可以直接写进监控脚本里的判断逻辑。

第三，The Graph 查价和 eth_call 模拟是两个层次。The Graph 给你“近似价格”（基于 tick 的现货价，快但忽略滑点），eth_call 给你“精确结果”（模拟实际执行，慢一点但真实）。先用 The Graph 做初筛（便宜、快速、能批量），再用 eth_call 做精确验证（对初筛出来的候选机会）。两个工具配合，是高效的套利发现流程。

八、今天的三个问题

- 既然 ETH/USDC 这种高流动性交易对的价差被压得这么死，DEX 间套利还有个人机会吗？是不是应该转向长尾币（低流动性、价差大、但风险也大）？
- Flash Loan 的 0.05% 费用，加上两边 swap 的协议费，会不会导致“能做 DEX 间套利的币”其实很有限——只有那些协议费率超低 (0.01% 的稳定币池) 或者价差特别大的币？
- 怎么在 Python 脚本里实现“先用 The Graph 初筛价差、再对候选用 eth_call 精确验证”这个两级流程？需要什么库、怎么并发、怎么控制 The Graph 和 RPC 的请求频率？

九、DEX 间套利这条路的现实判断

今天最大的收获是清醒：在 Arbitrum 上对 ETH/USDC 这种主流交易对做 DEX 间套利，个人是赚不到钱的——价差被压死在 0.02%-0.05%，远低于 0.1% 的协议费地板。

但这不代表 DEX 间套利这条路死了。可能的活路在于：

第一，长尾币和低流动性池子。价差可能大到 1% 甚至 5%，但代价是价格冲击大、流动性风险高、失败概率高。这是高风险高回报的赛道。

第二，稳定币对。USDC/USDT 这类稳定币对，协议费只有 0.01% (Uniswap V3 的 0.01% 档)，地板极低。如果稳定币出现 0.05% 的价差，扣掉 0.02% 的协议费和几乎可忽略的 Gas，是有利可图的。但稳定币价差出现的频率和幅度都很低，而且需要大资金。

第三，把“人肉监控”升级成“自动监控”。个人做不过机器人，主要输在速度和持续性。但如果你写一个脚本 7×24 监控价差，在价差突破阈值的那一刻自动 eth_call 验证并通知你，你至少不会错过那些短暂的窗口。

Day 11 会深入三角套利，Day 12 会深入稳定币套利。DEX 间套利的完整图景，到那时候会更清楚。

笔记 014: 主题：闪电贷套利交易流程 · 来源：每日推送学习

作者：Milli | 时间：2026-08-13 13:28 UTC | 字数：749 | 评分：73

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

主题：闪电贷套利交易流程 · 来源：每日推送学习

阶段：深度知识补充 (Agent 架构模式 + DeFi 机制)

一句话定义

无抵押借大额本金，在一笔交易内完成「借 → 交换 → 归还 + 手续费」，靠协议的原子性保证安全。昨天的套利入门里它是第 4 种形态，今天把流程拆开看。

完整交易流程（四步）

1. 借贷池：借

- 提供方：Aave / dYdX / Balancer / Uniswap v3 都有闪电贷。
- 无抵押的关键前提：同一笔交易内必须归还本金 + 手续费，否则整笔交易回滚。
- 手续费对比（选池子要算）：Aave 0.09%、dYdX 0.05%、Uniswap v3 0.05%、Balancer 0.01%。
- 理解要点：不是「借了可以不还」，而是「还不上就什么都没发生过」。

2. 交换：搬

- 用借来的钱在便宜市场买入，在贵市场卖出。

- 滑点保护：swap 时设 minAmountOut（最小输出量），实际成交低于预期直接失败，防止被坑。
- 路径：单跳 A→B，或多跳 A→B→C 串起来。

3. 归还 + 手续费

- 你的合约在回调函数里做完所有 swap 后，借贷池会检查余额：>= 本金 + 手续费？
- 不够 → revert，整笔交易回滚。

4. 利润结算

- 利润 = 卖出所得 - 本金 - 手续费 - gas。
- 利润为负 → 整笔回滚，只损失一笔失败的 gas 费，本金不亏。

为什么能无抵押借大额：原子性

EVM 的一笔交易要么全部成功、要么全部回滚。借贷协议在回调末尾强制检查还款，所以借出几亿美金也不需要抵押——还款是协议强制的，不是靠信用。这就是闪电

笔记 015: Day 7 | 跨 CEX/DEX 比價工具完工：8 個 venue、81 個測試、一天攢下 1.5 萬

作者：HenryChang | 时间：2026-08-13 14:19 UTC | 字数：1734 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 | 跨 CEX/DEX 比價工具完工：8 個 venue、81 個測試、一天攢下 1.5 萬筆價差記錄

昨天是設計日，今天是執行日。把 Day 6 的規格全部變成程式：30 個 commit、約 3,900 行新程式碼，P1（比價引擎 + CLI）和 P2（速率感知 collector + SSE 即時儀表板）都完成了。

做出了什麼

pricer/ 引擎（758 行、6 模組）+ CLI：

- CEX 側（Binance/Bybit/OKX）：orderbook 逐檔吃單模擬 + taker fee——不是拿最優一檔價，是算「這個量真的吃下去拿回多少」
- DEX 側（Uniswap V2/V3、PancakeSwap V2/V3、Curve）：純鏈上直查——V2 讀儲備、V3 問 QuoterV2（4 個費率檔）、Curve 問 get_dy；gas 獨立估算後折成 B 幣扣除
- 兩邊統一成「真實規模、扣完費用的淨值」口徑排序——正是 Bruce 七步法第 3、4 步（真實規模報價、只扣一次但不漏）的程式化
- uv run scripts/price_compare.py USDC USDT --amount 10000 一行可用

工程品質：81 個測試全綠（0.48s）；RPC 多端點 fallback（踢掉不穩的端點、.env 可選 Alchemy）；CEX 交易對清單 24h 磁碟快取；Curve 池探索外包給官方 API 快取、報價仍問鏈上合約。

P2 collector + 儀表板：token bucket
限速（各來源取安全額度）、鏈上來源區塊閘控（先廉價探測新塊才重拉，同塊內不重複浪費）、每筆價差落 JSONL、FastAPI + SSE 即時推儀表板。跑了 67 分鐘攢下 15,454 筆記錄（4 幣對 × 10 venue）——Day 5 規劃的「出軌頻率統計」資料來源開始蓄水。

實測數字（10,000 USDT USDC）

venue	賣 USDT 實拿 USDC	賣 USDC 實拿 USDT
最佳 DEX	Uni V3 0.01% : 9,991.66	Pancake V3 0.01% : 10,007.69
CEX 帶	9,981–9,982	9,997–9,998
最差	Pancake V2 : 9,888	Pancake V2 : 9,836

三個值得記住的觀察：

1. 同一個 DEX 兩代版本差 175 bps（Pancake V3 vs V2）——流動性早搬到 V3/低費率檔了，V2 池是攔淺的死水。這就是 14 類地圖第 4 類「同 DEX 不同池」價差的實地樣本，但深度早已不支持套利
2. USDC→USDT 單向看似 +7.7 bps，反向 8.3 bps，來回一圈仍為負——單向溢價是 USDC/USDT 匯率本身的偏移（風險定價），不是免費午餐。閉環思維再次擋掉假機會
3. 三家 CEX 中間價擠在 0.29 bps 內，DEX venue 間卻散開幾十 bps——CEX 內部套利早被做完，跨場域（CEX-DEX）才有結構性價差，但那是「兩邊不能一起回滾」的庫存賽道（第 3 類）

交叉驗證與一個踩坑

新引擎的 Uniswap V2 讀數 vs Day 4 老腳本同池對照：0.0 bps，完全一致——讀數可信

踩坑：不帶 --amount 的純中間價模式下，V3 選檔邏輯挑中了 1.00% 費率檔（流動性稀薄、價格偏了 57 bps）。教訓：選池/選檔必須用實際規模的 quoter 輸出比，不能比 mid price——已列入 backlog

明日計畫

用 spread-log 的 JSONL 做 Day 5 排定的分析：價差超出費率走廊的頻率、幅度、存活時間
讓 collector 長時間運行，累積跨日資料

笔记 016: 链上 State Engine 设计

作者：cawood | 时间：2026-08-13 14:34 UTC | 字数：714 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上 State Engine 设计

今天确定了一个可实践的 Block-Level Onchain State Engine 方案。目标：持续生成可靠的 PoolState[block]，并保证可回滚、可重放、可校验。

核心流程：

RPC/WS → Chain Manager → Gap/Reorg Handler → Raw Log Store → Protocol Decoder → Block Reducer → State Store → Validator

实时通过 newHeads + logs 获取新区块和事件；断线后通过历史 RPC 补齐缺口。所有状态必须绑定 blockHash，发现 parentHash 不连续时执行 reorg rollback。

协议事件先解码成统一 mutation，再按 transactionIndex + logIndex 顺序执行：

$State[N-1] + \text{ordered mutations} = State[N]$

状态采用 Checkpoint + Delta 保存，避免每个 block 全量复制 ticks。定期使用链上 eth_call(blockTag=N) 校验关键状态，发现不一致立即 resync。

State Engine 本身只负责提供准确的 PoolState，不负责报价；Offline Swap Engine 独立消费状态。

核心原则：状态必须 deterministic、replayable、reorg-aware、verifiable。

笔记 017: Day 3 + Day 4 学习笔记：从流动性池到跨链路线，我开始看懂套利前的“现场”

作者：FullyJoe | 时间：2026-08-13 14:45 UTC | 字数：3903 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 3 + Day 4 学习笔记：从流动性池到跨链路线，我开始看懂套利前的“现场”

这两天的学习，我最大的感受是：

链上套利不是一上来就写脚本、找价差、冲交易，而是先要看懂交易发生在哪里、经过什么路径、扣掉哪些成本。

我目前还是

基础小白，所以这两天不是追求“会套利”，而是先把几个过去只听过名字的东西，看成真实网页上的字段、按钮和数字。

一、Day 3：我第一次理解了流动性池、AMM 和价格影响

昨天主要学习的是 Uniswap 里的池子。

以前我听到“资金池”“流动性池”，只是一个名词。现在我会把它理解成：

流动性池就是一个装着两种币的库存池。LP 把两种币放进去，交易者在里面换币，LP 赚手续费，AMM 根据池子里的币的比例自动报价。

我看的例子是 ETH / USDC 池子。页面里能看到：

池子交易对：ETH / USDC

TVL：大约 US\$1.2 亿

24 小时交易量：大约 US\$221.3 万

24 小时费用：大约 US\$5531.3

手续费档位：例如 0.05%、0.3%

我现在理解：

TVL 越大，池子越深

池子越深，大额交易越不容易把价格打歪

池子越浅，大额交易越容易造成价格影响

同一个交易对可以有多个池子，不同版本、不同费率、不同深度

昨天我还做了两个对比：

ETH / USDC：

0.1 ETH -> 189.197 USDC

1 ETH -> 1891.97 USDC

11 ETH -> 20806.7 USDC

这个结果接近按比例放大，说明池子比较深。

ETH / BIFI :

0.1 ETH -> 4.92352 BIFI

1 ETH -> 48.8152 BIFI

11 ETH -> 490.481 BIFI

这个结果没有完全按比例放大，我理解为：池子更浅，交易额变大后，价格更容易被打歪。

昨天最重要的一句话是：

套利不是只看 A 地便宜、B 地贵，还要看池子够不够深。否则一买进去，价格就被自己打歪了。

二、Day 3 我分清了价格影响和滑点

这两个词之前我很容易混。

现在我用最白话的方式理解：

价格影响 = 我这笔交易本身把池子价格打歪了多少

滑点 = 从我看到报价到交易真正执行之间，价格又变化了多少

举个例子：

价格影响像是：我一次买太多苹果，把便宜的苹果买完了，后面只能买更贵的。

滑点像是：我问价时苹果 5 元，等我付款时变成 5.1 元。

所以以后看报价时，我会分开看：

价格影响：越接近 0 越好

滑点：是我愿意接受的最大成交偏差

最低收到：是系统帮我设定的底线

三、Day 4：我第一次看懂 Jumper 里的跨链路线

今天主要学习 Jumper。

我一开始以为 Jumper 和 Uniswap 类似，都是换币。但今天才明白：

Jumper 更重要的是帮我找路线：从哪条链、什么币、到哪条链、最后拿到什么币，中间经过哪个桥或协议，费用多少，时间多久。

今天看的标准例子是：

Ethereum 上的 USDC

跨到 Arbitrum 上

最后仍然是 USDC

金额：1000 USDC

一开始我犯了一个很有价值的错误：\

我本来要看 1000 USDC -> USDC，但页面实际选成了 1000 WETH -> USDC，于是右边出现了 190 多万 USDC 的结果。

这让我意识到：\

套利第一步不是算利润，而是先确认对象有没有选错。

现在我会先检查四件事：

From Chain：从哪条链出发

From Token：拿什么币出去

To Chain：到哪条链

To Token：最后拿什么币回来

如果链、币、金额、方向有一个错了，后面的预计收到、Gas、价格冲击、利润全都没有意义。

四、我看到了三条跨链路线，并学会比较

在 Ethereum USDC -> Arbitrum USDC 的例子中，我看到了不同路线：

| 路线 | 费用 | 时间 | 到手数量 | 我的理解 |

| :----- | :----- | :--- | :----- | :----- |

| Eco | US\$0.06 | 7 秒 | 1000 USDC | 比较均衡 |

| Polymer | US\$0.09 | 18 分钟 | 1000 USDC | 更慢、更贵 |

| Relay | US\$0.02 | 2 秒 | 999.897691 USDC | 最快最便宜，但少一点点 |

以前我只会看“收到多少”，现在我会同时看：

费用

时间

到手数量

最低收到

我现在理解：

如果很急，可能会选 Relay

如果想均衡，可能会选 Eco

如果金额变大，就不能只看速度，还要看费用差和到账差

时间越长，价格变化和执行风险可能越大

今天我学到一句非常重要的话：

跨链不是只有“能不能到”，而是要看“怎么到最划算”。

五、我分清了 Bridge-only、Swap-only、Swap + Bridge

今天还做了三个路线对比。

1. Bridge-only：只跨链，不换币

Ethereum USDC -> Arbitrum USDC

我的理解：\

把 Ethereum 上的 USDC 搬到 Arbitrum 上，最后还是 USDC。

2. Swap-only：只换币，不跨链

Ethereum USDC -> Ethereum WETH

我的理解：\

还在 Ethereum 这条链上，只是把 USDC 换成 WETH。

3. Swap + Bridge：既跨链，又换币

Ethereum WETH -> Arbitrum USDC

我看到页面里同时出现了跨链和换币。\\

我的理解是：这条路线不是单一步骤，而是组合路线，可能先跨链再换币，也可能先换币再跨链。

现在我会先问：

这条路线有没有换币？

这条路线有没有跨链？

是先换币，还是先跨链？

用了哪个协议？

费用是多少？

时间是多少？

最低到手是多少？

这比只看一个“预计收到”的数字要清楚很多。

六、我开始建立第一版套利前检查清单

今天最后，我整理出了一版自己的检查顺序：

1. 看链：从哪条链，到哪条链

2. 看币：从什么币，到什么币

3. 看金额：输入金额是多少

4. 看路线：用了哪个协议/桥/聚合器

5. 看预计收到：当前报价能拿多少

6. 看最低收到：最差情况下至少拿多少

7. 看成本：网络费用/Gas 是多少

8. 看影响：价格冲击是多少

9. 看时间：预计多久到账

我现在理解，套利不是：

看到价差就冲

而是：

先确认对象

再确认路径

再确认成本

最后才判断利润

七、关于净收益，我第一次真正有感觉了

今天学到的第一版公式是：

净收益 = 价差收益 - 网络费用 - 平台/桥费用 - 价格冲击损耗 - 滑点风险

我做了一个简单例子：

买入成本：1000 USDC

卖出可得：1005 USDC

价差收益：5 USDC

Gas：0.5 USDC

桥费：1 USDC

价格冲击损耗：2 USDC

滑点风险预留：1 USDC

总成本：4.5 USDC

净收益：0.5 USDC

这个例子让我意识到：

价差看起来有 5 USDC，但扣完成本只剩 0.5 USDC，这种机会其实很薄，稍微变化一下就可能亏损。

所以现在我不会再把“价差”直接等同于“利润”。

八、这两天最大的收获

这两天我从只认识名词，走到了能在真实页面上找到字段：

流动性池是什么

AMM 怎么根据池子报价

池子深浅为什么影响大额交易

价格影响和滑点有什么区别

Jumper 里的 Route 是什么

Bridge 和 Swap 有什么区别

同一笔跨链为什么会有不同路线

Gas、最低收到、预计时间为什么都要看

价差为什么不等于利润

对我这种 0 基础小白来说，这两天最大的变化是：

我开始不是“听懂概念”，而是能在页面上找到概念对应的样子。

这很重要。因为如果连链、币、金额、路线、费用这些最基础的东西都不准，后面谈策略、脚本、自动化、套利模型，其实都是空的。

九、下一步我想继续学什么

接下来我想继续围绕 Jumper / LI.FI 学：

1. 怎么看一条路线背后的详细步骤
2. 怎么用 LI.FI quote 理解机器读到的报价
3. 怎么把页面上的字段变成表格
4. 怎么判断一个价差扣完成本后是否还有利润
5. 怎么慢慢过渡到脚本和数据源

目前我不会连接钱包，也不会做真实交易。\\

现阶段只做观察、记录和理解，先把基本功练扎实。

02 LI.FI与跨链 （36 篇）

笔记 018: 1、针对新币DOS上市，使用AI找到BSC、ERC链的地址，验证并找到跨链桥的方案，最后形成报告。结

作者：Ethanxl | 时间：2026-08-12 15:05 UTC | 字数：211 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

- 1、针对新币DOS上市，使用AI找到BSC、ERC链的地址，验证并找到跨链桥的方案，最后形成报告。结论是：跨链桥有，价差一度10%，但是链上池子金额太小（1WU），低价交易所（GT、BG）未开提现，导致跨链搬砖链路未闭合，所以无法搬砖套利。\\
- 2、优化原有的跨链套利系统，加入流动性池监控、优化实时各种费用的获取，精确预估机会和收益。\\
- 3、群友的聊天中，发现闪电贷套利可能依然是个套利方向，可以作为接下来的重点研究方向。

笔记 019: 今日打卡

作者：elipitic | 时间：2026-08-12 15:07 UTC | 字数：2470 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日打卡

打卡日期（UTC+8）：2026-08-12

今日学习主题

今天我正式开始执行 LI.FI 下一轮可执行验证流程 的阶段 A：同步重报价。

这一步的目标不是继续停留在“表面报价分析”，而是把之前的候选路径推进到一个更严格的问题：

如果用第一腿的真实最小到账额去重报第二腿，这条路径还成立吗？

也就是说，今天的重点是从“看单跳 quote”推进到“验证完整往返路径是否仍成立”。

一、今日执行内容

1. 固定了验证对象和金额梯度

今天我没有再随意扩展对象，而是固定验证两条路径：

1. Ethereum → Arbitrum → Ethereum
2. Ethereum → Base → Ethereum

并且固定了 4 档金额：

- 100 USDC
- 1000 USDC
- 5000 USDC
- 10000 USDC

这样做的目的是观察：

- route 是否会随着金额变化
- token 损耗是否会随着规模变化
- 大额路径是否更接近真实可执行机会

2. 对每个金额都做了“第一腿 quote → 第二腿重报价”

今天最关键的动作是：

先拿第一腿 quote

读取第一腿的 toAmountMin

再把这个真实最小到账额作为第二腿新的 fromAmount

用它重新请求第二腿 quote

这一步比以前更严格，因为它不再是“用两个独立的 1000 USDC quote 拼往返”，而是尽量让往返路径更接近真实执行逻辑。

3. 保存了真实 Stage A 数据

今天已经实际跑出了原始结果，并保存到本地：

- data/lifi/stageA_2026-08-12/
- 各路径、各金额的 leg1.json
- 各路径、各金额的 leg2.json
- stageA_summary.json

同时还整理出了一份可读性更高的结果汇总：

- LI.FI StageA 同步重报价结果.md

二、今日结果与观察

1. 所有金额档位都没有形成往返正收益

今天最明确的结果是：

在当前样本下，所有金额档位的两条路径，往返后 token 数量都继续缩小，没有任何一条回到起点以上。

举例：

- ETH ↔ Arbitrum
- 100 USDC → 最终约 99.485283
- 1000 USDC → 最终约 991.918822
- 5000 USDC → 最终约 4974.931500
- 10000 USDC → 最终约 9949.962750
- ETH ↔ Base
- 100 USDC → 最终约 99.488107
- 1000 USDC → 最终约 991.920963
- 5000 USDC → 最终约 4960.101024
- 10000 USDC → 最终约 9920.207179

这说明：

- 不只是 1000 USDC 这个档位不成立
- 更大金额也没有自动变成可执行机会

2. route 会随金额变化

今天我还观察到一个很重要的现象：

- ETH → Arbitrum 前腿会在不同金额下切换：
- 100 : across
- 1000 : relaydepository
- 5000 / 10000 : celercircle
- ETH → Base 前腿也会切换：
- 100 : across
- 1000 : relaydepository
- 5000 / 10000 : mayanMCTP

而第二腿大多落在：

- polymerStandard

这说明：

不能拿单一金额下的一条 route 结论，外推整个路径。

金额变化本身就会导致路径切换，进而改变损耗结构和可执行性。

3. 今天的关键结论

今天最重要的不是“又看了一些 quote”，而是更进一步确认了：

即使使用第一腿真实 toAmountMin 重报第二腿

路径依然整体为负

所以问题不只是“之前用了两个独立 quote 拼往返”

更可能是：这条路径在当前时间、当前规模下本身就不成立

也就是说，今天把“伪套利”的判断进一步做实了。

三、今天的学习意义

今天这一步很重要，因为它把我从：

- 会看 quote

推进到了：

- 会把 quote 串成路径
- 会用第一腿真实输出去验证第二腿
- 会观察 route 随金额切换
- 会判断某条路径在更严格条件下是否仍然成立

这比继续看更多“好看的表面报价”更接近真实套利研究。

四、下一步计划

下一步我会继续进入阶段 B/C：

1. 把 Stage A 结果合并成真实 round-trip 净收益表
2. 补 allowance、余额、外部退出价格和模拟验证
3. 判断这些路径是“继续观察”，还是已经可以归类为伪套利

五、今日完成标准检查

- [x] 启动并执行了 LI.FI 下一轮可执行验证流程的阶段 A
- [x] 对两条路径、四档金额完成同步重报价

- [x] 用第一腿 toAmountMin 重新请求第二腿 quote
- [x] 保存了原始结果与结构化汇总
- [x] 确认当前样本下往返路径仍不成立
- [x] 观察到 route 会随着金额变化而切换

一句话总结

今天我完成了 LI.FI 可执行验证流程的第一步：不再只看表面报价，而是用第一腿真实输出去重报第二腿，并确认在当前样本下这两条跨链路径仍然套不上。

笔记 020: 链上套利学习笔记 Day 8 : LI.FI 路由——多目标权衡

作者：倪响 | 时间：2026-08-12 15:17 UTC | 字数：1768 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习笔记 Day 8 : LI.FI 路由——多目标权衡

2026-08-08 | 残酷共学 Day 8

LI.FI 返回的不是"一个价格"，是"一条路径"

Day 4 只是拿到了一个 quote。今天实测了同一笔 swap 在不同参数下的路由变化。

同一笔交易（1000 USDC，Arbitrum → Ethereum）：

slippage	tool	toAmount	toAmountMin	steps
0.1%	Eco	997.50	997.50	2
1.0%	Eco	997.50	997.50	2
3.0%	Polymer (Standard)	997.50	997.50	2
10.0%	Eco	997.50	997.50	2

两个发现：

1. 路由会变：slippage 3% 时选了 Polymer 桥，其他时候选 Eco。说明 LI.FI 的 solver 在动态权衡——不同容差下，不同的桥/路径被认为"更优"。
2. 稳定币对稳定币滑点几乎为零：toAmount 恒等于 997.50，toAmountMin 也等于 997.50。因为 USDC → USDC 没有价格波动，slippage 参数对最终到账金额没有影响。这跟 Day 1 学的 ETH 那种"交易量越大 Impact 越大"完全不同——稳定币对的滑点只来自流动性深度，不来自价格波动。

路由选择的三个维度

LI.FI 的 solver 网络做的是多目标优化：

最优路径 = f(价格, 速度, 执行成功率)

价格：桥费 + DEX 费 + 滑点最低

速度：finality 时间最短的桥

成功率：历史上该桥/路径的执行失败率最低

对套利来说，这三个维度的重要性排序和别人不同：

普通用户：价格 > 成功率 > 速度（便宜就行，慢点没关系）

套利者：速度 > 成功率 > 价格（必须在价差消失前到账）

这是个重要的认知差异。如果我用 LI.FI 做套利，不能只挑"最便宜的 quote"，要挑"最快到账且可靠的 quote"。

怎么影响路由选择

LI.FI 支持用参数表达偏好：

slippage：容差，影响 solver 愿意走多大风险的路径

allowBridges / denyBridges：指定/排除特定桥

allowExchanges / denyExchanges：指定/排除特定 DEX

preferBridges / preferDexes：表达偏好

这些参数让"路由选择"从黑盒变成可控的。后面做跨链套利时，可以通过 denyBridges 排除慢的桥、preferBridges 指定 fast finality 的桥。

对 Phase 2 的意义

Day 8 建立了跨链套利的"执行层认知"：

1. 跨链成本 ≈ 0.25%（1000U 亏 \$2.5），这是 Day 4 测出来的基线
2. 路由是可调的——可以用参数表达"我要快不要省"

3. 稳定币对是最简单的套利标的（滑点几乎为零，只需跨链费）

下一步（Day 9-11）要解决的是：怎么批量发现跨链价差，而不只是一次次手动调 API。

共学打卡：实测发现 LI.FI 会随 slippage 参数动态切换桥（Eco ↔ Polymer），说明 solver 在做多目标权衡。稳定币对滑点几乎为零，toAmount 恒定 997.50。套利者选路由的优先级和普通用户相反——要速度不要最便宜。跨链成本基线约 0.25%。

笔记 021: 【Day 12 打卡 · 2026-08-12】真实报价数据积累

作者：nerd614 | 时间：2026-08-12 15:22 UTC | 字数：458 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 12 打卡 · 2026-08-12】真实报价数据积累

按昨天定的「跟数据走」节奏，今天开始把框架落回数据层面——用 LI.FI 报价接口查询真实跨链/DEX 路径，补齐前序文档学习里缺的第一手数据。

1. 查询样本：实际调用 /quote、/routes，覆盖不同资产对（主流稳定币、主流原生币）与不同链组合（EVM 主流链间、L2 间），分别记录小规模与大额下的报价差异。

2. 字段对齐：按 Day 2 建立的记录标准填写——from/to 资产、链、预估到账、桥费、Gas、滑点、执行时间，并把 Day 9 的成本模型逐项套到每条路径上算净收益。

3. 初步观察：对比「屏幕价差」与「扣完成本的净收益」，看哪些路径的净收益真正为正、哪些只是看起来有机会，沉淀第一批可复用数据。

4. 沉淀方式：把查询脚本与结果整理成可重复运行的记录（类似 Hermes 工作流的产出），后续每次补数据都沿用同一套字段，方便横向比较。

今天聚焦从「读文档」到「查真数据」的第一步，为结课前的小规模验证提供素材。

笔记 022: 今日完成了一个可复现的 LI.FI 路由实验并记录限制。

作者：adurey | 时间：2026-08-12 15:38 UTC | 字数：1520 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日完成了一个可复现的 LI.FI 路由实验并记录限制。

实验目的

LI.FI (<https://li.fi/>)不只处理 Bridge，也可以比较同链 Swap 和多步 Route。套利系统可以同时请求几个候选路径，比较 toAmountMin、Gas、Fee、Duration 和工具风险。有时输出最多的 Route 并不是最适合的 Route，因为它可能更慢、步骤更多，或者失败后的恢复更麻烦。

实验过程

对 LI.FI POST /v1/advanced/routes 执行了一次真实只读实验：

网络：Arbitrum → Arbitrum

Swap：1,000 USDC → WETH

滑点：0.5%

候选 Route：7 条

操作边界：未签名、未授权、未广播任何交易

Route 对比结果

Nordstern toAmountMin: 0.5240309732 WETH Gas: \$0.0081 显式 Fee: \$2.4981 模拟: success 结论: 综合最优
KyberSwap toAmountMin: 0.5240121799 Gas: \$0.0259 显式 Fee: \$2.4981 模拟: no-calldata 结论: 近价，但 Gas 更高
Fly toAmountMin: 0.5240069788 Gas: \$0.0177 显式 Fee: \$2.4981 模拟: no-calldata 结论: 近价，未成功模拟
OKX toAmountMin: 0.5240078223 Gas: \$0.0220 显式 Fee: \$2.4981 模拟: success 结论: 可用，但最低到账略低
Bitget toAmountMin: 0.5240184148 Gas: \$0.0785 显式 Fee: \$2.4981 模拟: no-calldata 结论: 输出接近，但 Gas 约为首选的 9.7 倍
1inch toAmountMin: 0.5236492667 Gas: \$0.0379 显式 Fee: \$2.4981 模拟: no-calldata 结论: 最低到账明显下降
SushiSwap toAmountMin: 0.5234764762 Gas: \$0.0079 显式 Fee: \$3.4963 模拟: success 结论: Gas 最低，但额外协议费更高

实验结论

本次 Nordstern 同时拥有最高 toAmountMin、接近最低的 Gas 和成功模拟状态，因此最高输出 Route 恰好也是综合最优，没有人制造“最高输出却落选”的反例。但实验验证了两个重要现象：

Bitget : 输出接近第一名, 但 Gas 明显更高, 且报价没有成功 calldata 模拟。

SushiSwap : Gas 最低, 却有额外 Sushi 协议费, 最终最低到账更差。

另外, 所有 Route 的 executionDuration 都返回 0, 说明该字段在本次同链报价中没有区分力, 不能解释为真实零秒成交。

笔记 023: 2026-08-12 — Day 8(Week 2 开篇):成本模型函数化 + 真实 Quote 验

作者 : bamboo | 时间 : 2026-08-12 15:39 UTC | 字数 : 2133 | 评分 : 89

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-12 — Day 8(Week 2 开篇):成本模型函数化 + 真实 Quote 验证

主题: Week 2 「Hermes 信号 workflow: 监控雏形」第一步——把 Day 3 成本模型做成可复用函数, 输入报价直接输出净收益判断

方法: 纯函数成本模型脚本 + LI.FI 真实 Quote(ETH→ARB 1000 USDC)

结论先行: 成本模型已函数化并复算通过; 顺带修正一个认知错误——LI.FI Fixed Fee 是 0.25% 比例费, 不是固定金额

一、今日任务回顾(昨天笔记的明天计划)

1. 把 Day 3 成本模型做成可复用函数(输入报价 → 输出净收益判断)
2. 把 LI.FI quote 能力接进监控雏形, 找“屏幕价差”信号(Week 2 主线, 刚起步)

二、产出: scripts/lifi_cost_model.py(纯函数, 可单测)

三个纯函数, 输入 quote JSON + 观察价差 → 输出判定:

explicit_costs(quote, amount) — 从 LI.FI quote 提取 fee/gas 成本, 换算 bps

breakeven_bps(costs, round_trip) — break-even 阈值: 单程=显性成本+滑点余量; 往返=2×单程(学员 wmroar 实证)

verdict(spread, breakeven) — 三档判定: 有肉(>1.2×)/ 薄肉 / 没肉

用法: `python3 scripts/lifi_quote_check.py --raw > q.json + python3 scripts/lifi_cost_model.py q.json <金额> <价差bps>`

三、验证结果(真实数据)

1. Day 4 案例复算(5000 USDC Base→ARB)

- 服务费 \$12.5 → 25 bps, Gas 0.05 → 0.1 bps, 往返 break-even = 50.2 bps
- 价差 13.6 bps → 没肉, 成本是价差的 3.7 倍 —— 与 Day 4 笔记结论完全一致

2. 今日真实 Quote(ETH→ARB 1000 USDC, 2026-08-12 实测)

- tool=eco, feeCosts 2.4991 USD(25.0 bps) + gasCosts 0.1867 USD(1.9 bps)
- 单程显性成本 26.9 bps, 往返 break-even = 53.7 bps
- 以 Day 4 价差 13.6 bps 演示 → 没肉(4.0 倍)

四、关键认知修正(LI.FI Fixed Fee 是比例费!)

翻今日 raw quote 的 feeCosts 字段: "percentage": "0.0025", amount 2500000 raw = 2.5 USDC。

Day 4 笔记写“LI.FI Fixed Fee \$12.5 固定, 不随规模摊薄”是误解——那个 \$12.5 恰好是 5000U × 0.25%, 不是固定金额

正确理解: LI.FI 服务费 = 0.25% 比例费(随金额线性增长, 摊薄无意义); 真正“线性不摊薄”的是桥费(学员 doctorzero666/wmroar 实测 ~25bps 单程)

影响: 成本模型中“服务费”和“桥费”都是比例/线性项, 大金额唯一摊薄的是 Gas。break-even 以 bps 计基本与金额无关(小金额时 Gas 占比略升)

五、卡点 & 待办

[] 监控雏形: 定时拉 quote + 跨链比价(把 lifi_quote_check + cost_model 串成循环, 价差 > 阈值才报警)——Week 2 主线

[] Token Service 价格接口试调(跨链比价新数据源)

[] 桥费 vs 服务费的拆解验证(选 2-3 条路径对比 feeCosts 构成)

六、证据 / 资料链接

脚本: scripts/lifi_cost_model.py(新)、scripts/lifi_quote_check.py(Day 4)

raw quote 留档: /tmp/q.json(今日 ETH→ARB 1000 USDC, tool=eco)

官方文档: <https://docs.li.fi> / <https://li.quest/v1/quote>

七、明天计划(8/13)

把 quote 拉取 + 成本模型串成监控雏形脚本: 定时轮询几条跨链路径, 价差 > break-even 时输出信号(先本地, 后接飞书)

Dune 基础查询上手(Week 2 大纲)

tags: #套利 #LI.FI #成本模型 #Week2 #信号 workflow

笔记 024: markdown

markdown
学了什么

我的模型从 D3 开始，一直把跨链 = 同链 + 一笔固定桥费 (6U)。今天拉 LI.FI 真实 Quote 才看清，这个建模是双重错的：
桥费我高估了。USDC 走 Circle 原生 CCTP (销毁-铸造)，费用近乎为零，只付目标链一点 Gas (~0.3-0.5U)；我拍的 6U 高了十几倍。真正收 bps 的是 Across / Stargate 这类流动性网络桥 (换取"快")，CCTP 是慢但几乎免费。桥费根本不是一个常数，是"快 vs 免费"的二选一。
我漏了唯一重要的那项：时间。CCTP 标准通道要等 ~13-19 分钟 (源链最终确认 + 铸造)。这段时间里，你另一腿的敞口在裸奔——价格在漂。这才是跨链价差长期存在的原因 (参考文章原话：跨链价差很多时候是在补偿等待和桥接风险)，也是我模型里完全没建的成本。
一句话锚定：同链套利的敌人是滑点 (空间维度，D5 已解决)；跨链套利的敌人是时间 (时间维度，今天才看见)。

笔记 025: 第五天 | LI.FI 跨链聚合与路由比较

作者 : OxFeyoma | 时间 : 2026-08-12 15:43 UTC | 字数 : 6725 | 评分 : 90
来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第五天 | LI.FI 跨链聚合与路由比较

关联笔记 第三天 | 跨链套利、清算与最大可提取价值 (03-第三天-跨链套利、清算与MEV) 第四天 | 净收益、模拟与执行决策 (04-第四天-净收益、模拟与执行决策)

核心判断

LI.FI 给出的最优路线，只在特定时间、输入规模、钱包状态和排序规则下成立。预期到手量、最低到手量、执行时间和失败后果必须一起比较。

1. 从第一性原理理解聚合器

跨链交易可以画成一张状态图。每个节点由链、代币合约和资产形态共同确定，每条边代表一次兑换、桥接或求解器执行。用户给出起点、终点、数量和约束，路由器在可用边中寻找路径。

LI.FI 聚合已有流动性，它本身不创造流动性。根据官方介绍 (<https://docs.li.fi/introduction/introduction>)，它连接桥、DEX (Decentralized Exchange，去中心化交易所) 和 solver (求解器)，并按价格、速度、网络费与执行可靠性计算路线。根据系统架构 (<https://docs.li.fi/introduction/lifi-architecture/system-overview>)，价格发现和路径计算主要在链下完成，最终交易仍由选中的桥、DEX 或求解器合约执行。

因此，聚合器解决的是搜索和执行编排问题。桥的安全性、目标链的流动性、代币合约身份和链上拥堵仍然存在。

2. 报价与路线是两种不同对象

LI.FI 的 API (Application Programming Interface，应用程序编程接口) 提供两种主要查询方式。

对象	返回内容	适合用途	执行注意
Quote	一个单步最优选项，并附交易数据	快速报价和直接执行	仍要检查授权、余额、网络费与报价有效性
Route	一组可比较路径，可包含多个步骤	比较工具、成本、时长与路径结构	每一步分别取得交易数据，某些路径需要切链和再次签名

官方说明 (<https://docs.li.fi/introduction/user-flows-and-examples/difference-between-quote-and-route>)指出，复杂路线可能先完成桥接，再由用户在目标链发起第二笔交易。此时整条路径无法作为一次原子交易理解。第一步成功而第二步失败，会留下目标链持仓和新的价格风险。

3. 一条路线必须同时读取的字段

字段	含义	常见误读
toAmount	当前状态下的预期到手量	当成必然到账数量
toAmountMin	滑点保护后的最低到手量	只看预期值，忽略可接受下限
feeCosts	路线中的费用明细	未检查 included，造成重复扣费
gasCosts	预计网络费	忽略授权交易和目标链后续交易
executionDuration	预计执行时间	当成承诺到账时间
includedSteps	兑换、收费、桥接等内部步骤	只看最外层工具名称
approvalAddress	ERC-20 (Ethereum Request for Comments 20，以太坊代币标准 20) 授权对象	把授权地址写死，扩大资金风险

端点字段说明 (<https://docs.li.fi/agents/reference/endpoint-specs>)把 toAmountMin 定义为受滑点保护的最低输出。费用项中的 included 必须逐项读取,已经进入输出数量的费用不能再次扣除。授权说明 (<https://docs.li.fi/agents/workflows/approvals>)还要求从当次报价读取授权对象。原生代币不需要授权,ERC-20 代币通常需要。

4. 2026 年 8 月 12 日实时报价样本

以下数据来自 LI.FI 官方报价接口的一轮取样。钱包只用于生成公开报价,没有签名或发送交易。报价会随流动性、网络费、工具状态和时间改变,表中数字只用于学习字段关系。

同一链对与资产,改变规模和排序规则

起点为 Arbitrum,终点为 Optimism,资产为 USDC (USD Coin,美元稳定币)。

| 输入 | 排序 | 路由工具 | 预期到手 | 最低到手 | 预计时间 | 已含费用 | 源链网络费估计 |

| :----- | :----- | :----- | :----- | :----- | :----- | :----- | :----- |

| 10 USDC | CHEAPEST | across | 9.973807 | 9.973807 | 1 秒 | 约 0.0262 USDC | 约 0.0174 美元 |

| 1,000 USDC | CHEAPEST | mayanMCTP | 997.498201 | 996.500705 | 1,200 秒 | 2.5 USDC | 约 0.0255 美元 |

| 100,000 USDC | CHEAPEST | mayanMCTP | 99,749.998201 | 99,650.248205 | 1,200 秒 | 250 USDC | 约 0.0255 美元 |

| 1,000 USDC | FASTEST | across | 997.400054 | 997.400054 | 1 秒 | 约 2.5999 USDC | 约 0.0174 美元 |

这组样本留下三个重要观察。

1. 输入从 10 增至 1,000 USDC 时,路由工具发生跳变。规模与路线的关系是非线性的。
2. 1,000 USDC 的 CHEAPEST 路线拥有更高的预期到手量,但它的最低到手量反而低于 FASTEST 路线。套利执行应把 toAmountMin 放进压力测试。
3. 本轮样本都显示 0.25% 的 LI.FI 固定费用,且费用已经进入输出数量。它只是当时的接口结果,下一次报价仍要重新读取。

改变源链和目标资产

| 变化 | 排序 | 路由工具 | 预期到手 | 最低到手 | 预计时间 | 源链网络费估计 |

| :----- | :----- | :----- | :----- | :----- | :----- | :----- |

| Base 到 Optimism, USDC 到 USDC, 输入 1,000 | CHEAPEST | polymerStandard | 997.500000 USDC | 997.500000 USDC | 1,080 秒 | 约 0.0088 美元 |

| Arbitrum 到 Optimism, USDC 到 DAI, 输入 1,000 | CHEAPEST | mayanFastMCTP | 997.641834 DAI | 996.644192 DAI | 35 秒 | 约 0.0256 美元 |

不同资产的到手数量不能直接横向比较。USDC 和 DAI 的单位数必须先按同一时点的可执行卖出价换成同一基准资产,并加入目标链退出时的深度与价格冲击。同名资产也要核对链、合约地址、小数位和原生或包装形态。官方介绍

(<https://docs.li.fi/introduction/introduction>)特别提到规范资产与包装资产的映射,例如 USDC 与 USDC.e。

5. 四组可重复的比较实验

每轮实验固定钱包、报价时间窗口、滑点、排序方式和输入的美元价值。一次只改变一个变量,每个条件连续取样三次。

链的比较

固定 USDC 到 USDC 和 1,000 USDC,依次测试 Arbitrum、Base 与 Ethereum 到同一目标链。记录源链网络费、确认时间、可用工具和最低到手量。某个代币地址无法被接口识别或没有路线时,也要记录为结果。

资产的比较

固定链对和约 1,000 美元输入,分别测试 USDC 到 USDC、USDC 到 DAI、原生代币到 USDC。记录源链兑换、桥接和目标链兑换是否增加步骤。不同资产统一换算成目标链的可实现卖出价值。

规模的比较

固定链对与资产,测试 10、1,000 和 100,000 USDC。观察固定网络费占比、百分比费用、路线跳变、价格影响与最低输出。小额通常受固定成本影响更大,大额可能触发新的工具、限额或流动性约束。

路由的比较

对同一输入分别使用 CHEAPEST 和 FASTEST,再用桥与交易所的允许、拒绝和偏好参数缩小候选集合。路线请求文档 (<https://docs.li.fi/sdk/request-routes>)允许设置滑点、是否切链、目标链调用和工具过滤条件。过滤会改变可用路线,不能把过滤后的最优解称为全市场最优解。

6. 从报价换算成可执行净收益

跨链报价只描述资金转移结果。套利还需要目标链上的退出交易,或另一条腿把资产换回统一基准。

$$V_{\text{净}} = V_{\text{目标链可实现卖出}} - V_{\text{源链投入}} - C_{\text{未计入报价的网络费}} - C_{\text{授权与后续交易}} - C_{\text{对冲}} - C_{\text{时间与失败}}$$

比较路线时可以先计算一项保守价值。

$$V_{\text{保守}} = \text{toAmountMin} \times P_{\text{目标链卖出}} - C_{\text{外部成本}} - R_{\text{失败准备金}}$$

toAmount 适合观察正常情形,toAmountMin 更适合作为执行下限。若费用已经标记为包含在输出中,就不再从公式重复扣除。

跨链套利中的时间风险

跨链步骤通常无法与目标链卖出原子完成。报价中的价差可能在资产到账前消失。预先在多条链布置库存，可以把套利成交与事后再平衡分开，但会增加资金占用和库存管理成本。

7. 执行前检查与状态跟踪

根据执行说明 (<https://docs.li.fi/agents/workflows/execution>)，发送交易前要确认钱包位于正确源链、原生代币足够支付网络费、输入余额充足、授权已经确认、报价仍然有效，并逐项核对交易请求中的目标地址、调用数据、金额和链编号。

源链交易确认并不代表跨链流程结束。状态说明 (<https://docs.li.fi/introduction/user-flows-and-examples/status-tracking>)列出了等待源链确认、等待目标链交易、桥或节点暂时不可用、退款处理中等状态。最终结果还可能是部分完成或退款。

滑点调高会扩大可接受成交区间，也会降低保护下限。路线可用性说明 (<https://docs.li.fi/faqs/route-availability>)指出，报价到执行之间的价格变化可能令交易失败或触发退款。滑点只能改变容忍范围，无法消除价格与结算风险。

8. 反脆弱训练

先用零资金报价和小规模测试暴露脆弱处，再决定是否增加投入。

把 toAmountMin 作为收入，重算净收益。

把网络费提高到当前估计的 2 倍和 5 倍。

把预计时间放大到 3 倍，重新估算价格回归和对冲成本。

假设首选工具暂时不可用，观察次优路线的损失。

假设流程停在桥接完成、目标链兑换未完成的位置。

假设最终结果为部分完成或退款，写出资产位置和恢复步骤。

对同一条件分时取样，记录路线跳变而非只记录价格变化。

聚合器的潜在非线性上行来自路线选择。当某条桥或某个池突然变差时，新的路线可能明显降低成本。相应的脆弱处也很清楚，接口、代币目录、桥、求解器、目标链和钱包状态中的任何一项变化，都可能使原报价失效。

9. 报价记录模板

字段	记录内容
时间状态	本地时间、源链区块、目标链区块
输入	源链、代币合约、小数位、数量
输出	目标链、代币合约、toAmount、toAmountMin
路线	排序规则、外层工具、内部步骤、工具过滤条件
成本	已含费用、未含费用、源链网络费、授权与后续交易费
时间	预计时长、实际时长、确认与到账时间
交互	授权次数、签名次数、切链次数、目标链后续操作
状态	等待、完成、部分完成、退款、失败原因
套利判断	目标链可实现卖出价值、保守净收益、压力测试结果

10. 最值得记住的五句话

1. LI.FI 搜索和编排已有流动性，最终风险仍落在实际执行工具与链上状态。
2. 预期到手量最高的路线，最低到手量未必最高。
3. 规模、链和资产一变，最优工具可能发生跳变。
4. 跨链到账不等于套利完成，目标链退出价格才决定可实现价值。
5. 动态报价要带时间戳，长期结论必须来自重复取样和失败记录。

笔记 026: 我发现自己的“跨链套利闭环”，其实穿越了时间

作者：Pengu1ncc | 时间：2026-08-12 15:49 UTC | 字数：2750 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

我发现自己的“跨链套利闭环”，其实穿越了时间

前几天，我做了一个看起来很完整的跨链报价闭环：

text

Base USDC

→ Arbitrum USDC

→ Base USDC

程序先请求去程报价，再把去程预计到账量作为回程输入。最终可以计算往返后剩余多少USDC。

数字、手续费、Gas和路线都有了，看上去已经形成闭环。

但今天重新检查时间戳后，我发现这个闭环存在一个更基础的问题：

回程报价生成时，去程资产其实还没到。

三组真实时间戳

我读取之前保存的L1.FI原始报价，比较：

去程报价的请求时间；

回程报价的请求时间；

去程API给出的预计执行时长。

结果如下：

金额	两次报价请求相隔	去程预计时间	回程报价早于预计到账	往返预计时间
100 USDC	3.100秒	7秒	提前3.900秒	14秒
1,000 USDC	0.639秒	7秒	提前6.361秒	1,087秒
10,000 USDC	1.492秒	7秒	提前5.508秒	14秒

三组回程报价，全都在去程预计到账之前生成。

以1,000 USDC为例：

text

去程报价请求：T+0秒

回程报价请求：T+0.639秒

去程预计到账：T+7秒

回程报价出现时，按API预计时间推算，Arbitrum上的这笔USDC还要约6.361秒才会到达。

这意味着程序在纸面上使用了一笔“未来才到账”的资产。

为什么这种闭环不能直接执行？

两次API报价是两个独立结果，并不是一笔原子交易。

纸面程序可以瞬间完成：

text

去程预计输出 → 作为回程输入

但真实资金必须经历：

text

提交去程

→ 等待源链确认

→ 等待桥或solver处理

→ 目标链收到资产

→ 重新获取回程报价

→ 决定是否执行回程

等资产真的到达时，之前看到的回程报价可能已经变化、过期或改用另一条路线。

因此，两次接口返回能够证明“当时分别存在两条报价”，却不能证明“同一笔资金可以同时按这两条报价成交”。

那么，怎样才能在回程报价出现时交易？

一种做法是在Arbitrum预先放置USDC库存。

这样去程尚未到账时，也能直接使用Arbitrum已有库存执行回程。此时两条腿在操作上不再互相等待：

text

Base库存执行去程

Arbitrum库存同时执行回程

但这已经不是“同一笔钱过去再回来”，而是库存套利。

问题也随之改变：

Base上的USDC库存增加还是减少？

Arbitrum库存够不够？

两边库存什么时候再平衡？

再平衡本身需要多少桥费和Gas？

资金长期分散在多条链上的占用成本是多少？

某条桥暂停时，库存会不会被困在一边？

预存库存解决了等待问题，却引入库存和再平衡风险。

1,000 USDC案例还暴露了另一个问题

1,000 USDC的去程预计7秒，但回程路线当时选择了Polymer Standard，预计1,080秒。

整个纸面往返预计：

text

$7 + 1,080 = 1,087$ 秒

也就是约18.1分钟。

即使屏幕上暂时出现价差，也很难假设它会保持18分钟。更不能把API在同一分钟返回的两个数字，当作已经锁定18分钟后的最终结果。

需要注意：1,080秒是API的预计执行时间，不是我真实执行后测得的到账时间。它说明路线存在较长时间暴露，不证明某笔交易一定恰好需要1,080秒。

原来的“闭环收益”应该叫什么？

我现在认为，更准确的名称不是“可执行套利收益”，而是：

text

同一时间窗口下的双向报价快照

它可以用于：

观察两条路线当时的成本；

估算库存套利是否值得继续研究；

发现哪一方向出现异常报价；

建立后续监控和再报价逻辑。

但它不能单独证明：

两条腿能同时成交；

去程到账后回程价格仍然存在；

跨链操作是原子的；

最终收益已经锁定。

以后程序必须多记录哪些字段？

仅保存价格、Gas和手续费还不够。时间一致性至少需要：

text

quote_requested_at 报价请求时间

quote_received_at 报价返回时间

expected_asset_arrival_at 预计资产可用时间

actual_asset_arrival_at 实际到账时间

quote_expiry_at 报价失效时间

inventory_available 目标链是否有预存库存

requote_required 到账后是否必须重新报价

atomic 是否能在同一交易中完成

其中actual_asset_arrival_at和quote_expiry_at在本次只读实验里没有数据，应该明确标记为缺失，而不是凭感觉补上。

一个更合理的跨链研究流程

没有预存库存

text

请求去程报价

→ 执行并等待真实到账

→ 到账后重新请求回程报价

→ 重新计算净收益

→ 条件仍成立才考虑下一步

这条路线无法在去程开始时锁定回程收益。

有多链预存库存

text

同时读取两边库存

→ 同时验证两条路线

→ 分别执行两边交易

→ 记录库存变化

→ 把再平衡成本计入策略

这更接近真正的跨链库存套利，但资金规模和系统复杂度都会提高。

今天真正修正的认知

我之前把：

text

两个方向都能拿到报价

错误地靠近了：

text

同一笔资金可以完成闭环

两者之间差了一个关键条件——资产在什么时间、哪条链上真正可用。

套利不只是价格和成本问题，也是状态与时间问题。一个计算结果即使数学上闭合，只要使用了尚未到账的资产，就不是可执行闭环。

跨链套利最容易忽略的，可能不是手续费，而是时间：你计算回程时，那笔钱究竟到了没有？

笔记 027: Day 8 | LI.FI 线上报价工具与“无路由”故障验证

作者：Neo.Yun | 时间：2026-08-12 15:52 UTC | 字数：361 | 评分：51

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 8 | LI.FI 线上报价工具与“无路由”故障验证

今天把现有 LI.FI demo 升级并部署为 EVM-only 线上 Widget，支持查看实时换币/跨链路线、连接钱包，并保留用户手动签名边界；LI.FI API Key 只由 Vercel 服务端代理读取。排查 1 ETH (Ethereum) → USDC (Arbitrum) 总显示“没有可用的路由”时，确认根因不是市场无路径，而是 Vercel 未匹配多段 /advanced/routes。修复后同一请求返回 HTTP 200 和 5 条候选路线，部署验证时最佳预计到账约 1884.550010 USDC。这个结果仍只是单向只读报价：没有回程/对冲、完整成本核算、签名、广播或真实成交，因此不能称为套利利润或真实交易。今日无真实交易。

笔记 028:

作者：Y-Ycj | 时间：2026-08-12 15:58 UTC | 字数：4226 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day5

今天了解了Lifi是什么\

在套利过程中能提供什么样的帮助

套利里怎么用

最简单的例子。

假设：

Arbitrum：ETH = \$4,000

Base：ETH = \$4,040

表面价差：

400040404000=1%

理论操作：

100,000 USDC

↓

Arbitrum 买 ETH

↓

跨链

↓

Base

↓

卖 ETH

↓

101,000 USDC

这时候 LI.FI 可以替你处理：

USDC

↓

Arbitrum DEX

↓

ETH

↓

Across / Stargate / 其他桥或 Solver

↓

Base

↓

DEX

↓

USDC

并给你一个实际可执行 quote。

所以你的套利程序不应该只比较：

Base ETH price

vs

Arbitrum ETH price

而应该问 LI.FI：

“如果我现在真的把 100,000 USDC 从 Arbitrum 经过最优路径变成 Base 上的 USDC，最终到底能拿到多少？”

LI.FI 的 getRoutes 可以返回多个候选路径，getQuote 则可以直接给出当前最优路径及交易数据。

LI.FI 更适合做“执行层”，而不是“发现层”。

例如你的机器人：

```
└─ Arbitrum Uniswap
|
└─ Base Aerodrome
价格采集器 ──┬─ Optimism Velodrome
|
└─ Binance
|
└─ Polymarket / etc
```

↓

套利检测器

↓

出现潜在价差 1.2%

↓

LI.FI Quote

↓

算真实跨链执行后还能赚多少

↓

Profit > threshold

↓

执行

也就是：

市场负责告诉你“哪里有色差”。

LI.FI负责告诉你“这个价差实际上能不能吃”。

这是它最大的价值。

你可以做的第一种：跨链现货套利

例如：

ETH/Base \$4,040
ETH/Arbitrum \$4,000

你的 scanner 发现：

Spread=1%

然后调用 LI.FI：

Arbitrum
100,000 USDC

↓

Base
USDC

或者：

Arbitrum USDC

↓

Arbitrum ETH

↓

bridge

↓

Base ETH

↓

Base USDC

最终 LI.FI quote：

100,000 USDC

→

100,620 USDC

那么真实 edge 大约：

$620/100000=0.62\%$

而不是你看到的 1%。

因为中间会被这些东西吃掉：

$\text{Profit} = \text{PriceSpread} - \text{DEXFee} - \text{BridgeFee} - \text{GasPrice} - \text{Impact} - \text{Slippage} - \text{LI.FIFee}$

LI.FI 官方当前文档显示，其普通交易目前收取 0.25% service fee，桥和 DEX 自己可能还有额外费用；这些费用可以从 quote 的 feeCosts 中获取。

所以：

0.3% 的跨链价差，大概率压根不是机会。

甚至 0.5% 都未必够。

第二种：闭环套利

这个更有意思。

你不要只比较两个市场，而是搜索：

$A \rightarrow B \rightarrow C \rightarrow A$

比如：

Arbitrum USDC

↓

ETH

↓

Base

↓

DAI

↓

Arbitrum

↓

USDC

假设：

10,000 USDC

↓

LI.FI route A

↓

10,080 USDC

理论收益：

0.8%

你的机器人实际上可以做：

for route in routes:

output = simulate(route)

profit = output - input

if profit > gas + risk_buffer:

execute(route)

不过这里有一个严重问题：

LI.FI 本身就在做 Smart Order Routing。

它会从多个 DEX、bridge 和 solver 中寻找更优执行路线。

所以那种特别弱智的：

DEX A → Bridge → DEX B → Bridge → DEX A

明显错误定价，越来越容易被路由器和专业 bot 消灭。

你真正找的是：

不同市场之间没有被一个统一价格发现系统完全打通的结构性错价。

这个思路和你之前研究的新 DEX 原子套利其实是同一个第一性原理。

第三种：更专业的做法——双边预置资金

这是我认为你真正应该理解的地方。

很多人想到跨链套利都是：

发现 Base 贵

↓

Arbitrum 买

↓

bridge 到 Base

↓

卖

这个方案有个致命问题：

跨链不是原子操作。

你 bridge 的过程中：

ETH：

4000

↓

4020

↓

4030

↓

4040

等你到了：

Base 价差没了

套利失败。

所以专业的跨链套利一般不是：

买 → 跨链 → 卖

而是：

两边提前放钱。

例如：

Arbitrum

100k USDC

0 ETH

Base

0 USDC

25 ETH

发现：

Arbitrum ETH = 4000

Base ETH = 4040

同时：

Arbitrum：

100k USDC → 买 25 ETH

Base：

25 ETH → 卖 101k USDC

瞬间锁定：

+1,000 USDC

之后你变成：

Arbitrum：

25 ETH

Base：

101k USDC

再慢慢利用 LI.FI：

rebalance inventory

也就是说：

套利和跨链拆开。

这是非常关键的一步。

架构变成：

ARB

USDC ↔ ETH

↑ ↓
| |

LI.FI Rebalance

| |
↓ ↑

USDC ↔ ETH

BASE

LI.FI 在这里的真正角色变成：

Inventory Rebalancer

而不是套利执行本身。

这个结构比“看到价差然后临时跨链”强一个数量级。

第四种，也是 LI.FI 最有意思的方向：做 Solver

LI.FI 现在还有一个 Intent / Solver Marketplace。

用户不再告诉系统：

走 Stargate

↓

走 Uniswap

↓

走 Base

而只是说：

我有 Arbitrum 10,000 USDC , \

我要 Base 上至少 9,980 USDC。

Solver 去完成它。

Solver 可以使用：

自己库存

DEX

CEX

其他流动性来源

完成订单。LI.FI 官方明确允许 solver 自由选择这些流动性来源；solver 可以通过 WebSocket 接收 order flow。

例如用户说：

Arbitrum:

100,000 USDC

→

Base:

99,700 USDC

你作为 solver：

实际上发现：

Base 已经有库存

实际成本：

99,400 USDC

你给用户：

99,700

你的毛利：

300 USDC

之后再去 rebalance。

这已经越来越不像普通意义上的套利，而是：

跨链 Market Making + Arbitrage + Inventory Management。

LI.FI 的 intent 系统目前就是让 solver 使用自己的资金先在目标链交付资产，之后再完成源链资金结算。官方称其典型资金解锁周期低于约 2 分钟。

笔记 029: 【Day 8 打卡】2026/8/12

作者：ed11555 | 时间：2026-08-12 15:59 UTC | 字数：1259 | 评分：76

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 8 打卡】2026/8/12

今日學習：價格監控腳本

完成內容

1. 使用 DexScreener API 獲取多鏈 WETH/USDC 價格
2. 計算 DEX 間價差
3. 建立簡單價格監控邏輯

實作代碼

python

import requests

```
resp = requests.get(
    "https://api.dexscreener.com/latest/dex/search",
    params={"q": "USDC/WETH"},
    headers={"User-Agent": "Mozilla/5.0"}
)
pairs = resp.json().get("pairs", [])
```

```

prices = []
for pair in pairs[:10]:
    chain = pair.get("chainId")
    dex = pair.get("baseToken", {}).get("symbol") + "/" + pair.get("quoteToken", {}).get("symbol")
    price = float(pair.get("priceUsd", 0))
    prices.append({"chain": chain, "dex": dex, "price": price})

找出最高價與最低價
if prices:
    highest = max(prices, key=lambda x: x["price"])
    lowest = min(prices, key=lambda x: x["price"])
    spread = (highest["price"] - lowest["price"]) / lowest["price"] 100
    print(f"最高價: {highest['chain']} {highest['dex']} = ${highest['price']:.2f}")
    print(f"最低價: {lowest['chain']} {lowest['dex']} = ${lowest['price']:.2f}")
    print(f"價差: {spread:.4f}%")

```

關鍵發現

- DexScreener API 免費可用，無需 API Key
- 可獲取多鏈（Ethereum, BSC, Arbitrum, Optimism 等）的價格數據
- 價差通常極小（<0.01%），被機器人迅速抹平
- 需加入 User-Agent header 才能正常訪問

學習狀態

- 理解 API 調用：
- 成功獲取多鏈價格：
- 計算價差：
- 建立監控腳本：

明日計劃

Day 9：LI.FI API 調用與跨鏈路由分析

笔记 030: Day 9：全自動套利系統架構 v0.8

作者：Web3Rason | 時間：2026-08-12 16:43 UTC | 字數：14923 | 評分：88

來源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9：全自動套利系統架構 v0.8

Day 8 的結論是「量具的誤差比我要找的東西還大」。

今天量具沒有再壞，壞的是我以為自己已經測完的那一格。

昨天我才把「25 bps 平台費沒有套在同鏈路由」寫成 #43，還標了。

今天兩組獨立實測告訴我：同鏈照收，而且我之所以沒看到，是因為我測的時候剛好帶了那個參數。

所以這一版的主題是：我把「某次實測的條件」寫成了「世界的性質」。

一、最貴的一條：25 bps 免不免，取決於 integrator 字串是不是在白名單裡

我原本的理解有兩段，兩段都錯得剛好互補。

錯的第一段：以為「帶 integrator 參數」就免平台費。

有人做了 4 金額檔 × 4 個 integrator = 16 組的矩陣，同路徑同一刻打：

```

| integrator | feeCosts |
| :--- | :--- |
| jumper.exchange | 全檔歸零 (100 → 100.00) |
| socket | 照收 25 bps |
| lifi | 照收 25 bps |
| (不帶) | 照收 25 bps |

```

不是「帶參數就免」，是「這個字串在白名單上就免」。

我的成本模型把它記成一個布林開關，實際上它是一個字串維度。

錯的第二段：以為同鏈不收平台費。

另一個人打同鏈（Arbitrum→Arbitrum，1,000 USDC→WETH，7 條候選）逐條列顯式 Fee 欄位：

| 供應商 | 顯式 Fee |

| :--- | :--- |

| Nordstern / KyberSwap / Fly / OKX / Bitget / 1inch | \$2.4981 (= 0.24981%) |

| SushiSwap | \$3.4963 (多疊約 10 bps 自家協議費) |

同鏈一樣是 25 bps。

兩條合起來，我 #43 那筆「同鏈總損耗 0.115% / 0.175%」的解釋只剩一個：

我當時帶了白名單 integrator，而跨鏈那組沒帶。我把自己的實驗設定，讀成了世界的規則。

修正後：平台費維度不是 is_cross_chain，是 integrator 字串。

成本表的 key 要再加一維，而且這一維的取值是離散白名單，不能內插、不能推廣。

這條的殺傷力在於：Day 5 起「P0 成本工程：25 bps → 0」一直是整個架構排第一的動作，

而我對它的機制理解是錯的。結論（用 jumper.exchange 可以歸零）僥倖仍成立，

但理由錯了，換一個 integrator 就會靜默失效。

二、四個開關：三組人從三個方向撞到同一個洞

這是今天最漂亮的收斂 —— 三個彼此不知道對方在做什麼的人，測出同一件事。

| 誰 | 卡在哪 |

| :--- | :--- |

| A | 新幣價差一度 10%、橋也有，但低價那側的交易所尚未開放提現 → 搬不出來 |

| B | 該幣 Binance 只支援 BSC 提現、對手所只支援 ETH 充值 → 提幣 → 橋 → 充值 三段，任一段關掉整條死 |

| C | 新幣首日「交易 / 充值 / 提現」三個開關分鐘級反覆變化；某所只有永續沒有現貨 → 根本無法接收實物 |

我的閘門一直把「可執行性」當成一個靜態布林值。它其實是一條有時效、有方向、有分段狀態的路徑。

可執行性閘門改成四開關全真才算通：能買 → 能提 → 能跨 → 能賣。

而且每個開關的狀態必須連同時間戳一起記，因為它們會在分鐘級翻面。

同一天還有第四個人從時間軸的另一端補上第五個開關：

持倉期間那條腿會消失。而且「從看板消失」要再分兩種 ——

現貨腿真的停牌／退市（單腿裸敞口）vs 只是量跌破收錄門檻（還能平但滑點吃光利潤）。

決定敞口大小的不是開倉時的差價，是平倉時那條腿還在不在、還有多深。

三、報價的有效期不是一個數，是每個階段各一個

一個人連續幾天卡在「掃得到機會但永遠執行不了」，最後找到根因 —— 不在掃描器。

他原本讓整條候選共用一個 TTL。但一條候選要跑完：

買入路線生成 → 等另一側詢價 → 授權檢查 → 兩次鏈上模擬。

全部跑完時，最前面那筆報價已經被自己判過期了。

拆開之後：前置詢價容許較長等待，真正決定賣出的那筆維持極短有效期 —— 當天鏈路首次跑通。

這條和「四個開關」是同一個 bug 的兩張臉：

我把「可執行性」當成靜態判斷，但它是分段的、有時效的、會漂移的。

配套還有一個硬拒條件值得直接抄：TRANSFER_TOO_SLOW ——

預計轉移 1,200 秒 vs 機會窗口 60 秒，直接拒，不必算收益。

閘門順序固定成：① 同一經濟錨 + 計價單位一致 + 買賣端真能用 → ② 資產搬得過去 → ③ 才算賺多少。

四、慢路徑的第三次翻轉，以及一個我完全沒想到的觸發維度

Day 7 我說「≥1 萬 U 才慢」。Day 8 改成「兩端都會中」。今天是第三次。

| 觀測 | 結果 |

| :--- | :--- |

| 去程 Base → Arb, 100 / 1,000 / 10,000 | 三檔全是 7 秒 |

| 回程 Arb → Base | 只有 1,000 這個中間檔中招 (1,080 秒) |

「哪一檔慢」根本不是常數，它同時隨方向與時點漂移。

Day 8 我把「100 U → 1,080 秒」寫進表，那張表現在要整格刪掉。

而且往返總損耗也不是平的：同兩條路線四個金額檔，50–81 bps 之間非單調，中間檔最差，

而且損耗跳變與橋切換同步 (100=across、1000=relaydepository、5000/10000=celercircle 或 mayanMCTP) 。

最意外的是第三個觸發維度：slippage 參數也會靜默換橋。

同一筆 1,000 USDC，slippage 0.1% / 1% / 10% 都選 Eco，唯獨 3% 那檔跳去 Polymer(Standard)

—— 而四檔的 toAmount 與 toAmountMin 全部恆等於 997.50。

換橋 = 換執行時長，而 bps 欄位完全看不出來。

詢價維度從三元組膨脹成六元組：(from, to, 金額檔, slippage, integrator, 目標資產)。

最後一維也是新的：同金額 Arb→OP 走 USDC→USDC 需 1,200 秒、到手 997.498；

改成 USDC→DAI 走另一家 solver，只要 35 秒且到手更多 (997.642)。

目標資產不是計價問題，是路由與速度的獨立軸。

五、第七類（開盤前窗口）要砍成兩半，一半是裸方向不是套利

昨天我把「交易所開盤前窗口」記成正向候選。今天同一天出現方向相反的兩組實盤。

負的那組（自有資金，實盤）：某盤前永續 11:30 上線，開局幾乎沒價差，

幾分鐘後才出現 → 1 價差進 1 萬 U、1.5 補 5,000、2.5 再補 5,000 →

13:10 價差衝到 3.1-3.2，浮虧最高 350 U。另一個同類標的長期停在 1.5 不收敛。

根因是機制不是運氣：Pre-IPO 盤前永續沒有現貨、也沒有確定上市時間，

所以沒有任何強制收敛機制 —— 價差可以無限擴大。

正的那組：另一個人量到完整可執行路徑 ——

開盤前約 15 分鐘在鏈上買入 → 充值到交易所（實測到帳約 13 分鐘）→ 開盤用 WebSocket 依盤口掛 IOC 賣出。

容量瓶頸是那 13 分鐘的裸多敞口，不是開盤那幾秒的手速。

差別只有一個：有沒有確定性收敛錨點。

有現貨 + 有確定上市時間 → 是窗口；

沒有現貨 + 沒有確定時間 → 是裸方向暴露，只是長得像套利。

第七類拆成兩個 regime，後者移出候選清單。

還要記一筆持有租金：那類合約資金費 0.5 bp/8h ≈ 10 bp/週，

加上資金佔用機會成本，沒有收敛機制時光是等就在虧。

六、長尾跨鏈搬運：常態仍是負，但事件可以蓋過地板

我一直寫「長尾跨鏈搬運成本地板 198 ~ 563 bps，不成立」。今天出現一個正的複盤。

某新幣 ETH↔BSC (LayerZero OFT)：BSC 主池相對兩家交易所的 5 分鐘收盤溢價

峰值 14.94% / 14.80%，三個半小時後仍有 6.97% / 6.52%。

以 1,000 USDT 檔位試算，扣買入費、提現費、跨鏈訊息費、雙邊 gas、1.5% 波動緩衝後
計畫淨利 +480~510 bps，摩擦約佔毛差 27~31%。

但同一批人給出的反例更多，而且指出殺手換了：

- 掃 361 個已驗證 OFT 資產 / 704 條活躍路徑，80 個有量的資產無一有可執行正價差。

冒頭那個毛價差 +438.8 bps，\$500 檔路由損耗 1,065 bps 直接抹平。

- 另一個案例卡在鏈路閉合（見第二節），不是價差不夠。

修正後：不是「長尾跨鏈搬運不成立」，是「常態分布下不成立」。

事件窗口的毛差可以大到蓋過地板，此時瓶頸從「價差夠不夠」換成

「鏈路閉不閉合」與「哪條鏈能放量」。

而且容量比百分比重要：同一類幣同一個事件，ETH 側 1,000 USDT 就已經損 3%，BSC 側池深 35 倍以上。

「哪條鏈價差大」和「哪條鏈能放量」是兩個獨立問題。

順帶把 OFT 這條路本身補進資料層 —— 它是聚合器完全看不到的第三類跨鏈路徑：

聚合器對該方向回全 404，但 OFT 走得通，成本是近似固定的訊息費 + gas (約 \$0.69)，

與 25 bps 比例抽成無關，所以帶不帶 integrator 都不會變便宜。

它自帶三個坑：入口只有 send() (普通 ERC20 轉進 Adapter 地址 = 把幣鎖死)；

decimals=18 但 sharedDecimals=6 (最後 12 位小數跨不過去，會被截掉)；

「來源鏈鎖倉 == 目標鏈總供應」不能當硬不變量 (在途訊息、誤轉、精度截斷、狀態時間差都會造成差額)。

七、幣股 RWA：舊價窗口回來了，但出場腿不見了

Day 6 我收窄過一次：「協議會在 oracle stale 時主動降權，舊價窗口分鐘級就被關掉」。

今天有人證明那條只對鏈上協議成立。

CEX 股票永續的實測 (184 個商品、4,471 筆事件觀測，單一事件還原)：

| 觀測 | 值 |

| :--- | :---: |

| Index 完全不更新持續 | 481 分鐘 (約 8 小時) |

| 最大 Last Index | 5.14% |

| 最大 Mark Index | 4.80% |

| 期間 Funding | 照常結算，且為正 |

沒有任何 stale 判定、降權或關窗機制介入。

修正成分場所判準：鏈上協議會降權（分鐘級關窗）；CEX 股票永續不會（小時級不動）。
不可再當通則引用。

但同一個人立刻把它自己按住了：週末美股不開盤，無法即時用現貨股票對沖，
而 Mark Price 仍在動且直接決定清算。

所以這是「窗口確實存在，但出場腿不存在」——

只能算 Basis / Funding 研究，不能當無風險套利。

這跟第四類「確定性強制平倉時刻」正好相反：那個是機制逼對手方成交，這個是機制讓你沒得對沖。

分類方式也要改：「單一指數來源」不等於「某個特定供應商」。

把事件條件寫死成供應商名稱會漏掉大部分樣本——

要存每次 Index 的組成與更新時間，用「來源是否縮減／切換／停滯」這種結構條件分類。

八、資金費率：連「當庫存閒置收益增強」都要再打折

我把 funding 從 edge 降級成「現金管理」已經兩版了。今天三組實測說連這個定位都太樂觀。

① funding 收益與價差變動是負相關，不是兩個獨立加項。

你收費那一腿，正是價差已經擴大的那一邊；價差回歸時，收斂全額變成虧損。

| 項目 | 值 |

| :--- | :---: |

| 開倉價差 | 66.7 bp |

| 平倉價差 | 40 bp |

| 收窄部分 (全是虧損) | 26.7 bp |

| 雙邊 3.2 萬 U 名目下價差腿虧 | ≈ 84 U |

| 同期收到的資費 | 80.98 U |

| 最終 | 仍虧近 30 U 出場 |

② 可見費差 $\neq$ 拿到的費差，差一個數量級。

入場時看到中位費差 0.14%，實際只拿到 0.01%（衰減約 14 倍），36 筆只贏 1 筆，策略已被作者廢除。

③ 高年化的衰減是小時級的。

兩腿淨差 $0.1443\%/h = 14.43$ bp，直接年化 1,264%；

但只撐到第 4 個結算週期就衰減到年化約 40%。

建模修正：funding 收益（入場價差 出場價差）才是淨值。

入場門檻不可以用「當前年化」，要用「每個結算週期能收多少 bp $\times$ 預期還能收幾輪」，
而且必須事先算好費率反轉率。

作用域再加一維「標的類別」：美股永續的正費率是相對乾淨的 regime；

山寨幣的負費率可被聯合調整資金費計算規則與收取間隔來反覆收割 $\rightarrow$ 不可上大倉位。

九、幾個把「已知」再往下鑿一層的

- 成功交易不是有效樣本。同一個高頻套利熱錢包，鏈上 status 成功的 18 筆裡

只有 5 筆有可認列毛利，其餘 13 筆（72%）毛利幾乎為 0 卻照付 gas；

18 筆合計淨 $2.8e-4$ ETH，而且還沒算失敗交易的 gas。

$\rightarrow$ 失敗率基準的分母錯了：成功筆數裡要再分「有效毛利 / 零毛利」兩類。

- 跨區塊幻覺：固定 blockNumber 或 blockHash 都不夠。

正解是 EIP-1898 的 {blockHash, requireCanonical: true} ——

否則讀到的可能是已被重組掉的分叉區塊，而節點不會報錯。

- 費用欄位解釋不掉全部損耗。同一次查詢裡，某條路線的損耗與三筆 included 費用

逐 raw unit 完全對上、殘差 0；但意圖型那條 $100 \rightarrow 99.729806$ 只列 0.25，

少了 0.020194 USDC (≈ 2.02 bps) 不出現在任何費用欄位。

$\rightarrow$ 加一條硬不變量：residual = 輸入 預計輸出 included feeCosts，

殘差 $\neq 0$ 標 CHECK（是「需要調查」不是「路線錯誤」），且不可在無證據下把它歸進任何一類。

- gas 估算有兩層，相加就是重複計費。高層 route 的 gasCostUSD 已經是各 step 的匯總。

另外「Relayer gas fee」（在 feeCosts、以 USDC 收、已從 toAmount 扣）與

「錢包實付源鏈 gas」（在 gasCosts、以 ETH 計、沒有從 toAmount 扣）名字像但不可互相取代。

- toAmount 與 toAmountMin 的排名可以反向。同一輪取樣，最便宜那條預期到手 997.498 但保底只有 996.501；

另一條預期較低（997.400）保底卻等於預期。只用 toAmount 挑路由，會在承保口徑下選到最差的那條。

- 同鏈的第一級損益來源與跨鏈相反。跨鏈是「到帳量差 1%」；
同鏈是「到帳量幾乎沒差、但 gas 差 9.7 倍、額外協議費差 10 bps」。
→ 同鏈排序要改用 (toAmountMin gasUSD 額外協議費)。
另：executionDuration 在同鏈全部回 0，該欄位在同鏈沒有區分力，0 不代表零秒成交。
- 毛價差分布這一格終於有數字了。三條鏈對持續監控，|價差| 中位數只有 0.4–0.5 bps、均值 <1 bps。
→ 比單邊成本小兩個數量級，等同於在成本歸零的假想下也不成立。這補上我先前只量成本側與淨值側的缺口。
- 缺值要四態不是三態：KNOWN / ASSUMED / NOT_APPLICABLE / MISSING。
關鍵是把「本次路由沒有 swap 步驟所以 DEX 費不適用」（可合法記 0）
與「這項應該存在但沒資料」（整筆回 INCOMPLETE + netProfit=null + NO_DECISION）分開。
- 出口 IP 是一票否決的前置閘門，15 分鐘就測得完。
某地區出口 IP 打合約與股票端點全部 451 restricted location，換一個地區全通。
這與帳號地區資格是兩回事，應該排在任何成本建模之前。

十、清算線：兩個把毛機會再砍一刀的東西

雖然清算仍是「不碰」，但今天多了兩條會改變任何人估算的參數。

① 協議端的價值回收 (SVR) 會收走絕大部分。
有人算出某條線「有 \$25 萬/年，但 92.3% 被 SVR 回收」——
這是與 builder 競價彼此獨立的第二道抽成。若成立，「毛機會 × 留存率」要再乘一層 7.7%。

② 一個倉位不是一次機會，是一串遞減的機會。
因單次清算比例上限，某 206 ETH 倉位被 9 筆 LiquidationCall 級聯清到歸零
(單筆從 85.7 ETH 遞減到 0.59 ETH) —— 健康因子被清回 1 之後又跌破，往復。
→ 以「倉位」計數會低估筆數，以「首筆規模」估收益會嚴重高估後續各筆。

③ 出場通道類型是我成本模型缺的一維。
拆解某清算機器人的四步 (閃電貸 → liquidationCall → RFQ 精確換回還款資產 → 還貸) ——
它走 RFQ 不走 AMM，因為 RFQ 給確定性報價、防夾、具名對手。
我一直預設出場腿走 AMM/聚合器。

平靜期的密度也被量到了：某主網借貸協議 7 天全量只掃到 3 筆清算 (0.4 次/天，最大單筆約 \$6 萬)；
另一條鏈枚舉 1 萬個借款倉位，當前可清算 0，但有 65 個落在離清算線 70–100% 的危險區。
→ 危險區存量可當「波動一來就有機會」的先行指標，平靜期就是 0。

十一、風控：把「代理被污染」從機率問題變成結構問題

Day 8 補了合約權限面。今天補的是上游——不讓 AI 有能力做壞事，而不是叮嚀它別做。

做法是：不讓代理持有私鑰，改用多簽金庫，並在鏈上用 Module 寫死 Policy Gate
(目標地址白名單、單筆與時段金額上限)。

即使代理被 prompt injection 餵進假資料 (例如假的 500% APY 金庫) 而發出惡意交易，
合約層直接 revert。

這與 Day 8 的准入檢查清單是兩層不同的東西：那層是「檢查我寫的合約有沒有洞」，
這層是「假設我的判斷會被污染，在它下游放一道我改不動的閘」。

十二、參數驗證進度 (接續 Day 8)

#	參數	值	狀態
1	聚合器平台費	25 bps，同鏈跨鏈都收；免除與否取決於 integrator 白名單 (見 #64)	v0.7 的作用域錯了
2	聚合器 rate limit	匿名 75 次/2h (實測 header)；「200 次/2h」是文檔轉述，降權	
3	直連橋費 (USDC/L2)	1.0–1.6 bps	
4	V3 深度計算法	liquidityNet 前綴和	
5	MEV tip 比例	策略 × 通道 × 鏈 × 是否垂直整合；Jito 的 15% 只是低額讀數 (見 #71)	加規模維度
6	CEX 費率階梯	一般 10/10 bps ~ VIP 1.1/2.3 bps	
7	規模曲線	$q = R_{\text{eff}} \times (\text{價差} \text{ 費率}) / 2$	
8	長尾同鏈成本地板	198 ~ 563 bps	
9	同鏈雙邊費底線	Σ (每跳實際 fee tier)，Curve 檔可低到 4.5 bps	
10	LP 覆蓋率分水嶺	日波動 1.5%	
11	MEV 佔優先費支出	66–80%，前 15 名後斷崖	
12	兩端滑點	—	必須即時算
13	規模窗口上保本點	有，與下保本點差約 94 倍	
14	跨鏈成本方向性	5 萬 U 時方向反轉；1 ETH 檔位有向性被 25 bps 比例費淹沒 (0.25/0.26/0.26%)	加條件

15	同資產路徑成本穩定度	同路徑 142 次極差 0.11 bps	
16	時間權重溢價	1.00–2.79 bps	
17	預測市場費率公式	$100股 \times 0.07 \times p(1p)$ (我方舊式已證偽)	
18	integrator 對平台費的影響	~帶上就免~ → 只有白名單字串免 (見 #64)	機制理解錯誤
19	gas 是否已計入報價輸出	兩層 gas 估算, 相加 = 重複計費; skipSimulation=true 時官方明示 gasLimit 不準	可結案
20	原子套利實測失敗率	EVM 67%/Solana 60–73%/某鏈三角 55%	
21	失敗成本兩級落差	Gas 級 ~0.2 bps vs 單腿級 ~40 bps; 同鏈原子失敗實測 \$0.3–1.5	
22	下架結算前期現價差	峰值 439 bps/最後 6 小時均值 226 bps	
23	跨所插針幅度離散度	19.6 倍 vs 0.33	
24	報價取得延遲	本地狀態幀 p50 48 ms / p90 110 ms	
25	跨鏈價差翻轉時間尺度	31 秒可從 0.12% 翻到 +0.04%	
26	oracle stale 降權時間常數	$\tau = 1$ 分鐘 —— 只對鏈上協議; CEX 股票永續不降權 (見 #67)	分場所
27	借貸替代等橋的窗口壓縮	7 分鐘 → 27 秒	
28	資金費率年化	淨年化 7.16% (2026 年度毛 +1.28%) —— 且需再減價差收斂項 (見 #72)	再降級
29	速度溢價	恆定比例 1.00 bps (小額檔 1.26)	
30	executionDuration 規模斷點	~兩端都會中~ → 「哪一檔慢」隨方向與時點漂移, 不可入表 (見 #65)	第三次翻轉
31	builder refund 佔競價比例	1.1%; 只對非垂直整合者成立	
32	清算競價留存率	小額恆定約 2%; 斷點 100 WETH	
33	閃電貸費實測	Balancer 0%/Aave V3 0.05% (premium 需合約自帶餘額, 否則必 revert)	
34	v3 fee 欄位單位	pips 不是 bps	
35	多費率檔位深度比	最低費率池深度只有 1/950	
36	靜態相乘 vs 逐腿模擬落差	+28.83 → 63.40 bps	
37	Mark / Index 1 異常量級	實見 +298%	
38	名目對沖的裸倉殘留	各投 1000 U → 留 3,521 顆裸多	
39	公共 RPC context 取得延遲	1,013–1,306 ms	
40	資金費率證券化的兩個 APR	Implied vs Underlying (Fixed APY $\equiv$ Implied APY, 不是第二訊號)	收窄
41	滑點容忍是否為成本項	否, 是回滾邊界。誤當成本會抬出假的 160 bps 盈虧線	可結案
42	Partner Key 的 quote 窗口	未壓測	
43	平台費是否套用於同鏈路由	~否~ → 是, 同鏈也收 0.24981%	已證偽 (我自己的)
44	口徑誤差: USD vs token	USD 13.94 vs token +26.01 bps, 差 40 bps 且反號	
45	同次查詢內路由到帳離散度	17 條候選差 1% (100 bps)	
46	報價噪聲帶寬的決定因素	solver 數量, 不是深度	
47	事後再平衡成本	50–70 bps (與直連橋物理摩擦 2 bps 是兩件事, 引用時要標)	加標註
48	預置庫存模式的預篩門檻	現貨階段需正 Edge 80.61–93.08 bps	
49	延遲風險換算式	$\sigma \times \sqrt{t}$ (等待/年); 穩定幣腿實測漂移 std $\approx$ 0.001%, 趨近 0	實測佐證
50	獨立清算人數	183 / 184 / 159	
51	清算單筆中位金額	\$20,266 / \$19,857 / \$22,396	
52	可清算倉位的自行退出率	約 2/3 靠價格回升自己消失	
53	清算前是否有向下跳空	70–80% 沒有	
54	清算成交機率迴歸係數	規模 +5.58/MEV-Boost +1.81/gas 0.60/波動 0.565/oracle 偏離 0.092	
55	CEX-DEX 平均毛報酬率	9.7 bps; builder tips 佔 61.5%	
56	跨鏈淨利的獨立複現	中位 182 ~ 213 bps; 「最好 59 bps」停止引用 (尾部極值對樣本極敏感)	收緊引用
57	CEX 新幣 ↔ AMM 窗口壽命	小時級 (新增獨立樣本: 峰值 14.94% → 3.5 小時後仍 6.97%)	加樣本
58	v3 虛擬儲備在 tick 邊界失真	$L/\sqrt{P}$ 高估 9 億倍	
59	開清之差 (CEX 對沖)	開 3.25% vs 平 4.09%, 差 0.84% 是必付往返摩擦	
60	價格衝擊的凹性	名義 4 倍時 impact 只 2.3 倍	
61	資料缺口率的否決門檻	58.12% → NO_GO	
62	資金費率年化的結算頻率	必須按 8h/4h/1h 分別調	
63	第三方套利工具的服務費佔比	吃掉毛利 83–94%	
64	平台費的真實維度是 integrator 字串白名單	4 金額 $\times$ 4 integrator 矩陣: 只有一個字串全檔歸零, 其餘照收 25 bps; 同鏈亦收 0.24981%, 某供應商再疊 10 bps	新增 (推翻 #18/#43)
65	慢路徑的觸發維度	(from, to, 金額檔, slippage, 目標資產) 五軸皆會靜默換橋; slippage 3% 那檔跳慢橋而 toAmount/Min 恆等 997.50	新增
66	盤前窗口的 regime 切分	有現貨+有確定上市時間 = 窗口; Pre-IPO 永續 (無現貨無時間) = 無收斂機制, 實盤價差擴大到 3.1–3.2、浮虧 350 U	新增
67	CEX 股票永續的 Index 凍結	481 分鐘不更新, LastIndex 5.14% / MarkIndex 4.80%, 照常結算正	

Funding，無任何降權（184 商品 / 4,471 事件） | 新增 |

| 68 | 股票永續的出場閘門 | 休市時無法用現貨對沖，而 Mark 仍決定清算 → 只能算 Basis/Funding 研究 | 新增 |

| 69 | 事件型長尾跨鏈的摩擦佔比 | 毛差的 27–31%（1,000 USDT 檔），最大單項是「延遲/波動緩衝」不是手續費 | 新增 |

| 70 | OFT 原生橋的成本結構 | 近似固定費（訊息費+gas ≈ \$0.69），聚合器回 404 也走得通，帶不帶 integrator 都一樣 | 新增 |

| 71 | Jito tip 的規模相依 | 單筆 1.73 萬 U 獲利付 35.8 SOL → 隱含 25–40%，遠高於 15% 的總體讀數 | 新增（單樣本） |

| 72 | funding 收益與價差變動負相關 | 開 66.7 bp → 平 40 bp，收窄的 26.7 bp 全是虧損；收到 80.98 U 資費仍虧 30 U 出場 | 新增 |

| 73 | 可見費差 vs 實得費差 | 0.14% → 0.01%（衰減 14 倍），36 筆只贏 1 筆 | 新增 |

| 74 | 高年化 funding 的衰減速度 | 淨差 14.43 bp/h（年化 1,264%）只撐 4 個結算週期降到年化 40% | 新增 |

| 75 | 成功交易中的零毛利比例 | 18 筆成功交易只有 5 筆有可認列毛利，13 筆（72%）零毛利照付 gas，整體淨為負 | 新增 |

| 76 | 跨鏈毛價差的分布 | 三鏈對持續監控：\價差\ 中位 0.4–0.5 bps、均值 <1 bps | 新增 |

| 77 | 意圖型路由的費用殘差 | 100 → 99.729806 只列 0.25，少 0.020194 USDC（2.02 bps）不在任何費用欄位 | 新增 |

| 78 | 同鏈選路的第一級損益來源 | 到帳量幾乎無差，gas 差 9.7 倍、額外協議費差 10 bps；executionDuration 同鏈全回 0 | 新增 |

| 79 | SVR 對清算毛機會的回收率 | 92.3% 被協議端回收，與 builder 競價是獨立的第二道抽成 | 新增 |

| 80 | 單一倉位的清算次數 | 因單次比例上限，206 ETH 倉位被 9 筆級聯清到歸零（85.7 → 0.59 ETH 遞減） | 新增 |

| 81 | 平靜期清算密度 | 某主網借貸協議 7 天全量僅 3 筆（0.4 次/天）；另一鏈枚舉 1 萬倉位當前可清算 0、危險區 65 個 | 新增 |

| 82 | 價差壽命與池深的分層 | 501 樣本：薄池對出軌率 100%、持續 3–7 小時；深池對 54%、持續 <2 小時；容量邊界約 100 SOL/筆 | 新增 |

| 83 | Solana 的最終性 | 400 ms 是出塊不是最終性，實際確認約 12–13 秒 → 跨所腿暴露窗口按 12–13 秒算 | 新增 |

| 84 | 代幣化股票的 multiplier | 67 個標的中 4 個 multiplier ≠ 1，不歸一化會憑空造出 50.1 / 23.5 / 6.0 / 1.1 bps 假價差 | 新增 |

| 85 | 出口 IP 閘門 | 某地區出口 IP 對合約與股票端點全回 451，換地區全通；與帳號資格是兩回事 | 新增 |

十三、一句話

Day 5：我可能一直在量一個被自己參數截斷過的世界。

Day 6：有些東西我根本沒有量具。

Day 7：我有量具、數字也讀對了，但我差點把它讀成相反的意思。

Day 8：量具本身的誤差，比我要找的東西還大。

Day 9：我把「我當時的實驗設定」寫成了「世界的性質」。

25 bps 免不免，取決於一個我當初隨手填的字串。

慢路徑哪一檔慢，取決於方向、時點、和我沒想到會影響它的 slippage 參數。

舊價窗口關不關，取決於我測的是鏈上協議還是 CEX。

這三條的共同形狀是：我的結論裡藏著一個沒有被寫下來的條件。

而它們全都通過了驗證——因為驗證的時候，那個條件剛好也是真的。

Day 8 我學到「不會報錯的誤差最危險」。

今天再往下一層：不會報錯的誤差，往往是我自己的實驗設定變成了隱含假設。

所以這一版新增了一條驗收規則，排在所有實測之前：

每寫下一個結論，同時寫下「這次實驗中有哪些我沒有改變的東西」。

那份清單就是這條結論的作用域。沒寫的話，作用域預設是「全世界」——而那幾乎永遠是錯的。

笔记 031: https://github.com/ZeroNullNone/onchain_arbitrage_

作者：Nothing | 时间：2026-08-13 02:37 UTC | 字数：2348 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/docs/daily/day_09.md

Day 9 — LI.FI Route Dispersion

状态与结论

- 状态：Day 9 frozen-universe acceptance scope 已完成。

- LI.FI Base USDC→WETH 的 size sequence 为 Fly → Fly → KyberSwap，Switch Rate 为 0.5；Fly Provider Concentration 为 2/3。

- Independent Aerodrome direct source 按 Day 3 frozen 50 bps minimum-output policy 计算后，在三个 size 的 minimum output 排名均高于 LI.FI，但证据相隔约一天，100 USDC 的精确间隔为 98,225.491580 秒。

- 因此 Candidate 被 direct-source gate 否证为 stale_quote，不是可交易套利；本日没有盈利结论。

完成项

- 新增 route observation boundary，保留 request ID、Raw reference、UTC time、latency、integer token units、decimals、provider、fingerprint、duration 与 fee evidence。
- 新增以 minimum output 排名的 Best/Second-best，以及 Route Switch Rate、Observed Lifetime、Provider Concentration、Duration/Fee Dispersion 与 Size Sensitivity。
- 新增 routing improvement、temporary subsidy、stale quote、token mapping difference、unavailable route、tradable edge 六类显式分类。
- tradable_edge 必须同时有 fresh independent refresh、complete cost ledger 与 complete executable cycle；Route Difference 单独永远不会设置 is_arbitrage=true。
- Fee evidence 缺失时返回 unknown，不把缺失成本写成零。

Evidence 与验证

- 完整语义与 frozen evidence : docs/day09_route_dispersion.md
(https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/docs/day09_route_dispersion.md)。
- LI.FI Raw fixtures : tests/fixtures/lifi/
(https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/tests/fixtures/lifi/)。
- Independent direct Raw fixture :
tests/fixtures/amm/base_aerodrome_weth_usdc_block_49641814.json (https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/tests/fixtures/amm/base_aerodrome_weth_usdc_block_49641814.json)。
- Acceptance tests : tests/test_route_dispersion.py
(https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/tests/test_route_dispersion.py)。
- uv run pytest : 57 passed。

Gate 边界

- 当前 Switch Rate/Lifetime 来自连续的不同 size probe，只描述 size sequence，不代表固定 size 的时间序列 reliability。
- LI.FI 与 Direct Source 不同时，因此 Best/Second-best 只用于 route analytics，不能声称同步可执行。
- 本日没有新增外部请求、Aggregator、Bridge PnL assumption、签名或广播交易。

下一步

- Day 10 只基于已证据化的 cross-chain candidate 建立 two-chain virtual balance sheet；任何 cross-chain signal 都必须说明利润锁定时点与 initial inventory requirement。
- Provider Reliability Time Series 与 Subsidy Detection 保留在 backlog。

笔记 032: LI.FI 的切面合约 (Facet Contracts) 是基於 EIP-2535 多切面代理標準建立

作者：Yolandatsai | 时间：2026-08-13 02:42 UTC | 字数：805 | 评分：73

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LI.FI 的切面合约 (Facet Contracts) 是基於 EIP-2535

多切面代理標準建立的專用鏈上合約。它們各自對接不同的流動性來源，負責將交易路由至指定的橋接、DEX 或求解器，以實現高效、安全的跨鏈與同鏈資產互動。 \[1

(<https://docs.li.fi/zh-Hans/introduction/lifi-architecture/system-overview>),

(<https://docs.li.fi/zh-Hans/introduction/lifi-architecture/smart-contract-overview>)]

切面合约的主要类型

桥接切面合约 (Bridge Facet)

路由交易到特定跨鏈橋 (如 Bridge A 或 Bridge B)。

執行跨鏈轉賬。

確保橋接兼容性與資產轉移安全。 \[1 (<https://docs.li.fi/zh-Hans/introduction/lifi-architecture/system-overview>)]

DEX 切面合约 (DEX Facet)

路由交易到特定 DEX 合約。

處理同鏈代幣交換。

根據參數優化流動性利用率。 \[1 (<https://docs.li.fi/zh-Hans/introduction/lifi-architecture/system-overview>)]

求解器切面合约 (Solver Facet)

路由交易到求解器合約。

訪問擴展的流動性來源。

基於協議實現高級路由與定價。 \[1 (<https://docs.li.fi/zh-Hans/introduction/lifi-architecture/system-overview>)]

笔记 033: 学习心得：从套利概念学习到 USDT 跨链观察框架

作者：Chic | 时间：2026-08-13 02:55 UTC | 字数：835 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习心得：从套利概念学习到 USDT 跨链观察框架

今天的学习，我把方向收敛到更适合自己的 USDT 稳定币跨链 Quote/Route 观察。选择 USDT 是因为自己平时最常用这个资产，也曾经遇到过“币种对了但网络错了”的实际问题。通过学习，我现在明白链上转账不是简单地“转 USDT”，而是必须同时确认币种、网络和地址。比如对方要 Optimism 上的 USDT，就不能只准备 Ethereum 主网上的 USDT。

我还学习了原生资产和桥接资产的区别。同名稳定币不一定同风险，像 USDC.e 这种资产要先怀疑是桥接或包装版本，需要额外核对来源桥、合约地址、流动性和兑回风险。也学习了 L1/L2 的概念，知道 Ethereum 是 L1，Optimism、Arbitrum、Base 是 Ethereum 的 L2。后续观察路线优先选择 Ethereum USDT -> Optimism USDT、Arbitrum USDT -> Optimism USDT、Base USDT -> Optimism USDT 这类主流、可解释、和真实场景相关的路线。

目前我已经建立了一个 USDT 跨链观察框架：先确认币种、网络、地址和资产版本，再看 toAmount、toAmountMin、Gas、feeCosts、执行时间、Route 经过的桥和 DEX，最后用保守净收益判断是否值得继续观察。这个阶段的目标不是实盘套利，而是建立一个自己能看懂、能解释、能持续记录的跨链成本数据库。

今天最大的收获是，我开始从“看到价格差”转向“验证路径、成本和风险”。现在我更清楚地知道，很多看起来有利润的机会，扣完成本和风险后并不值得做。对我来说，当前最重要的能力不是执行套利，而是能判断哪些机会不该做、为什么不该做，以及还需要补充哪些数据。接下来我希望继续补强区块链基础、DeFi

基础、金融交易基础和链上数据分析能力，让自己对这些概念背后的原理理解得更扎实。

笔记 034: LayerZero 多链资产价差扫描：从资产发现到可执行报价

作者：QQQ | 时间：2026-08-13 06:46 UTC | 字数：2694 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LayerZero 多链资产价差扫描：从资产发现到可执行报价

本节对应项目：w3player/arbitrage-playbook/layerzero-multichain-arbitrage。系统先从 LayerZero 消息和 OFT Metadata 中发现真实多链资产，再用 LI.FI 对不同链上的同一数量 Token 进行买入、卖出报价，判断扣除直接成本后是否仍有利润。

一、系统的完整流程

LayerZero 消息扫描 → 读取 OFT Metadata → 识别同一资产在多链上的合约 → 验证 OFT/Adapter/Peer 关系 → 建立双向路径 → LI.FI 查询两链真实买卖报价 → 统一 Token 数量 → 扣除 Gas 和额外费用 → 输出正价差或负价差。

二、跨链资产发现结果

本轮共发现 224 个可扫描跨链资产、350 条双向路径、518 个链内部署，覆盖 Ethereum、BNB Chain 和 Base 三条链；另有 130 条路径待补全，9 个资产被排除。DOS 已验证存在 BNB Chain ↔ Ethereum 双向路径，使用 OFT Adapter 模式。它不是因为 ticker 相同就被当作同一资产，而是通过 Token/OFT、Adapter 和 Peer 配置确认两侧属于同一个 LayerZero 跨链资产。

图 1：LayerZero 跨链资产管理页面，展示 224 个资产、350 条双向路径及 DOS 的已验证部署关系。

三、LI.FI 同数量报价：大多数路线仍然亏损

系统使用约 500 USDC 的测试档位：先查询买入链最多需要支付多少 USDC，并取得实际 Token 数量；然后在卖出链按完全相同的 Token 数量查询最少可以收到多少 USDC。FXRP 的三个示例方向均为负：BNB→Base 最终约亏 7.0691 USDC (141 bps)；Base→BNB 约亏 8.1441 USDC (162 bps)；BNB→Ethereum 约亏 8.1618 USDC (163 bps)。这说明多数表面价差会被双边 DEX 手续费、滑点、价格冲击、Gas、路由费和报价时差吃掉。

图 2：LI.FI 跨链价差扫描页面，FXRP 多个方向在扣除直接成本后均为负。

四、利润计算口径

毛利润 = 卖出最小到账 - 买入最大支付。直接净利润 = 毛利润 - 买入 Gas - 卖出 Gas - 买卖两侧额外费用。直接净价差 (bps) = 直接净利润 ÷ 买入成本 × 10,000。只有净利润为正且报价未过期，才能进入下一步验证。

五、H420 的正价差为什么仍不能直接执行

截图中 H420 的方向为 Base → Ethereum：买入最大支付约 426.8217 USDC，卖出最小到账约 438.9878 USDC，毛利润 12.1661 USDC；扣除约 0.4127 USDC 的 Gas 和额外费后，直接净利润约 11.7534 USDC，对应 +275 bps (约 +2.75%)。

但这一信号高度可疑：Token 数量达到约 512 亿枚、单价被显示为 0、极小单价容易因 decimals、精度或报价取整产生假价差；还可能存在转账税、卖出限制、黑名单、流动性异常或模拟与实际执行不一致。因此它只能列为候选信号，必须先做链上模拟和小额

端到端测试。

图 3：H420 显示约 +2.75% 的直接净价差，但数量和单价存在明显异常。

六、PENGU 与 DRV 候选机会

本轮文字记录还发现：PENGU 在 Ethereum 与 BNB Chain 之间约有 1% 表面价差；DRV 在 Ethereum 与 Base 之间约有 2.65% 表面价差。PENGU 的 1% 很可能不足以覆盖完整执行成本；DRV 更值得继续模拟，但两者都不能直接当作最终利润。仍需加入真实跨链费、跨链到账损耗、等待时间、资金档位价格冲击、库存占用和失败风险。

七、代码中的关键执行假设

LI.FI 报价代码把 fromChain 和 toChain 都设置为当前链，因此当前扫描实际计算的是：链 A 上 USDC→Token 的本链买入报价，加上链 B 上 Token→USDC 的本链卖出报价。它没有把“链 A 买入→LayerZero 跨链→链 B 卖出”作为一笔完整路径计价。因此页面中的直接净利润尚未完整扣除 LayerZero quoteSend 费用、跨链时间和跨链失败风险。

当前系统更适合双链预置库存模式：买入链预放 USDC，卖出链预放 Token；机会出现时两边近乎同时成交，之后再低频跨链补充库存。这样可以把跨链延迟移出关键成交窗口，但需要计算库存占用和再平衡成本。

八、LI.FI API Key

项目通过环境变量 LIFI_API_KEY 配置密钥。有 Key 时可以进行完整扫描，默认并发数为 4；无 Key 时只能匿名增量扫描，并发数为 1、每批约 6 个资产，容易受到匿名报价限制。持续扫描 224 个资产及数百条方向时，API Key 基本属于必要配置。

九、后续改进方向

- 1\、接入 LayerZero quoteSend/quoteOFT，计算真实跨链费用和最小到账量。
- 2\、检测 Token 转账税、黑名单、暂停状态、卖出限制和 decimals 异常。
- 3\、设置最大报价时间偏差、有效期和连续采样条件。
- 4\、扫描 100、500、1,000、3,000、5,000 USDC 等多个资金档位。
- 5\、增加双边并发下单、单腿成交保护、余额对账和熔断机制。
- 6\、将库存资金成本和跨链再平衡成本计入 NetEdge。
- 7\、链上模拟通过后，先用小额资金进行端到端测试。

十、结论

这套系统最有价值的地方不是偶尔出现的 1%~2.75% 数字，而是建立了“LayerZero 验证同一跨链资产 + LI.FI 验证相同数量真实买卖报价”的筛选管线。DOS 属于少见的事件型机会；PENGU、DRV 和 H420 都只能先作为候选。真正可执行的套利必须同时满足：合约与 Peer 正确、两侧流动性足够、完整成本后净利润为正、跨链或库存路径可用、报价在有效期内、两边成交风险可控。

笔记 035: Day 9 (08-13)：阶段复盘收官 + 执行层全面升级

作者：Paxon Qiao 乔鹏军 | 时间：2026-08-13 08:00 UTC | 字数：2798 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 (08-13)：阶段复盘收官 + 执行层全面升级

主线：L0009 阶段复盘·知识图谱 v1

为何做：官方线阶段二 (Day

5-9) 收官，把「机会地图+报价对比+候选清单」收成一张可导航的知识图谱——不是知识清单，是证据网络。

怎么做：四层结构 (概念→机制→工具→决策)；输入 = L0001 七类机会结构 × Bruce 14 类对照总表 (存活状态) × L0008 候选清单 v1 × 6 天互证网。

结果：

- 存活判定三支柱：①毛价差<手续费 (bStock 四层市场「连肉都没有」) ②价差≠可执行 (桥费+四开关, IMD +438bps 被 -1065bps 吃 2.4 倍) ③返佣撑表面收益 (PM 0x204f 返佣 1.9× 纯交易)
 - 14 类存活地图：只有事件驱动类活着；预测市场类全灭 (论文 99% 机会未被执行 + 盘口定价准)
 - 个人 edge 收敛：事件驱动 > 常驻价差 > 预测；带方向 LP 是唯一结构性候选 (吃流动性激励=非预测)
 - 七条认知互证网：每条结论 ≥2 独立来源 (IMD/048/117 三方证桥费杀手；GMEB/008 互证上市窗口寿命)
- 意义：方法论第一次完整闭环——被打脸 2 次 (DOS integrator 假设、IMD 桥费) 都变成了产出，证伪也是有效产出。

个人线：AMM 数学 2 (CLMM 正式化)

为何做：D9 主线——集中流动性是当前 Solana 主流 AMM 形态 (Orca Whirlpool/Raydium CLMM)。

怎么做：amm_clmm_visualize.py (tick 推导、虚拟储备、swap 公式、出区间单边资产) + 可视化 PNG。

结果：资金效率 21.5x/19.5x (窄区间 vs 全区间) ; ±10% 区间 = 2007 ticks; 价格出区间后池子只剩单边资产 (无常损失的本质) ; 30bps 档位选择影响走廊宽度。

意义：与无套利雷达打通——走廊宽度 = 费率函数, CLMM 低费率池 = 走廊更窄 = 出轨频率更高 = 机会频率更高。

个人线：广度——链上 perp 资金费率机制

为何做：候选清单「链上 perp funding」待验证, 先补机制再谈套利。

怎么做：HL (CEX 搬上链、1h 结算) / Drift (利用率+持仓不平衡双因子) / Zeta (纯订单簿) 三家机制对比 + 哨兵实测。

结果：三家资金费驱动因子不同 → 同一时点 funding 方向可以背离; HL 首扫抓到 KAITO -1187%/yr 极端空头拥挤 (长尾资产 funding 极端 = 拥挤信号, 与 CEX TUT 陷阱同构)。

意义：funding 高 ≠ 可套 (执行层容量才是杀手) ; HL funding 快照 cron 已上线 (8h) , 与 CEX funding 哨兵并排 = 跨所 funding 价差 z-score 是下一步。

超前预习：D10 套利利润模拟器 v0 + 聚合器路由

为何做：明天主线提前做 (AMM 预习模式复用) ; 模拟器 = 把「池子参数+价格+Gas → 扣费后利润」变成可判定工具。

怎么做：arb_profit_simulator.py (V2 带费 swap 精确解、单腿往返/跨池搬/三角预留、价格偏离注入、gas 折算)。

结果 (三场景实测) :

- 平价 10 SOL : 全路径负 = 基线正确 (无价差=无肉, 0 信号是正确输出)
- 薄池 -5% 价差 10 SOL : 薄池→深池 +427.5bps 可执行 (5% 费率 滑点 ≈ 4.3%)
- 同价差 1000 SOL : 全部变负——薄池滑点吃掉 5% 价差

意义：滑点曲线 = 容量曲线 (#13 结论的数学演示) ——同一条价差, 10 SOL 可执行、1000 SOL 不可执行; D10 正式版待补三角路径/真实数据源/聚合器对比列。

广度：聚合器路由读完——聚合器报价 = 市场共识价, edge 在它看不见的地方 (新池子/私有流动性/更新延迟) ; 定位坐标：跨链搬砖适合、毫秒级赛道只当参考价。

工程攻坚 (4 件)

1. 雷达 v2.1 硬上限 + v3 连续确认 : |spread|>5000bps 一律标损坏 (根治 Aquifer 150 万 bps 假出轨) ; 连续 2 次采样 (30min×2=1h 持续) 才推送, 首次静默 + 池名归一化防断连续性 + 旧格式兼容——端到端实测通过
2. #13 执行层卡点攻坚 (深度阶梯) : swapInfo 暴露 ammKey 但 DexScreener 未索引新 AMM ; protocols 单池报价有偏差弃用 (Aquifer 限定下首腿走 ZeroFi) ; 深度阶梯方案 : corridor 3 天数据按池×金额档频次 → HumidiFi 100 档 186 次 = 薄池深度冠军, 全部池都能接 100 SOL = 薄池是价格敏感不是容量敏感 (修正假设)
3. 信息差分层补全 : 公告层 (listing_monitor 15min) → 快讯层哨兵 v1 (The Block RSS 30min, 首跑命中 NYC PM 监管调查 + Kalshi 全订单簿流) → 推特层 (人工) ——三层齐了
4. bStocks 事件窗口 watchdog (每小时) : 首跑 GMEB 两腿 ±70bps (闭市漂移实况) ; bapi 详情接口修好 (articleCode+Referer 头) ——公告→ticker 全自动管线打通, GMEB 上市 3.5h 后溢价已被吃掉 (窗口寿命数据 : 热股数小时收敛 vs 冷股挂几天)

归档与发布

- 6 条推文/链接归档 (老白访谈 v4 Hook/开源 PM bot 生态/信息差源/Alloomi 自进化 Agent/PM 晒单核验/新币教程) + μToken 新币套利准备清单 (DEX 新池子源/四开关/假币清单/流动性真空窗口)
- Day 8 增量 PDF (180 页 121 篇) 消化 → digest (DOS 加错池子=信息差活教材、大价差≠好机会)
- D7 X thread 已发布 (「看起来赚了≠真的赚了」)
- 连续 5 天 ICL 全勤打卡

今日总结

认知三连：被打脸=产出 (DOS/IMD 证伪进图谱) ; 薄池是定价错误不是流动性真空 (可执行性比预想好) ; 信息差=速度分层 (公告/快讯/推特三层各司其职)。阶段三 (Hermes 研究工作流) 骨架已立起——L0010 直接基于知识图谱开工。

笔记 036: D7 学习进展：第一周混合检索与知识地图 v1

作者：Qun Wang | 时间：2026-08-13 08:08 UTC | 字数：1600 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D7 学习进展：第一周混合检索与知识地图 v1

今日问题

面对一条新的套利主张，能否把来源、新鲜度、固定状态、AMM

报价、条件成本、跨链终态和最终结论重新连接，并区分支持、否定与证据不足？

使用的一手资料或区块/数据

D7 新的教学用虚构观察包，用于避免复述 D1–D6 原题。

D3 固定 CPMM 公式与 D5 固定 LI.FI route snapshot 中的 toAmount、toAmountMin、fee、Gas 和 priceUSD 口径。

D6 的 Ethereum JSON-RPC、The Graph 与聚合器多源证据边界。

本地 Worksheet 与基线后生成的知识地图 v1；无钱包、签名或广播。

做了什么

先提交六题无提示混合基线，原样保留不会的长题和错误数字。

将来源/延迟、CPMM、费用/Gas、跨链终态与三值结论拆成小题纠错，再用新数字和新资产交错迁移。

把 receiving/refund 都整理为“链—交易—回执—币—人—量”。

整理知识地图 v1，明确 The Graph、JSON-RPC、聚合器与 transaction/receipt/logs 分别直接支持什么、延迟什么、不能证明什么。

最后单独复测唯一剩余的 minimum 阈值判断。

结果或失败

首次基线未通过：第 2 题因过长未答；CPMM 把剩余储备当输出；净值遗漏未嵌入 fee；跨链回答越过目标链证据；实际与条件报价混层。

纠错后能算出无费/含费输出约 0.0990099009901 / 0.09871580343971 WETH、条件 expected/minimum net = -0.01 / -0.14 USD-at-quote-time 与 receipt Gas 0.0024 ETH。能完成 Graph → JSON-RPC → 聚合器 → transaction/receipt/logs 路由，并在 receiving/refund 分支守住六字段边界。

三次综合迁移仍各有错误，尤其反复混淆“证据不足”和“否定”。最后只剩报价 minimum 与阈值：对门槛 ≥ 0.01 ，新报价 minimum +0.02 判支持、+0.005 判否定，最终通过。

这里的原始字段是 toAmountMin；minimumNet 是用最低报价输出减输入、报价 Gas 与未嵌入费用得到的 USD-at-quote-time 条件筛选下界。它不是 API 原生保底利润、赔付承诺或实际执行证明。

证据链接或本地路径

evidence/0007-first-week-review/worksheet.md

evidence/0007-first-week-review/knowledge-map-v1.md

learning-records/0007-d7-first-week-review.md

reference/0006-lifi-route-net-value.html

reference/0007-multisource-state-evidence.html

下一假设

如果先固定对象、方向、规模、状态/时间、阈值、反证与停止规则，再让辅助检索来源，就能避免先收集大量材料、最后才补一个不可证伪的问题。

Hermes

风险认识的变化

“没有证据”通常是证据不足，不是失败已经发生；minimum 低于门槛能否定报价层的保证，但不能证明实际亏损；publisher status、成功 receipt 与正条件净值都不能单独证明指定资产到账和实际盈利。

笔记 037: 今天查阅了 LI.FI 官方 Status Tracking 相关说明。跨链交易提交后，不能只依据源

作者：jason52012345-bot | 时间：2026-08-13 08:19 UTC | 字数：309 | 评分：45

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天查阅了 LI.FI 官方 Status Tracking 相关说明。跨链交易提交后，不能只依据源链交易已确认来判断结束；应通过 /status 结合 txHash、bridge、fromChain 追踪跨链进度，并以返回的最终状态和子状态为准。

其中 PENDING 表示仍需继续查询；DONE + COMPLETED 才对应完成，DONE + PARTIAL 表示完成但收到的代币可能不同，DONE + REFUNDED 表示未完成但资金已退回，FAILED 则需要检查错误与后续处理。后续记录跨链路径时，会把报价比较与执行后状态核验分开，避免把预估结果误当作实际到账。

笔记 038: 建立跨链监控

作者：hello16 | 时间：2026-08-13 09:09 UTC | 字数：2140 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

建立跨链监控

Scanner 核心逻辑

Chain A price

vs

Chain B price

↓

difference (价差)

↓
cost (全部成本)

↓
EV (期望净收益)

这是跨链套利监控的最小闭环。任何信号最终都要落到：价差能否覆盖真实成本，并留下正的期望值。

1. 价格层 (Price Layer)

监控同一资产在不同链上的有效价格：

Chain A：源链现货价 (DEX 或聚合器报价)

Chain B：目标链现货价

优先使用可执行深度内的价格 (而不是单纯 mid price)

常见数据源：

LI.FI /quote (直接拿到可执行路径和预估到账)

DEX 聚合器 (1inch、ParaSwap、Jupiter 等)

链上池子直接读取 (Uniswap V3 slot0 / reserves)

2. 价差计算 (Difference)

$\text{difference} = (\text{Chain B 可卖价格} - \text{Chain A 可买价格}) / \text{Chain A 可买价格}$

或用绝对数量：

$\text{gross_profit} = \text{目标链卖出所得} - \text{源链买入成本}$

注意：

必须用「可执行规模」下的价格，而不是无限深度的理想价格

考虑买卖方向 (买 A 卖 B，或反向)

3. 成本层 (Cost)

必须扣除的全部真实成本：

| 成本项 | 说明 | 来源示例 |

| :----- | :----- | :----- |

| 桥费 / Bridge Fee | 跨链协议费用 | LI.FI feeCosts |

| LI.FI / Aggregator Fee | 固定或比例费用 | LI.FI Fixed Fee |

| 源链 Gas | 发起交易的 Gas | estimate.gasCosts |

| 目标链 Gas | 如果需要 claim 或二次 swap | 估算 |

| 滑点 | 实际成交偏离 | slippage 参数 + 深度 |

| 失败成本 | 交易失败浪费的 Gas | 历史失败率 × Gas |

| 资金占用成本 | 资金锁定期间的机会成本 | 时间 × 资金量 × 利率 |

| 其他 | 授权费、中继费等 | 视路径而定 |

$\text{total_cost} = \text{桥费} + \text{聚合器费} + \text{Gas} + \text{预期滑点损失} + \text{失败预期成本} + \text{资金占用}$

4. 期望值 (EV)

$\text{EV} = (\text{difference 对应的毛利润}) - \text{total_cost}$

或更严谨：

$\text{EV} = (\text{成功概率} \times \text{成功时净收益}) + (\text{失败概率} \times \text{失败损失})$

判断标准 (建议)：

$\text{EV} > 0$ 且 覆盖安全边际 (例如要求 $\text{EV} > \text{总金额的 } 0.1\% \sim 0.3\%$)

同时满足最小绝对利润 (覆盖 Gas 波动)

执行时间在可接受范围内

简易 Scanner 伪代码结构

text

for each monitored pair (Token, ChainA, ChainB):

price_A = get_executable_price(ChainA, Token, size)

price_B = get_executable_price(ChainB, Token, size)

difference = calculate_spread(price_A, price_B)

quote = get_lifi_quote(ChainA → ChainB, amount)

cost = sum(quote.feeCosts + quote.gasCosts + estimated_slippage + opportunity_cost)

EV = gross_profit(difference, amount) - cost

if EV > threshold:

alert / record / paper trade

笔记 039: 2026-08-13

作者：Sansho2 | 时间：2026-08-13 10:41 UTC | 字数：686 | 评分：55

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-13

1. 今天学/做了什么

Day9：完成 案例 B 复盘卡 notes/cases/case-B.md——Base 同链 ETH→USDC 多路由路径差（~35–45bps / LI.FI 验证最优 ~38bps）。与案例 A 对照入库；强调「数字相同、语义不同」。

2. 得到什么结果

- 与 A 的一句差异：A 用跨链 ~25bps 地板否决微价差搬砖；B 证明同链 ~38bps 是换汇执行质量，不是无库存双边印钞。
- 正确语义五步：Quote 确认最优≈fly → 无桥地板 → 相对劣路径净节省可正 → 单腿换汇即可 → 小额可练。
- 错误语义：缺第二腿 → F2 结构差；表上 38bps 不得报成已锁双边利润。
- 可复制性：作执行案例约 4/5。
- A/B 成对：跨链否决尺子 + 同链选路尺子。

3. 失败或不定点

路径差时变、放量冲击会吃边（F5）；本日以 Day1 已验证采样 + v1/失败模式归档为主，未声称新的实盘成交。

4. 对净收益模型的更新

同链无 25bps 地板只去掉一项否决，不自动产生双边边；先贴语义标签再算净利（巩固 v1）。

5. 明天要验证的最小问题

Day10 案例 C：抽「可迁移检查清单」10 条（从 A/B + failure-modes 提炼），为 Day11 Agent 流程做输入。

笔记 040: 链上套利残酷共学 · Day 8 心得

作者：kako | 时间：2026-08-13 11:04 UTC | 字数：401 | 评分：47

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 8 心得

今日主题：机会候选清单 v1

前七天学理论、做实验、写脚本，今天终于到了“筛选真机会”这一步——从观察到的跨链价差中，挑出
3
个值得持续追踪的候选机会，并用统一模板记录下来。

说实话，第一反应是“平时看群里的价差截图感觉到到处都是机会”，但真要自己从零开始筛选时才发现，符合“值得追踪”标准的机会其实很少。真正的筛选逻辑不是“价差大就入选”，而是先过一遍成本拆解：桥费多少、两端 Gas 多少、滑点多少、有没有失败历史记录。几轮筛选下来，大部分“看起来不错”的机会都被淘汰了，剩下的那三四个才是值得放进清单的。

记录模板也挺关键的，每条机会要填时间、链、资产、规模、路径和成本明细。填完才发现，有些机会在特定时间段（比如亚洲凌晨）表现很好，但欧美时段就完全不行；有些路径在小额时表现优异，但大额时滑点失控。这些规律不记录就不会意识到。

笔记 041: 2026年8月13日 | 链上套利学习笔记

作者：WikiToTheMoon | 时间：2026-08-13 12:11 UTC | 字数：435 | 评分：58

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026年8月13日 | 链上套利学习笔记

今天继续学习链上套利，对套利机会中的执行条件和风险控制有了进一步理解。

之前理解套利时，更多关注市场之间的价格差异。但进一步学习后发现，真正能够实现的套利，需要建立完整的交易闭环。

一个套利机会需要考虑：

交易双方是否能够顺利完成；

资产是否可以快速转移；

交易过程中价格是否会发生变化；

手续费、Gas、滑点等成本是否可接受；

如果其中一笔交易失败，风险如何处理。

因此，链上套利并不是简单地发现两个市场价格不同，而是需要分析整个过程：

发现差异 → 判断机会 → 执行交易 → 控制风险 → 实现收益。

这让我进一步认识到，套利和传统金融中的交易、清算、结算逻辑有很多相似之处。表面的价格机会只是开始，更重要的是背后的资金流、交易规则和执行能力。

下一步继续学习不同套利模式的具体流程，例如 DEX 内套利、DEX 间套利和跨链套利，理解不同模式分别利用了什么市场差异。

笔记 042: 2026-08-13 : Day08 打卡内容

作者：Flxxx | 时间：2026-08-13 12:54 UTC | 字数：1389 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-13 : Day08 打卡内容

今天进入 Day 8，主题是“可执行报价标准化”。我没有把重点放在找利润，而是先解决扫描器未来一定会踩的坑：不同 API、不同 route、不同 integrator 返回的报价，必须统一成同一张证据表，才能比较。

我尝试复跑 LI.FI /advanced/routes，但今天访问 li.quest 失败：沙盒内 DNS 解析失败，联网重试后连接超时。所以今天没有新增实时行情样本。我改用 2026-08-11 已保存的原始响应做标准化练习，并明确标注它不是今天行情。

今天复盘的是 Arbitrum USDC -> Base USDC、1,000 USDC、order=CHEAPEST 的多候选 route。baseline 前 5 条分别是 eco、polymerStandard、mayanMCTP、lifilIntents、across。按保守口径：

text

strictNet = toAmountMin - fromAmount - gasCostUSD - 未包含额外成本

这 5 条全部为负，约从 -25.14 bps 到 -75.09 bps，因此都只能标记为 reject。其中 mayanMCTP 特别值得记：如果用 toAmount 算，净值约 -2.5212 USDC；如果用 toAmountMin 算，净值变成 -7.5087 USDC。这说明扫描器不能偷懒用乐观到账量。

我还对比了 baseline 和 integrator=jumper.exchange。前 5 条路径顺序一致，但 top route eco 的费用呈现明显不同：baseline 里 toAmountMin=997.5、feeUSD=2.4984、严格净值约 -2.5142；jumper 样本里 toAmountMin=1000、feeUSD=0、严格净值约 -0.0094。结论不是“jumper 一定更便宜”，而是 integrator 会影响费用口径，后续每条记录必须保存请求参数和原始响应。

今天写入的规则：

text

缺少 toAmountMin、token address、decimals、gasCostUSD 或费用包含状态 -> REJECTED_DATA

routeRank 只是聚合器排序，不是策略排序

候选净收益优先用 toAmountMin

跨场景比较必须保存 endpoint、integrator、order、slippage 和 raw response

旧样本只能用于格式学习，不能当作今日实时信号

今日产出：

Day08 任务：每日任务/Day08-可执行报价标准化.md (../每日任务/Day08-可执行报价标准化.md)

Day08 知识库：知识库/每日/Day08-可执行报价标准化.md (../知识库/每日/Day08-可执行报价标准化.md)

Day08 研究记录：研究记录/2026-08-13-Day08-可执行报价标准化.md (2026-08-13-Day08-可执行报价标准化.md)

笔记 043: Day 9 : 固定 1,000 USDC 获取一份 LI.FI 单向 Quote。

作者：daohuang1982-beep | 时间：2026-08-13 13:13 UTC | 字数：608 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 : 固定 1,000 USDC 获取一份 LI.FI 单向 Quote。

今天我先核对 LI.FI 官方 /v1/tokens 与 /v1/quote 文档，再用公开非个人占位地址，对 Ethereum → Arbitrum、1,000 USDC、CHEAPEST、0.5% 滑点条件调用了一次只读 Quote API。

采集时 LI.FI 选择 Eco 路线：预计与最低输出都是 997.5 USDC，包含 2.5 USDC 的 LI.FI 固定费；Ethereum 侧 Gas 估算约 0.1347 USD，预计时长 7 秒。按 Token 数量，单向最低输出损耗为 2.5 USDC (0.25%)。Quote 的两端 USD 价格快照显示价值差为 3.4296 USD，加所列 Gas 后约 3.5643 USD，但这只是瞬时报价口径，不是已实现 PnL。

这条路线只能从 needs_quote 进入 quote_observed，仍然停在 needs_round_trip：没有同步回程 Quote，就没有资产闭环，也没有覆盖两腿 Gas、费用、MEV、失败与资金占用。API 虽返回

transactionRequest，但今天没有保存可执行

calldata，也没有连接钱包、授权、签名、广播或成交。当前没有发现或确认任何可执行套利机会；产物是一份可复核的证据快照。

Quote

笔记 044: 一、今天建立了什么

作者：rakisa | 时间：2026-08-13 13:32 UTC | 字数：2550 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、今天建立了什么

数据库包含五张表和一个视图：

1. raw_quotes：保存完整API请求和响应；
2. normalized_quotes：保存链、Token、金额、最低到账、桥、fee、Gas和预计时间；
3. signals：连接去程和回程，保存净结果、决策与失败原因；
4. cost_components：保存每一项成本及证据状态；
5. timeline_checks：保存回程报价是否早于去程预计到账、库存需求和缺失的时间证据；
6. signal_replay_view：一条查询重放去程、回程、成本、时间和最终判断。

二、实际导入结果

| 数据 | 数量 |

| :--- | -: |

| 原始报价 | 6 |

| 标准化报价 | 6 |

| 闭环信号 | 3 |

| 成本明细 | 30 |

| 时间检查 | 3 |

六条报价来自之前保存的三档Base→Arbitrum和三档Arbitrum→Base真实LI.FI响应。数据库没有连接钱包或保存任何私钥。

三、如何承接第八天的时间问题

timeline_checks保存：

text

去程报价时间

回程报价时间

两次请求间隔

去程预计完成时间

回程报价提前多少秒

预计往返时间

是否时间不一致

是否需要目标链预存库存

是否实际观察到库存

实际到账时间是否缺失

报价失效时间是否缺失

是否原子

实际查询结果：

| 金额 | 去程→回程 | 净结果 | 回程报价提前 | 时间状态 |

| :-----: | | :-----: | | :-----: | | :-----: |

| 100 USDC | Eco→Eco | -0.349877 | 3.900秒 | TIME_INCONSISTENT |

| 1,000 USDC | Eco→Polymer | -2.231773 | 6.361秒 | TIME_INCONSISTENT |

| 10,000 USDC | Eco→Eco | -16.968222 | 5.508秒 | TIME_INCONSISTENT |

三条闭环都需要目标链预存库存，才能在原回程报价时点进行第二条腿。但本次只读实验没有读取钱包余额，所以数据库保存：

text

prefunded_inventory_required = 1

inventory_observed = 0

inventory_observed=0表示当前没有观察证据，不是断言目标链一定没有资产。

四、哪些关键数据仍然缺失

API延迟

Day04没有采集延迟：

```
text
api_latency_ms = NULL
api_latency_status = MISSING_NOT_CAPTURED
```

共6条。

实际到账时间

没有发送交易，所以只有预计时间，没有实际结算时间：

```
text
actual_forward_arrival_at = NULL
actual_arrival_status = MISSING_READ_ONLY_EXPERIMENT
```

共3条。

报价失效时间

保存的报价没有可直接使用的过期时间：

```
text
quote_expiry_at = NULL
quote_expiry_status = MISSING_NOT_RETURNED
```

共3条。

这些字段不能用0或自己猜测的时间替代。

五、今天遇到的真实问题

升级数据库时第一次重建失败：

```
text
view signal_replay_view already exists
```

原因是旧视图不会随着相关表重建自动消失。修复方式是在schema开始处显式执行：

```
sql
DROP VIEW IF EXISTS signal_replay_view;
```

然后再按依赖顺序重建表和新版视图。这让我意识到，数据库结构升级不仅是增加字段，还要处理旧对象和依赖关系。

六、数据完整性与安全

数据库使用以下约束：

- 报价ID唯一；
- 金额不得为负；
- 请求和响应必须是合法JSON；
- 标准化报价必须关联原始报价；
- 信号必须关联去程和回程；
- 时间检查必须关联信号；
- MISSING数据必须使用NULL；
- 未产生信号时不能获得执行资格。

导入前还扫描疑似私钥、助记词、API Key、授权头和Cookie字段。本次没有发现敏感字段。

验证结果：

```
text
SQLite完整性检查：ok
外键错误：0
孤儿记录：0
产生的真实信号：0
允许执行信号：0
时间不一致闭环：3
需要预存库存：3
实际观测库存：0
实际到账时间缺失：3
报价失效时间缺失：3
原子闭环：0
```

新版数据库运行18项自动化断言，全部通过。

七、今天最重要的认识

数据库不应该只保存“预计赚多少钱”。它还必须保存：

这个数字来自哪两条报价；
哪些成本是真实数据，哪些是假设；
回程报价出现时资产是否已经可用；
如果资产没到，是否有预存库存证据；
实际到账和报价过期时间是否缺失；
最终为什么接受或拒绝。

即使未来某次净收益计算为正，只要时间不一致、没有库存证据或报价有效期未知，也不能直接产生执行信号。

我的当前数据链已经变成：

text

原始报价

→ 标准化报价
→ 成本模型
→ 时间一致性检查
→ 信号决策
→ 拒绝原因

八、下一步

下一步准备写定时报价采集器，持续写入今天的数据库，并补齐当前缺少的API延迟、HTTP状态、重试次数、错误类型和去重指纹。采集器需要支持单次失败不崩溃和Ctrl+C优雅退出。

笔记 045: Day 9 · 2026-08-13 · 把第一周的工具压一压,再用 24 小时真实市场数据测它一遍

作者：rslofty | 时间：2026-08-13 13:50 UTC | 字数：13939 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 · 2026-08-13 · 把第一周的工具压一压,再用 24 小时真实市场数据测它一遍

一句话总结

今天完成了 Day 8 计划里的三件事,得到三个新认识:(1) net_pnl v2 给 amt > E_x,eff 加了红色警告,并把"跨 tick 累计冲击"从单段近似升级为按 N tick 模拟累加 — 一个本来报告 net_final = +\$1.87 的 case,跨 tick 后真实冲击从 \$0.0007 放大到 \$0.005,7.5 倍的"隐藏成本"被工具自己识别出来;(2) LI.FI v1 API 在没有 API Key 的情况下,36 条回归 quote 中只有 2 条 ETH→Arb 跑通,其余 34 条全部 code=1005 Rate limit exceeded, retry in 2 hours — 这意味着 Day 1–Day 6 笔记里所有引用 LI.FI quote 的数字,今天没有任何一条能在不付费 / 不带 key 的情况下复现;(3) Hyperliquid BTC 24 小时永续 funding rate 在 23 个数据点上几乎恒定 0.0000125/小时 ($\approx 10.95\%$ 年化),premium 全部为负 (-0.02% ~ -0.05%),意味着做空永续 + 持现货是当下唯一有 funding 收益的方向。

1. 今日目标 (来自 Day 8 §6 计划 #1, #2, #3)

| 编号 | Day 8 计划 | 今天做完了吗 |

|---|---|---|

| #1 | net_pnl v2:加 amt > E_x,eff 红色警告 + tick 跨越后的冲击重算 | |

| #2 | 回归 LI.FI v1 schema 变更,把 Day 1–Day 6 所有 quote 用新字段名重跑一遍 | 部分(限流) |

| #3 | Hyperliquid xyz paper track 第一段:info API 拉 24h mark/oracle/funding 时间序列,存到本地 CSV | |

本篇就这三件事,不再扩张范围。其余两条 (tick 跨越 / Hyperliquid paper trading 实操) 留 Day 10+。

2. net_pnl v2 — 三个改动全部落地

2.1 改动清单

工具代码:/tmp/day9/net_pnl_v2.py(基于 Day 8 /tmp/day8/net_pnl.py,把三个改动直接补进原文件,而不是 fork 一个新工具)。

| 改动 | 对应 Day 8 §4 失败 #2 / #3 | 实现位置 |

|---|---|---|

| (a) amt > E_x,eff 红色警告 | #2 | v3_calc 在 E_x,eff 计算后,加 if amt > E_x,eff: warnings.append(RED...),并把 warnings 字段加进 V2Result 和 V3Result dataclass |

| (b) --simulate-ticks N 跨 tick 累计冲击 | #2 (后半) | v3_calc 加 simulate_ticks 参数,默认 0 (单段不变),>0 时把 [p_cur, p_upper] 等分成 N 段,每段算 E_x,eff_i 和 amt_i,累计 impact |

| (c) --target-fee-pct 让 break-even 可配置 | #3 | xchain_calc 加 target_fee_pct 参数,默认 0.05% (兼容 Day 8);break-even 反推公式改为 (bridge_fixed + gas_usd) / (target_pct/100) |

| (d) (顺手) combo verdict 加警告聚合 | — | combo_calc 把 pool 的 warnings 也聚合到顶层, verdict=LOSS 时加红色告警 |
combo 的 verdict 逻辑保持 Day 8 的 "只看 net_final 符号", 没有做 "warning 但 verdict 仍 PROFIT" 时的二次降级 — 这是 Day 10 要补的 (见 §6 计划 #1)。

2.2 Day 8 V3 combo case 重跑 — 一个数字对照

| 场景 | Day 8 输出 | Day 9 v2 输出 | 差异 |

| ---|---|---|---|

| 1k USDC, V3 5bp 池, 0.5% 价差, E_x,eff=749, Arb→BSC \$2.63 跨链 | PROFIT, net_final=\$1.87, 无警告 | PROFIT, net_final=\$1.869, 1 条 RED 警告 | 数字一致, \$1.87 vs \$1.869 是 round 精度差; 但 v2 现在告诉用户 "amt > E_x,eff 必须跨 tick" |
| 同上, 加 --simulate-ticks 10 | (Day 8 没这功能) | PROFIT, net_final=\$1.865, 2 条 RED; price_impact_simulated = \$0.005 vs 单段 \$0.000667 | 跨 tick 累计冲击是单段近似的 7.5 倍 — Day 8 漏算的隐藏成本 |

那个 7.5 倍数字本身就是今天最重要的工具升级验证: 本来 v1 会告诉用户 "能赚 \$1.87", v2 在 simulate-ticks=10 下告诉用户 "能赚 \$1.865, 而且工具警告你 E_x,eff 不够, 跨 tick 后真实冲击是 \$0.005 而不是 \$0.0007"。注意 net_final 数字本身变化很小 (\$1.87 → \$1.865 = -\$0.005), 这是因为这个 case 的价差 0.5% 远大于 fee + impact; 但如果价差只比 fee + impact 大一个零头, 这个 7.5 倍就会把 PROFIT 翻成 LOSS。

2.3 6 个数字证明 v2 比 v1 更狠

| Case | 工具输入 | v1 verdict | v2 verdict | 警告数 |

| ---|---|---|---|

A	V2 100k 池, 30bp fee, 1% 价差, 1k, \$2.63 跨链	PROFIT \$4.37	PROFIT \$4.370	0 (符合预期, V2 模型不需警告)
C	V3 300k L, 1.0→1.005, 5bp fee, 1k, \$2.63 跨链	PROFIT \$1.87	PROFIT \$1.869	1 (amt > E_x,eff)
B	C + --simulate-ticks 10	(无此功能)	PROFIT \$1.865	2 (含 "10 tick 仍不够")
G	V3 300k L, 1.0→1.005, 5bp fee, 600 (在 E_x,eff 内), \$2.63	(v1 不会警告)	PROFIT \$0.070	0 (关键: amt < E_x,eff 不警告)
H	V3 300k L, 1.0→1.005, 5bp fee, 2000 (远超), --simulate-ticks 20	(v1 不会警告)	PROFIT \$6.36	2 (含 "20 tick 仍不够")
D	xchain Arb→BSC 1k, \$2.60 bridge + \$0.028 gas, target 0.3%	(v1 target 写死 0.05%)	fee% 0.26, break-even \$876	—
E	同 D, target 0.05% (Day 8 默认)	break-even \$5,256	break-even \$5,256	(v2 数字 = v1 数字)
F	xchain 50k, target 0.3%	—	fee% 0.26, break-even \$43,356	—

注意 G vs C: amt 600 落在 E_x,eff 749 内, v2 不警告; amt 1000 超 E_x,eff, 立刻警告。这是 v1 缺失的 "门槛检测"。

也注意 E_x,eff 这个数值本身很有意思: 300k L, 1.0→1.005, 意味着区间宽度只有 0.5%, 但价格是 1.0, 所以 $\Delta p/p = 0.5\%$, 在这种窄区间里 E_x,eff 只 749 — V3 集中流动性并不一定比 V2 池深, 只是把流动性集中在窄区间。Day 5 §1.1 写过这点, 今天用 v2 工具把它变成可执行的 "区间容量检测"。

3. LI.FI v1 schema 回归 — 大部分失败, 这是发现本身

3.1 计划与现实

Day 8 §6 计划 #2 说: "回归 LI.FI v1 schema 变更, 把 Day 1–Day 6 所有 quote 命令用新字段名重跑一遍, 看哪些数字今天跑不出来了"。今天跑了 6 条路径 × 6 个金额 = 36 条 quote, 目标是把 Day 5 / Day 6 / Day 8 的 quote 数字全部对账一遍。

3.2 实际跑出来的

只有 2 条 quote 拿到了 200 OK, 其余 34 条全部返回 {"code": 1005, "message": "Rate limit exceeded, retry in 2 hours"}。

| 路径 | 100 | 500 | 1k | 5k | 10k | 50k |

| ---|---|---|---|---|

ETH → Arb		429	429	429	429	
Arb → BSC	429	429	429	429	429	429
ETH → Poly	429	429	429	429	429	429
Arb → ETH	429	429	429	429	429	429
Poly → Arb	429	429	429	429	429	429
Poly → BSC	429	429	429	429	429	429

注: 429 = Rate limit exceeded, LI.FI v1 的真实代码是 1005 但语义等价。

3.3 这 2 条 ETH → Arb 数据点的对照

| amt (USDC) | 工具 | bridges | fee% | feeUSD | gasUSD |

| ---|---|---|---|---|

| 100 | across | (空) | 0.2654% | \$0.2655 | \$0.1035 |
| 500 | across | (空) | 0.2611% | \$1.3057 | \$0.1035 |

和 Day 8 同路径 (Arb → BSC) 对照:

- Day 8 Arb → BSC 100 USDC: fee% 0.2885%, feeUSD \$0.2633

- Day 9 ETH → Arb 100 USDC:fee% 0.2654%,feeUSD \$0.2655

两条 fee% 都在 0.26-0.29% 区间,与 Day 8 的"fee% 在 100-50k 区间稳定在 0.26-0.29%"结论一致。这是今天唯一能拿到的对账数据。

3.4 关于"bridges"字段为什么是空

新版 LI.FI v1 的 quote response 把 bridges 数组的细节塞到了 includedSteps[].toolDetails.name 和 includedSteps[].tool 里,顶层 bridges 字段要么是 null,要么只填 bridge name 数组。今天我的 parse.py 只读了顶层 bridges,所以是空。下次(等限流解除)要从 includedSteps 里 parse 详细桥列表。

3.5 这意味着什么

Day 1-Day 6 笔记里所有引用 LI.FI quote 的数字,在没有 API Key 的公开端点上,今天没有一条能在 2 小时内复现。也就是说,我作为个人研究者,如果想真做回归,要么等 2 小时(75 次 / 2h 的公开窗口),要么去申请 API Key(100 req/min,需 Portal 注册)

Day 8 §4 失败 #1 提到的 schema 变更:fromChainId → fromChain、fromTokenAddress → fromToken、fromAddress 必须小写 — 今天用 GET + data-urlencode 全小写地址跑通了 2 条,这三个 schema 变更今天仍然有效,没有二次回归

USDC 在 BSC 是 18 decimals 而不是 6 — 之前 Day 1-Day 6 笔记里都没注意这个细节,但 feeUSD 和 fee% 字段是 API 自己换算的 USD 价值,decimals 差异只影响 fromAmount 字符串格式。今天的 feeUSD 字段已经按 USD 计,数字看起来仍然正常,所以对账其实没被 decimals 问题影响

3.6 Day 10/11 要做的事

1. 申请 LI.FI Portal API Key(应该 1 分钟搞定),然后重跑完整 24 条 quote
2. 把 bridges 字段解析从顶层挪到 includedSteps,这样可以拿到完整桥链路
3. 把今天这 36 条 quote 的 rate limit 记录作为"已知边界"写进 net_pnl 的 README(虽然 net_pnl 不联网,但抓数据的脚本应该 fallback + 退避)
4. Hyperliquid 24h paper track — 三个数字 + 一个观察

4.1 抓了什么

metaAndAssetCtxs:BTC + ETH 当前的 markPx / oraclePx / midPx / funding / openInterest / 24h notional volume

fundingHistory BTC 24h:23 个 hourly funding event(1786543200026 → 1786626000026,跨度 23 小时)

candleSnapshot BTC 1h:25 根 1h K 线(最近 24 小时 + 1 根"上一个未完成小时")

4.2 关键数字

BTC 当前快照(metaAndAssetCtxs):

字段 值
--- ---
markPx \$63,665.00
oraclePx \$63,695.90
midPx \$63,664.50
impactPx (bid / ask) \$63,661.90 / \$63,665.00
当前 hourly funding 0.0000108383
openInterest 40,679.15 BTC
24h notional volume \$1,542,709,663
premium (mark - oracle) / oracle -0.04851%

ETH 当前快照:

字段 值
--- ---
markPx \$1,890.30
oraclePx \$1,891.01
当前 hourly funding 0.0000125
premium (mark - oracle) / oracle -0.03755%

BTC funding rate 时间序列(23 个 hourly 事件):

统计 rate premium
--- --- ---
min 3.846e-07 -4.969e-04
max 1.250e-05 -2.044e-04
mean 1.069e-05 -3.308e-04
末 3 个 rate 9.799e-06, 8.155e-06, 1.250e-05 (—)
末 3 个 premium (—) -4.216e-04, -4.348e-04, -3.639e-04

BTC 1h K 线(25 根):

```
| 字段 | 值 |  
|---|---|  
| first close | $63,820.00 |  
| last close | $63,665.00 |  
| min close | $63,340.00 |  
| max close | $63,911.00 |  
| 24h 振幅 (max-min)/first | 0.8947% |  
| 24h 变化 (last-first)/first | -0.2429% |
```

4.3 关键观察

(1) 永续普遍折价

24 小时内 BTC premium 全部为负 (-0.02% ~ -0.05%), ETH premium 也是负 (-0.04%)。这意味着:

做空 BTC 永续 + 持 BTC 现货 在过去 24h 每小时收 funding rate 平均 0.0000107 $\approx$ 9.36% 年化

做空 ETH 永续 + 持 ETH 现货 当前 funding rate 0.0000125 $\approx$ 10.95% 年化

折价幅度很小(几十 bps),所以单纯"funding 套利"在不动用高杠杆、不付跨链费的情况下收益不高 — 但风险极低,因为是 delta-neutral(永续空 + 现货多,价格波动抵消)

(2) funding rate 几乎恒定在 1.25e-05 / 小时

23 个数据点里,rate 在 [3.8e-07, 1.25e-05] 之间,均值 1.069e-05,只有 1 个点偏离到 3.8e-07(可能是某次极端 premium 闪崩导致 funding cap)。这意味着 HL 上 BTC 的 funding 上限被 peg 在大约 1.25e-05 / 小时(平台风险参数),不会因为市场情绪大幅波动。

(3) premium vs funding 不是线性关系

funding rate 平均 1.07e-05, premium 平均 -3.3e-04。如果 funding 严格跟随 premium, funding 应该更负(对应空头付费)。但实际 funding 是正(空头收)。这说明 HL 的 funding 不只是 premium $\times$ clamp_factor, 还叠加了利率 component(HL 文档里叫 interest rate component, 通常是正的 1.25e-05 / 小时量级)。所以即便 premium 为负, funding 仍然可能为正。

(4) 今天没做的事(坦白)

没拉 funding 时间序列到本地 CSV(虽然 raw JSON 存了,但没转成 pandas DataFrame / 没画图)

没拉 mark / oracle / impactPxs 的时间序列(只拿了 snapshot)

没拉 ETH 和 SOL 的 funding(只看了 BTC)

完全没有触发 paper trade(只读数据,没下任何模拟单)

(5) 5 个 candidate 套利方向(未验证)

```
| 方向 | 思路 | 估算收益 | 验证状态 |
```

```
|---|---|---|---|
```

```
| Cash-and-carry BTC | 现货买入 + 永续做空 | 9.36% APR funding | 未验证(只读了 HL 一边,没看现货/Binance/coinbase 深度) |
```

```
| Cash-and-carry ETH | 同上 | 10.95% APR funding | 未验证 |
```

```
| 跨所 funding 套利 (HL vs Binance) | HL funding 10.95% APR, 问 Binance / OKX 同标的 funding | ? | 未验证(还没拉 Binance funding) |
```

```
| mark vs oracle 套利 | mark - oracle = -0.05%, 但 funding 是 9% APR | (—) | mark 偏差被 funding 套利吃掉,不是单独机会 |
```

```
| impactPxs 套利 | bid 63661.9 / ask 63665.0, spread $3.10 (0.005%) | 微薄 | spread < 跨链成本,无利可图 |
```

Day 10+ 选 1-2 个深入。

5. 失败 / 卡壳 / 反例

1. LI.FI 限流阻塞 34/36 quote:这是今天最大的"工具失败",导致 schema 回归不完整。根本原因:LI.FI v1 公开端点 75 次 / 2 小时,我的 36 条 quote 一口气全跑出去,前 2 条 200 OK 之后立刻被限流,后面 34 条全 fail。改进:抓 quote 的脚本必须做"每跑 5 条 sleep 10 秒"的退避,或者先申请 API Key。

2. bridges 字段解析失败:今天 parse.py 读了顶层 bridges,但新版 LI.FI v1 quote response 把桥细节塞到了 includedSteps[].toolDetails。改进:Day 10 parse 改成读 includedSteps。

3. Hyperliquid paper trade 没启动:今天只拉了数据,没做任何"开仓 / 平仓 / 记账"的 paper track。改进:Day 10 至少搭一个最小 paper track 的 SQLite 表(schema:open_time, coin, side, size, mark_at_open, funding_paid_so_far, status)。

4. net_pnl v2 没做"warning + verdict 二次降级":今天 C 和 H 两个 case 都是 verdict=PROFIT 但 warnings=[2 RED],v2 没把 warnings 升级成"PROFIT_WITH_WARNING"。这意味着用户可能看到 PROFIT \$1.87 就忽略了 RED。改进:Day 10 加一个 verdict_with_warning 字段。

5. (坦白)Day 3 仍漏打:Day 3 (8/7) 漏打,本周请假期额剩 1 次。本周还剩 Day 10 (8/14), Day 11 (8/15) 是周末,看 ICL 周报怎么算。

6. 净收益自检 (套利专项)

用今天 v2 工具跑一组典型 case,自检数字是否站得住:

Case 1: V3 5bp 池, E_x,eff 749, amt 600 (区间内)

毛利 = $600 \times 0.5\% = \$3.00$

池 fee = $600 \times 5bp = \$0.30$

price_impact = $600 \times 5bp / 749 = \$0.0004$

net_pre_gas = \$2.70

跨链 (Arb→BSC) = \$2.63

net_final = $\$2.70 - \$2.63 = \$0.07$

→ 这个 case 几乎被跨链费吃光,V3 池子的"fee 低"优势在 600 规模下荡然无存

→ tool: net_pnl_v2.py combo --liq 300000 --p-cur 1.0 --p-upper 1.005 --fee-bps 5 --amt 600 --gas-usd 2.63 --bridge-fixed 0

Case 2: 同 Case 1 但 amt 1000 (跨 tick 警告)

毛利 = \$5.00

池 fee = \$0.50

price_impact (单段近似) = \$0.0007

price_impact (simulate-ticks=10 累计) = \$0.005 7.5× 放大

net_pre_gas = \$4.495

跨链 = \$2.63

net_final = \$1.865

→ Day 8 数字 \$1.87 是错的(漏算跨 tick),今天 \$1.865 才是真值

→ 差额 \$0.005 看似小,但如果 amt=2000 + 价差只比 fee 大 0.1%,这个 7.5× 会把数字翻负

Case 3: Cash-and-carry BTC (Hyperliquid funding)

现货买 1 BTC @ \$63,665 = \$63,665

永续空 1 BTC @ mark \$63,665 = 0 保证金 (假设 1× 杠杆不计成本)

每小时 funding = $\text{mark} \times 1.069e-05 = \0.681

24h funding = $\$0.681 \times 24 = \16.34

APR = $\$16.34 / \$63,665 \times 365 = 9.37\%$

→ 不计现货端的借贷成本(如果用自有币就是 0 成本)

→ 风险:价格 1 小时内闪崩 / 强平线触发,但 1× 杠杆不会强平

→ 真正的风险:HL 这边 9.37% APR,但 Coinbase spot 不付息 → 资金利用率 = 100% 在 BTC 上

→ 如果 BTC 1 年后价格不变,净赚 $\$16.34 \times 365 = \$5,964$ per BTC

→ 1 BTC 的下行风险敞口依然在(虽然 delta 中性,但要保证金 / 借款成本)

7. 下一周 Day 10 计划 (不超过 3 条)

1. net_pnl v2.1:加 verdict_with_warning(warning + verdict 二次降级) + bridges 字段从 LI.FI includedSteps 解析

2. LI.FI API Key 申请 + 完整 24 条 quote 回归:把今天限流的 34 条跑完,完整对账 Day 5 / Day 6 / Day 8 的 quote 数字

3. Hyperliquid paper track 启动:搭 SQLite 表,先做 ETH cash-and-carry 的虚拟开仓(只读 HL 行情 + 假装 0.1 BTC 当 margin),跟踪 24h 后的 funding 累积和 mark vs oracle 偏移

证据与可复跑入口

| 文件 / 端点 | 内容 |

|---|---|

| /tmp/day9/net_pnl_v2.py | Day 9 升级版 CLI(基于 Day 8 + 3 项改动) |

| /tmp/day9/net_pnl_v2_runs.json | 8 个 case 的完整 JSON 输出 |

| /tmp/day9/day9_eth_arb_quotes.json | 今天跑通的 2 条 ETH→Arb quote |

| /tmp/day9/day9_rate_limited.json | 34 条限流 quote 的错误记录 |

| /tmp/day9/regression/quotes.csv | 36 条 quote 完整 raw(2 OK + 34 ERR) |

| /tmp/day8/lifi_quotes_arb_bsc.json | Day 8 Arb→BSC 6 金额 quote (今天继续引用) |

| /tmp/day9/hl_ctxs.json | HL 232 个币的 mark/oracle/funding snapshot |

| /tmp/day9/hl_funding_btc.json | HL BTC 23 小时 funding 时间序列 |

| /tmp/day9/hl_btc_1h.json | HL BTC 25 根 1h K 线 |

| /tmp/day9/hl_summary.json | BTC/ETH 当前快照 + funding 统计 |

| LI.FI v1 quote (限流) | GET <https://li.quest/v1/quote?fromChain=42161&toChain=56&fromToken=0xaf88d065e77c8cC2239327C5EDb3A432268e5831&toToken=0x8ac76a51cc950d9822d68b83fe1ad97b32cd580d&fromAmount=1000000000&fromAddress=<lowercase>&slippage=0.005> |

| HL info metaAndAssetCtxs | POST <https://api.hyperliquid.xyz/info> body {"type":"metaAndAssetCtxs"} |

	HL	info	fundingHistory		POST	https://api.hyperliquid.xyz/info	body
{"type":"fundingHistory","coin":"BTC","startTime":<ms>,"endTime":<ms>}							

	HL	info	candleSnapshot		POST	https://api.hyperliquid.xyz/info	body
--	----	------	----------------	--	------	---	------

```
{"type":"candleSnapshot","req":{"coin":"BTC","interval":"1h","startTime":<ms>,"endTime":<ms>}} |
```

安全边界

本篇全部工作以公开 RPC、LI.FI 公开报价读取、Hyperliquid 公共 info API 和本地 CLI 计算为主;没有使用私钥、助记词或钱包签名,也没有进行真实资金操作。所有"套利"案例都是纸面推演或本地 CLI 计算,未实际广播任何交易,也未在 Hyperliquid 上开任何真实或虚拟仓位(虽然明天 Day 10 会建 SQLite 表开始 paper track,但今天没启动)。

今天我学到的(非数据,只是 mental model)

工具的"红色警告"比"verdict 数字"更重要。C case 里 v1 说 PROFIT \$1.87,v2 说 PROFIT \$1.869 + RED。数字差异小,但警告代表对模型假设的破坏。下次写套利评估时,我会先看 warnings 而不是 verdict。LI.FI 公开端点对个人研究者不可用。75 次 / 2 小时意味着我每天最多能跑 75 条 quote,今天 36 条就用掉了 48%。如果要做真实回归测试,必须申请 API Key(下一步)。Hyperliquid 的 funding 几乎恒定。这反而是好事 — 意味着 cash-and-carry 套利的预期收益是 deterministic 的,可以做资金利用率计算。坏消息是,所有看到 funding 的人都看到了这个 9-11% APR,套利空间会被市场参与者挤压,直到 funding 收敛到接近 0(arb-free equilibrium)。

笔记 046: 链上套利学习笔记 Day 9 : 信号 vs 策略

作者: 倪响 | 时间: 2026-08-13 13:54 UTC | 字数: 1759 | 评分: 86

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习笔记 Day 9 : 信号 vs 策略

2026-08-08 | 残酷共学 Day 9

信号和策略不是一回事

这是量化里最基本也最容易混淆的一对概念:

信号 (signal): "这里可能有东西"

策略 (strategy): "满足 X 条件就执行 Y 操作"

信号是策略的原料。一个策略通常由多个信号 + 执行规则组成:

策略 = 信号1(价差>阈值) AND 信号2(流动性充足) AND 信号3(成本覆盖)

→ 执行买入卖出

前八天我一直在"看信号"——手动调 API、读池子、看价差。今天开始把它变成"系统自动看信号"。

好信号的标准

一个值得自动化监控的信号,应该满足:

1. 可量化:能用明确的数值表达("价差 > 0.5%"),而不是"感觉有点便宜"
2. 可自动化:能通过 API / 链上数据计算出来,不需要人工判断
3. 低误报:触发时大概率是真机会,而不是噪音
4. 有响应时间:从信号出现到机会消失,有足够时间让人反应

结合前八天的学习,候选信号:

| 信号 | 阈值 (初版) | 依据 |

| :----- | :----- | :----- |

| 跨链价差 | > 0.5% | Day 6 成本模型:跨链成本约 0.25%, 0.5% 才有利可图 |

| DEX 间价差 (L2) | > 0.5% | Day 2: 但需 L2 且池子深度足够 |

| 稳定币脱锚 | 偏离 > 1% | Day 5 延伸:脱锚持续久,个人可反应 |

跨链价差信号的设计

跨链是 Phase 2 的主方向,信号应该围绕它设计。

核心思路:同一个 token (稳定币最优) 在不同链上有价格。当某条链上的价格显著偏离其他链,就是跨链套利信号。

USDC 在 Arbitrum 的价格 = \$1.001

USDC 在 Ethereum 的价格 = \$0.995

价差 = 0.6% > 0.5% 阈值 → 触发信号

怎么算跨链价差:LI.FI 的 quote 本身就是最好的信号源。它返回的 toAmount 除以 fromAmount,就是实际能拿到手的比率。如果这个比率显著偏离 1 (对稳定币),说明跨链存在价差。

但要小心:quote 里的 toAmount 已经扣了桥费。所以:

quote 比率 = toAmount / fromAmount

如果 quote 比率 > 1.005 (对 USDC→USDC)，说明"跨链后反而多了"

这才是真正的套利信号，而不是简单的链间价格差

不过 LI.FI 作为聚合器，它给的 quote 已经是最优路径了。用 LI.FI 找价差，等于"用别人的最优结果找机会"——信号会很弱。真正要自己算的是不同链上同 token 的现货价，然后判断价差是否覆盖 LI.FI 的成本。

今天的代码

写 signal_monitor.py，先实现最简单的版本：

1. 定时（每 5 分钟）调 LI.FI quote，USDC Arbitrum → Ethereum
2. 算 quote 比率 (toAmount / fromAmount)
3. 如果比率偏离 1 超过阈值，打印告警
4. 记录到 CSV（时间戳 + 比率），为后续分析积累数据

这是监控系统的最小闭环。跑起来之后，数据会积累下来，Day 13 再分析。

共学打卡：区分了信号和策略——信号是"可能有东西"，策略是"满足条件就执行"。好信号要可量化、可自动化、低误报。跨链信号用 LI.FI quote 比率作为观测值，阈值 0.5%（基于成本模型）。开始写第一个定时监控脚本。

笔记 047: 今日学习：

作者：derenjie | 时间：2026-08-13 14:34 UTC | 字数：299 | 评分：55

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习：

1. 同一种资产在不同链上的价格为什么会不同
2. 如何比较不同跨链路线的到账数量
3. 如何拆分跨链总成本：
 - 源链 Gas
 - 桥费
 - 目标链 Gas
 - DEX 手续费
 - 滑点
 - 等待时间和资金占用成本
4. 如何区分"表面价差"和"真实可执行收益"
5. 跨链路由失败、流动性不足、代币脱锚等风险

今天的练习

选一个稳定币USDC，观察一条：

Ethereum → Arbitrum

记录：

出发数量

预计到账数量

使用的桥和 DEX

总费用

预计到账时间

最终净价差

是否值得执行

笔记 048: 一、今日主题

作者：oetn | 时间：2026-08-13 14:36 UTC | 字数：766 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、今日主题

从发现价差到执行：如何接入 LI.FI 实现跨链套利 + AI Agent 集成

二、核心认知 / 一句话总结

发现价差 $\neq$ 能赚钱。套利 = 发现价差（策略层）+ 算透所有成本到 Break-even（执行层）。LI.FI 是执行层基建，但默认 25bps 服务费会直接抬高 Break-even，套利前必须先算透成本。

三、关键概念与要点

LI.FI 定位：链上流动性“路径与执行层”，把 Chain/Bridge/DEX/Solver 统一到一个 API。是执行基础设施，不是策略系统。

bps 基点：1 bps = 0.01%；25 bps = 0.25%；1000 USDC 的 25bps = 2.5 USDC。

真实案例：ETH USDC $\rightarrow$ ARB USDC 1000U，Route=Eco，到手 997.5U，LI.FI 服务费 0.25%，Gas $\approx$ \$0.25，耗时 7 秒。

关键教训：套利边际常只有几十 bps。价差 20bps 连服务费都覆盖不了；价差 40bps 扣完 Gas + 滑点 + 延迟也未必剩利润。

四、公式 / 实操

Break-even Spread（最低所需价差）：

plaintext

= LI.FI 服务费 + Bridge/DEX/Solver 费 + 两端 Gas + 两端滑点/Price Impact + 延迟/失败/资金占用成本

成本项：toAmount（预计到手）/ toAmountMin（最低保证）/ feeCosts / gasCosts / executionDuration / tool+includedSteps

笔记 049: August 13, 2026 · Day 9 / 21 · 时间分配：先动手 60min $\rightarrow$ 后概

作者：clare ma | 时间：2026-08-13 14:37 UTC | 字数：5135 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

August 13, 2026 · Day 9 / 21 · 时间分配：先动手 60min $\rightarrow$ 后概念 40min $\rightarrow$ 打卡 20min

SUCCESS | Day 9 验收通过

- [实测] 已完成第一个 LI.FI quote 采集脚本，能够主动请求新的时点报价并追加落盘。
- [实测] 共留下 7 次观察记录，其中 4 次成功获得可用 quote。
- [实测] 已完成 --interval 60 的两次连续采集，并通过 Ctrl-C 安全停止。
- [实测] 已保存结构化 JSONL、成功及 HTTP 错误的原始响应和 SHA-256。
- [实测] 请求仅使用 POST <https://li.quest/v1/advanced/routes>，无 API key；没有调用交易生成端点，没有签名、广播或真实交易。
- [Hermes 报告] 12 项离线测试通过，py_compile 通过；测试使用 fake transport，没有访问真实 LI.FI API。
- [未知] 所有报价字段仍是 LI.FI API estimate；没有真实执行，因此不能据此判断盈利、真实成交结果或真实交易成本。

学了什么

今天理解了三个脚本在数据链路中的不同职责：

parse_lifi_routes.py：解析已经保存的 LI.FI 响应；

analyze_lifi_costs.py：分析已有报价中的费用、精确 token unit 和成本字段；

quote_collector.py：主动请求新时点的只读 quote，记录观察时间，并将结构化记录和原始响应落盘。

我也确认了以下边界：

1. API quote 是报价估算，不是真实成交，也不能直接当作套利盈利。
2. 不同时间的 route_count 可能变化，因为路由器在不同观察时点返回的候选路线集合会变化；这不一定是脚本错误。
3. 同名 USDC 不能只根据 symbol 当作同一个精确 token unit。本次分别记录了两端的 chainId + decimals + token address：
 - Ethereum USDC：chainId 1，decimals 6，地址 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48
 - Arbitrum USDC：chainId 42161，decimals 6，地址 0xaf88d065e77c8cC2239327C5EDb3A432268e5831
4. HTTP 请求成功与“存在可用 quote”是两层状态：脚本分别使用 request_status、quote_available 和 quote_status 表达。
5. tool 和 first_step_tool 只表示第一步工具，tools 才按 route step 顺序保留整条路线的工具列表。

做了什么

固定采集参数：

参数 固定值 状态
--- --- ---
来源链 Ethereum，chainId 1 实测配置
目标链 Arbitrum，chainId 42161 实测配置
来源资产 Ethereum USDC 实测配置
目标资产 Arbitrum USDC 实测配置
输入规模 名义面额 100 USDC 实测配置
amount_in_raw 100000000 实测配置
请求端点 POST https://li.quest/v1/advanced/routes 实测配置
认证 无 API key 实测配置

| 交易生成 | 未调用 /steps/transaction | 实测安全边界 |
| 真实交易 | 未签名、未广播、未执行 | 实测安全边界 |

完成并使用了以下文件：

scripts/quote_collector.py
tests/test_quote_collector.py
data/day9-quotes.jsonl
data/day9-raw/

采集器支持单次采集、固定间隔定时采集、JSONL 追加写入、Ctrl-C 安全停止，以及 HTTP 错误、网络异常和无效响应的独立状态。HTTP 错误和非法 JSON 会保存 raw response；网络异常没有 response body 时会记录明确原因。成功观察保存 UTC 时间、请求状态、两端精确 token unit、raw amount、路线字段、费用字段、原始响应文件和 SHA-256。

其中 to_amount_raw、to_amount_min_raw、gas_cost_usd 和 execution_duration_seconds 都明确属于 LI.FI API estimate，不是实际执行结果。

得到了什么结果

data/day9-quotes.jsonl 中共有 7 条追加记录：

观察结果	次数	证据分类
本机 CA 证书链错误，network_error	2	实测请求状态
Cloudflare HTTP 403 Error 1010	1	实测 HTTP 响应
HTTP 200 且 quote_available=true	4	实测请求状态 + LI.FI API estimate

Cloudflare 403 的原始响应已保存，内容明确标记为 error_code=1010、error_name=browser_signature_banned、retryable=false、owner_action_required=true。该失败记录没有被删除或伪装成成功 quote。

四次成功观察：

observed_at_utc	route_count	quote_status	分类
2026-08-13T12:45:56.765721Z	16	available	实测观察 + API 路线估算
2026-08-13T13:05:55.637626Z	17	available	实测观察 + API 路线估算
2026-08-13T13:19:42.310182Z	17	available	实测观察 + API 路线估算
2026-08-13T13:20:45.690716Z	16	available	实测观察 + API 路线估算

这里的 route_count 是各观察时点 LI.FI 返回的候选路线数量，不是成交数量。

最后两次通过 --interval 60 连续采集：配置间隔为 60 秒，两条 UTC 时间戳之间的实测间隔为 63.380534 秒，随后通过 Ctrl-C 安全停止，之前的 JSONL 记录得到保留，没有被覆盖。

成功与 HTTP 错误响应共留下 5 个 raw 文件：4 个成功 quote 的原始响应和 1 个 Cloudflare HTTP 403 原始响应。两次 CA 证书链失败发生在取得 HTTP response body 之前，因此对应记录中的 raw_response_file 和 response_hash 为 null，并保存了无 response body 的原因。

测试结果：

[Hermes 执行并报告] tests/test_quote_collector.py 的 12 项离线测试通过。
[Hermes 执行并报告] py_compile 语法检查通过。
[Hermes 执行并报告] 测试使用 fake transport，没有访问真实 LI.FI API。
[Hermes 执行并报告] 测试覆盖 raw response、从磁盘重新计算 SHA-256、错误状态、no_routes、单 step 与多 step route、User-Agent、无认证请求和端点安全。

以上测试结果是 Hermes 执行并报告，不是 Codex 在本次 REVIEW 中独立重跑所得。

遇到什么失败 / 卡点

1. 前两次请求因本机 CA 证书链问题失败，状态为 network_error。由于没有收到 HTTP response body，raw 文件和 hash 保持为 null。这不是 LI.FI 返回的失败 quote，也不是交易失败。
2. 第三次请求收到 Cloudflare HTTP 403 Error 1010。原始响应已保存并计算 SHA-256；没有把失败记录伪装成成功，也没有使用代理、IP 轮换或其他绕过方式。
3. --interval 60 的两次观察实际相差 63.380534 秒。当前循环是完成请求和落盘后再等待 60 秒，因此观察间隔还包含请求与写盘耗时；本次没有单独记录各阶段耗时，无法进一步拆分。

八项成本状态：

1. 价差收益：未知 / 未计算。本次只有单向跨链 route quote，没有构造套利闭环。
2. Gas：LI.FI API estimate。每条路线保存了 gas_cost_usd，不是真实交易 receipt 中的 Gas 支出。
3. 协议费：LI.FI API estimate，尚未统一汇总。已保存 feeCosts 原始字段，没有重复扣减已包含在 toAmount 中的费用。
4. 桥费：LI.FI API estimate，尚未统一归类。已保存跨链路线相关 feeCosts，没有从无关字段猜测费用性质。
5. 滑点和价格冲击：LI.FI API estimate + 真实执行部分未知。已保存 toAmount、toAmountMin；没有真实执行，因此 quote 到 execution 之间的真实滑点未知，不能把 toAmountMin 写成已发生的滑点损失。
6. 延迟成本：LI.FI API estimate，金额未知。已保存 execution_duration_seconds，但尚未换算成金额。
7. 失败交易成本：未知 / 未量化。本次记录的是 HTTP、证书链和 Cloudflare 请求失败，不是真实链上交易失败；没有真实失败交易 Gas 成本。
8. 资金占用成本：未知 / 未量化。没有投入真实本金，也没有完成套利闭环或确定资金占用周期。

最大不确定性：当前拿到的是 LI.FI 在四个观察时点返回的路线估算，没有真实执行证据，因此真实成交数量、真实 Gas、真实执行时长、quote-to-execution 滑点、失败交易成本和最终盈利均未知。

下一步验证什么

下一步先继续做离线复读，不执行交易：

1. 重新读取 data/day9-quotes.jsonl，确认 7 条观察均能稳定解析；
2. 按 observed_at_utc 对齐四次成功观察；
3. 比较不同观察时点的 route_count、route ID、工具列表和 API estimate 如何变化；
4. 保持 Ethereum USDC 与 Arbitrum USDC 的 chainId + decimals + token address 区分；
5. 不把路线数量变化或 API 报价变化解释成真实盈利；
6. 若后续继续采集，先明确采集频率与停止条件，仍只调用只读 Routes API。

在构造往返闭环并逐项处理八项成本之前，不判断是否存在可执行套利机会。

笔记 050: 2026-08-13 — Day 9(Week 2 第 2 天):监控雏形落地 — 跨链路径轮询脚本

作者：bamboo | 时间：2026-08-13 14:41 UTC | 字数：2146 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-13 — Day 9(Week 2 第 2 天):监控雏形落地 — 跨链路径轮询脚本

主题: Week 2「Hermes 信号 workflow:监控雏形」——把 Day 8 成本模型函数接进轮询脚本,4 条跨链路径定时扫描,价差 > break-even 才出信号

方法: 复用 lifi_quote_check.fetch + lifi_cost_model 纯函数,串成 lifi_monitor.py

结论先行: 监控雏形 v0.1 已跑通(单轮/轮询/JSON 留档),4 条路径实测——全部没肉,但"没肉"本身就是信号系统要输出的常态结论

一、今日任务回顾(昨天笔记的明天计划)

1. 把 quote 拉取 + 成本模型串成监控雏形脚本:定时轮询几条跨链路径,价差 > break-even 时输出信号(先本地,后接飞书)
2. Dune 基础查询上手(Week 2 大纲) — 未做,顺延

二、产出:scripts/lifi_monitor.py(监控雏形 v0.1)

- 复用:lifi_quote_check.fetch(本机直连) → 失败自动服务器中转)+ lifi_cost_model

三个纯函数(explicit_costs/breakeven_bps/verdict),不重复造轮子

- 内置 4 条 USDC 代币跨链路径:ETH→ARB / BASE→ARB / POL→BASE / ETH→BASE(金额默认 1000 U)

- 模式:

- 单轮扫描:输出 路径/route/耗时/服务费bps/Gas/显性成本/往返break-even

- --spread <bps>:传入当前观察价差 → 逐条三档判定(//)

- --watch --rounds N --interval S:轮询模式(限流保护 ≤5 轮,未认证 quote 2h/75 次)

- --json:输出 JSON 留档(为后续接监控存储做准备)

- 已登记 scripts/README.md

三、真实数据(2026-08-13 22:39 实测,4 条路径全通)

| 路径 | route | 耗时 | 服务费 | Gas | 显性成本 | 往返 BE | 判定(价差13.6) |

| --- | --- | --- | --- | --- | --- | --- | --- |

| ETH→ARB | eco | 7s | 25.0 | 1.5 | 26.4 | 52.9 | 3.9× |

| BASE→ARB | eco | 7s | 24.9 | 0.1 | 25.0 | 50.0 | 3.7× |

| POL→BASE | eco | 7s | 25.0 | 0.2 | 25.2 | 50.4 | 3.7× |

| ETH→BASE | eco | 7s | 25.0 | 1.5 | 26.4 | 52.9 | 3.9× |

三个观察:

1. 4 条路径全走 eco 路由、全 ~25bps 服务费——再次验证 Day 8 认知:LI.FI 服务费 = 0.25%

比例费,与金额/路径几乎无关,是跨链的"地板税"

2. 往返 break-even 50~53 bps;跨链搬运要肉,观察价差必须 > 50bps;13.6bps 只是零头

3. Gas 随链差异明显(ETH 1.5bps vs BASE/POL 0.1~0.2bps)——L2 便宜是结构性的,但相对 25bps 服务费可忽略

四、卡点 & 待办

[] 观察价差仍是 --spread 手动传入——自动价差源(Token Service 两链同币比价,或 Pulse 对子)是 v0.2 关键

[] 接飞书推送:轮询到 信号才推(用户嫌吵原则:常态没肉不打扰)

[] Dune 基础查询上手(昨天顺延)

[] 桥费 vs 服务费拆解验证(选 2-3 条路径对比 feeCosts 构成,Day 8 遗留)

五、证据 / 资料链接

脚本: scripts/lifi_monitor.py(新)、scripts/lifi_cost_model.py(Day 8)、scripts/lifi_quote_check.py(Day 4)

实测输出: 上文表格(22:39,4 条路径);JSON 留档用 --json 可复现

官方文档: <https://docs.li.fi> / <https://li.quest/v1/quote>

六、明天计划(8/14)

监控雏形 v0.2:接 Token Service / 链上价格做自动价差源(两链同币实时比价 → 价差 bps 自动喂给判定)

Dune 基础查询上手(补昨天)

视进度:飞书推送通道接上(有信号才推)

tags: #套利 #LI.FI #监控 #Week2 #信号 workflow

笔记 051: 1、使用跨链系统进行套利实践, 24小时内, 只发现了2次净利润为正的套利机会, 一次成功, 一次失败。成功

作者: Ethanxl | 时间: 2026-08-13 14:46 UTC | 字数: 164 | 评分: 50

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1、使用跨链系统进行套利实践, 24小时内, 只发现了2次净利润为正的套利机会, 一次成功, 一次失败。成功的那次是出现了5%的净利润, 最后执行完成后, 利润大概3%。失败的那次只有0.3%的净利润, 执行之后, 由于中途两边池子的变动, 最后利润为-0.5%。

2、继续进行特定token跨链监控与执行的泛化工作, 出现大量问题需要持续优化。

笔记 052: \

作者: CnHhm | 时间: 2026-08-13 14:47 UTC | 字数: 1635 | 评分: 63

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

\

之所为套利, 就是同一标的在不同市场产生了不同的定价, 在扣除搬运损耗之后还有利润的情况。

套利学习开篇-背景介绍

套利要满足的条件一定是不同市场间可流通, 扣除流通成本还有利润。(常见的同一名称代币高价差很多都是不同交易所关了充提币成了单机币, 或者是跨链成本高, 跨链没有流动性)。

本人是去年6月份被朋友带入开始接触加密市场的, 在接触了许多不同的坑与骗局之后, 误打误撞接触到套利的概念。套利主要是去中心化的透明与低效形成的市场错位, 而套利收益就是奖励去帮市场恢复效率的人的。所以套利不会消失, 但会越来越卷。我目前在做一个小容量的套利, 可以明显感觉到在和同一个市场的同行相互竞争, 最后会卷到利润边际点(一开始卷算法优化, 算法优化到头后开始卷基建, 之后就开始拼概率)。下图是我近两个月的套利收入, 贴上来是为了证明我真的不是在空谈, 虽然收入不多但也算是入行了。行相互竞争, 最后会卷到利润边际点(一开始卷算法优化, 算法优化到头后开始卷基建, 之后就开始拼概率)。下图是我近两个月的套利收入, 贴上来是为了证明我真的不是在空谈, 虽然收入不多但也算是入行了。[图片]

为什么要加入残酷共学, 想得到什么, 能付出什么?

上面说到我入了行有了收入, 但是我的基础是很薄弱的, 薄弱到看到bruce发的学习清单就知道一定要借此机会把清单上的东西快速过一遍(所以清单上的概念都是我的薄弱项)。因为套利的价差只是表现, 同一个市场大家都能看到价差, 但是需要知道这个价差是什么结构的, 是市场的什么低效结构导致了这个价差。拿前段时间两个例子说明一下:

1.2026 年 8 月 3 日, CZ 发帖引发“币友”集中买盘, 由于币友/SPYB 与 SPYB/BNB 两个链上池子的价格更新和流动性深度发生错位, 套利者通过 BNB→低价买入币友→币友换成高估的 SPYB→SPYB卖回BNB 完成纯链上三角套利。

这个案例中, 能最先看到价差的人一个是冲meme的玩家, 第二个是平时有做lp的玩家。

2.7月28日约07:00 (UTC+8), 有人通过韩国券商在盘前市场以约867美元成交了一股海力士, Trade\XYZ的预言机引用了这个价

格，带动标记价下跌并触发连环强平；由于各平台的定价和清算能力不同，价格恢复速度不一致，产生了跨平台价差，当时的套利方式就是在cex做空、在Hyperliquid做多，等待价差回归。\\

这个案例中，能吃到的是对海力士有价格监控的人，海力士最近是极度热门标的。\\

市场是非常大的，你不可能监控所有标的，但是你要在市场最可能发生结构性低效的地方，加大你的监控投入，才能在这个市场上吃一口肉。所以要系统性的认识这个市场，知道市场的运行规则，再通过规则去找可能的漏洞。这也是我加入共学的目的之一：发现新的市场。所以我来共学大目标就是补全我的链上知识图谱，大概就像bruce的那样清单。\\

具体到细节：\\

1.我最想拿到链上报价模型的方案。链上交易是受amm池影响的，聚合器又是多池路由，计算报价越准确，套利收入越高。\\

2.发现跨链桥的思路，还有在IO实现跨链桥。\\

这两点是最想实践的，第一点目前感觉非常难，第二点应该是可以实现的。\\

关于我会分享什么\\

我绝对不会分享我已经盈利的策略（这是饭碗），但是我会分享很多有效的工具和套利系统的框架，ai和tg的运用技巧，ai是你思路的放大器、加速器、执行器，tg是你实施观察和复盘策略的窗口。如果上面链上报价方案和IO跨链桥我实现了的话，也会分享出来实现步骤。\\

~~结尾：今天突然改过不再划水打卡是因为发现bruce在群里说只拉有效笔记的一些剩余的人组个小群，不知道现在努力还来得及不。~~

笔记 053: Day 9 · 从可行告警推进到执行前决策闸门

作者：Junfan Chen | 时间：2026-08-13 14:53 UTC | 字数：1722 | 评分：84

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 · 从可行告警推进到执行前决策闸门

日期：2026-08-13

主题：把告警、模拟验证与真实执行明确分层

今日整理 / 推进

昨天把告警定义为一条完整证据链：候选机会不仅要有正的 `executable_bps`，还要经过成本校验、时效检查、连续性确认，并拥有 `opened` → `updated` → `resolved` 生命周期。今天继续向后推进：告警成立后，系统凭什么允许进入执行准备？

今天没有新的链上交易、代码运行或报价实验，因此本次是对执行闸门与复盘结构的设计，不把计划写成已经完成的实操结果。

1. 告警不等于下单指令

`qualified` 只说明样本通过了 `collector` 的数据和成本规则；`alerted` 说明它值得关注，但二者都不代表可以使用真实资金。跨链路径还有报价过期、桥接延迟、目标链状态变化、资金占用和回程不确定性。

拟增加独立状态：

text

`qualified` → `alerted` → `reviewed` → `simulated` → `approved` / `rejected`

没有模拟或小额验证证据时，最多推进到 `reviewed`，不能标为“可实盘”。

2. 执行前的五个问题

每条告警进入复核后，应形成 `decision record`：

1. 报价是否仍新鲜？复核时重算 `quote_age_ms`，不能沿用告警生成时的值。
2. 边际是否覆盖安全垫？除 `gas`、协议费和桥费外，还要预留延迟衰减、失败交易及未知成本缓冲。
3. 路径能否原子完成？非原子路径必须记录中间资产暴露和回程方案。
4. 资金会占用多久？不能只看单次 `bps`，还要考虑周转时间。
5. 失败后如何退出？若桥接成功但目标链换回失败，需要预先定义等待、改路由或止损条件。

3. “拒绝执行”也是有效产出

执行闸门需要独立于 `collector` 的拒绝原因：

text

`edge_below_safety_buffer`

`non_atomic_exposure`

`bridge_latency_unknown`

`no_exit_route`

`simulation_missing`

`manual_review_timeout`

这样才能区分：候选是被数据质量过滤、被市场成本吃掉，还是报价虽成立但不符合自身风险边界。拒绝记录不是失败，而是策略约束的证据。

4. Paper Trading 的记录口径

下一阶段先做 paper decision，不直接追求真实收益。每次告警保存：

text

alerted_at / reviewed_at

edge_at_alert_bps / edge_at_review_bps

decision / decision_reason

signal_resolved_at

hypothetical_fill_window

重点比较告警到复核期间的 edge_decay_bps，并观察信号寿命能否覆盖完整决策延迟。如果多数机会在复核前消失，应先改采样与通知链路，而不是放宽执行阈值。

关键判断

当前研究可收敛为三层：collector 生成可信证据，alert 层筛选值得关注的信号，execution gate 保护资金。三层必须保留各自状态与拒绝原因。一个好告警最终可以被拒绝执行；这不代表告警错误，而代表风险闸门发挥了作用。

下一步

- [] 定义 reviewed / simulated / approved / rejected 状态与时间戳
- [] 将市场拒绝原因和执行风险拒绝原因分开
- [] 为非原子跨链路径增加中间资产暴露与退出方案字段
- [] 用 paper decision 记录告警至复核的边际衰减
- [] 以信号寿命反推可接受的决策延迟
- [] 没有模拟证据前，不把候选描述为可实盘机会

03 MEV与抢跑 （4 篇）

笔记 054: 1. BuilderNet refund 机制

作者：Oxmigueleth | 时间：2026-08-13 03:35 UTC | 字数：5476 | 评分：89
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. BuilderNet refund 机制

规则变更：2026 年 6 月至今未找到官方对核心 refund rule (Flat Tax / contribution-based pro-rata) 的变更公告。最新官方文档 (2026-07-24) 仍描述：BuilderNet 在 MEV-Boost 中竞标，保留 surplus，按每笔 tx/bundle 对 block value 的边际贡献 (simulate with/without) 按比例退还；public mempool 或非 exclusive 流不退。支持 refundPercent/refundRecipient 及 delayed/combined refunds（早期试验延续）。\

来源类型：官方文档。 \

链接：<https://buildernet.org/docs/refunds> (2026-07-24) ；API
(<https://buildernet.org/docs/refunds%EF%BC%882026-07-24%EF%BC%89%EF%BC%9BAPI>) 参考
<https://buildernet.org/docs/api> (2026-07-25) 。论坛有
(<https://buildernet.org/docs/api%EF%BC%882026-07-25%EF%BC%89%E3%80%82%E8%AE%BA%E5%9D%9B%E6%9C%89>)
2026-07-08 关于 builder-funded gas sponsorship 的提案，但非 refund rule 变更。

实际 refund 比例统计：未找到社区公开的系统性实际比例统计（如平均 % of contribution 或 searcher 净收益占比）。仅有 Dune 总 refund 数据 (all-time gas refunds 约 2,866 ETH，max 单笔约 35.66 ETH，数据持续更新至 2026)。规则本身是贡献比例分配 remainder，实际取决于 competition 与 BuilderNet 利润。 \

来源类型：官方仪表板 / 研究者数据。 \

链接：<https://dune.com/flashbots/buildernet>（聚合自
(<https://dune.com/flashbots/buildernet%EF%BC%88E8%81%9A%E5%90%88%E8%87%AA>) 2024-11，持续更新）。

2. Aave V3 清算赛道竞争格局

活跃搜索者数量：一研究者分析（覆盖 Aave V3 + SparkLend + Morpho + Fluid，约 39k 事件）显示共 569 个 unique liquidator 地址；月活跃从 2024 的 20–30 升至 early 2026 超 100。Top 5 捕获约 47.4% 总 gross profit，Top 20 约 89%，其余绝大多数亏损。Aave V3 占清算量约 75%。 \

来源类型：研究者一手数据（从业者/分析帖）。 \

链接：X @vself_meta highlights 相关分析（内容引用 early 2026 数据）。另有 arXiv 论文记录跨链 speculative liquidators（2025

年 10 月事件 64 个)。

liquidation bonus 调整：核心 V3 参数多为资产/市场特定，主流仍约 5% (部分 6–8%)。有针对性调整：如 Polygon V3 WBTC LB 从 6.5% 提至 8.5% (提案 2026-02-08)。V4 规划中引入 variable/Dutch-style bonus，但 V3 未全面调整。\\

来源类型：官方治理。\\

链接：<https://governance.aave.com/t/direct-to-aip-increase-wbtc-liquidation-bonus.../24029> (2026-02-08)；风险讨论 (<https://governance.aave.com/t/direct-to-aip-increase-wbtc-liquidation-bonus.../24029?BC%882026-02-08?BC%89?BC%9B%E9%A3%8E%E9%99%A9%E8%AE%A8%E8%AE%BA>)

<https://governance.aave.com/t/optimal-liquidation-parameters.../24294> (2026-03-14)。

(<https://governance.aave.com/t/optimal-liquidation-parameters.../24294?BC%882026-03-14?BC%89%E3%80%82>)

大额清算事件：2026-01-31 至 02-05 市场下跌期间 Aave 清算约 \$429M (\\~12,500 txs, 破纪录)。SVR 累计处理约 \$675M liquidations、recapture \\~\$16M (至 early Feb 2026)。近期有 reserve deprecation 提案 (影响约 \$98M 供应)，但非清算事件。\\

来源类型：官方博客 + 研究。\\

链接：<https://aave.com/blog/historical-liquidations>；<https://blokz.dev/articles/oev-liquidation-recapture/> (2026-06-19)。(http://aave.com/blog/historical-liquidations?BC%9Bhttps://blokz.dev/articles/oev-liquidation-recapture/?BC%882026-06-19?BC%89%E3%80%82)

3. Bebop PMM RFQ 接入门槛

现状：仍需 API key (生产级 tight quotes + 完整 rate limit)。无 key 仅 demo mode (widened quotes、重限速，不适合生产)。通过 support 页面申请，团队直接 provision。文档明确 Authentication 要求。\\

来源类型：官方文档。\\

链接：<https://docs.bebop.xyz/core-concepts/authentication>；<https://docs.bebop.xyz/rfq-api/introduction> (更新至 (<https://docs.bebop.xyz/core-concepts/authentication?BC%9Bhttps://docs.bebop.xyz/rfq-api/introduction?BC%88%E6%9B%B4%E6%96%B0%E8%87%B3>) 2026-03/06/07)。

同类替代 (对小团队开放)：未找到明确对小团队完全 permissionless、无需商业协议的同类 PMM RFQ / intent 结算层直接替代。Bebop 自身有 solver 申请通道；UniswapX 等 intent 层存在，但机制不同 (非纯 PMM RFQ)。\\

来源类型：官方文档检索结果。

4. Robinhood Chain 现状

活跃度：2026-07-01 公网 mainnet 上线 (Arbitrum Orbit)。首周/月 TVL 快速超 \$100M–\$300M+，DEX 周 volume 曾超 \$3B (短暂进 top 5)，日活地址数十万级，交易量初期由 memecoin 主导 (约 79%)，RWA/Stock Tokens 占比低但增长。\\

来源类型：研究/新闻 (Galaxy、The Block、CoinMetrics 等)。\\

链接：<https://www.galaxy.com/insights/research/robinhood-chain-launch-analysis...> (2026-07-31)；<https://www.theblock.co/post/408024/...> (2026-07-13)；<https://coinmetrics.substack.com/p/state-of-the-network-issue-373> (2026-07-22)。(https://www.galaxy.com/insights/research/robinhood-chain-launch-analysis...?BC%882026-07-31?BC%89?BC%9Bhttps://www.theblock.co/post/408024/...?BC%882026-07-13?BC%89?BC%9Bhttps://coinmetrics.substack.com/p/state-of-the-network-issue-373?BC%882026-07-22?BC%89%E3%80%82)

主要 DEX：Uniswap (v2/v3/v4 + UniswapX) 为主 (约 99.5% 早期 volume 声明)，另有 Pleiades、Lighter (perps)、Morpho 等。\\

来源类型：官方 + 研究。\\

链接：<https://blog.uniswap.org/robinhood-chain-is-live> (2026-07-01/02)。

(<https://blog.uniswap.org/robinhood-chain-is-live?BC%882026-07-01/02?BC%89%E3%80%82>)

Sequencer 架构：中央化 sequencer (Offchain Labs 代表 Robinhood 运营)，FCFS (first-come-first-served)，目标 \\~100ms block time。\\

来源类型：研究/文档。\\

链接：同上 Galaxy、Routescore 等。

公开 mempool：无。交易直接到 sequencer，经典 public-mempool sandwich 不适用。\\

来源类型：研究分析。\\

链接：<https://www.routescore.io/blog/does-mev-exist-on-robinhood-chain> (2026-07-07)。

(<https://www.routescore.io/blog/does-mev-exist-on-robinhood-chain?BC%882026-07-07?BC%89%E3%80%82>)

MEV 竞争程度：经典 MEV (sandwich) 极低；可能存在 latency/sequencer 优先、atomic arb 或 private flow 竞争，但整体低于有公开 mempool 的链。初期流动性薄，执行成本以 slippage/impact 为主。\\

来源类型：研究者分析 (可信度中等，独立博客)。\\

X 相关讨论多为营销/价格噪音，可信度低。

5. 2026 年以来 MEV 圈对“长尾链 atomic arb 是否还有 edge”的主流看法

主流倾向：仍有 edge，尤其对中小资本。L2/长尾链资本门槛低、竞争相对分散 (atomic arb 比 stat arb/sandwich 更分散)，残值更高；但 probing 失败率高、薄利高频。L1 更适合大资金 liquidations/大 arb。研究强调 long-tail 机会 (niche

pairs、新池) 在 alpha 未泄漏前 bribe % 更低。 \

来源类型：研究者报告 / 论文 (非纯观点)。 \

链接：Greenfield Capital Part I/II (2026-08-06/10, <https://greenfieldcapital.com/...>) ; arXiv (<https://greenfieldcapital.com/...%EF%BC%89%EF%BC%9BArXiv>) 相关 atomic arb 归因与 L2 MEV 研究 (2026) ; FRB Agent L2 vs L1 对比 (2026-05) ; Oxapriori 报告 long-tail 描述 (2026-03) 。

笔记 055: \

作者：0x2077 | 时间：2026-08-13 12:53 UTC | 字数：861 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

\

死亡禁区 (不要做)：

ETH 主网原子套利/三明治：你无法战胜 BuilderNet 的模拟速度，Gas 费会让你破产。

高频 CEX-DEX：没有 CEX 的 VIP0/免费率，你的手续费会吃掉所有利润。

破局策略：做“长尾”与“拾荒者”\

步骤 1：赛道选择 - 避开锋芒 (L2 Long-tail)

目标：选择 Base, Arbitrum, 甚至更小的新兴 L2 (如基于 OP Stack 的新链)。

策略：关注新上线的、未经审计的、或者机制复杂的 DEX (如 Solidly forks, Camelot forks)。大玩家的通报系统往往忽略这些小池子。 \

步骤 2：技术栈 - 必须硬核

语言：放弃 Python web3.py, 必须使用 Rust (Alloy/Reth SDK) 或 Huff。在 L2 上，垃圾交易 (Spam) 极多，节点的处理速度决定了你是否能看到最新的状态。

节点：不要用 Infura/Alchemy。租用一台裸金属服务器 (如 Hetzner)，自己跑一个 Reth 节点。 \

步骤 3：策略 - "The Janitor" (清道夫)

不要做夹子 (Sandwich) 。

专注于清算 (Liquidation) 或特殊的套利模式：

例如：监控某些带税代币 (Tax tokens) 的特殊转账逻辑。

例如：Uniswap v4 Hooks 的特定逻辑漏洞。

理由：顶级 Searcher 的通用机器人往往过滤掉带税代币或复杂逻辑合约，这是你的 Alpha。 \

步骤 4：执行 - 概率性发送

在 L2 上，Gas 便宜。你可以采取“机枪扫射”模式，针对同一笔机会发送多笔不同参数的交易，博取被 Sequencer 打包的概率。

笔记 056: 2026.08.13

作者：Johnova | 时间：2026-08-13 13:25 UTC | 字数：2398 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026.08.13

Day 9：链上套利残酷共学

今日核心进展

2026.08.13

Day 9：Gas 策略模块 + GasOracle 合约 + RBF 失败重试

今日完成

- [x] 实现 GasOracle 合约：链上维护 baseFee 历史窗口 (滚动 50 块)、EIP-1559 baseFee 预测算法
- [x] 实现 GasStrategy 模块：estimateMaxFee(baseFee, priorityFeeP99, bufferBps)、estimateMaxPriorityFee(networkCongestion)
- [x] 集成链下 p99 PriorityFee 统计：从近 100 块提取 priorityFee = maxFeePerGas - baseFee，计算分位数
- [x] 实现动态 Gas 策略：根据套利预期利润 expectedProfit 自动调整 maxPriorityFeePerGas 上限 (利润越大越激进)
- [x] 实现 RBF (Replace-By-Fee) 失败重试机制：nonce 管理、replaceTransaction 签名替换、逐级提升优先费
- [x] GasEstimator 统一接口：estimate(swapParams, strategy) → (maxFee, maxPriorityFee, gasLimit, confidence)
- [x] 单元测试：Anvil fork 验证 BaseFee 预测准确率、RBF 替换成功率、Gas 成本 vs 利润对比

核心洞察

1. EIP-1559 BaseFee 预测只需看近期趋势 — $\text{nextBaseFee} = \text{baseFee} (1 + (\text{gasUsed} - \text{targetGas}) / \text{targetGas} / 8)$ ，滚动 20 块加权平均足够，无需复杂 ML
2. PriorityFee p99 而非均值 — 均值会被低优先费交易拉低，p99 能保证 99% 情况下不因 Gas 竞争失败，套利场景值得多付这 10-20% Gas

3. 动态优先费上限 = $\min(p99 \ 1.5, \text{expectedProfit } 0.3)$ — 保证 Gas 成本不吃掉超过 30% 预期利润，同时在拥堵时有弹性
4. RBF 替换需控制 nonce 复用窗口 — 同一 nonce 最多替换 3 次，每次 maxPriorityFeePerGas 提升 $\geq 12\%$ ，避免 mempool 拒绝
5. Gas 估算必须包含「回滚风险溢价」 — 复杂多跳路径失败率 $\sim 5\%$ ，预留 gasLimit 1.2 作为 buffer

技术决策落地

| 模块 | 方案 | 关键参数 |

|-----|-----|-----|

GasOracle 合约	baseFeeHistory[50] 环形缓冲 + updateBaseFee() 回调	BASE_FEE_WINDOW=50, ELASTICITY=8 (EIP-1559)
PriorityFee 统计	链下索引器聚合近 100 块 priorityFee，计算 p50/p90/p99	SAMPLE_BLOCKS=100, PERCENTILES=[50,90,99]
GasStrategy	DynamicPriorityFee(profitUSDC, networkCongestion)	MAX_PROFIT_RATIO=0.3, CONGESTION_MULTIPLIER=1.5
RBF 管理器	NonceManager + ReplaceByFeeSigner	MAX_REPLACES=3, MIN_FEE_BUMP_BPS=1200
GasEstimator	统一入口：estimate() → GasQuote	GAS_LIMIT_BUFFER=1.2, CONFIDENCE_THRESHOLD=0.8

遗留疑问

1. BaseFee 预测在区块间隔不稳定链（如 BSC 3s 出块波动）上准确率如何？
2. eth_maxPriorityFeePerGas RPC 返回值在不同客户端一致性如何？
3. RBF 替换时 maxFeePerGas 是否也需同比例提升，还是仅提升 maxPriorityFeePerGas？

明日预习 (Day 10)

- [] MEV-Boost 集成：mev-share / Flashbots Protect RPC 路由
- [] 闪电贷回调 Gas 优化：receiveFlashLoan 最小化存储读写
- [] 批量套利执行器：单笔交易打包多个独立套利路径
- [] 利润分配合约：团队/协议/质押者按比例自动分账

共学第 9/21 天 | 链上套利残酷共学

关键洞察

- 待补充...

技术决策落地

| 模块 | 方案 | 关键参数 |

|-----|-----|-----|

| 待补充 | | |

遗留疑问

- 1.
- 2.

明日预习

-

共学第 9/21 天 | 链上套利残酷共学

笔记 057: Day 8 (8/13) 打卡

作者：littlefyer007-igtm | 时间：2026-08-13 14:17 UTC | 字数：579 | 评分：61

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 8 (8/13) 打卡

今日完成

第一个套利合约从零到部署

- 搭建了 Hardhat 开发环境 (Node.js v24 + Hardhat 2.28)
- 编写了 FlashArbitrage.sol 闪电贷套利合约
- 核心逻辑：flash借USDC→便宜DEX买ETH→贵DEX卖ETH→还贷→留利润
- 编译通过 (solc 0.8.20)
- 本地测试网部署成功 (10000假ETH)

单元测试

- 写了4个测试：owner权限、非owner禁止调用、代币提取保护
- 4个测试全部通过

踩坑与解决

- Hardhat 3插件机制大改，import ethers/viem都报错→降级到Hardhat 2
- npm版本冲突(ERESOLVE)→对齐hardhat@2.28.0+toolbox@5.0.0

- ESM/CommonJS切换问题→删除type=module
- 公网水龙头领币失败→改用本地测试网

关键认知

- 教学合约有3个坑：无滑点保护、无防三明治、未审计
- 本地测试网比公网测试网更适合学习
- 私钥暴露过就不能放真钱

里程碑

一周前还是零编程，今天写出了可编译、可部署、可测试的闪电贷套利合约

明天

学习mempool监控，或给合约加滑点保护+防三明治

04 做市与LP（3篇）

笔记 058: 最后还是要学rust!

作者：James Wong | 时间：2026-08-13 12:09 UTC | 字数：1084 | 评分：55

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

最后还是要学rust!

高频基建一 服务器层面。做加密高频做市，一切从行情延迟开始。很多人以为策略是核心，其实不是，基建才是。你的策略再强，行情比别人慢一拍，单子挂进去就是被吃的那个。跟Cto聊完，直接放几个硬指标。服务器：AWS东京 ap1a-az4，EC2 c7i/c8i xlarge起步。别省钱用小土豆，做市不是跑网站，CPU缓存和内存带宽直接决定你的解析速度。我们20个instance一组，秒来秒关做ec2 hunting。行情延迟 benchmark：Futures p50 local ts-T：1.3-1.8ms Spot SBE+Fix：400us-900us 怎么压下来的？第一，把行情模块打包成蒸馏好的低延迟探针，直接上传到服务器，辅助 EC2 搞延迟 hunting。第二，Codec 零拷贝解析JSON，别让数据在内存里搬来搬去。

第三，双轨订阅。同一个交易所的行情，两条线同时拉。为的是万一其中一条出现丢包，另一条能瞬间补上。解析完的行情通过IPC 回传给策略进程，本地维护订单簿深度，用 ringbuffer 管理。为什么选 ringbuffer？因为它不需要锁，写和读可以同时进行。还有个细节：depth\@0ms。这东西不在交易所的API 文档里，熟悉底层的人知道有这一档位。我们要维护的是绝对“当下”的快照，策略看的是此刻的盘口，给它50毫秒前的，决策就是错的。行情快只是一半，另一半瓶颈在发单执行。

连接池必须预热保活，指令出来直接裸写。更关键的是网络分离：策略的发单绑定 EIP（弹性公网IP）发送订单组，这跟接收行情的ENI（弹性网卡）必须彻底分开。只有这样，遇到极端大行情爆发时，才能保证整个服务器上所有instance

的定价依然准确，不会因为网络拥堵互相踩踏。另外就是订单状态维护。高频里的订单状态转换必须依靠极其严苛的状态机流转。Pending、New、Partial、Canceled，每一步都不能有模棱两可的灰色地带。遇到极端行情拔网线或者接口抽风，单子卡在未知状态，你的状态机怎么兜底处理，直接决定了下一秒是继续做市还是单边止损。最后，为什么实盘核心代码的尽头依然是Rust？

很多语言的 GC在极端插针行情里只要卡你两三毫秒，单子就被吃得骨头都不剩。Rust 虽然没有 GC的烦恼，但底层原则是一致的：实盘中绝不临时 new 或者 malloc。所有内存必须在系统启动时一次性分配好，对象用完放回内存池，坚决不让操作系统在盘中去给你找内存碎片。

笔记 059: 链上套利残酷共学 | 2026-08-13 打卡

作者：koyo922 | 时间：2026-08-13 13:47 UTC | 字数：377 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 | 2026-08-13 打卡

学习记录

1. 财务状况梳理

今天主要用 ChatGPT 梳理了自己的财务状况。下一步先把基础理财规划建立起来，避免资金长期闲置。

2. 社群与资讯获取

听完了群内消息。

阅读了一些 Twitter 内容，也看了“白云”的推文和相关总结。

今天接触到的部分内容涉及跨链资产、窄区间做市、Token Tax 和跨链执行等背景知识，目前还不能完全理解，需要继续补充基础。

3. 方向调整

现阶段先不盲目自己寻找套利机会，优先寻找别人已经明确说明的具体案例，再交给 AI 分析和复现。

明日计划

继续刷 Twitter，重点寻找具体案例，记录：

交易所；

币种；

具体时间；

买卖方向和价格差。

然后选择案例进行分析和复现。

笔记 060: 链上套利残酷共学 · Day 9 打卡草稿：Curve LLAMMA 的软清算为什么依赖套利者

作者：jinzy | 时间：2026-08-13 14:44 UTC | 字数：4784 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 9 打卡草稿：Curve LLAMMA 的软清算为什么依赖套利者

日期：2026-08-13 (Asia/Shanghai)

主题：Curve crvUSD LLAMMA 的 bands、软清算、反向恢复与套利驱动再平衡

边界：只读机制研究；没有扫描实时仓位、寻找盈利机会、连接钱包或提交打卡。

选题说明

今天课程 events API 仍为空。公开打卡中已经大量出现跨链、清算、资金费率、机器人和 AMM，但尚未看到系统讲解 Curve LLAMMA 的文章。

LLAMMA 很适合接在 Aave 清算之后学习：Aave 更像健康因子跌破阈值后的离散清算，而 Curve crvUSD 在进入某个价格区间后，会把抵押品放进多个 price bands，通过套利者交易逐步将抵押品换成 crvUSD；价格恢复时还可能反向换回抵押品。

核心问题是：

LLAMMA 为什么故意制造内部价格与外部预言机价格的差异？套利者怎样利用差异获利，同时替借款仓位完成软清算或去清算？这种“平滑清算”减少了什么风险，又把哪些损耗留给借款人？

可直接打卡草稿

Curve LLAMMA：套利者如何推动软清算与反向恢复

今天学习 Curve crvUSD 的 LLAMMA (Lending Liquidating Automated Market Maker Algorithm)。它不是等价格跌到单一清算线后一次性卖掉抵押品，而是把抵押品分布在一组价格 bands 中，在价格进入清算区间后连续再平衡。

一、Bands：清算不是一个点，而是一段价格区间

LLAMMA 类似集中流动性 AMM。用户建立贷款时，抵押品会被平均分到多个连续 band 中，每个 band 都有自己的上下价格边界。

text

较高价格

Band 1：抵押品

Band 2：抵押品

Band 3：抵押品 ↔ crvUSD 交易中的 active band

Band 4：crvUSD

Band 5：crvUSD

较低价格

Curve 官方文档显示，用户创建贷款时可选择 4–50 个 bands。多个 band 合在一起，构成这笔贷款的软清算价格区间。

价格高于整个区间：各 band 主要持有原抵押品；

价格进入某个 band：该 band 成为 active_band，同时可能持有抵押品和 crvUSD；

价格继续下穿：已经经过的 band 逐步变成 crvUSD；

价格回升：过程可以反向发生，crvUSD 被重新换成抵押品。

这意味着 LLAMMA 的“清算”首先是仓位构成变化，不一定立即关闭贷款。

二、软清算与去清算

假设抵押品是 ETH，借出资产是 crvUSD。

当 ETH 价格下跌并进入贷款的 bands：

text

ETH → crvUSD

LLAMMA 会逐步降低仓位对 ETH 的暴露，使贷款同时由 ETH 和 crvUSD 支撑。这个过程称为 soft-liquidation。

如果价格随后恢复：

text

crvUSD → ETH

之前软清算得到的 crvUSD 可以逐步换回 ETH，这称为 de-liquidation。理论上，只要价格重新穿过相关 bands，仓位可以恢复为原抵押品。

与单点硬清算相比，它把一次剧烈卖出拆成跨多个 band 的连续转换，减少在某一个价格点集中抛售的压力。

三、为什么一定需要套利者

软清算不是合约看到 oracle 更新后自动替用户换币。真正推动资产转换的是外部套利者。

LLAMMA 中需要观察两个价格：

price_oracle：来自外部价格源的抵押品价格；

get_p：LLAMMA 内部 AMM 的交易价格。

协议会让内部价格对 oracle 变化更敏感，从而主动制造可套利差异。

价格下跌：推动软清算

当 oracle 价格下降时，get_p 会降得更快。例如：

text

LLAMMA 内 ETH = 830

外部市场 ETH = 850

套利者可以：

text

在 LLAMMA 用 crvUSD 低价买 ETH

→ 到外部市场卖出 ETH

结果是 LLAMMA band 内 ETH 减少、crvUSD 增加，借款人的抵押品被逐步软清算。

价格上涨：推动去清算

当 oracle 价格上升时，get_p 会涨得更快。例如：

text

外部市场 ETH = 850

LLAMMA 内 ETH = 860

套利者可以：

text

在外部市场买 ETH

→ 卖给 LLAMMA 换出 crvUSD

结果是 band 内 ETH 增加、crvUSD 减少，仓位逐步恢复抵押品。

因此：

text

get_p < price_oracle：买 LLAMMA 抵押品，推动软清算

get_p > price_oracle：卖抵押品给 LLAMMA，推动去清算

get_p = price_oracle：理论上没有该价格差套利

套利者赚取内部价和外部价的差额；协议利用他们追逐利润的行为完成仓位再平衡。

四、套利收益从哪里来

对套利者而言，简化净收益为：

text

净收益

≈ LLAMMA 与外部市场的可执行价差

- LLAMMA swap fee

- 外部市场交易费

- 双边价格影响与滑点

- Gas

- MEV / priority fee
- 报价变化与失败成本
- 资金或闪电贷成本

关键不是只比较 `get_p` 和 `price_oracle`，因为 `oracle` 是价格参照，不一定是套利者能真实成交的外部价格。真正执行时必须比较 LLAMMA 的 `get_dy` 与外部场所同一规模的 `bid/ask` 或 `quote`。

机会容量也有限：只能够交易 `active band` 及后续可跨越 `bands` 中的实际资产。Curve 合约提供 `get_amount_for_price(target)`，用于估算把内部价格推向某个目标需要买卖多少资产。规模继续增加会跨 `band`、支付更多 `fee`，并改变自己的成交均价。

五、软清算不是无损清算

套利者的利润来自谁？很大一部分来自 LLAMMA 给出的有利成交价格，因此借款仓位会承受再平衡损耗。

Curve 官方文档明确说明，`soft-liquidation` 和 `de-liquidation` 都可能产生损失。尤其当价格反复穿越清算区间时，仓位可能出现类似：

text

价格下跌时卖出 ETH

价格回升时更高价买回 ETH

即“低卖高买”的路径损耗。虽然借款人能获得 AMM swap fee，但费用不保证覆盖所有再平衡损失。

官方总结的影响因素包括：

- 使用更多 `bands`，单次转换通常更细，损失可能更低；
- 市场流动性越充分，损失可能越低；
- 套利越高效，内部价越快回到 `oracle` 附近，损失可能越低；
- 价格来回震荡、频繁穿越 `bands`，会累积更多转换损耗；
- 借款利息持续增加债务，不论是否处于软清算都会降低 `health`。

更多 `bands` 也不是免费午餐。官方给出的近似公式显示，`band` 数 `N` 越高，最大 `LTV` 越低：

text

$$LTV \approx 100\% - \text{loan_discount} - 100 \times N / (2 \times A)$$

也就是在更平滑的清算范围和更高的借款能力之间做取舍。

六、软清算与硬清算的区别

| 机制 | 触发/发生方式 | 结果 |

|---|---|---|

| 软清算 | 抵押品价格进入用户 `bands`，通过套利交易连续再平衡 | 贷款继续存在，抵押品逐步变成 `crvUSD`，也可能反向恢复 |

| 硬清算 | 贷款 `health` 低于 0%，外部清算人替用户偿债 | 贷款被强制关闭，清算人取得抵押品 |

最重要的区别是：进入软清算区间不等于已经可以硬清算。硬清算资格取决于 `health`，而不是简单取决于价格进入了哪个 `band`。

但软清算的往返损耗和持续利息都会降低 `health`。如果累计损失过大，仓位最终仍可能进入硬清算。

七、今天的核心收获

LLAMMA 把套利嵌入了风险管理机制：

text

oracle 变化

- 内部价格产生可套利偏差
- 套利者交易
- `band` 资产构成改变
- 贷款完成软清算或去清算

它的优势是把单点清算变成一个连续价格区间，并允许价格恢复时反向恢复抵押品；代价是借款人要承担套利成交让利、交易费以外的路径损耗和利息。

所以“软清算”不等于“没有损失”，“可以去清算”也不等于“一定恢复原资产数量”。从套利角度看，内部价差只是候选信号，还必须核对真实可交易数量、外部可成交价、费用、Gas、竞争和 `band` 容量。

下一步可以选一个 Curve `crvUSD` market，只读记录：

text

timestamp / block

price_oracle / get_p

active_band / p_oracle_up / p_oracle_down

bands_x / bands_y
fee / external executable quote
get_amount_for_price
gross edge / gas / net edge

只建立观察表，不连接钱包、不执行交易。

官方资料

Curve LLAMMA Explainer (<https://github.com/curvefi/docs/blob/main/docs/developer/crvusd/llamma-explainer.md>)
Curve LLAMMA AMM Contract Documentation (<https://github.com/curvefi/docs/blob/main/docs/developer/crvusd/amm.md>)
Curve crvUSD Controller Documentation (<https://github.com/curvefi/docs/blob/main/docs/developer/crvusd/controller.md>)
Curve Stablecoin Source Code (<https://github.com/curvefi/curve-stablecoin>)

证据边界

文中的 830/850/860 是 Curve 官方解释采用的教学型价格例子，不代表实时市场。

本文没有读取具体 market、用户仓位、band 余额或真实外部报价，因此没有确认当前套利机会。

price_oracle 是协议价格输入，不等于外部市场可成交价格。

“更多 bands、更多流动性和更高效套利通常减少损失”来自官方机制说明，不是对任意仓位损失率的保证。

实际 hard-liquidation、health、费用和动态参数应以目标 market 当前合约状态为准。

本文是协议机制学习，不构成交易建议。

05 其他（1 篇）

笔记 061: 学习记录

作者：capromka | 时间：2026-08-12 15:17 UTC | 字数：484 | 评分：59

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习记录

今天主要学习了LP份额、手续费分配与 APR 计算。

主要收获

理解了手续费怎么分：交易量 × 手续费率 × LP分配比例 × 我的流动性份额 = 我的手续费收入。

V2 按 LP Token 份额分配手续费；V3 按当前价格附近的活跃流动性份额分配。出区间后，V3 仓位不再赚手续费。

理解了窄区间 vs 宽区间：窄区间资金效率可能更高，但更容易出区间；宽区间相对稳定，但单位资金手续费效率可能更低。

开始理解 APR ≠ 保证收益。APR只是按照当前/近期收益速度进行年化估算，还会受到交易量、价格区间、LP竞争、手续费率、奖励和无常损失等影响。

也区分了 LP Fee APR、Farm APR、APR 和 APY，了解到 V3 手续费通常需要手动领取，再重新添加流动性才能复投。

今日总结

LP真正能赚多少，核心还是看“有效流动性” + “真实交易量” + “自己的份额”，APR只能作为参考，不能只看APR判断一个池子值不值得做。

06 套利框架（43 篇）

笔记 062: Day 8 · 把散落的策略证据归进一张表,然后剪枝

作者：99hansling | 时间：2026-08-12 15:00 UTC | 字数：1503 | 评分：69

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 8 · 把散落的策略证据归进一张表,然后剪枝

这篇在讲什么

Week 1 用数字杀死了好几条路(静态 AMM、跨链搬运、稳定币池……)，

但这些结论散在不同文件里,没有统一管理。今天做的事不性感但很重要:

把所有策略发现整理成一张“真值表”,给每条策略标上明确状态,

然后剪掉重复和无效的开户。

核心目的:防止一个已经被数字杀死的命题,换条链或换个名字又重新当成新机会来研究。这是研究里最常见的时间浪费——不是机会没了,是记账出了问题。

为什么要整理

打个比方:你上周证明"在公开 AMM 上做同链闭环套利,净亏 100 bps"。这周换个链再跑一遍,发现"还是亏"——这不是新结论,是同一个结论。但如果没有一张表记着"这条已经死了、死因是 X",你就会反复做同样的实验。

今天建的这张表解决三件事:

1. 每个策略有唯一身份:不是"借贷套利"这种模糊大类,而是精确到"在 Base 链上借 Moonwell 的 WETH、存进 Aave"这种具体计划
2. 每个计划有四个独立状态:还值不值得找证据?有没有取得采集资格?已有证据证明/否定了什么?现在被授权做什么?——这四个问题分开记,不能混为一谈
3. 死掉的不能换名复活:同一类静态闭环(Base/ARB/BSC)共享同一个被证伪的前提,换链名不算新策略,不再重复开户

今天具体做了什么

策略真值表重构:把 10+ 个策略计划按"策略族 → 具体计划"两层组织。每个计划一行,带发现状态(还该不该找证据)、验证状态(能不能采集)、证据判定(证据说了什么)、授权(现在能做什么)。

试验登记册改为单向冻结:以前"冻结假设"和"记录结果"混在同一行,可以偷偷回填。现在拆成两类记录——先冻结假设(写下命题、样本、阈值、证伪条件,然后锁死),再追加结果(指向那条冻结记录,不能改冻结卡)。这保证了"先写假设再跑实验"的纪律,防止看完结果再编假设。

加固验证卡的草案边界:借贷利差那条线(Week 1 唯一的 candidate)有一张完整的验证卡,但今天明确标注它是草案、未冻结、不得采集——抵押品选择、清算模型、价格源这些会主导结论的参数还没定,不能"先采 9 个点再决定怎么算"。

一个反直觉的发现:剪枝本身是产出

整理后看到,当前没有任何一条策略处于"正在验证"状态——所有重型采集槽位是空的(WIP = 0/1)。

这不是坏事。它说明:Week 1 杀死的那些路真的死了(没有偷偷复活),而唯一还活着的借贷利差,老老实实停在"草案"阶段等人工批准。一个诚实的研究系统,大部分时候应该是"没有在跑的东西"——而不是塞满候选但没一条经得起查。

为什么这件事对套利研究重要

套利研究最大的风险不是"没找到机会",而是把已经被否定的东西当成新发现反复投入精力。尤其当你在不同的链、不同的币对上看到"好像有价差"时,如果没有一张表告诉你"这个前提已经被 22 对 0 正的数据杀死了",你就会陷入无限重复劳动。

今天这张表的价值不在它新增了什么,而在它阻止了未来的无效重复。

验证

真值表覆盖了广度扫描的全部策略,每条带状态和证据指针
试验登记册的冻结→结果单向链路已写入规则,回填不可表达
当前 WIP = 0/1:没有未授权的采集在进行
这张表本身不产生新的正/负结论,它是对已有结论的诚实记账

笔记 063: 11:30上新anthropic盘前合约, 开局基本没有价差, 大概过几分钟才有币安多-ok空的一点价差

作者: unikzz2026 | 时间: 2026-08-12 15:05 UTC | 字数: 496 | 评分: 50

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

11:30上新anthropic盘前合约，开局基本没有价差，大概过几分钟才有币安多-ok空的一点价差，此时的策略已经上了1价差的1wu，1.5左右补了5000u，2.5左右补了5000u，在13:10左右价差到达巅峰3.1-3.2，最终浮亏时高达-350u左右。

这件事的教训：

- 1：盘前合约，哪怕是股票，如果没有预期上市时间，风险很大，参考openai，开局有价差，后来一直维持在1.5左右价差
- 2：在开局没有很好价差插针机会的时候，就应该首先意识到这个标的不是很好。
- 3：开盘的标没有很好做好梯度准备，比如1进3000,1.5进2000，2进5000等预设好心理防线。
- 4：没有做好仓位管理，对于盘前合约，而且流动性不是非常好，市场没有那么多，比如没有hl市场的，要做好赔钱打长久战的准备，不应该头铁上2wu，而是应该1wu就停止。
- 5：没有考虑到资金费的影响，8h收0.5bp，一天收1.5bp，大概一周就收10bp，也就是千分之1，在资金量不足的情况下有资金流动性限制，错失机会成本，有盘前合约作恶风险，有资金费损失，因此盘前合约如果不能有确定性收敛的机会，就不应该进。

笔记 064: 链上套利学习 Day 7 笔记

作者：Harper | 时间：2026-08-12 15:26 UTC | 字数：5365 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习 Day 7 笔记

今日主题

今天主要学习：

ETH 和 WETH 的区别

为什么 DeFi 中经常使用 WETH

Wrap / Unwrap 是什么

Price 和 Quote 的区别

为什么套利机器人不能只比较价格

如何利用 Quote 计算真实套利路径

为什么套利应该比较 Input 和最终 Output

一、ETH 和 ERC-20 Token 的区别

ETH 是：

Ethereum 网络的原生资产 (Native Asset)。

ETH 本身不是普通的 ERC-20 Token。

而：

USDC

USDT

UNI

LINK

这些 Token 通常都是 ERC-20 Token。

可以理解为：

Ethereum

|

|—— 原生资产

| |—— ETH

|

|—— ERC-20 Token

|—— USDC

|—— USDT

|—— UNI

|—— LINK

|—— WETH

二、为什么 ETH 会带来一个问题？

ERC-20 有统一的接口，例如：

transfer()

approve()

balanceOf()

DEX 智能合约可以按照统一方式处理 ERC-20 Token。

例如：

USDC

USDT

UNI

LINK

都可以使用类似的逻辑。

但是：

ETH 本身不是 ERC-20 Token。

所以如果 DEX 直接处理 ETH，就需要为 ETH 编写额外的特殊逻辑。

于是出现了：

WETH

三、什么是 WETH？

WETH：

Wrapped Ether

中文：

封装 ETH。

简单理解：

WETH 是把 ETH 包装成符合 ERC-20 标准的 Token。

基本关系：

1 ETH

↓

Wrap

↓

1 WETH

反过来：

1 WETH

↓

Unwrap

↓

1 ETH

正常情况下：

$1 \text{ ETH} \approx 1 \text{ WETH}$

四、Wrap 和买入有什么区别？

ETH → WETH 并不是普通意义上的：

用 ETH 买 WETH

而是：

ETH

↓

包装

↓

WETH

可以理解成：

同一种资产换了一种链上表示方式。

所以：

ETH → WETH

叫：

Wrap

而：

WETH → ETH

叫：

Unwrap

五、为什么需要 WETH？

因为 WETH 符合 ERC-20 标准。

DEX 智能合约就可以统一处理：

USDC

WETH

UNI

LINK

而不需要：

如果是 ETH：

使用特殊逻辑

如果是 ERC-20：

使用另一套逻辑

所以：

WETH 的重要作用，是让 ETH 可以像普通 ERC-20 Token 一样参与 DeFi 智能合约。

六、为什么 DEX 经常看到 WETH/USDC？

以后查询 Ethereum 上的 DEX Pool 时，经常会看到：

WETH / USDC

而不是：

ETH / USDC

这是正常的。

因为：

WETH 是 ETH 的 ERC-20 表示形式。

所以套利监控工具查询 Ethereum DEX 时，经常需要监控：

WETH/USDC

七、什么是 Price？

Price：

当前市场状态下，一个 Token 相对于另一个 Token 的价格。

例如：

1 WETH $\approx$ 3000 USDC

这告诉我们：

ETH 当前大约值 3000 USDC。

但是：

这不代表：

10000 USDC

一定可以换到

3.333 WETH

因为真实交易还会受到：

流动性

滑点

Price Impact

DEX Fee

等因素影响。

八、什么是 Quote ？

Quote ：

给定一个具体输入金额后，预计实际能够得到多少输出 Token。

例如：

不是问：

ETH现在多少钱？

而是问：

如果我现在拿10000 USDC交易：

实际能得到多少WETH？

DEX Quote 可能返回：

Input：

10000 USDC

↓

Estimated Output：

3.27 WETH

这就是：

Quote。

九、Price 和 Quote 的区别

Price 回答：

1 ETH大约多少钱？

例如：

1 ETH = 3000 USDC

Quote 回答：

如果我真的拿10000 USDC交易，

能得到多少ETH？

例如：

10000 USDC

→

3.27 WETH

所以：

Price

\=

当前市场价格状态

而：

Quote

\=

针对具体交易金额的预计成交结果

十、为什么套利机器人更应该关注 Quote ？

假设：

DEX A：

ETH Price：

3000 USDC

DEX B :

ETH Price :

3050 USDC

表面价差 :

3050 - 3000

\= 50 USDC

如果只看 Price :

可能认为存在套利机会。

但是 :

不能直接交易。

因为真正需要知道 :

10000 USDC

在DEX A到底可以买多少WETH ?

然后还需要知道 :

这些WETH

在DEX B到底可以卖多少USDC ?

十一、正确的套利 Quote 流程

假设本金 :

10000 USDC

第一步 :

询问 DEX A :

Quote :

10000 USDC

↓

3.28 WETH

然后 :

第二步 :

询问 DEX B :

Quote :

3.28 WETH

↓

10040 USDC

所以完整交易路径 :

10000 USDC

↓

DEX A

↓

3.28 WETH

↓

DEX B

↓

10040 USDC

毛利润 :

10040 - 10000

\= 40 USDC

但是 :

40 USDC 还不是最终利润。

十二、还需要扣除 Gas

例如：

毛利润：

40 USDC

Gas：

30 USDC

那么：

净利润：

40 - 30

\= 10 USDC

最终预计：

+10 USDC

十三、如果 Gas 大于毛利润怎么办？

例如今天的练习：

DEX A Quote：

10000 USDC

→

3.25 WETH

DEX B Quote：

3.25 WETH

→

10020 USDC

所以毛利润：

10020 - 10000

\= 20 USDC

Gas：

30 USDC

最终：

20 - 30

\= -10 USDC

所以：

最终亏损：

10 USDC

因此：

不应该执行这次套利。

虽然：

DEX A Price = 3000

DEX B Price = 3060

看起来存在：

60 USDC

价差。

但真实 Quote + Gas 告诉我们：

最终 = -10 USDC

这就不是套利机会。

十四、套利思维需要发生的变化

最开始：

DEX A：

3000

DEX B：

3060

↓

有60美元价差

↓

套利

这种判断是错误的。

现在应该变成：

我有多少本金？

↓

DEX A Quote

↓

实际得到多少WETH？

↓

DEX B Quote

↓

实际得到多少USDC？

↓

减去Gas

↓

减去其他成本

↓

计算最终净利润

十五、套利真正比较的是什么？

不是简单比较：

Price A

vs

Price B

而是比较：

Initial Input

vs

Final Output

例如：

Initial Input：

10000 USDC

经过：

USDC

↓

WETH

↓

USDC

最终：

Final Output：

10080 USDC

那么毛利润：

80 USDC

然后再扣 Gas 等成本。

十六、套利利润基本公式

可以开始记这个公式：

Gross Profit

\=

Final Output

\-

Initial Input

也就是：

毛利润

\=

最终得到的资金

\-

初始投入资金

然后：

Net Profit

\=

Final Output

\-

Initial Input

\-

Gas

\-

其他额外成本

即：

净利润

\=

最终输出

\-

初始输入

\-

Gas

\-

其他执行成本

十七、目前完整的套利监控逻辑

经过前 7 天学习，现在套利监控工具的逻辑已经可以串起来：

获取不同DEX数据

↓

确认Chain

↓

确认Token Contract Address

↓

处理Decimals

↓
检查Pool
↓
检查TVL / Liquidity
↓
发现Price差异
↓
确定套利本金
↓
请求DEX A Quote
↓
得到实际Token Output
↓
把Output作为DEX B的Input
↓
请求DEX B Quote
↓
得到最终USDC Output
↓
计算Gross Profit
↓
估算Gas
↓
计算Net Profit
↓
判断是否达到最低利润要求
↓
发送套利提醒

十八、为什么不能直接使用 Price 算套利？

因为：

Price 只是：

当前市场的边际价格。

而真实交易：

会受到交易金额影响。

例如：

Price：

1 ETH = 3000 USDC

理论：

$10000 \div 3000$

≈ 3.333 ETH

但真实 Quote：

可能：

10000 USDC

→

3.27 ETH

所以：

理论数量 $\neq$ 真实预计成交数量

十九、Price、Quote、Swap 的区别

Price

回答：

现在大约多少钱？

例如：

1 ETH $\approx$ 3000 USDC

Quote

回答：

如果我现在拿10000 USDC交易，预计能得到多少？

例如：

10000 USDC

→

3.27 ETH

Quote 是：

交易前的预计结果。

Swap

真正执行：

10000 USDC

↓

智能合约

↓

实际获得ETH

Swap 是：

真正发生的交易。

所以：

Price

↓

发现机会

Quote

↓

模拟/预计交易结果

Swap

↓

真正执行交易

二十、今天最大的认知升级

以前：

套利

\=

低价买

\+

高价卖

现在：

套利

\=

确定本金

\+

获得买入Quote

\+

获得卖出Quote

\+

计算完整交易路径

\+

扣除Gas和成本

\+

判断最终净利润

所以：

套利机器人真正寻找的不是“价格差”，而是“从一笔资金出发，走完整个交易路径以后还能多出来的钱”。

Day 7 一句话总结

WETH 是 ETH 的 ERC-20 包装形式，使 ETH 可以方便地参与 DeFi 智能合约。套利机器人不能只比较两个 DEX 显示的 Price，而应该针对具体交易金额获取 Quote，把第一个 DEX 的实际预计 Output 作为第二个 DEX 的 Input，计算最终 Output，再扣除 Gas 等成本。只有最终净利润大于 0，并达到设定的利润和风险要求，才是真正可执行的套利机会。

笔记 065: 今日打卡 | 从单次机会 PnL 走向完整库存循环 PnL

作者：Lizzie | 时间：2026-08-12 15:31 UTC | 字数：2843 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日打卡 | 从单次机会 PnL 走向完整库存循环 PnL

今天完成了什么

今天基于本地链上套利 Wiki，重点整理了成本模型、执行原子性、跨链库存和报价口径，把“某一笔看起来赚钱”进一步拆成“完整资金循环是否真的赚钱”。

本次没有新增实时行情、链上交易、回测或模拟结果；完成的是知识提炼与研究框架升级。核心变化是：套利评估不能停在单次机会，而要追踪资产最终能否回到目标市场和目标库存。

关键发现

1. 单次机会利润不等于完整循环利润

跨链两端预置库存可以让低价链买入和高价链卖出近似同时发生，避免等待桥接期间价差消失，但交易结束后会留下库存偏移：

text

低价链：目标资产增加、计价资产减少

高价链：目标资产减少、计价资产增加

因此真正需要计算的是：

text

Cycle PnL

= 单次机会利润

- 恢复目标库存的桥费与 Gas
- 再平衡期间的价格变化
- 资金占用成本
- 失败概率 × 失败损失

如果只记录 Opportunity PnL，不记录库存归位，可能把尚未兑现的单边敞口误认为利润。

2. 跨链套利首先是库存管理问题

跨链没有共同回滚按钮。源链成功后，目标链可能延迟、失败或价格反转。预置库存降低了即时断腿风险，却增加了：

多链资金占用；

库存失衡；

桥与稳定币版本风险；

再平衡费用和等待时间；

某条链暂停时的资金被困风险。

所以策略必须设置每链、每资产的目标库存、偏离阈值、最大单边敞口和停止条件，而不是看到价差后临时桥接资金。

3. 报价比较必须使用同一资产与同一计价口径

同币种跨链比较应优先使用 Token 数量口径，并单独记录两链价格源差异。已有笔记案例显示，同一笔 100 USDC Quote，Token 数量口径约为 -25 bps，而 USD 口径约为 -35.69 bps，其中约 8.37 bps 来自两链 priceUSD 差异。

这说明 USD 估值会把预言机或报价源噪声混进跨链成本。字段缺失时应记录为 null 或报错，不能静默当成真实的零。

4. 最优路径是交易规模 and 时间的函数

固定 Gas 对小资金影响更大，比例费会持续影响大资金，金额放大又会产生更强价格冲击。已有共学笔记的报价矩阵显示，小额和中大额交易可能选择不同 Bridge；某些路线只节省少量 Gas，却把预计时间从约 7 秒提高到约 1,080 秒。

因此不能为所有金额固定一条“最优桥”。至少应比较：

text

route = f(amount, fee, gas, liquidity, duration, failure_rate)

预计时间也不是最终性保证，必须与价差寿命和失败分布一起评估。

5. 失败成本必须进入期望值，而不是事后复盘

同链原子交易失败时资产状态可回滚，但 Gas 仍可能损失；跨链或 CEX-DEX 的一条腿失败会留下方向性仓位和二次对冲成本。

最小模型应记录：

两腿成交率与单腿失败率；

报价到执行延迟；

对冲耗时；

失败平均损失；

P95/P99 极端损失；

库存恢复时间与成本。

当目标利润只有几个基点时，一次断腿损失可能抵消许多次成功。

证据在哪里

第一性原理总笔记：/Users/lizzye/LLMwiki/web3/arbitrage/index.md

完整成本模型：/Users/lizzye/LLMwiki/web3/arbitrage/concepts/arbitrage-cost-model.md

执行、原子性与库存：/Users/lizzye/LLMwiki/web3/arbitrage/concepts/execution-atomicity-and-inventory.md

验证工作流：/Users/lizzye/LLMwiki/web3/arbitrage/concepts/arbitrage-validation-workflow.md

实证与纠错整理：/Users/lizzye/LLMwiki/web3/arbitrage/queries/high-value-cohort-findings-2.md

今日笔记：/Users/lizzye/LLMwiki/web3/arbitrage/checkin-2026-08-12.md

课程来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

假设与风险

费用：Quote 中已包含的 DEX、桥或聚合器费用只能扣一次；报价外的 Gas、再平衡和失败准备金不能遗漏。

滑点：不能固定假设为 0.5%；应按金额、池曲线和 Re-quote 历史分位数估计。

延迟：报价只对应特定状态，预计执行秒数不保证目标链成交价或最终性。

流动性：TVL 不等于可用深度；V3 要看 active liquidity 和 tick，订单簿要逐档计算。

同步：比较必须记录同一 block/slot 或统一时间戳，避免把时间差当空间价差。

资金费率：若涉及永续或借贷，必须加入费率反转、基差、借币和保证金成本；本次未测试此类策略。

库存：预置库存会占用资本并产生偏移、再平衡和资金被困风险。

本次数字来自已有共学笔记，未逐项独立复现，不能作为当前可交易 Edge 或收益证明。

当前仅做研究与知识整理，不连接钱包、不签名、不广播交易、不进行 Paper Trading 或 Live Execution。

还没完成

尚未建立按链和资产记录目标库存、实际库存及偏离阈值的数据表。

尚未用真实只读 Quote 测量不同金额下的路径切换、Re-quote 消失率和完整 Cycle PnL。

尚未得到失败概率、再平衡延迟与 P95/P99 损失的真实样本。

下一步

建立一个最小只读“库存循环账本”，固定两条链、一个资产对和三个交易规模，记录：

text

初始双边库存

→ 同步 Quote 与独立 Re-quote

→ 两端可执行输出

→ Opportunity PnL

→ 库存偏移

→ 再平衡路径与成本

→ 失败成本缓冲
→ Cycle PnL
→ CONTINUE / ITERATE / KILL

今天最重要的判断是：跨链套利不是完成两笔方向相反的交易，而是在可承受风险下完成一次可重复的库存循环。

笔记 066: 链上套利 Day 5 学习笔记

作者：1505zhou | 时间：2026-08-12 15:50 UTC | 字数：1528 | 评分：79

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利 Day 5 学习笔记

从观察到筛选：建立套利机会日志

一、今日学习目标

前四天完成了：

Day 1：理解链上套利与机会来源

Day 2：理解流动性、AMM、滑点和价格冲击

Day 3：认识 DEX、跨链、三角、稳定币、清算、MEV 等套利类型

Day 4：建立成本模型，计算真实净收益

今天进入第一阶段最后一步：

如何持续观察市场，并把零散的价格差转化为可研究的套利机会？

二、为什么需要“机会日志”？

链上套利机会通常具有：

短暂性

高频性

重复性

高噪声

一次看到的价差，不一定是真正的 Edge。

因此不能只记录：

“某 Token 出现 2% 价差。”

而应该记录完整的：

时间 → 市场 → 路径 → 成本 → 净收益 → 执行结果

长期积累后，才能判断：

哪些机会具有重复出现的规律，哪些只是偶然噪声。

三、建立 Opportunity Journal

每天记录至少 10 个潜在机会。

核心字段：

时间：

资产：

链：

DEX：

交易路径：

买入价格：

卖出价格：

交易规模：

Gas：

手续费：

滑点：

Price Impact：

预估净收益：

ROI：

持续时间：

是否可执行：

最终结果：

失败原因：

四、从“发现”到“筛选”

可以建立四层过滤机制：

Level 1：Price Spread

首先发现：

是否存在明显价格差？

↓

Level 2：Liquidity

判断：

是否有足够流动性支持交易？

↓

Level 3：Cost

计算：

Gas、手续费、滑点等成本。

↓

Level 4：Execution

最终判断：

是否能够真实完成交易并获得正向净收益？

形成：

Price Spread

↓

Liquidity

↓

Cost

↓

Execution

↓

Net Profit

五、如何判断机会是否有价值？

一个值得进一步研究的机会，通常需要满足：

① Edge 明确

知道利润来自哪里。

② 净收益为正

扣除所有成本后仍然盈利。

③ 流动性充足

不会因为交易规模导致严重滑点。

④ 可执行

不是只能在网页上看到，而是能够真实成交。

⑤ 具有重复性

机会能够反复出现，而不是一次性事件。

六、建立“机会评分”

可以简单设置：

指标

评分

Spread

1-5

Liquidity

1-5

Cost

1-5

Execution

1-5

Repeatability

1-5

总分越高：

越值得进入下一阶段研究。

这样可以逐步从：

人工观察

升级为：

系统化机会筛选。

Day 5 总结

第一阶段的核心逻辑已经形成：

认识市场 → 理解流动性 → 分类机会 → 计算成本 → 筛选机会

链上套利研究不应该停留在“发现价差”，而应该逐步建立：

Opportunity → Evidence → Cost → Execution → Net Profit

经过持续记录，发现要寻找的不是“今天赚不赚钱的机会”，而是：

能够被重复发现、验证和执行的套利 Edge。

笔记 067: 8月11日（周二）——、

作者：Cool-CoCo | 时间：2026-08-12 15:52 UTC | 字数：467 | 评分：48

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月11日（周二）——、

今天有点其他事情 处理时间比预计多了，没有留下太多时间空闲汲取新知，进度延后一天，明天补上。

交流群里最近又有好多新讯息、信息，也需要及时补一下、分析一下的。

看到有同学对经典的链上套利交易进行了案例，我想我也应该利用善于搜集研究的优势对此类案例进行浏览、观察，应该也能从中总结出一套自己的分析结论。先把“MEV交易法医”案例分析Flag立下。

我准备梳理案例中关于闪电贷借款、跨DEX兑换、还款、矿工付款和搜索者利润的完整资金路径，并理解其内部交易与ERC-20转账记录的区别。

可能的收获会体现出前几天的基础知识的学习扎实与否。比如 链上套利不仅是发现价差，还包括交易模拟、执行排序和区块空间竞价。搜索者也就是准备交易者通常需要提前计算好利润上限，并在看不到竞争者报价/交易池报价的情况下，决定愿意分给矿工或Builder多少“手续费”。报价越高，中标概率越大，但自己留下的收益也会越少。

希望通过这类案例的分析进一步加深对于MEV套利背后的市场结构和竞争机制等内容的理解。

笔记 068: Day 5 : Hermes与套利研究 workflow

作者：JWiang | 时间：2026-08-12 15:53 UTC | 字数：1569 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 : Hermes与套利研究 workflow

一、核心概念

套利研究的核心不是偶然发现一个价差，而是建立一个能够：持续发现、记录、验证、监控和复盘机会的流程。

二、套利研究的基本流程

发现机会 - 记录机会 - 验证资产 - 验证流动性 - 模拟执行 - 计算成本 - 计算净收益 - 判断是否存在Edge - 持续监控 - 复盘

三、Edge

定义：指在考虑交易成本、风险和执行条件之后，策略仍然能够产生正向预期收益的优势。

一次盈利： $\neq$ 存在稳定Edge。真正需要验证的是：大量样本+重复出现+成本调整+风险调整

四、Hermes的作用

Hermes可以作为套利研究工作流中的Agent，辅助：信息发现，寻找：

价格异常

交易量异常

稳定币偏离

跨链价差

研究拆解

将一个机会拆解成：

Asset

Chain

DEX

Pool

Size

Price

Liquidity

Gas

Bridge

Path

Cost

Net Profit

数据整理：将不同时间、不同链、不同资产的机会统一记录。

持续监控：按照设定条件重复查询。

告警：当满足条件时： $\text{Spread} \geq 0.5$ 、 $\text{Net Profit} > \$20$ 时，触发提醒。

五、Opportunity Record

每一个候选机会记录：

Timestamp

Chain

Asset

Token Contract

DEX

Pool

Trade Size

Price

Liquidity

Execution Price

Path

Gas

DEX Fee

Bridge Fee

Slippage

Price Impact

Net Profit

Execution Time

Status

Failure Reason

六、Price at Size

必须记录：\$1,000交易规模下：Execution Price = \$3,001；

\$100,000交易规模下：Execution Price = \$3,020

因为：套利机会的价值取决于“这个价格能够成交多少”。

七、Net Profit

基本模型：Net Profit = Gross Profit - DEX Fees - Gas - Bridge Fees - Slippage - Price Impact - Failure Cost - Capital Cost

最终应该研究：Net Profit / Capital。而不是单纯研究：Price Spread

八、Failure Reason分类

建立标准标签：

PRICE_MOVED

LOW_LIQUIDITY

HIGH_SLIPPAGE

HIGH_GAS

BRIDGE_DELAY

BRIDGE_LIQUIDITY

TX_FAILED

MEV_COMPETITION

WRONG_ASSET

INSUFFICIENT_CAPITAL

统计策略到底死在哪里？

九、重要认知

1：套利机会不是一个价格，而是一组条件。Price+ Liquidity + Size + Path + Cost + Time + Risk

2：屏幕上的价差只是研究的起点。

真正需要研究：Executable Spread也就是：在特定交易规模和具体路径下，实际能够获得的价差。

3：套利策略的价值来自重复性，而不是偶然性。

笔记 069: Day07 纸面模拟系统

作者：tutu | 时间：2026-08-12 15:55 UTC | 字数：660 | 评分：55

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day07 纸面模拟系统

今天把前面学过的价差、资金费和风控，落到了一套 10,000 USDT 的纸面账户里。单笔双腿合计保证金上限为 5,000 USDT，每条腿杠杆不超过 2 倍，总名义仓位不超过 10,000 USDT，最长持有 72 小时。

我把止损口径改成“按当前盘口退出双腿后的预计净亏损”，而不是只看页面浮亏。这个数字要扣手续费、滑点、资金费和其他待付成本。预计亏损达到 160 USDT 时冻结开仓和补仓，重新按盘口计算退出成本，并将双腿同步减仓 50%。达到 200 USDT 时执行 Kill Switch。

薄盘口是今天纠正的一个重点。流动性枯竭时，不能把大额市价单当作默认动作，因为市价单可能把滑点直接打穿止损。应使用可成交限价、IOC 或拆单，并尽量同步处理两条腿，先消灭裸露方向敞口。

我也把单笔记录的标准定下来了。未来复盘时，不依赖记忆也应该能看懂三件事：当时为什么做以及逻辑有没有被证伪，实际怎样成交和执行风控，最终盈亏是判断、执行还是随机性造成的。预期成交价、按盘口逐档计算的模拟 VWAP 和真实成交价必须分开，不能拿 K 线中间价假装成交。

今天最重要的理解是，“不套利是为了更好的套利”。低质量机会会占用资金容量、风险预算和注意力。主动放弃成本覆盖不住、深度不足或风险说不清的机会，真错价出现时才还有保证金、容错空间和执行精力。

明天进入期现套利，用这套账户规则做一笔“现货多

+

永续空”的完整模拟，把基差、资金费、手续费、双腿滑点和最大回撤放进同一张账里。

笔记 070: 今天继续学习 DEX-CEX

套利，主要思考了一个问题：拿到的两个价格是不是同一时刻的价格。

作者：zgos | 时间：2026-08-12 16:14 UTC | 字数：241 | 评分：53

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续学习 DEX-CEX 套利，主要思考了一个问题：拿到的两个价格是不是同一时刻的价格。

如果先查询 DEX 报价，过几秒再查询 Binance，市场价格可能已经发生变化。这样计算出来的价差看起来很大，但其实是两个不同时间点的数据，不能直接当成套利机会。

今天的收获是，记录价格时除了交易对和价格，还应该记录：

报价时间；

交易方向和数量；

DEX 的预计到账金额；

Binance 的买一价和卖一价；

Gas、手续费和滑点；

报价经过多久会失效。

笔记 071: 8 月 13 日：先把新渠道放进观察层

作者：xiaoquanyo | 时间：2026-08-12 16:24 UTC | 字数：1080 | 评分：74

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8 月 13 日：先把新渠道放进观察层

今天主要在扩充报价来源，但没有急着把新渠道接入自动交易，而是先建立一套独立的只读观察程序。这样既能比较价格，也不会因为新接口不稳定、报价口径不同或风控限制，影响正在运行的套利主程序。

观察一段时间后，新渠道确实出现过一次扣除 Gas 后仍为正的报价组合，收益空间也超过了当前的最低门槛。但这个信号持续时间很短，后续大部分报价重新计算后都接近持平或为负。单次出现优势只能说明它值得继续观察，还不能证明存在稳定的套利路径。

今天比较重要的收获是：接入更多渠道不等于直接增加套利次数，首先要解决报价是否可比较的问题。

不同渠道返回的报价可能存在以下差异：

有的只给参考价格，有的已经包含真实路由和滑点。

有的返回速度快，但构建交易时会重新询价。

买入和卖出报价的生成时间不同，组合起来可能形成一个实际上不存在的价差。

小金额存在利润，不代表放大金额后仍然成立，池子深度会改变最终结果。

某些接口在本地可以访问，但服务器出口 IP 会触发风控，不能算作稳定可用的渠道。

因此，今天给观察逻辑增加了即时复核：发现候选后，不直接采用扫描阶段的旧报价，而是重新获取买入和卖出两边的报价，并检查报价年龄。只有两边都足够新，扣除 Gas、滑点和闪电贷费用后仍有利润，才记录为有效候选。这样会减少表面上的“机会数量”，但留下来的信号更接近真实可成交结果。

目前执行层采用闪电贷完成套利：在同一笔交易中借入资金，完成买入和卖出，再归还本金及费用。交易结束前会检查最终余额，利润不足或任意一步失败，整笔交易都会回滚。这种方式能够避免长期占用本金，也能防止只完成买入、无法卖出的单边持仓风险。

不过闪电贷只能保证交易的原子性，并不能制造利润。它同样无法解决过期报价、流动性不足、价格冲击和抢跑等问题。真正需要满足的条件是：

text

卖出所得

闪电贷本金

+ 闪电贷费用

+ Gas

+ 交易费用

+ 滑点与安全缓冲

因此，发现价差之后仍要使用最新区块状态重新构建并模拟完整交易。只有模拟能够归还借款，并且剩余利润超过最低门槛，交易才有执行价值。

新渠道目前仍保持观察状态。接下来会继续统计每个资产、不同金额档位下的正收益次数、持续轮数和最大净收益。只有某条路径能够多次复现，并且报价、构建和模拟都稳定后，才考虑加入闪电贷执行层。

今天最大的感受是：套利监控真正需要优化的不是“发现更多价差”，而是更快地区分价格差异、过期报价和能够在一笔交易中完成的真实利润。

笔记 072: 稳定币与锚定资产套利 (Stablecoin / Peg Arbitrage)

作者：zym89861-wq | 时间：2026-08-12 16:30 UTC | 字数：343 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

稳定币与锚定资产套利 (Stablecoin / Peg Arbitrage)

原理：捕捉理论上应 $\$1:\1 锚定、但短期因市场恐慌或流动性倾斜而脱锚 (Depeg) 的资产。

典型场景：

稳定币脱锚：如 USDT/USDC/DAI 在 Curve 3pool 中比例失衡，导致某稳定币跌至 $\$0.985$ 。

LST/LRT 衍生品：如 stETH/ETH 或 eETH/ETH 出现折价。

套利路径：在公开市场折价买入（如 $\$0.98$ ），通过协议官方的赎回机制 (Redemption Mechanics) 按 $\$1:\1 提回底层资产，或等待市场流动性自动修复（套利周期较长，需承担资金沉淀成本）。

笔记 073: 今天学到的东西 (方法论层面)

作者：bxynhs-del | 时间：2026-08-12 18:02 UTC | 字数：947 | 评分：60

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天学到的东西 (方法论层面)

1. MVP 不是“缩小版的完美产品”

错误理解：

MVP = 完美产品的 0.5 版

所有指标都要达到“及格线”才能往前走

正确理解：

MVP = 验证核心假设的最小实现

只验证“能不能做”，不验证“做得好不好”

具体到这个项目：

核心假设：Solana 上存在跨池套利机会，且我能检测到

MVP 目标：构造一笔交易，simulate pass，证明“交易结构正确”

不是：第一天就要达到 J7GR6X 的 5% 成功率

2. 门槛设置要分“生死线”和“优化线”

实施方案的问题：

把“命中率 $\geq 50\%$ ”设为阶段 0 的生死线

但这其实是“优化指标”，不是“可行性指标”

重新分类：

生死线（不过就是系统不可行）：

WebSocket 能稳定监听池子

能解析 J7GR6X 的交易结构

能构造一笔完整交易并 simulate pass

优化线（不过说明还不够好，但不代表不可行）：

命中率 $\geq 50\%$

simulate 通过率 $> 30\%$

成功率 $> 2\%$

关键差异：

生死线决定“要不要继续”

优化线决定“什么时候放大资金”

3. “对标”是陷阱

实施方案写的：

“不需要知道正确答案，只需要知道谁做对了，然后对齐。”

听起来很合理，但有个隐藏前提：

假设你和他同一个起跑线

J7GR6X 有自研链上程序、durable nonce、多版本并发

你什么都没有，就一个 WebSocket 监听器

让你对齐他，等于让自行车对齐高铁

正确的做法：

阶段 0：不对标任何人，只验证“机会存在”

阶段 1：simulate pass，验证“能构造交易”

阶段 2：小额实盘（0.05 SOL/笔），验证“能赚到钱”

阶段 3 再去对标 J7GR6X（此时你已经有数据，知道差距在哪）

笔记 074: Day 9（08/13）学习总结，素材来自今日同学精华：

作者：cjdaocoin | 时间：2026-08-13 01:01 UTC | 字数：409 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9（08/13）学习总结，素材来自今日同学精华：

1. Web3Rason：拆解某清算机器人的四步（閃電貸 → liquidationCall → RFQ 精確換回還款資產 → 還貸）——
 2. xiaoquanyo：今天最大的感受是：套利监控真正需要优化的不是“发现更多价差”，而是更快地区分价格差异、过期报价和能够在的一笔交易中完成的真实利润
 3. zgos：今天继续学习 DEX-CEX 套利，主要思考了一个问题：拿到的两个价格是不是同一时刻的价格
 4. Courtney Snyder：每个候选先请求买入链 USDC → WETH 和卖出链 WETH → USDC 两条现货报价，计算保守 pre_rebalance_edge_bps；只有接近历史完整成本门槛时，才追加 WETH/USDC 两条库存再平衡报价
- 以上为今日从同学打卡中学到的知识点，继续积累执行与验证能力。

笔记 075: 加密市场里的“期货/合约套利”，本质上是：

作者：hellingercheri-gif | 时间：2026-08-13 01:17 UTC | 字数：4035 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

加密市场里的“期货/合约套利”，本质上是：

同时建立方向相反、价格相关的仓位，尽量消除币价涨跌风险，赚取基差收敛、资金费率或不同合约之间的相对价差。

它与链上原子套利最大的区别是：两条腿通常不能原子成交，而且仓位需要持续持有。

因此即使最终理论上盈利，也可能中途因保证金不足被强平。

一、现货—交割合约基差套利

这是最接近传统“无方向套利”的方法，也叫 cash-and-carry。

假设：

BTC 现货：100,000 USDT

90 天交割合约：103,000 USDT

合约处于升水，也就是期货高于现货

操作：

1. 买入 1 BTC 现货，花费 100,000 USDT。
2. 同时做空 1 BTC 的 90 天交割合约，价格 103,000。
3. 持有到交割，期货价格向现货收敛。

到期时无论 BTC 涨跌，理论结果都是：

$$[\text{总毛利润} = \text{期货开仓价} - \text{现货买入价}]$$

也就是 3,000 USDT。

如果到期 BTC 跌到 80,000：

现货亏 20,000

期货空单赚 23,000

合计赚 3,000

如果到期 BTC 涨到 130,000：

现货赚 30,000

期货空单亏 27,000

合计仍赚 3,000

90 天毛收益率：

$\lfloor 3,000 / 100,000 = 3\% \rfloor$

简单年化约：

$\lfloor 3\% \times 365 / 90 \approx 12.17\% \rfloor$

实际净收益还要扣除：

现货和合约开平仓手续费

买卖价差与滑点

资金占用成本

借款利息

提币、托管成本

合约交割规则导致的误差

交割合约越接近到期，理论上越应向现货收敛；否则持续存在可套利价差。CME：期货基差与到期收敛

(<https://www.cmegroup.com/education/courses/introduction-to-fx/importance-of-fx-futures-pricing-and-basis>)

最大风险：中途爆仓

假设你买了现货并做空合约，后来市场大涨：

现货在赚钱；

合约账户却产生大量浮亏；

如果现货和合约资金分属不同账户，现货盈利未必能自动补充合约保证金。

所以可能出现：

最终到期本应赚 3,000，但在到期前空单先被强平。

“最终方向中性”不等于“中间不需要现金”。这是合约套利最容易被忽视的问题。

二、反向基差套利

如果交割合约低于现货，例如：

BTC 现货：100,000

90 天期货：97,000

理论操作是：

1. 借入 BTC 并卖出现货；
2. 买入相同数量的期货；
3. 到期用期货收益或结算头寸覆盖现货空头。

毛价差约为 3,000 USDT。

但反向套利通常比正向套利困难，因为涉及：

借币成本；

可借数量不足；

借币利率突然升高；

借贷方要求提前归还；

现货做空权限与强平风险。

因此负基差不代表一定有可执行利润。

三、永续合约资金费率套利

永续合约没有到期日，所以交易所使用资金费率让永续价格靠近现货。

通常：

资金费率为正：多头向空头付款；

资金费率为负：空头向多头付款。

不同平台结算间隔可能是 1、2、4 或 8 小时，规则也可能动态调整。OKX

(<https://www.okx.com/en-gb/help/funding-fee-mechanism-eea>)

正资金费率套利

当资金费率持续为正时：

资金费率机制

1. 买入 1 BTC 现货；
2. 做空价值 1 BTC 的永续合约；
3. 现货和空单抵消方向风险；
4. 空单定期收取资金费。

假设：

对冲名义价值：100,000 USDT

每 8 小时资金费率：+0.01%

一天结算三次

理论每日资金费收入：

$[100,000 \times 0.01\% \times 3 = 30 \text{ USDT}]$

简单年化：

$[0.01\% \times 3 \times 365 = 10.95\%]$

但这个 10.95% 只是把当前费率机械年化，并不代表未来收益，因为资金费率会不断变化，甚至转负。

完整收益公式

假设永续价格为 (P)，现货价格为 (S)，做多现货、做空永续：

$[\text{净收益} = \text{初始基差} - \text{退出基差} + \text{累计资金费} - \text{全部交易成本}]$

也就是：

$[(P_0 - S_0) - (P_1 - S_1) + \text{Funding-Cost}]$

因此资金费率套利不只是“收 funding”：

如果退出时永续升水扩大，基差亏损可能吃掉资金费；

如果升水缩小，还可以额外赚取基差收敛；

如果长期持仓，资金费率可能从正数变成负数。

所以它更准确地说是市场中性 carry trade，不是完全锁定的无风险套利。

四、负资金费率套利

资金费率明显为负时，理论上反向操作：

1. 做多永续合约，收取空头支付的资金费；
2. 做空现货或用其他合约建立空头对冲。

难点仍然是做空现货：

需要借币；

借币利率可能高于资金费收入；

某些小币借不到；

做空现货也可能被强平。

另一种做法是在交易所 A 做多负 funding 的永续，在交易所 B 做空价格相近的永续。但此时引入了跨交易所风险。

五、跨期套利

也叫日历价差套利，交易同一资产、不同到期月份的合约。

例如：

9 月 BTC 期货：102,000

12 月 BTC 期货：108,000

如果认为远月升水过高，可以：

买入 9 月合约；

卖出 12 月合约。

你主要关心的不是 BTC 涨跌，而是：

$[12 \text{月合约价格} - 9 \text{月合约价格}]$

是否缩小。

CME 将这种“买入一个月份、卖出另一个月份”的操作定义为 intramarket/calendar spread。CME 期货价差介绍 (<https://www.cmegroup.com/education/courses/understanding-futures-spreads/futures-spread-overview>)

但跨期价差不一定会在你需要的时间内收敛。远月可能继续升水，因此它更接近相对价值交易，而不是锁死利润的套利。

六、跨交易所合约套利

例如同时刻：

交易所 A 的 BTC 永续：101,000

交易所 B 的 BTC 永续：100,500

可以考虑：

在 A 做空；

在 B 做多；

等两个市场价差缩小后同时平仓。

也可以根据资金费率差异操作：

在高正资金费率平台做空；

在低资金费率或负资金费率平台做多。

但这类策略风险很实在：

两边不能同时成交；

一边下单成功、另一边失败；

两个平台标记价格不同；

保证金不能即时跨平台调拨；

某个平台暂停提币、宕机或限制账户；

资金费率和价差同时反向变化；

交易所存在信用风险。

因此跨所套利通常需要提前在两边分别放置资金，而不是发现机会后再转账。

七、几种策略的“锁定程度”

| 策略 | 收益来源 | 锁定程度 |

| :----- | :----- | :----- |

| 现货 + 做空交割合约 | 到期基差收敛 | 最高 |

| 反向基差套利 | 负基差收敛 | 较高，但借币困难 |

| 现货 + 做空永续 | 资金费 + 基差变化 | 中等 |

| 跨期套利 | 期限结构变化 | 较低 |

| 跨交易所永续套利 | 价差和资金费差 | 较低 |

| 单纯抢结算前 funding | 一次资金费 | 很低，容易被价格变化反噬 |

最适合用来理解合约套利的是第一种：买现货、空交割合约、持有到期。

八、实际做系统时最重要的计算

不能只看页面显示的“价差”或“资金费年化”，而应计算：

$$[\text{预期净利润} = \text{基差收益} + \text{预期资金费} + \text{现货持有收益} - \text{四次交易手续费} - \text{滑点} - \text{借币成本} - \text{资金成本} - \text{预留风险成本}]$$

其中“四次交易”是：

1. 开现货；

2. 开合约；

3. 平现货；

4. 平合约。

还要统一比较：

USDT 本位还是币本位；

线性还是反向合约；

合约面值和乘数；

标记价格还是成交价格；

资金费率结算周期；

到期结算采用哪个指数；

两边真实可成交深度。

九、与原子套利的本质区别

原子套利：

同一笔交易内全部成交，否则回滚

合约套利：

建立两条腿

→ 长期持仓

→ 持续管理保证金

→ 收取资金费或等待基差收敛

→ 分别平仓

所以合约套利的核心竞争力不是智能合约原子性，而是：

更低的交易费率；
更好的双腿成交能力；
更便宜的资金和借币；
更合理的保证金管理；
更准确地预测资金费和基差；
更可靠的多交易所基础设施。

一句话概括：

交割合约套利主要赚“确定性较强的到期收敛”；永续合约套利主要赚“不断变化的持仓补偿”。前者更像锁利，后者更像动态利差交易。

笔记 076: 两次都读取 latest，不保证两个池来自同一个区块；应先固定 chain ID、区块号和区块哈希。

作者：Montaire | 时间：2026-08-13 01:37 UTC | 字数：278 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

两次都读取 latest，不保证两个池来自同一个区块；应先固定 chain ID、区块号和区块哈希。

同一区块号出现不同哈希时，可能发生链重组，应作废旧结果并重新读取全部池。

reserve0/reserve1 必须按合约的 token0/token1 对应，不能按页面上的交易对名称猜顺序。

原始整数必须按各自 decimals 换算；地址、顺序或 decimals 缺失时不能继续计算。

任一必读字段失败，就终止整次机会计算、不报警、不拿旧缓存补位。

同块重复结果一致，只证明该历史快照可复现，不证明当前仍存在套利。

笔记 077: 今天把套利机会评估流程固化成可复用的 Hermes 技能「arbitrage-opportunity

作者：wuyuandao | 时间：2026-08-13 02:16 UTC | 字数：735 | 评分：74

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天把套利机会评估流程固化成可复用的 Hermes 技能「arbitrage-opportunity-evaluation」，包含四步 SOP：核来源→补成本→查流动性→给判断。这是 Bruce 说的「训练专属套利 Agent」——把判断逻辑写成技能，而非写代码。然后用同一个 DOS 代币实测了这套 Agent：第一步「核来源」就戳破了两个幻觉——① DEX 显示 12.31% 价差其实是 TVL 仅 \$9 的死池幻觉，真实可成交价差仅 0.93%；② 核实 Bitget 真实订单簿后，发现 CEX 最新价 \$0.3145 与 DEX \$0.3157 几乎一致，之前以为的 10% 价差根本不存在。订单簿前 10 档累计仅 \$8000，深度极薄。按真实 0.35% 价差重算，扣除交易费/提币费/协议费后净亏 \$2.50。核心收获：口述的「10% 价差」经 Agent 核实只剩 0.35%——这正是「核来源」这一步的价值。屏幕上看见价差只是线索，不是机会。套利 Agent 现在能自动完成：核来源戳破幻觉 → 补成本算净收益 → 查流动性看深度 → 给三档判断（可试/观察/放弃）。另外迭代了 SOP：补充「按套利类型选数据源」——有桥才用 LI.FI，单链 CEX↔DEX 套利用 DexScreener+CEX API 即可，跨链套利才需要 LI.FI。这是从「为什么没用 LI.FI」的追问中发现的 SOP 漏洞，说明技能需要在使用中持续迭代。困难：Bitget 订单簿深度极薄（\$8K），CEX 侧实际可成交量和挂单价差差异大。编程零基础靠 Hermes 写脚本。下一步：Day 10 数据采集脚本——多链多资产报价自动采集，保存历史数据用于回测。

笔记 078: D11 (8/13) 案例对比分析 + 清算监控实操

作者：peanutwen00 | 时间：2026-08-13 02:26 UTC | 字数：643 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D11 (8/13) 案例对比分析 + 清算监控实操

- 1) 三维框架（策略×基础设施×运气窗口）对比三大赛道：A 大额闪电贷套利（毫秒级窗口+军备竞赛）/ B 三明治（最卷）/ C 清算长尾（分钟~天级窗口+长尾真空）
- 2) 核心理念：价差“可见性≠可捕获性”——所有人看到同一个价差，可捕获性零和；0x8d64 付 BuilderNet 买的是“比别人快”
- 3) 大机器人“看不上”长尾清算（机会成本：同样资源大笔赚 \$5,000 vs 小笔赚 \$50）→ 散户的缝在小仓位/小链/冷门资产
- 4) 实操：Morpho GraphQL API 扫描 Base 60 市场（marketCollateralAtRisk）→ 0 风险；Aave V3 Base 最近 9,000 块

LiquidationCall 0 个 (Borrow 163 条验证数据流通) ——市场平静时空手而归是常态

5) 踩坑：Aave Pool 地址记错一位 (getCode 验证为空才发现, 已修正)；DefiLlama 页面被 Cloudflare 拦；The Graph 免费层需 key；Base RPC getLogs 限 10,000 块

6) 产出：morpho_monitor.py 监控脚本 + API 工具清单笔记 (Morpho/The Graph/DefiLlama/LI.FI/RPC 全景)

7) 下一步：D12 Hermes 套利 workflow 设计 (把"监控→算账→判断"流程化)

笔记 079: Day 8 打卡 | 鏈上套利殘酷共學

作者：roy328line | 时间：2026-08-13 06:15 UTC | 字数：1204 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 8 打卡 | 鏈上套利殘酷共學

今日實作：p_win sigmoid 校準

資料來源 (無需 Dune/API key)：

- 公開 RPC 掃 200 個最新 block
- Transfer event → WETH 淨流入 = gross_profit 下限
- bribe = priority_fee x gas_used (不含 coinbase.transfer, 是下限)
- 有效樣本：356 筆

校準結果 (按規模分桶)：

- S 桶 (< 0.003 ETH gross)：n=211, k=3.228, midpoint=0.353, 可信度：下限
- M 桶 (0.003-0.015 ETH)：n=17, k=50.0, midpoint=0.016, FIT_FAILED
- L 桶 (0.015-0.06 ETH)：n=50, k=50.0, midpoint=0.012, FIT_FAILED
- XL 桶 (> 0.06 ETH)：n=78, k=50.0, midpoint=0.001, FIT_FAILED

M/L/XL 桶：大額 bot 用合約內 coinbase.transfer 付 bribe, priority fee 約等於 0, 量到的只是雜訊。

校準對 best_ev 的影響 (同一組池, gross=\$11.73, gas=\$3)：

- 舊猜測 k=5.65, mid=0.95, venue=public：r=1.000, EV=-\$1.93 (no-go)
- S桶下限 k=3.228, mid=0.353, venue=public：r=0.621, EV=+\$0.46 (go) ← 翻轉！
- 中間估計 k=4.0, mid=0.60, venue=public：r=0.819, EV=-\$0.46 (no-go)

midpoint 在 [0.353, 0.95] 之間, 決策在區間內翻轉。

結論：EV 絕對值不可用於決策, 只可用於同類別內的相對排序。

今日最重要的一件事

大額 bot 用 coinbase.transfer 付 bribe, priority fee 約等於 0。

只看 priority fee 等於只看到「基礎設施弱的小額 bot」, 競爭看起來比實際容易。

正確 bribe 方向：Flashbots bundle + coinbase.transfer, 不是 priority fee 競標。

Day 9 方向：放棄繼續校準, 回到 Leg 抽象

midpoint 在 [0.35, 0.95], 再花兩天也收不緊, 邊際報酬接近零

改用 bundle venue (f_cost=0), go 條件簡化為 net_raw > gas_cost, 不再依賴 sigmoid

先用 0 bribe 確認機會真的存在過, 再考慮最優 bribe 策略

笔记 080: 今天研究期权波动率曲面的静态套利。重点不是判断隐含波动率看起来高或低，而是检查同一批期权价格是否违反

作者：dyhbrewer | 时间：2026-08-13 07:11 UTC | 字数：1142 | 评分：60

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天研究期权波动率曲面的静态套利。重点不是判断隐含波动率看起来高或低，而是检查同一批期权价格是否违反无套利约束。

一、执行价单调性

相同标的与到期日下, 执行价越高, Call 价格理论上不应越高：

$K_1 < K_2$, 则 $\text{Call}(K_1) \geq \text{Call}(K_2)$ 。

Put 则方向相反。若按实际 Bid/Ask 建仓、扣费后仍违反, 才可能通过垂直价差锁定收益。

二、Butterfly 凸性

对同一到期日的三个执行价 $K_1 < K_2 < K_3$, 等距时构造：

买入 K_1 Call、卖出两份 K_2 Call、买入 K_3 Call。

它的到期收益在所有标的价格下都应大于等于零, 因此可成交建仓成本不应为负。如果用

Ask

买入两翼、用

Bid

卖出中间两份后，组合还能收到超过全部费用的净权利金，才可能属于静态套利。

非等距执行价不能简单使用 1:-2:1，需要按执行价间距调整权重，使组合保持凸性。验证器也不能只抽查几个价格点，而应按执行价分段求出整个到期收益函数的全局最小值。

三、Calendar Spread

相同执行价的长期期权通常不应简单地比短期期权便宜，但加密期权中 Calendar 更容易被误判。不同期限具有不同 Forward Price，同一执行价不等于同一 Moneyness；短期腿到期后，长期腿仍暴露于 Delta、Vega 和波动率曲面变化。因此 Calendar 多数是带模型风险的相对价值交易，不应直接标记为严格套利。

四、完整判定标准

雷达必须使用可成交 Bid/Ask 和真实深度，而不是 Mark Price 或 Mid Price。压力净收益应计算为：

最小到期收益 - 建仓净支出 - 交易费 - 交割费 - 滑点 - 保证金成本 - 多腿失败准备金。

还要排除这些假机会：

1. 混用不同到期日、结算币或合约面值；
2. 多条腿最优报价无法同时成交；
3. 忽略卖出期权的峰值保证金；
4. 币本位和 USDC 线性期权直接比较；
5. 盘口陈旧、深度不足或报价规模不一致；
6. 组合虽有正的理论价值，但扣费后为负。

下一步准备设计只读期权无套利验证器：按资产、结算币种和到期日分组，扫描 Call/Put 单调性、Butterfly 凸性、Put-Call Parity 和 Box Spread；记录最大可成交规模、全局最坏到期收益、Return on Margin、连续存在时间及主要阻塞原因。Calendar 只列为研究信号，不列为严格套利。

今天的结论：波动率曲面不平滑不等于套利。只有组合在所有到期价格下都不存在负现金流，而且以真实盘口扣除多腿成本后仍为正，才属于严格静态套利；其余只是带模型和执行风险的相对价值交易。

笔记 081: markdown

作者：darven | 时间：2026-08-13 08:33 UTC | 字数：2123 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

markdown

f(x) Protocol：Stability Pool 與代幣經濟

今天的重點

今天是「掃描 f(x) Protocol」的正式開場。這是 AladdinDAO 做的協議，核心是把生息資產 (stETH/WBTC) 拆成「零波動穩定幣 fxUSD + 高波動槓桿代幣 xPOSITION/sPOSITION」。本來以為今天會一路掃完整個白皮書，結果一天下來，最深的是把 Stability Pool (穩定池) 這塊跟 Liquity 的源頭做了一整輪對比，最後收在 FXN 代幣經濟。

我學到了什麼

1. 抓 GitBook 文件的坑。f(x) 官方文件在 GitBook，有 .md 全文端點可以一次抓 72 頁。結果快速連抓會被防爬——回傳 HTTP 200 但內容是假的「Page Not Found」，第一次單抓又正常，超迷惑。我交叉試了 execute_code sandbox (全被擋)、背景 process (網路被封)、最後才發現終端前台 curl + 每頁間隔幾秒 + 偵測 404 重試才穩。這坑我直接寫進 `whitepaper-scanning skill` 了，下次不再踩。

2. Stability Pool 的定位，三家完全不同。這是我今天最核心的認知：

- Liquity V1：SP 是純清算吸收器——銷毀 LUSD、換來被清算 Trove 的 ETH。不主動買賣 LUSD 調 peg。

- Liquity V2：演化成每個抵押品一個池，收該市場 75% 的借款利息 + 清算 gain (無代幣排放)，這才是「Earn」賣點。但還是不碰價格調節，錨定靠獨立的 redemption。

- f(x)：SP 是主動 peg keeper——用 Chainlink 定價，0.3% 閾值內自動買賣 fxUSD/USDC 套利，官方明說這是協議主要收入來源。

所以「SP 做不做價格調節」是關鍵分水嶺，只有 f(x) 主動做。Liquity 兩代都把它丟給 redemption。

3. Liquity 的真正競爭力是純去中心化，不是收益。我起初傾向「f(x) SP 更優」，但其實 f(x) 的優勢是用引入 USDC 依賴換來的，而 USDC 正是 Liquity 極力避免的 (Liquity 只收 ETH/wstETH/rETH、無 RWA、無中心化託管)。對「非基本教義派」用戶，f(x) SP 收益更高更受歡迎；但重視去中心化純度的人會選 Liquity。這是定位之爭，不是功能之爭。

4. f(x) 引入 USDC 依賴多了一層風險。SP 套利是官方合約內部原子執行，外部根本沒有介入空間，所以真正的風險是在 oracle 一層；就算 USDC 脫錨停掉主動 peg 維護，Brake 和 Redemption 還在跑，等同退回 Liquity 式功能。

5. fxMINT「0% 利率」是行銷話術。實質是用「抵押品收益」換「免息」——抵押 wstETH，質押收益歸 Stability Pool。而且使用者的關鍵洞察更狠：失去的是整個抵押品的收益 (3%)，但借款面額只有抵押品 60%，換算成借款利率其實是 $\$90 / \$1800 = 5\%$ /年——比在 Aave 借還貴。fxMINT 只有對 WBTC (無質押收益) + 超長期才是真便宜。

6. FXN 是標準的 ve 經濟學。49% 流動性激勵、50 年攤完，但靠 veFXN 把持有者和協議 75% 收入分潤綁起來。跟 Curve 的最大區別是 rollover：沒人領走的 FXN 加回池子，不讓有 boost 的人搶走——這對散戶農夫比較友善。

我做了什麼

掃完 f(x) 官方 GitBook 全部 72 頁正文，落盤成 `research/Topics/fx-protocol/references/fx-protocol-spec.md` (297KB)。

子代理濃縮成機制摘要 `fx-protocol-mechanism.md`。

建 Liquity V1 / V2 兩個 research topic (Stability Pool 的源頭)，各抓完官方 docs，確立對比參照系。

跟 agent 一輪深談：SP 跨協議對比 (已總結存檔)、fxMINT 真實成本 (已存決策筆記)、FXN 代幣經濟。

把 GitBook 防爬解法寫進 `whitepaper-scanning skill`。

收斂

今天最大的收穫不是「f(x) 比 Liquity 好」，而是學會看的軸：SP 做不做價格調節、中心化 vs 純去中心化的取捨、隱藏機會成本怎麼換算成真實借款利率。規則的形狀，決定了玩家的行為——而選擇站在哪個「形狀」裡，比在形狀裡怎麼玩更重要。

笔记 082: 原子套利:一笔交易里买完就卖

作者：starkxun | 时间：2026-08-13 08:45 UTC | 字数：3405 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

原子套利:一笔交易里买完就卖

起因:共学群分享的 Solana 地址 J7GR6XoJ...Tx6Nf

(<https://solscan.io/account/J7GR6XoJfCPwZumW9xZ1nbWkyaq1oYSQuSVmCdBTx6Nf>)\

分析日期:2026-08-06 | 数据来源:Solana 公共 RPC 直读(非二手说法)

先给结论

那个地址是一个正在运行的原子套利机器人。61.7 小时里打了 6000 笔交易，\ 其中 5656 笔失败(94.3%),靠剩下的 344 笔赚钱。

成功率 5.73%

频率 $\approx$ 97 笔 / 小时

10 笔成功交易合计 +0.094 SOL

94% 的失败率不是问题,而是这个策略的设计前提。下面解释为什么。

一、「原子」是什么意思

对比你已经测过的跨链搬砖

跨链搬砖(你 Week 1-2 测的):

① 在 A 链买 $\rightarrow$ ② 等桥(7 ~ 1080 秒) $\rightarrow$ ③ 在 B 链卖

中间有真实的时间间隔。价差可能跑掉,资金可能卡在半路。

原子套利:

[① 买 + ② 卖] 打包进同一笔交易

$\rightarrow$ 成功:两步都成,赚差价

$\rightarrow$ 失败:整笔回滚,像没发生过

「原子」(atomic)就是不可分割——没有“买完了但还没卖”这种中间状态。

这带来的最大改变:失败成本 $\approx$ 0

回顾你的成本模型:

净收益 = 屏幕价差 - 桥费 - 滑点 - gas - 延迟风险

失败交易成本(失败概率 $\times$ 已花 gas + 占用资金) $\leftarrow$ 这一项
资金占用

原子套利把失败交易成本压到只剩手续费。资金没有被占用过,\n因为那笔交易根本没执行成功。

所以它能承受 94% 的失败率。试错是免费的,那就疯狂试 ——\n每小时 97 笔就是这么来的。

二、拆一笔真实交易

签名 5rpAyf7XYLVZSaiSEEipPbmw4hjMRqr...,一笔交易里同时做了两件事:

① 从池子 Fex8CSk 买入: 付出 5.6259 WSOL → 得到 81,172 个 memecoin

② 卖给池子 2TD1fMP: 付出 81,154 个币 → 得到 5.6887 WSOL

差价 +0.0628 WSOL,外加剩下 18.25 个币、

同一个币在两个池子里价格不一样,低买高卖,一笔搞定。

它的技术构造

签名钱包 J7GR6Xo...(普通钱包,持有 46.4 SOL)

└─ 调用 BaMpent4Np6pYZ7JLDmtZNFJ8o3VBnHdy6A9mGDyZRW8 ← 它自己部署的程序

└─ CPI → LBUZKhRxPF3XUpBCjp4YzTKgLccjZhTSDM9YuVaPwxo ← Meteora DLMM

└─ CPI → pAMMBa...(Pump AMM)等其它 DEX

注意中间那层:它部署了自己的 Solana 程序。因为 Solana 的一笔交易里\n要把多个 DEX 的 swap 串起来、并且让第二步用上第一步的结果、\n需要一个合约来编排。(这一点在 Sui 上不一样,见第五节。)

失败长什么样

AnchorError caused by account: bin_array

Error Code: AccountNotEnoughKeys. Error Number: 3005

Meteora DLMM 是"分箱"做市 —— 流动性按价格区间(bin)存放。 \

机器人发交易前要先算好会用到哪些 bin,把这些账户列进交易里。

交易上链时价格已经走出那个范围 → 需要的 bin 账户没列进来 → 整笔失败。

这就是 94% 失败的主因:预测账户时的价格,和上链时的价格不一样了。

三、我在分析过程中读错了一次

第一遍我只看原生 SOL 余额变化:

交易 原生SOL净变化

5rpAyf7XYL -0.014629 ← 看起来在亏钱

2PrEykhmoQ -0.013635 ← 也在亏

10 笔合计: -0.040895 SOL ← "这机器人一直在亏"

结论是反的。因为:

利润以 WSOL(包装 SOL)到账 —— 那是个 token,不算在原生 SOL 余额里

小费用原生 SOL 付 —— 所以原生余额在减少

两个一起算才对:

交易 原生SOL WSOL 合计 小费

5rpAyf7XYL -0.014629 +0.031686 +0.017057 0.014624

2hNaDLCWja -0.000603 +0.030704 +0.030101 0.000000

4qQQhnPoaM -0.001071 +0.017480 +0.016409 0.001000

2PrEykhmoQ -0.013635 +0.025645 +0.012010 0.013630

...

10 笔合计: +0.094360 SOL

这是铁律 1 的翻版:只认一种计价口径,就会得出符号相反的结论。 \

你 Day-0 栽在"美元 vs token",我这次栽在"原生 SOL vs WSOL"。 \

同一个坑,换了张皮。
通用判据:分析任何链上账户前,先问"钱可能以几种形式存在"。\
原生币、包装币、各种 token —— 漏掉一种,结论就可能翻。
另外注意那个小费:0.0146 SOL 是付给验证者/区块构建者买优先权的。\
在总利润 0.0317 里占了 46% —— 速度是明码标价的。
四、和你测过的东西对比

 	跨链搬砖(你测的)	原子套利(这个)
失败成本	高 —— 桥费已付、资金在途	≈ 0 —— 交易回滚
能承受的失败率	几乎不能失败	94% 都在失败
延迟	7 ~ 1080 秒	同一区块内
门槛	25 ~ 60 bps	覆盖手续费 + 小费即可
谁在竞争	慢,人也能参与	纯拼速度,机器人战场
需要的工程	会调 API 就行	部署合约 + 账户预测 + 延迟优化
最本质的差异是"失败成本"。这一项从"高"变成" ≈ 0 ",整个策略的形态就变了:\
从"必须算准才敢做",变成"算不准就多试几次"。

笔记 083: 2026-08-13 學習打卡

作者: cht+ | 时间: 2026-08-13 09:19 UTC | 字数: 729 | 评分: 51
来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-13 學習打卡

今天完成了什麼

完成跨市場套利監控的韌性修正與驗證:行情斷線、資料庫失效連線或通知失敗時,不再讓整體掃描直接中止;未送達的事件也不會被誤標成已通知,保留後續重試機會。

建立「研究候選通知」與「正式套利訊號」的隔離機制。研究模式具備期限、通知額度與獨立狀態;正式模式維持原本的嚴格門檻與 Shadow Mode,不會自動交易。

完成審查後的三項補強:研究模式到期自動停止、通知額度採互斥保護避免競態,以及正式/研究模式的設定與狀態完全分離。

證據在哪裡

今日重新執行完整驗證:87 項單元測試與 2 項公開行情 Live Smoke 全數通過,靜態檢查與變更格式檢查亦通過。

正式監控服務目前持續運作,重啟次數為 0;限時研究驗證程序已停止。

已確認正式模式仍使用原定門檻,研究設定不會滲入正式掃描;程式工作區亦維持乾淨狀態。

尚未完成/下一步

持續跨越完整美股交易時段觀察服務,累積候選漏斗、行情重連與通知失敗統計,確認韌性修正在長時間運作下仍成立。

研究通知預設保持關閉;若再次驗證,只採有期限、有額度且明確標示為研究用途的方式,不降低行情新鮮度、時間對齊、商品映射與深度檢查。

累積更多有效樣本後,再估計正式門檻下的合理通知頻率,避免用短期零通知誤判系統故障。

今日學習心得

今天更確定「路由可達」不等於「完整通知機制正常」。可靠驗收必須拆開觀察:單次評估合格、通過持續性門檻、通知呼叫、實際送達與人工確認。另一方面,為了測試而降低收益門檻可以接受,但資料品質與風控閘門不能一起拆掉;研究訊號也必須和正式訊號在生命週期、額度與狀態上徹底隔離。

笔记 084: 期现套利要算完的几笔账

作者: tutu | 时间: 2026-08-13 10:38 UTC | 字数: 3393 | 评分: 74
来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

期现套利要算完的几笔账

期现套利刚看明白的时候,很容易产生一种错觉。现货买一份,永续空一份,两边数量配平,涨跌互相抵消,接下来等着收基差和资金费就行了。

这套结构确实成立。麻烦也恰好藏在这里。两条腿把BTC的共同涨跌抵消了，基差仍然会动，资金费也会变。永续账户的保证金、盘口深度、手续费、滑点，任何一项处理不好，都能让一笔看起来稳稳当当的套利提前散架。

今天把几组数字完整算下来，我对期现套利有了一个很朴素的认识。会搭两条腿只是起点，一套仓位能在坏行情里活着退出，才值得开进去。

两条腿到底赚什么钱

先看最常见的情况。

BTC现货价格是100000 USDT，永续价格是101000 USDT。永续比现货贵1000 USDT。买入0.033 BTC现货，同时做空0.033 BTC永续，两边数量相等。

后来现货和永续都走到100500 USDT。现货赚了16.5 USDT，永续空单也赚了16.5 USDT，合计赚33 USDT。

这33 USDT来自开仓时锁住的价差。开仓基差是1000 USDT，平仓基差归零，1000乘以0.033，刚好是33。

把公式写下来很简单。

\$\$

基差 = 永续价格 - 现货价格

\$\$

\$\$

基差盈亏 = (开仓基差 - 平仓基差) \times 数量

\$\$

对现货多、永续空这套组合来说，基差往下走会赚钱，基差往上走会亏钱。

另一份收入来自资金费。正资金费期间，永续多头向空头付款。假设空单名义价值约3333

USDT，每8小时收到0.01%，持有三天结算九次，大约能收3 USDT。

这里得把一句口头说法改准。资金费不能简单叫过夜费。它按照交易所规定的周期结算，方向和数值都会变化。开仓时空头收钱，几小时后也可能轮到空头付钱。

基差收益和资金费也不能混成一笔糊涂账。资金费有明确的直接付款方。基差收益来自两种价格之间的错位逐渐缩小，无法固定指给某一个现货空头。卖出现货给我的人可能只是在减仓，成交结束以后，他未必还承担任何风险。

BTC跌了，组合未必亏

同一套仓位开好以后，现货从100000跌到96000，永续从101000跌到98880。

现货多单亏132 USDT。永续空单赚69.96 USDT。双腿合起来亏62.04 USDT。

乍看之下，亏损像是BTC下跌造成的。换一个数字就能把原因看清。

假如现货跌到96000以后，永续仍然保持开仓时的1000 USDT溢价，永续价格应该是97000。这样现货亏132 USDT，永续空单赚132 USDT，组合毛盈亏刚好为零。

实际永续价格是98880，基差已经从1000扩大到2880。扩大部分是1880 USDT，乘以0.033 BTC，正好得到62.04 USDT。

这笔亏损来自基差扩大。BTC的共同下跌已经被两条腿互相抵消。

这也是期现套利里最容易漏掉的一层。双腿数量相等，只能把方向敞口压低。基差仍然是一项独立风险。BTC可以大跌，组合一分钱不亏。BTC也可以待在原地，永续溢价突然拉大，组合照样出现明显回撤。

页面浮亏不能拿来做风控

上面的62.04 USDT还只是毛亏损。

假设开仓已经花掉6 USDT，期间收到1 USDT资金费，现在平掉双腿还需要5 USDT手续费和滑点，那么当前可执行的退出净盈亏是负72.04 USDT。

\$\$

-62.04-6+1-5=-72.04

\$\$

页面上的浮动盈亏没有算完这笔账。我该盯住两条腿按照当前盘口立即退出以后，账户会少多少钱。

昨天给纸面账户定的规则很明确。组合可执行净亏损达到160

USDT，进入红色预警，停止开仓和补仓，两条腿同步减掉一半。达到200 USDT，Kill Switch启动，剩余仓位无条件退出。

按照这组0.033 BTC的仓位计算，当前基差扩大到约5545 USDT时，会触发红色预警。扩大到约6758 USDT时，会触发硬止损。

这两个数字让我对所谓的等收敛谨慎了很多。价差以后可能会回来，仓位未必能撑到那一天。

基差如果继续扩大，永续空单会持续浮亏。现货账户里即使还有值钱的BTC，也不代表永续账户自动拥有足够保证金。两条腿分在不同账户或不同交易所时，资金来不及划转，空单可能先被强平。空腿一消失，手里只剩现货多单，整套套利当场变成方向仓。

资金费也可能转负。原来每天收钱，后来每天付钱。收敛需要多久又没人保证。持仓拖上几个月，资金、风险预算和注意力全被锁住。极端行情再碰上盘口变薄，想走的时候还要挨一刀滑点。

收敛两个字也有方向

再看一组容易做反的数字。

现货价格100000，永续价格98000，开仓基差是负2000。仍然买入0.033 BTC永续。后来两者都走到99000。

BTC现货，同时做空0.033

现货亏33 USDT，永续空单也亏33 USDT，合计亏66 USDT。

价差的确归零了，这组合却站在错误的一边。基差从负2000上升到零，现货多、永续空赚的是基差下降，两者方向正好相反。

想交易负基差回升，理论持仓应当换成空现货、做多永续。空现货又会带来借币利息、借币容量、利率跳升和提前召回等问题。公式可以一行写完，能否借到币、借多久、最后怎样买回，才会决定这笔生意是否存在。

以后再看到价差收敛，我得先问两个问题。当前基差是正还是负，我的仓位赚的是基差上升还是下降。方向不对，收敛得越干脆，亏得越完整。

一笔33 USDT的利润该不该赚

最后一组案例最有趣。

某币现货价格100 USDT，永续价格101.2 USDT，计划交易50枚。预计72小时后基差从1.2缩小到0.2，预计收到18 USDT资金费。开平仓手续费20 USDT，双腿滑点15 USDT。

基差收益是50 USDT，加上18 USDT资金费，扣掉35 USDT成本，预计净赚33 USDT。

单看这一步，交易似乎还能接受。

压力情景里，基差可能从1.2扩大到4。扩大2.8 USDT，乘以50枚，毛回撤是140 USDT。把手续费和滑点算进去，如果18 USDT资金费还没有到账，真实可退出亏损会达到175 USDT。

50枚本身也违反了账户规则。双腿总名义价值是10060 USDT，超过10000 USDT上限。数量压到49.8枚仍然略超，单按名义仓位计算，最多约49.70枚。

风险预算给出的数量更小。假设35 USDT成本暂时固定，只要求不碰160 USDT红色预警，最多可以做约44.64枚。还想给模型误差留20 USDT空间，让计划最坏亏损控制在140 USDT以内，最多只能做37.5枚。40枚也会亏到约147 USDT，已经不合要求。

这些数字最后指向同一个决定。放弃。

50枚预计赚33 USDT，压力情景亏175 USDT，风险约是收益的5.3倍。用一个很简化的二元模型计算，收敛成功率要超过84.13%，期望值才刚刚转正。真实市场还有肥尾、滑点恶化和无法退出，要求只会更高。

手上没有足够证据证明成功率高到这个程度，33 USDT就不值得赚。拿大炮打蚊子，炮弹还是自己买的。

我以后会怎样检查期现套利

第一步先算利润。只认基差变化和资金费，手续费、滑点、借币和转账成本全部提前扣掉。净利润薄到不如安稳拿着USDT，这笔机会直接跳过。

第二步检查双腿。现货和永续按照标的数量匹配，不能只看两边放了多少USDT。成交没有同步完成时，先处理裸露方向敞口。

第三步按照目标数量读盘口。开仓和平仓都计算真实可成交价格，不能拿页面中间价安慰自己。

第四步做压力测试。基差继续恶化，资金费转负，盘口深度下降，把这些情况放进同一张账里。尚未到账的资金费不拿来垫风险预算。

最后写好退出条件。基差走到哪里减仓，亏到多少启动Kill

Switch，持有多久仍不收敛就离场。条件触发以后同步处理两条腿，不给仓位编故事。

学到这里，期现套利在我眼里已经没那么神秘了。它能把方向风险压下去，也会留下另外几张账单。账单一张都不能漏。

算出利润以后，还要把坏行情里的退出路径算清，随后才能决定是否开仓。很多时候，最好的执行就是把页面关掉，留着资金等下一次够厚的错价。

笔记 085: Day 9 | 非线性滑点下的最大套利规模

作者：chrise | 时间：2026-08-13 11:03 UTC | 字数：662 | 评分：75

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 | 非线性滑点下的最大套利规模

今日研究

把滑点改为随资金规模平方增长，计算同时满足“净收益至少 10 美元且至少 10 bps”时，35 bps 毛价差可以承受的资金规模区间。

套利逻辑

模型以 10,000 美元规模下 5 美元滑点为锚点。毛收益、协议费、失败期望损失和资金成本按本金变化；gas、桥费与 3 美元延迟成本按固定金额处理。规模太小时固定费用占比过高，规模太大时价格冲击加速增长。

数据与结果

100 美元步长扫描得到满足双重利润门槛的区间为 8,700–29,100 美元；连续模型上界为 29,165.9542 美元。
29,000 美元规模的滑点为 42.05 美元，净收益为 29.1803 美元，比 29 美元门槛高 0.1803 美元。
30,000 美元规模的滑点升至 45 美元，净收益为 29.0521 美元，比 30 美元门槛低 0.9479 美元。
50,000 美元规模的滑点达到 125 美元，净收益仅 5.4868 美元。

成本与风险

平方滑点是参数化假设，尚未用真实池深度或报价校准。可执行上界会随流动性、路由、交易方向、失败概率和延迟成本变化。

今日结论

规模扩大存在收益上限。当前假设下，接近 29,166 美元后，新增毛收益已经无法覆盖加速增长的滑点和 10 bps 利润门槛。

下一步验证

选定一个具体池或报价接口，采集多个交易规模的只读报价，拟合实际滑点曲线并重新计算规模上界。

笔记 086: Day 9 | 闪电贷、盈亏比与跨链套利

作者：fangliaofan | 时间：2026-08-13 11:26 UTC | 字数：506 | 评分：60
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 | 闪电贷、盈亏比与跨链套利

今日学习

今天补了三块拼图。第一，闪电贷的原子性机制：借助 Aave/dYdX 的 flashLoan，借、套利、还、手续费四步在同一笔交易内完成，任一环节失败即整笔回滚，等于用「无本金、无风险敞口」的方式验证套利路径。第二，盈亏比核算：把 gas 成本、滑点、池子手续费、可能的抢跑损耗都计入成本端，只有当价差收益 > 总成本时才值得动手，养成「先算账再出手」的习惯。第三，跨链套利：同一资产在不同链的定价偏差，需借助跨链桥或消息协议锁定两边价格，难点在确认时间与流动性错配。

关键认知

闪电贷的价值不在「空手套白狼」，而在于把资金门槛从约束条件里剔除，让策略验证回归到纯粹的执行与信息差。套利不是「有价差就能做」，而是「价差覆盖成本后仍有正期望」才做——gas 与滑点才是沉默的对手。跨链套利的真正壁垒不是发现价差，而是如何在两边都拿到可成交、且不被抢跑的确定性。

下一步

在本地用 foundry 写一个最小化闪电贷套利合约的测试用例，量化一笔真实价差在扣除 gas 与手续费后的净收益，验证「先算账」的结论。

笔记 087: 【Day 13 打卡 · 2026-08-13】数据分析与候选验证

作者：nerd614 | 时间：2026-08-13 12:00 UTC | 字数：477 | 评分：51
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 13 打卡 · 2026-08-13】数据分析与候选验证

延续「跟数据走」的节奏，今天从「积累报价」推进到「分析数据、验证候选」——把前面积累的真实路径数据拿来检验假设，看哪些值得进入结课前的小规模验证清单。

- 数据回顾：把已记录的跨链/DEX 路径按「净收益」排序，标记出扣完成本后仍为正收益的候选，以及那些只是屏幕价差、实际不成立的路径，找出规律（例如某类链间组合、某类资产对更容易出现真机会）。
- 信号对照：把候选路径与 Day 8 建立的信号规则做对照——触发条件是否命中、净收益阈值是否达标、路由是否可执行，筛选出「信号触发且数据成立」的交集。
- 假设检验：用候选数据回看 Day 9 提出的套利假设，验证「特定稳定币对/链间组合价差是否常超跨链成本」是否得到数据支持，未支持的部分调整假设方向。
- 验证清单：把通过数据筛选的候选整理成待验证清单，标注资产、链、路径、净收益、风控边界，为下一步 Paper Trading 或小额试跑做准备。

今天聚焦让数据真正驱动判断——从「记录了很多」到「知道该验证什么」。

笔记 088: 今天继续围绕链上套利策略做了一些整理，重点思考了一个问题：

作者：Oxgen3 | 时间：2026-08-13 12:14 UTC | 字数：625 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续围绕链上套利策略做了一些整理，重点思考了一个问题：

如果每天能发现很多信号，怎么判断哪些值得继续研究？

之前比较关注的是有没有价差，但这几天慢慢发现，真正有价值的并不是“看到价差”，而是这个价差能不能稳定出现，并且在实际执行之后还能留下利润。

所以今天开始尝试给机会做分类。

有些机会属于瞬时行情波动，出现时间很短，等发现的时候基本已经没了；有些虽然价差比较明显，但流动性不足，实际成交之后利润会被滑点吃掉；还有一些机会虽然利润不高，但是出现频率比较稳定，执行条件也比较成熟，这种反而更值得长期研究。

也就是说，后面不能只看单次收益，而是要综合考虑：

出现频率

持续时间

可执行规模

实际成本

成交后的真实收益

失败概率

今天也进一步理解了为什么之前一直强调“复盘”。

每完成一次模拟或者真实验证，都应该把结果记录下来，看看原来的判断和实际执行到底差在哪里。时间长了之后，才能知道哪些条件真的有用，哪些只是自己主观认为有用。

目前的思路也越来越清晰了：

先发现信号 → 过滤无效机会 → 计算真实成本 → 验证执行 → 记录结果 → 找出重复出现的规律 → 最后形成策略。

感觉现在已经开始从单纯学习套利知识，慢慢转向研究“怎么建立属于自己的套利系统”。

下一步准备继续把这些判断条件整理成更加明确的规则，然后尝试用程序或者 Agent 自动监控，减少人工盯盘和重复分析的工作。

笔记 089: 先说个我得出的结论：链上套利这行，看着是捡钱，其实全是坑，而且坑几乎都长在一个地方——你看到价差的那

作者：Joneloo | 时间：2026-08-13 12:32 UTC | 字数：1979 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

先说个我得出的结论：链上套利这行，看着是捡钱，其实全是坑，而且坑几乎都长在一个地方——你看到价差的那一刻，心里已经把成本省了。我翻了一堆案例和代码，把我学到的写下来，算是自己的复盘。

价差为什么存在

我之前老想不通：如果真有钱赚，为什么别人不赚？后来想明白了——价差不是白给的，它是市场在给「摩擦」定价。桥又慢又贵、不同链流动性互相不通、机器人卷成红海、风险还没人愿意承担……这些都是摩擦。你赚的钱，本质是替市场扛了其中一种摩擦的报酬。所以看到一个「诱人」的机会，第一反应不该是兴奋，而是问：我是不是漏掉了它的成本？

套利 vs 赌博，一线之隔

套利说白了就四个字：价差、等价、锁定、净收益。

价差：有机会的线索

等价：确认你在比「同一个东西」（同名代币不代表同一个——跨链包装、脱锚稳定币都可能骗你）

锁定：两条腿能不能一起锁死（买入后要等跨链到账才能卖、那就不叫锁定）

净收益：扣完所有成本还剩多少

单腿 = 赌方向。只有一条腿，剩下的都在赌未来价格。这在套利圈有个很直白的话：「套利是一条腿加另一条腿，把 LP 当一条腿找到另一条腿才叫套利，单腿都是在赌方向。」我深以为然。

最容易翻车的地方：五种幻觉

1. 幻觉价差——看着有差价，你真上去成交，没了
2. 成本低估——Gas、滑点、资金占用，一样样把你利润吃干净
3. 规模错觉——小资金回测赚钱，放大资金被价格冲击打穿。成本-规模是个 U 形：太小被固定成本吃掉，太大被滑点吃掉
4. 模拟盘陷阱——模拟盘赚钱不代表实盘赚钱，因为没算排序竞争、成交和卡单
5. 执行失控——策略逻辑全对，结果订单失败、gas 不够、卡单，照样白搭

我把成本模型抄到这了，以后每个机会都往里套

预期净收益 = 成功概率×成功净收益 - 失败概率×失败损失 - 资金占用 - 尾部缓冲

然后每个数字都要贴标签：

- 已观察：Quote 里直接给的、收据里能查的
- 已估计：历史数据推的（失败概率、延迟），要留方法
- 未知：没有数据的。未知不是零，别拿个漂亮的缓冲区间掩盖它

还有一个铁律：先找会推翻结论的那个变量。一次只改一项，找到「改到多少就亏」的反转点。单点回测好看没用。

最震撼我的一个案例：LP 收费站

有个很聪明的思路：卷不过套利机器人，那就当收费站——在机器人必经之路上放个高费率池收过路费。听着特别美好，对吧？结果有人真拿历史数据回测了，打脸打得非常结实：

高费率池根本没啥流量：Base 上 1% 费率的池，成交额只有 0.3% 池的 0.03%。因为聚合器只认最终到手金额，你一贵就被绕过一个「开张很成功」的下跌池：手续费赚了 +180 bps，但存货亏了 -1157 bps，净亏 9.8%

最反直觉的是：手续费和存货亏损是一回事。成交越频繁 = 价格穿越越狠 = 你被逆向选择打得越惨。「生意好」和「亏钱」是同一件事——这就很绝望了

唯一的正收益场景是最窄区间 -5%~1%，赚 +1.4 bps。「窄」才有流量，但「窄」也最容易被一根大阴线扫穿变全仓接盘。两个要求直接冲突，没有中间解。

教训：你想赚机器人的钱，其实你自己在当庄——赚的是点差，扛的是库存和 LVR。点差能不能覆盖「被更聪明的人挑着打」的损失，才是全部。而高费率池能吃流量，仅限于「没有更便宜的路」的时候……这个条件本身会自我毁灭。

我整理的入门路径（按难度排）

1. 资金费套利：统一账户，现货买入 + 合约反向开仓，吃每 8 小时的资金费率结算。机制最简单，可惜正费率机会真不多
2. CEX-DEX 价差发现 MVP：拿币安 API + DEX 报价拉价格，超阈值告警。先做发现，不做执行，跑通再说——这应该是新手最该从这起步
3. 小资金组 LP：感受一下无常损失疼不疼，重点监控大额撤池（跑路信号）
4. 跨链/聚合器：用它拿可执行报价（别信页面展示价，看 toAmountMin），跨链按「多段状态」算账
5. MEV：主网三明治贿赂已经 95% 起步了，公共 mempool 被 builder 把持得死死的。正面卷不过，只能差异化——复刻大佬吃尾气、走私有通道

最后几句掏心窝的话

这套玩法真的要大本金。小资金套的利润，经常连手续费都不够。先想清楚自己处在什么生态位
回测赚钱 ≠ 真能赚钱。研究阶段的纪律，就是实盘阶段的防火墙——这句话我抄下来的，因为它太真了
稀缺的不是知识，是方向，和敢不敢否定一个看起来很诱人的机会

我给自己定的规矩：每看一个新机会，填五个空——我在比的是不是同一个东西？我真实的成交规模是多少？哪条腿锁得住？我还有哪些成本没扣？它凭什么还没被别人拿走？

最后这个空最难填。填不上，就别上。

笔记 090: 链上套利共学 · Day9 打卡笔记 (08/13)

作者：haibian-slowly | 时间：2026-08-13 12:39 UTC | 字数：3207 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 · Day9 打卡笔记 (08/13)

今日信息

日期：2026-08-13 (周四)

主题：采集 Gas 数据

核心问题：一笔交易实际要花多少 Gas？预估和实际差多少？Gas 成本在套利全成本里占多大？

打卡正文

【第 9 天】

1. 今天要验证的问题

Day3 的净收益模型里有 gas 这一项，但一直是个"估算"。今天把它变成实测数据：BSC 上一笔普通转账、一笔 ERC20 转账、一笔复杂合约调用，到底各花多少 Gas？花多少美元？Gas 成本对套利到底算不算事？

2. 今天实际做了什么

(1) 先补概念 (Gas 的三个数字要分清)：

| 概念 | 是什么 | 谁说了算 |

|---|---|---|

Gas Limit (预估)	你愿意为这笔交易支付的最大 Gas 数	发交易的人/钱包/协议填的
Gas Used (实际)	交易真的消耗了多少 Gas	链上执行完才知道 (收据里)
Gas Price	每单位 Gas 付多少钱 (Gwei)	市场决定, BSC 用建议值 0.05 Gwei
Base Fee	每区块的固定基础费 (EIP-1559)	BSC 当前读出来是 0 (低负载)

(2) 写交付物 gas_collector.py (基于 web3.py + 现有 .env 的 BSC RPC) :

- 读最新区块 : gas_limit / gas_used / 利用率 / base_fee / gas_price ;
- 扫最近 15 个区块抽 45 笔真实交易, 按「普通转账 / 简单合约 / 复杂合约」分类统计 ;
- 每类抽一笔, 对比「交易里的 gas 字段 (预估)」vs「receipt.gasUsed (实际)」 ;
- 用 eth_estimateGas 模拟三笔典型交易 (BNB 转账 / ERC20 转账 / 复杂调用) ;
- 读 PancakeSwap V2 两个池 (BUSD/WBNB、USDT/WBNB) 的 reserves 算 BNB 价格, 把 Gas 折算成 USD。

(3) 真跑了一遍, 拿到的实测数据 (2026-08-13 12:38 UTC, BSC 区块 115,697,964) :

交易类型	实际 Gas (中位/样本)	折算 USD
普通 BNB 转账	21,000 (恒定)	$\approx \$0.00064$
ERC20 转账	21,996 (模拟)	$\approx \$0.00067$
简单合约调用	48,984 / 121,575	$\approx \$0.0015$
复杂合约调用 (swap 级)	35,080 / 51,840	$\approx \$0.0011 \sim 0.0016$

(4) 预估 vs 实际的关键对比 (真实链上收据) :

样本交易	预估(gas字段)	实际(gasUsed)	余量
普通转账	23,000	21,000	9%
简单合约调用	300,000	121,575	147% 富余
复杂合约调用	55,000	51,840	6%

3. 证据 (本次真实运行结果)

- 交付物 : E:\链上套利共学\gas_collector.py (~9KB, 已运行验证) ;
- 运行输出关键行 :
 - 最新区块 #115697964 利用率 37.5% ; gas_price(建议)=0.05 Gwei ; baseFeePerGas=0 ;
 - 45 笔抽样分类 : 复杂合约调用 38 / 简单合约 6 / 普通转账 1 ;
 - 收据对比 3 行 (预估 vs 实际, 见上表) ;
 - 折算 : 普通转账 \$0.00064、ERC20 \$0.00067、复杂调用 \$0.00114 ;
 - BNB 价格交叉验证 : BUSD/WBNB 池 608.98 USD、USDT/WBNB 池 608.94 USD (两个独立池一致) ;
 - 数据存档 : E:\链上套利共学\打卡笔记\Day9_gas_data_20260813.json。

4. 得到了什么结论 (来自数据)

1. BSC 上 Gas 成本几乎可以忽略 : 一笔复杂合约调用 (swap 级) 实际只要 \$0.001 左右——比一条推文便宜得多。这直接回答了 Day3 净收益模型里的疑问 : 在 BSC 跑套利, gas 不是门槛, 滑点和手续费才是 ;
2. Gas Limit 的“预估”普遍虚高 : 钱包/协议默认给的 gas 字段往往比实际多 6%~147% (抽样里简单合约调用给了 300K 实际只用 121K)。这带来两个套利含义 : ① 失败交易会烧掉整个 gas limit (不是 gas used), 所以失败成本 = gas limit $\times$ price ; ② 一些 MEV/套利机器人会把 gas limit 压到贴近实际值来省成本 ;
3. 普通转账 21,000 Gas 是协议刚性成本, 任何链都躲不掉, 但 BSC 把它折算成 \$0.0006——跨链比价时, 同样一笔转账在 ETH 主网可能 \$2~5, 在 BSC 是 \$0.001, 这就是“链的选择本身就是套利空间”的注脚 ;
4. baseFeePerGas=0 (BSC 低负载时) 说明 : gas price 完全由市场/建议值决定, 套利时机选在链上空闲时, gas 成本还能更低。

5. 今天失败了什么

web3 7.x 的 POA middleware 改名了 : geth_poa_middleware 在 7.16 里叫 ExtraDataToPOAMiddleware, 直接 import 旧名会崩 (BSC 是 POA 链, extraData 超长, 不加这个 middleware 连 get_block 都过不去) ——这是 web3 版本迁移的典型坑 ;
BUSD/WBNB 池 token 顺序踩坑 : 第一次想当然认为 token0=BUSD, 结果把价格算成了 0.0016 (方向反了)。教训 : 合约数据不能凭名字猜, 要现场调 token0()/token1() 验证 ;
复杂合约调用的 eth_estimateGas 模拟用了假 calldata, 只验证了“长度影响”——真实 swap 的 gas 应该直接从链上收据拿 (已用抽样收据补齐)。

6. 明天只解决什么

统一数据结构 : 把这几天散落的数据 (LI.FI 报价、Gas、滑点、跨链时间) 对齐成一张表, 让“净收益 = 价差 - 全成本”能直接算出来 (Day10 预告主题)。

一句话总结

实测结论：BSC 上 gas 便宜到可以忽略（复杂调用一笔 $\approx$ \$0.001），Gas 成本在 BSC 套利里不是门槛，滑点和手续费才是；同时发现“预估 gas limit 普遍虚高 6%~147%”，失败交易烧的是整个 gas limit——这两个点都是净收益模型里 gas 项的真实形状。今天交付 gas_collector.py 并把 Gas 折算 USD 补进了成本模型。

明日预告

Day10 (08/14)：统一数据结构——让所有数据能合并计算。

笔记 091: Day 9 (2026-08-13) · 阶段二开启：案例 + 工具 + Hermes Agent (8/

作者：gh25888 | 时间：2026-08-13 12:44 UTC | 字数：409 | 评分：51

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 (2026-08-13) · 阶段二开启：案例 + 工具 + Hermes Agent (8/13-8/19, 7 天)

今天没有固定任务卡，按阶段二方向开工：

1. 不再背名词，开始拆真实案例——还原市场/资金/路径/成本/风险/失败条件，把「策略 vs 基建 vs 运气」分开看
2. 主攻方向 (4 选 1)：① Robinhood 案例补全 ② The Graph 还原真实套利 tx (路径/gas/滑点/净收益) ③ Hermes 机会核对清单 v0.1 (来源/成本/流动性/执行限制) ④ Taoli/QQQ 半自动工具思路落地
3. 阶段二总产出：案例拆解模板 + 至少 1 个完整案例还原 + Hermes 机会核对清单 v0.1 + 工具评估笔记
4. 昨日遗留：8/12 打卡已提交但实际学习待对证追补；8/11 漏卡补卡方案待定

证据：vault 链上套利共学/每日/2026-08-13.md

笔记 092: Day 9 | DEX 间价差：从「看到」到「敢做」的可执行报价

作者：zhanglianzhong | 时间：2026-08-13 13:01 UTC | 字数：962 | 评分：73

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 | DEX 间价差：从「看到」到「敢做」的可执行报价

今日目标：把 DEX 间套利从概念落到判断标准——同一链上两个 DEX 出现价差时，到底多大的价差才值得动手。

学到了什么

1. 价差存在是常态，可执行才是稀缺。同一链上 Uniswap 和 SushiSwap 对同一个交易对的报价经常短暂错开，原因和 DEX 内的多费率池错位一样：流动性分散、没有套利者时刻拉平。但「屏幕上看到的价差」和「扣完成本后的净收益」是两回事，这是第一周就立下的原则，今天把它量化了。
2. 可执行价差阈值的思路。一笔 DEX 间套利的硬成本包括：两个 DEX 各自的手续费（常见 $0.3\% \times 2 = 0.6\%$ ）、Gas (L2 上几美分到几毛钱，主网可能直接吃掉全部利润)、以及执行期间价差收敛的风险。粗算下来，在 Arbitrum 这类 L2 上做主流币对的 DEX 间套利，价差至少要稳定在 0.8% 以上才有讨论价值——0.6% 覆盖双边手续费，剩下 0.2% 是给滑点和延迟的容错空间。这个数字比我之前直觉想的要低，但命中率问题更大：这种价差通常几秒到几十秒内就被抹平。
3. 信号真实性比价差大小更重要。结合我最近在 CEX 合约上踩的坑——OI 从 4000 万瞬间被野庄对开砸到 200 万——我更确定一件事：价格可以画、成交量可以刷，但持仓量的真实变化很难长期造假。链上同理，一个「价差」如果来自流动性极浅的池子，那它不是机会，是陷阱的诱饵。以后我的观察表里，每个价差信号旁边必须标注池子深度，浅池价差直接过滤。

认知收获

套利的第一性原理不是「找价差」，而是「排除假信号」。社群里有同学总结得很到位：ETH 主网的原子套利是死亡禁区，打不过 BuilderNet 的模拟速度。散户的活路在长尾——新链、新 DEX、浅竞争，但长尾恰恰也是假信号密度最高的地方。过滤能力就是护城河。

明日计划

用 The Graph 拉一次 Uniswap V3 和 SushiSwap 在 Arbitrum 上的实时报价，亲手算一笔 ETH/USDC 的 DEX 间价差，把 0.8% 阈值放到真实数据上验证，顺便记录两个池子的 TVL 作为深度参照。

笔记 093: 2026-08-13 打卡

作者：syndoom | 时间：2026-08-13 13:05 UTC | 字数：157 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-13 打卡

价差监控持续运行：今日 74 个有效样本 (14 个币 $\times$ 5 链)。

- 毛价差 中位 97 bps / 最大 10638 bps

- 全成本净利 中位 -213 bps / 最好 557 bps (WBTC optimism→polygon)

首次出现净利为正的窗口，值得深挖！

笔记 094: 参考

作者：mookixiao | 时间：2026-08-13 13:24 UTC | 字数：527 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

参考

Gemini

问题

1. ERC-20是什么？ERC下还有什么别的种类？

1. ERC：表示Ethereum Request for Comments，即以太坊那个意见征求草案，是以太坊社区的技术接口规范，可以简单理解为以太坊世界的“统一标准”

1. ERC这些标准只在以太坊世界通用（无论是L1还是L2），BTC、Solana世界有另外的标准体系

2. ERC-20：同质化代币（Fungible Token）标准规范

1. 同质化（Fungible）：是说每个代币都完全一样、可自由替换且可无限分割

3. ERC-721：非同质化代币（NFT）标准规范，每个代币有唯一的tokenId，不可分割且独一无二

笔记

1.

eth_call：以太坊JSON-RPC的制只读/沙箱模拟接口，在本地节点EVM中运行，它无需签名、不赏脸、0Gas消耗。套利中的应用：

1. 零成本监控：高频轮训DEX资金池储备量，计算即时价差与滑点

2. 交易预演与避险：在提交真实交易前先用eth_call本地跑一遍，若计算出亏损或Revert，直接放弃发单，时间零成本避险

笔记 095: 今日实践：给 LayerZero 跨链套利监控程序加上「实时询价」能力

作者：Zhou Junjie | 时间：2026-08-13 13:33 UTC | 字数：1355 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日实践：给 LayerZero 跨链套利监控程序加上「实时询价」能力

在 arbitrage-playbook 项目上，把监控程序从「只发现机会」推进到「发现后实时询价」，补齐了套利决策链路。

实现的功能

新增 OpportunitiesService，把一次跨链套利拆成三条腿并行询价：

买腿 LI.FI quoteExactOutput USDC → OFT token（源链 DEX）

跨链腿 OFT quoteTransfer OFT token 源链 → 目标链（LayerZero）

卖腿 LI.FI quoteExactInput OFT token → USDC（目标链 DEX）

暴露两个 API：POST /opportunities/quote（指定路由询价）和 POST /opportunities/scan（扫描全部活跃路由，按利润排序）。

利润计算模型

$$\text{netProfit} = \text{sellProceeds} - \text{buyCost}$$

marketExplicitCost（双边 gas + 未含 fee）

crosschainTokenCost（OFT amountSent amountReceived）

crosschainNativeFee（LayerZero 消息费）

executionGasBuffer（OFT 发送 + approve gas）

关键设计决策

1. BigInt 精确计算：金融计算全程用 bigint 规避浮点误差，金额解析拒绝超 decimals 位数的输入

2. 保守估算：卖出用滑点下限 toAmountMin，买入用 exact-output 保证拿够数量，再叠加 gas buffer

3. 只读报价边界：executable=false，不生成 calldata、不签名、不执行；报价 15 秒过期，执行前必须重询

4. 数据可信：只对 scan 阶段 VERIFIED + 双向 active peer 的资产询价

技术栈

LI.FI SDK 做 DEX 聚合询价

viem 直读 OFT 合约的 quoteOFT/quoteSend 获取链上真实跨链费率

每链配置 USDC 作为计价基准（注意 BSC 用 Binance-Peg USDC，18 decimals）

收获与反思

套利决策的本质是「把所有成本项都找齐」：DEX 价差只是毛利润，gas、跨链消息费、滑点、approve
这些隐性成本才是决定净利的关键
询价必须保守：宁可低估，执行时再实测
三腿并行 + Promise.all 显著降低询价延迟，但要注意 LI.FI/RPC 的限流（scan 并发度设 2）
遗留 TODO：报价 → 生成 calldata → 签名执行 的自动化闭环；scan 从手动触发改为定时全市场扫描 + 利润阈值告警

笔记 096: 套利基础学习打卡④ | 看到价差≠能赚钱：从毛价差到真实套利利润

作者：cfanboy | 时间：2026-08-13 13:33 UTC | 字数：7109 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利基础学习打卡④ | 看到价差≠能赚钱：从毛价差到真实套利利润

前三篇分别学习了：

- ① Funding Rate（资金费率）\
- ② Basis（基差）\
- ③ Delta Neutral（Delta 中性）

今天继续研究一个真正决定套利策略能不能落地的问题：

交易成本（Trading Cost）。

刚开始研究套利的时候，很容易产生这样的想法：

A 市场 BTC = 100,000\

B 市场 BTC = 100,500\

中间有 0.5% 价差，所以存在 0.5% 的套利机会。

但真正开始计算以后会发现：

Spread ≠ Profit。

我们看到的 0.5% 只是毛价差（Gross Spread）。

真正能赚到的钱应该是：

Net Profit = Gross Profit - All Costs

而很多套利机会，就是在“All Costs”这里消失的。

一、先从一个最简单的例子开始

假设：

交易所 A：

BTC = 100,000 USDT

交易所 B：

BTC = 100,500 USDT

表面价差：

500 USDT

价差率：

0.5%

于是我们同时：

A：买入 1 BTC

B：卖出 1 BTC

理论毛利润：

500 USDT

看起来非常不错。

但这只是最理想的数学模型。

真实交易过程中至少还需要考虑：

交易手续费

买卖价差

滑点

Funding

借币成本

资金占用成本

提现 / 跨链 / Gas 成本

对冲误差

执行延迟

最终真正应该计算的是：

500 USDT - 所有成本

剩下来的部分，才是真正的套利利润。

二、第一项成本：Trading Fee

最容易理解的是交易手续费。

假设两个交易所 Taker Fee 都是：

0.05%

A 买入：

$100,000 \times 0.05\%$

= 50 USDT

B 卖出：

$100,500 \times 0.05\%$

≈ 50.25 USDT

仅仅开仓：

手续费就已经超过：

100 USDT

原来的：

500 USDT

已经变成：

约 400 USDT

如果最终还需要平掉两个仓位：

又会产生一次手续费。

于是一个完整套利周期可能需要：

开仓两笔 + 平仓两笔

总共：

4 次交易。

这让我意识到：

套利系统不能只看单边手续费。

而应该计算：

完整生命周期的 Round-Trip Cost。

三、Maker 和 Taker 对套利影响非常大

这里又涉及两个重要概念：

Maker

和

Taker

简单理解：

Maker：

挂单等待别人来成交。

Taker：

主动吃掉订单簿中的已有挂单。

通常：

Maker Fee < Taker Fee

所以对于高频套利系统来说，Maker / Taker 的选择可能直接决定策略是否盈利。

例如：

套利毛利润 = 0.15%

如果：

双边手续费合计 = 0.10%

那么还剩：

0.05%

如果再加一点滑点：

策略可能直接变成负收益。

所以：

低费率本身就是套利策略的竞争优势。

四、第二项成本：Bid-Ask Spread

这是一个很容易被忽略的问题。

假设交易所显示：

BTC 最新价格：

100,000

但订单簿实际上是：

Bid：

99,990

Ask：

100,010

那么：

100,000 并不是你一定能够成交的价格。

如果你立即买：

需要支付：

100,010

如果立即卖：

只能卖：

99,990

这里已经存在：

20 USDT 的 Bid-Ask Spread。

所以套利系统真正应该比较的不是：

Last Price

而是：

Executable Price。

例如：

买入应该看：

Best Ask

卖出应该看：

Best Bid

这也是我今天觉得非常重要的一个认知：

行情页面显示的价格，不等于真正可以成交的价格。

五、第三项成本：Slippage

即使看 Best Bid / Best Ask，还不够。

因为：

Best Price 不代表你所有数量都能以这个价格成交。

例如订单簿：

Ask Price	Quantity
100,000	0.1 BTC
100,050	0.2 BTC
100,100	0.3 BTC
100,200	1 BTC

如果我要买：

1 BTC

显然不可能全部以：

100,000

成交。

真正成交价格需要把订单簿逐层吃掉。

最终可能平均成交价格：

100,140

这就是：

Slippage（滑点）。

所以一个机会可能看起来：

Spread = 0.5%

但真正按照目标交易数量模拟订单簿以后：

Effective Spread

可能只剩：

0.2%

甚至：

< 0

这意味着：

套利机会必须和交易规模一起讨论。

同一个机会：

交易 1,000 USDT

可能赚钱。

交易 100,000 USDT

可能已经亏钱。

六、第四项成本：Funding Rate

如果套利涉及永续合约：

Funding 就必须加入成本模型。

例如：

Long Spot

[图片]

Short Perp

如果 Funding：

+0.03%

Short Perp：

收到 Funding

那么 Funding 是：

收益。

但如果持仓期间 Funding 变成：

-0.03%

Short Perp：

就开始：

支付 Funding。

所以 Funding 在不同情况下可能是：

Revenue

也可能是：

Cost

因此真正的模型应该计算：

Expected Funding

而不是简单使用：

Current Funding。

这和第一篇 Funding Rate 的学习正好连接起来了。

七、第五项成本：Borrow Cost

如果套利需要：

Short Spot

那么通常需要：

借币。

例如：

借入 1 BTC

↓

卖出 BTC

↓

建立 Short Spot

↓

另一边 Long Perp

如果借币利率：

8% APR

那么这部分成本必须从套利收益中扣除。

这也解释了为什么：

Long Spot + Short Perp

通常比：

Short Spot + Long Perp

执行起来更加简单。

因为前者：

直接买现货即可。

后者：

通常涉及借币。

所以即使两种套利结构理论收益相同：

实际净收益也可能完全不同。

八、第六项成本：资金占用

这是我以前比较容易忽略的一项。

假设一个套利策略：

投入：

100,000 USDT

一个月以后：

赚：

500 USDT

看起来：

稳赚 500。

但问题是：

这 100,000 USDT 被占用了整整一个月。

收益率：

0.5% / 月

简单年化大约：

6%

那么就应该进一步问：

如果资金放在其他低风险策略中：

是否可以获得更高收益？

所以套利策略不能只看：

Absolute Profit

还应该看：

Return on Capital。

甚至进一步看：

Capital Efficiency。

这也是为什么很多套利系统会使用一定程度的杠杆。

不是为了赌方向，而是为了：

提高资金利用率。

当然，杠杆同时会增加强平和保证金风险。

九、DEX 套利还要增加 Gas 成本

如果从 CEX 进入 DEX / 链上套利，还会增加新的成本：

Gas Fee。

例如发现：

Uniswap：

ETH = 4,000

另一个 DEX：

ETH = 4,010

看起来存在：

10 USDT

套利空间。

但是执行需要：

Swap A

[图片]

Swap B

[图片]

可能还有 Approve

[图片]

Flash Loan Fee

[图片]

Gas

最终：

Gas = 8 USDT

DEX Fee = 3 USDT

那么：

理论利润：

10 USDT

实际：

-1 USDT

套利机会直接消失。

所以链上套利必须计算：

Profit After Gas。

十、跨链套利还要增加更多成本

如果继续扩展到：

Cross-Chain Arbitrage

事情会更加复杂。

除了交易手续费和 Gas：

还需要考虑：

Bridge Fee

跨链时间

目标链 Gas

资金预部署成本

跨链失败风险

价格变化风险

例如：

Arbitrum ETH = 4,000

Base ETH = 4,030

表面：

30 USDT Spread

但如果套利流程是：

Arbitrum 买 ETH

↓

Bridge 到 Base

↓

Base 卖 ETH

等真正跨过去以后：

可能已经过去几分钟。

此时：

Base ETH = 4,005

套利机会已经消失。

所以真正高效的跨链套利通常不会：

发现机会以后才开始跨链。

而是：

提前在不同链部署资金。

发现机会以后：

两边直接同时交易。

后续再进行：

Rebalance。

这个思路和跨交易所套利其实非常类似。

十一、套利系统真正应该计算什么？

经过今天的学习，我觉得一个套利 Scanner 不应该只输出：

Spread = 0.8%

而应该输出类似：

Gross Spread

↓

减：

Trading Fee

↓

减：

Slippage

↓

加 / 减：

Funding

↓

减：

Borrow Cost

↓

减：

Gas / Bridge Cost

↓

减：

Expected Execution Loss

↓

最终得到：

Net Profit

甚至进一步计算：

Net Profit %

以及：

Annualized Return

和：

Return on Capital

这才是一个真正有意义的套利信号。

十二、举一个完整例子

假设发现：

Gross Arbitrage Spread = 0.60%

交易金额：

100,000 USDT

理论毛利润：

600 USDT

成本：

Trading Fee：

120 USDT

Slippage：

80 USDT

Funding :

30 USDT

Borrow Cost :

20 USDT

其他执行成本 :

20 USDT

那么 :

总成本 :

250 USDT

最终 :

$\text{Net Profit} = 600 - 250 = 350 \text{ USDT}$

真正收益率 :

0.35%

所以 :

页面看到 :

0.60%

真正赚到 :

0.35%

已经少了接近一半。

如果市场稍微波动一下 :

Slippage 从 :

80 $\rightarrow$ 250

那么整个套利机会可能只剩 :

180 USDT

甚至直接消失。

十三、因此需要一个 Min Profit Threshold

这让我进一步想到 :

真正的套利程序不能 :

$\text{Net Profit} > 0$ 就交易。

因为预测永远存在误差。

例如计算出来 :

$\text{Expected Net Profit} = 0.03\%$

理论上是正数。

但只要 :

滑点多一点

或者 :

行情延迟几十毫秒

或者 :

一条腿成交价格差一点

就可能亏损。

因此应该设置 :

Minimum Profit Threshold

例如 :

$\text{Expected Net Profit} :$

0.20%

才允许执行。

中间留出的空间就是：

Safety Margin。

于是策略真正判断的应该是：

Expected Net Profit > Min Profit Threshold

而不是：

Spread > 0。

十四、今天最大的认知变化

今天最大的收获，是进一步理解了：

套利不是寻找“价差最大的地方”。

而是在寻找：

扣除全部成本以后仍然存在正收益的价格关系。

所以：

Gross Spread $\neq$ Arbitrage Profit

真正应该关注的是：

Net Profit After All Costs

从系统角度来看：

以前可能会写：

if spread > 0.5% $\rightarrow$ trade

现在更合理的逻辑应该是：

获取真实订单簿

↓

模拟目标数量成交

↓

计算 Effective Buy Price

↓

计算 Effective Sell Price

↓

计算 Trading Fee

↓

计算 Slippage

↓

计算 Funding

↓

计算 Borrow Cost

↓

计算 Gas / Bridge

↓

计算 Expected Net Profit

↓

如果：

Net Profit > Safety Threshold

↓

Execute

这已经开始从：

“发现套利机会”

进入：

“判断套利机会是否真的可执行”。

十五、四篇内容终于可以串起来了

目前已经学习了：

① Funding Rate

理解：

永续市场多空双方之间的资金交换。

↓

② Basis

理解：

现货与合约为什么会产生价格偏离。

↓

③ Delta Neutral

理解：

如何降低币价涨跌带来的方向风险。

↓

④ Trading Cost

理解：

如何从理论价差得到真正可以赚到的 Net Profit。

于是一个最基础的套利框架已经逐渐形成：

Find Spread

↓

Understand Why

↓

Hedge Delta

↓

Calculate All Costs

↓

Estimate Net Profit

↓

Execute

今天最值得记住的一句话：

套利机会不是“存在价差”，而是“存在可成交、可对冲、扣除全部成本后仍然为正的价差”。

笔记 097: 【8月13日 学习打卡】链上套利 Day 9：基础设施运维与套利赛道认知

作者：feiy2165-blip | 时间：2026-08-13 13:45 UTC | 字数：871 | 评分：71

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【8月13日 学习打卡】链上套利 Day 9：基础设施运维与套利赛道认知

一、基础设施运维（实战经验）

- API 认证过期问题：OAuth 令牌有时效，过期导致服务 401，切换为 API Key 方式解决
- 守护进程崩溃教训：手动后台进程不可靠，升级为系统服务（开机自启 + 崩溃自启）根治
- 已验证自动恢复机制：进程被杀后系统服务自动拉起
- 经验：关键基础设施必须服务化，避免“人不在就失联”

二、清算机会扫描与分析

- 多链扫描当前可清算仓位，发现 HF<1 的仓位大多为“资不抵债坏账”
- 典型案例：借款远大于抵押品价值，即使全部抵押品也不够还债，清算必然亏损
- 无流动性抵押品（无交易对/脱锚）同样不可清算
- 结论：系统正确拦截所有无利可图的清算，等待真实机会

- 认知：坏账≠机会，清算前必须验证抵押品流动性和利润

三、群聊学习（套利赛道进阶）

资金门槛争论

- "小资金没必要套利" vs "每天 1-2% 收益已经无敌"
- 真实案例：有人从 5 千 U 做到 6 万 U (9 个月)
- 结论：资金不是门槛，策略才是，小资金更灵活

闪赚套利（年化 130%）

- 路径：现货做多 + 合约做空对冲 + 现货存入闪赚吃高额利息
- 风险：合约价差、资金费、做空山寨的心理压力
- 容量有限：知道的人一多就无币可借

闪电贷套利（小资金最优解）

- 无本金风险（还不了钱交易自动回退）
- 配置要求极低，成本只有 gas
- 原子执行，失败不成交

其他认知

- 跨链价差方向实践结论：有价差的基本没流动性（与我的实践一致）
- CEX-DEX 套利：波动剧烈时才有机会，窗口极短
- 服务器：闪电贷套利低配就够，延迟敏感才需要好机房
- 用 AI 开发：不要给固定指令，要反问式交互

四、核心认知

1. 基础设施必须系统服务化，崩溃自启是底线
2. 坏账不是机会，清算前必须算清利润
3. 闪电贷套利是小资金最优解（无本金风险）
4. 公开策略都有时效性，借鉴思路不照抄

笔记 098: 2026-08-13 | Day 9 | 四平台永续资金费率监控：费率差不等于套利利润

作者：Neo.Yun | 时间：2026-08-13 14:11 UTC | 字数：365 | 评分：59

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-13 | Day 9 | 四平台永续资金费率监控：费率差不等于套利利润

今天完成 Binance、Bybit、OKX、Hyperliquid 四个平台、10
个主流币的永续资金费率只读监控，并将不同结算周期统一为每小时费率。最新快照覆盖 40/40 个市场：SOL 最高为 Hyperliquid 0.001228%/h，最低为 Binance -0.000567%/h，屏幕差约 0.001795%/h；按每边 \$10,000 名义价值，仅对应约 \$0.179473/h 的理论资金费差。要利用它，需要 Hyperliquid 做空、Binance 做多并匹配名义价值；还必须扣除开平仓费、价差、滑点和资金成本，并考虑费率反转、保证金与清算风险。因此这只是候选信号，不是保守净收益或可执行套利。今日无真实交易。

笔记 099: 早上先处理了一个装错工具的插曲。群里提到的 Astro 搬砖工具，我先做安全审计，仓库里只有安装脚本

作者：Twooweeks | 时间：2026-08-13 14:14 UTC | 字数：478 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

早上先处理了一个装错工具的插曲。群里提到的 Astro 搬砖工具，我先做安全审计，仓库里只有安装脚本，真正的程序在 Docker 镜像里。好在镜像构建是带 SHA256 校验从 GitHub Releases 下载的，供应链比想象中干净。在 Mac 上用 Docker 模拟跑起来了，面板也通了。然后发现装错了地方。这工具该装服务器上。全卸了，清干净。一早上折腾完，安全流程完整走了一遍，也值。

白天补了跨所套利的完整拼图。价差公式的窍门是用中点归一化，两个价的差距除以两个价的平均值，对称不偏心。照着案例手算了一遍，开仓价差 0.16%。再拿它和 break-even 比，0.044% 的价差，门槛是 0.27%。赚的方向，但赚不回成本，不做。

下午把合约三件套补齐了。做空平仓就是买入平仓。之前纠结为什么加“买入”两个字，后来明白，平仓是中性的，必须指明方向。做多和买现货的关系也理清了，都是赌涨，但合约做多带杠杆、会爆仓、有资金费率。爆仓判断就是保证金跌破维持线。10x 杠杆，价格跌 9.5% 就爆。套利只用低杠杆，要的是活着等到价差收敛。

笔记 100: 今日打卡 | 从“发现价差”到“建立证伪机器”

今日打卡 | 从“发现价差”到“建立证伪机器”

今天完成了什么

今天基于本地链上套利 Wiki，复盘了 2026-08-12 全量共学语料整理出的实证、负结果和主动纠错，进一步明确安全学习路线：链上套利研究的第一目标不是找到一个看起来赚钱的机会，而是建立一台能快速识别假套利、拒绝坏交易的证伪机器。

本次没有新增实时行情、签名、广播、Paper Trading 或实盘结果；以下结论来自已有本地研究材料和其中记录的参与者实验，不能视为我独立复现或当前市场 Edge。

关键发现

1. 大多数表面价差会被完整成本吞掉

已有语料中的典型负结果包括：跨 L2 稳定币配对在当前费用下全部为负；SOL/USDC 五档完整环为负；固定区块双池回放在扣除协议费、价格冲击和教学 Gas 后为负；跨链完整库存恢复候选没有通过。

这说明“看到价差”只是在发现候选，不是发现利润。真正要计算的是：

text

完整循环净收益

= 最终回到起始计价资产的增量

- DEX / 桥 / 聚合器 / 借贷费用
- Gas 与优先费
- 自身价格冲击与滑点
- 延迟与波动缓冲
- 失败、部分成交与再平衡损失

2. 量具错误比策略参数更危险

语料中多次出现：

同一报价源“低买高卖自己”，制造虚假的正信号；
USD 估值和 Token 数量口径相差几十个基点，甚至符号相反；
decimals 错误制造数量级价格异常；
静态价格相乘为正，但逐腿更新池状态后为负；
HTTP 200 的索引器响应实际包含错误或空历史；
WS 浅盘口与 HTTP 深盘口的前缀兼容问题，被误诊为交易策略失败。
因此，研究顺序应该是：先审计数据和测量过程，再优化策略参数。

3. “更快”不等于“更赚钱”

跨链路线比较显示，FASTEST 可能显著缩短预计时间，却因为协议费、库存恢复成本和路线条件不同，净 Edge 反而更差。速度必须放进完整成本模型，而不能作为单独的优化目标。

同理，预置双链库存可以减少等待桥接的风险，却产生资金占用、库存偏移、再平衡、稳定币版本和资金被困风险。跨链套利首先是库存管理问题，其次才是价格问题。

4. 失败必须进入分母

真实机器人样本中，失败交易可能占多数，且失败 Gas 能消耗总 Gas 的很大部分。跨所系统中，陈旧盘口导致的候选刷新后经常消失；IOC 还可能出现一腿成交、另一腿零成交。

所以应记录：

text

p_success × 成功净利

- p_fail × 失败损失
- p_partial × 部分完成损失
- 正常执行成本

不能只统计成功交易，更不能把未完成的单腿资产变化当成完整套利利润。

5. Quote、模拟和成交必须分级

observed quote：某个状态、金额和路由下的估算；
re-quote：检查机会是否只是陈旧或离群；
fixed-block replay：证明指定历史状态下的计算结果；
paper trading：观察时序和失败路径；
live execution：证明某次真实执行发生；

realized PnL：还要完成资产归位、成本结算和资金核对。

其中任何一层都不能单独证明未来长期可重复盈利。

最小可复用实验

固定一条链、一个资产对、两个 V2-compatible 池和三个输入规模，只实现五个离线/只读模块：

1. Snapshot：固定 block，读取资产元数据和池状态；
2. Quote：用 bigint 和 Solidity 一致的整数公式计算两跳闭环；
3. Ledger：费用只扣一次，未知成本明确标记并设置保守缓冲；
4. Replay：同一快照重跑结果逐 raw unit 一致；
5. Decision：输出 KILL / ITERATE / PAPER_TEST，允许稳定输出 NO_TRADE。

必须包含负样本、边界样本、合成正样本和

decimals/reserve/block

错误样本。合成正样本只能证明计算链路能识别正值，不能证明存在可交易 Edge。

证据在哪里

第一性原理：</Users/lizyye/LLMwiki/web3/arbitrage/queries/arbitrage-from-zero-first-principles.md>

安全学习路线：</Users/lizyye/LLMwiki/web3/arbitrage/queries/arbitrage-safe-learning-roadmap.md>

全量语料实证：</Users/lizyye/LLMwiki/web3/arbitrage/queries/full-cohort-evidence-2026-08-12.md>

2026-08-12 打卡：</Users/lizyye/LLMwiki/web3/arbitrage/checkin-2026-08-12.md>

课程来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

假设与风险

本次是本地 Wiki 的知识提炼，没有新增真实行情、交易或回测。

全量语料包含教程、计划、二手整理和参与者自报实验；重复表述不等于独立验证。

历史回放、Quote、Fork、模拟和单次成交都不等于未来可重复 Edge。

当前不连接钱包、不请求授权、不签名、不广播、不接触私钥或助记词。

还没完成

尚未完成统一数据规范下的长时间窗口下一块存活率统计。

尚未获得足够的跨链部分完成损失、库存资本成本和再平衡尾部样本。

尚未独立复现 Wiki 中所有参与者提供的外部交易、API 或代码证据。

下一步

继续建设最小只读验证器，输出每个候选的：

text

资产身份 → 同步状态 → 实际规模 Quote → Re-quote

→ 完整成本瀑布 → 失败期望值 → 库存循环

→ 证据等级 → KILL / ITERATE / PAPER_TEST

今天最重要的结论是：一个可靠的套利系统，不是每天都说“能赚”，而是能用可审计证据解释为什么绝大多数机会不应该做。

笔记 101: Day 9 | 2026.08.13 | 免费 RPC 降级陷阱: 一个进程活着, 但搜索已经死了

作者：xzone911 | 时间：2026-08-13 14:32 UTC | 字数：1499 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 | 2026.08.13 | 免费 RPC 降级陷阱: 一个进程活着, 但搜索已经死了

背景

8/11 Helius 主 RPC 被 429 硬封锁后，用免费端点做了双数据面应急拆分：Foundation 公共端点负责池发现，PublicNode 负责状态账户读取(每批 8 个，因为 12+ 就返回 403)。当时验收窗口：state-selection error 0、HTTP 403/429 全 0、PID 正常、NRestarts=0、12/12 广播端点预热成功。看起来恢复得很好。两天后复查，发现这是一个陷阱。

复查：一切指标都正常，除了搜索本身

| 指标 | 状态 |

|-----|-----|

| 进程 PID | 没变, NRestarts=0 |

| 钱包余额 | 1,184,283,122 lamports, 没变 |

| StorkFun 候选榜 | 217-218 个, 持续更新 |

| provider 预热 | 6 组, 12/12 成功 |

| 主服务/health timer/momo | 全 active |

光看这些, 系统完全健康。但深入报价漏斗:

- 一小时内 646 次 state 尝试, 195 次失败(30.2%), 192 次是 PublicNode getMultipleAccounts 403
- slot skew p50/p90 = 20,798 / 24,545 slots, 最大 25,358——旧 push 缓存和新 HTTP 补读被混进了同一个不可变帧
- 443 份 quote summary: with_any_quote=0、fully_failed=443
- 正毛利、eligible、signal、broadcast、submitted: 全 0

结论: 这个 bot 已经死了——它没有在等机会, 它在失败地读数据。只是没人发现, 因为健康脚本报绿色。

为什么健康检查是盲的

健康脚本只检查最近有没有出现任意 state-selection 事件, 有就绿色。它没有检查:

1. state error rate(30% 失败率被无视)
2. slot skew(2 万个 slot 的差距 = 把 40 分钟前和现在混在一起)
3. 有效 quote 数(443 份 summary 全是废的)
4. 池队列年龄(queue wait p50 高达 45.9 分钟)

进程活着 $\neq$ 系统在工作。一个心跳只证明心脏还在跳, 不证明它还在思考。

免费 RPC 的三个门槛

把单批从 100 缩到 8, 只绕过了账户数量门槛, 没解决:

1. 频率门槛——持续负载下 WAF 限流(getSlot 单发 96ms 正常, 持续读就 403)
2. 并发门槛——16 分片的并发请求触发速率限制
3. 一致性门槛——PublicNode 慢读取导致同一帧内 slot 跨度巨大

今日认知

零利润这个数据, 在数据面坏掉的时候, 不具备统计意义。过去两天可能会误读成市场没有套利机会, 实际上是系统根本没形成过可用报价——不是没有肉, 是眼睛瞎了。

监控的正确姿势不是有没有心跳, 而是: 有效报价率 > 0 且 slot skew $<$ 阈值 且 state error rate $<$ 阈值。

可见的错误比隐藏的错误好。一个 crash 的进程你会立刻修; 一个活着但瞎了的进程能骗你两天。

笔记 102: 一、Delta-Neutral Funding Arb 的两个方向

作者: amazing-lzh | 时间: 2026-08-13 14:50 UTC | 字数: 1201 | 评分: 75

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、Delta-Neutral Funding Arb 的两个方向

| 场景 | 资金费率 | Perp 端 | 现货端 | 净头寸 | 收益 |

| :----- | :----- | :----- | :----- | :----- | :----- |

| A. 牛市常态 | 正 (+) | 空 | 多 (买现货) | $\Delta \approx 0$ | 收 funding |

| B. 熊市/极端反向 | 负 (-) | 多 | 空 (难点) | $\Delta \approx 0$ | 收 funding |

场景 A 简单——现货直接钱包持有或存 Aave 生息。场景 B 的"链上做空现货"才是真正需要设计路径的。

二、链上怎么做空现货?(场景 B 展开)

核心思路: 借币 $\rightarrow$ 卖币 $\rightarrow$ 未来低价买回还币。三条主流路径:

路径 1: 借贷协议 (Aave / Morpho / Compound) —— 最主流

\

存 USDC 做抵押品 (Aave)

1. 借出 ETH (借贷利率 x%) (Aave)
2. 卖掉 ETH 换 USDC (DEX)
3. 同时开多 ETH perp (Hyperliquid / dYdX / GMX)

↓

Delta 中性, 收资金费率

成本: 借币利率 抵押品存款利率。只有当 |负 funding| $>$ 借贷净成本 才有利可图。

路径 2: Perp DEX 直接开空 (跳过借贷)

如果你的 perp 平台本身就在链上 (Hyperliquid、GMX、dYdX), 可以:

在 A 平台开多 perp (收负 funding)

在 B 平台开空 perp (付另一边的 funding)

套两个平台之间的 funding rate spread

这是 funding rate arbitrage across venues，实际上是主流玩法之一。

路径 3：合成资产 / Delta-Neutral 协议（专业化）

Ethena / Resolv / Elixir 这类协议本质是“打包好的资金费率套利”：

Ethena (USDe)：用户存 ETH → 协议在 CEX 开等额空 perp → 收 funding → 分给 USDe 持有者
已经把上面的 1-4 步做成产品，用户只需持有稳定币

对散户/产品来说，最有意义的路径其实是 接入这类协议做收益增强，而不是自己搭建套保。

笔记 103: 今天没有开始新版 Day 10，先完成课程与研究方向的复审。

作者：shiyang07ca | 时间：2026-08-13 15:00 UTC | 字数：304 | 评分：45

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天没有开始新版 Day 10，先完成课程与研究方向的复审。

旧课程的问题是把“修代码、补测试”当成学习主线，知识停留在表层，无法训练对 perp 套利机会的独立判断。拟重构的主线是：以 Lighter XAU ↔ Hyperliquid xyz:GOLD 为连续研究对象，BTC/ETH 作控制组，XAG/SILVER 作迁移测试；依次完成市场可比性、同步行情、可执行净收益、机会证伪、双腿执行和实时模拟。

今天只确认了学习目标、候选市场和下一步实验设计；没有完成策略有效性验证，也没有发送真实订单。明天从 XAU/GOLD 的市场定义表、同步行情采集和 Opportunity Memo v0 开始。

笔记 104: 本地跑 Reth 节点后，用 REVM + CacheDB 做模拟最快。通过本地 RPC 拉状态即可

作者：LeonardChen666 | 时间：2026-08-13 15:00 UTC | 字数：190 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

本地跑 Reth 节点后，用 REVM + CacheDB 做模拟最快。通过本地 RPC 拉状态即可，延迟很低，没必要一上来就直接读内存。

套利逻辑最好封进一个通用合约，路径、池子地址、金额都当参数传，支持不同数量和混合类型的池子。调用时只发一笔交易就行。

闪电贷能无本金套利，但有手续费（常见 0.05%）和 gas 成本，失败只亏 gas。先把模拟和合约跑通，再优化性能。

07 学习计划（5 篇）

笔记 105: 统计了热钱包 0xd712...9b96 最近一批调执行器且链上成功的交易。计划 25 笔，receipt

作者：catcashx | 时间：2026-08-12 15:55 UTC | 字数：472 | 评分：48

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

统计了热钱包 0xd712...9b96 最近一批调执行器且链上成功的交易。计划 25 笔，receipt 解析成功 18 笔。

毛利优先看执行器利润事件里的 ETH/WETH； $Gas = gasUsed \times gasPrice$ ；扣 gas 后 = 毛利 Gas。小于约 $1e-9$ ETH 的数值当作几乎为 0，不计入有效利润。

18 笔里：

- 有效毛利（能当真的）5 笔：其中 4 笔扣 gas 后仍为正，1 笔毛利盖不住 gas
- 其余 13 笔几乎没有利润，但仍付 gas，扣 gas 后全是负的

5 笔有效毛利合计大约 $3.76e-4$ ETH，对应 Gas 大约 $1.62e-4$ ETH，这 5 笔本身合计是赚的。

但 18 笔放一起（无利润的按 0 算），Gas 更重，净大约 $2.8e-4$ ETH，整体没赚。

我的判断：链上成功 ≠ 赚钱。少部分单子闭环有利润且盖得住 gas；多数成功单利润极薄或几乎没有，gas 把结果打成负。这还没算失败交易的 gas。样本只是最近窗口、单一热钱包；若利润进了金库，毛利可能被低估。

笔记 106: Day 9 — DeFi 协议间套利

作者：pennchester | 时间：2026-08-13 01:08 UTC | 字数：1253 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 — DeFi 协议间套利

今日核心收获

1. 简单借贷利差在高效市场中为负

- Aave 借 USDC 6.5% vs Compound 存 5.5% = 净利差 -1.0%
- 借贷协议间利率被套利者迅速收敛，看到的正利差要么已消失要么是暂时的
- 唯一可行的简单利差：协议代币激励期（如 Morpho 上线时 +5% 奖励，窗口仅几天）

2. 循环杠杆放大微薄利润，但清算风险是生死线

- Compound V3 存 ETH 借 USDC 循环 5 轮：杠杆 2.45x，净 APY 从 5.5% 提到 8.2%
- 健康因子随轮次递减 2.86→2.00，ETH 跌 25% 即触发清算
- 收益本质是承担清算风险换来的

3. LST 折溢价套利是最稳定的协议间套利

- stETH 理论锚定 1:1 ETH，二级市场折价 1-5% 常见
- Curve stETH/ETH 池失衡 6.67% 折价时，100 ETH 可换 107.06 stETH，利润 ~\$24.7K
- Curve 池提供即时退出，优于 Lido 排队赎回（1-14 天）
- rETH 溢价最多（去中心化叙事稀缺性），cbETH 折价最多（无赎回+监管不确定性）

4. 稳定币脱锚的生死线 = 硬赎回

- USDC 脱锚 10%（有 Circle 1:1 赎回）：回归概率 85%，预期 +10.3% → 可做
- 算法稳定币脱锚 10%（无硬赎回）：回归概率 30%，预期 -52.7% → 千万别碰
- 历史复盘：UST/LUNA 归零 vs USDC/SVB 48h 恢复，差异全在硬赎回

5. 收益农耕表面 APY 靠协议代币奖励撑起来

- 净 APY = (LP 费 + 协议奖励) - (借款利息 + 无常损失)
- 代币奖励随时间衰减、高 APY 吸引更多 LP 稀释手续费、IL 是滚动的
- Delta 中性农耕（LP + 永续空单对冲）会被资金费率吃掉 30-50% 收入

6. Curve 是 DeFi 套利的“基础设施”

- stETH/ETH 池做 LST 套利，3pool 做稳定币套利，低费率+高深度
- 3pool 是稳定币套利的“自动套利机”，池子失衡时任何 swap 都在套利

综合风险矩阵

- 借贷利差(杠杆) / LST 折价 / 稳定币脱锚 / 收益农耕 / Curve 3pool

产出

- 学习笔记：Day9_DeFi协议间套利.md（借贷利差/LST套利/稳定币脱锚/收益农耕/风险矩阵）
- 代码：defi_protocol_simulator.py（四大场景：循环杠杆清算/LST折溢价/脱锚硬赎回对比/Delta中性农耕）

明日计划

Day 10：MEV 深度攻防 — Searcher 视角的套利竞争与博弈

笔记 107: Day 09 · 2026-08-13 · 用 AI Agent 搭建案例研究 workflow

作者：twone | 时间：2026-08-13 01:50 UTC | 字数：1785 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 09 · 2026-08-13 · 用 AI Agent 搭建案例研究 workflow

今日目标

1. 把昨天手工做的“案例拆解”流程固化成可重复的 Agent workflow
2. 明确 Agent 在这套研究里干什么、不干什么
3. 用 Day 8 的产出反向验证 workflow 的有效性

一、为什么要先把流程固化

昨天的 USDC 案例拆解是纯手工做的：找资料、对时间线、套成本模型、列失败条件。做完发现一件事——这套动作的 80% 是机械劳动：搜证、交叉验证来源、按固定结构成文。真正需要人判断的只有两处：选案例的眼光，和对失败条件的压力测试。

这种“机械劳动占比高、判断点清晰”的流程，正是该交给 Agent 的。共学 21 天里至少要拆 5~10 个案例，不流程化的话每个案例都要重新发明一遍轮子。

二、产出：案例研究 SOP (workflows/case-research.md)

固化为仓库里的一份 SOP 文件，使用时把它作为上下文喂给 Agent（Hermes 或任何通用 Agent），附一句话任务即可。核心设计：

五段流程：取证 → 路径还原 → 套成本模型 → 失败条件清单 → 归档。每段的输出格式都写死，Agent 只需要填空。

三条硬约束，针对 AI 做研究的三个常见病：

1. 来源分级 + 交叉验证：链上数据 > 官方公告 > 媒体报道 > 社群传闻，后两级必须标注"未经链上验证"。AI 最容易把传闻写成事实，这条是防幻觉的
2. 失败条件 ≥ 3 条，禁止只写成功路径：事后视角会系统性低估风险，人懒的时候会跳过，机器必须强制
3. 缺证据就标注，不许脑补：每个关键数字必须有来源，没有的标"约/传闻"，并沉淀到"待验证问题"一节留给链上数据复查

三、反向验证：Day 8 的案例符合 SOP 吗

拿昨天的 USDC 笔记对照 SOP 验收标准：

时间线、价格区间、储备数字均有公开来源

失败条件 4 条

但有两处不合规：做市商利润"数亿美元级别"没有给出具体来源标注（应标"约/媒体报道"）；链上交易级证据缺失（没有具体的交易哈希佐证路径 C）。这正是 SOP 要防的问题——如果没有流程，这些缺口就会被忽略过去

四、Agent 在这套研究里的分工

| 环节 | 谁做 |

| :----- | :----- |

| 选案例、定研究方向 | 人（需要市场嗅觉） |

| 取证、交叉验证、按 SOP 成文 | Agent |

| 套成本模型、生成失败条件初稿 | Agent |

| 压力测试失败条件、定仓位规则 | 人（亏钱的是人） |

| 链上数据复查（The Graph / RPC） | Agent + 脚本（Day 10 衔接） |

分工原则一句话：Agent 负责把证据摆全，人负责在证据不全时下注。

五、与 Hermes 的衔接说明

SOP 本身与具体 Agent 工具解耦——它是一个 Markdown 上下文文件，喂给 Hermes Agent、Kimi Code 或任何带搜索/链上工具能力的 Agent 都能跑。后续如果侧有稳定的调用方式（定时任务、工具链配置），再把触发层补上；SOP 这层不用改。

明日衔接（Day 10）

The Graph / 链上数据查询：把 SOP 里"待验证问题"的复查能力建起来，用 USDC 脱锚期的池子数据做一次案例还原实测。（前置待办：注册 The Graph Studio API Key。）

参考资料

SOP 文件：workflows/case-research.md

首个按此流程产出的案例：notes/2026-08-12-day08.md

笔记 108: Day 9 - 比较不同路由

作者：MountainE | 时间：2026-08-13 09:49 UTC | 字数：579 | 评分：61

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 - 比较不同路由

今天把路由比较的条件先固定下来，避免把不同金额、资产或时间的报价直接混在一起。本次比较模板固定为：源链 Ethereum，目标链 Arbitrum，资产 USDC，交易金额 1,000 USDC。

我为每条候选路由建立了对比表字段：

- 路由来源：桥、DEX 和具体步骤。
- 预计到账数量：不只记录界面标价，还要扣除路由费用、Gas 和滑点。
- 总费用：分开记录源链 Gas、目标链 Gas、桥费、DEX 费和其他费用。
- 预计时间：包括等待确认和桥接到账的时间。
- 失败条件：报价过期、余额不足、滑点超限、桥或 DEX 不可用。
- 报价时间与区块状态：后续复核时必须重新查询。

路由对比的核心不是选择界面上输出金额最大的方案，而是比较同一时间、同一金额下的净到账数量和执行条件。如果一条路由输出更高，但耗时更长、费用更高或失败条件更多，它不一定是更好的执行方案。

今日产出：完成了一份路由对比表模板，可以用来记录同一组条件下的不同方案。当前不根据静态报价直接判定最优路由，而是将实时报价、费用和失败条件一起复核。

今日结论：路由对比必须先做条件标准化，再做成本和风险比较；只看输出金额容易把一次性报价误判成套利机会。

明日计划：观察同一路由在不同时间的变化，分析报价、流动性和失败率变化的可能原因。

笔记 109: 如果你那么有钱，为什么还那么不聪明？

作者：x-tan-dev | 时间：2026-08-13 12:44 UTC | 字数：9776 | 评分：77

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

如果你那么有钱，为什么还那么不聪明？

《随机漫步的傻瓜》第一章通常题为“如果你那么有钱，为什么还那么不聪明？”。塔勒布主要讨论一个问题：成功究竟来自能力，还是运气？

他通过交易员的故事说明：

有些人靠高风险、杠杆或恰好赶上行情迅速致富，看起来非常聪明，但这种成功可能只是某一种有利结果。

真正评估一个人，不能只看已经发生的结果，还要考虑“其他可能发生的历史”：如果市场换一种走法，他是否仍能生存？

稳定、可重复的赚钱能力，比短期暴富更能证明实力。一个收入普通但长期稳定的牙医，其财富质量可能高于偶然赚到巨款的交易员。

人们习惯“以结果论英雄”，容易把好运误认为能力，把倒霉误认为愚蠢。

这一章的核心可以概括为：

财富只能说明一个人赢过，不能单独证明他懂得为什么会赢。

它也为全书建立了判断框架：评价决策时，不仅要看收益，还要看承担了多大风险、结果能否重复，以及失败的可能性是否被隐藏。

奇怪的会计方法

《随机漫步的傻瓜》第二章通常译作“奇怪的会计方法”。它进一步解释：怎样区分真正的能力与偶然的好运。

塔勒布提出一种不同于普通会计的思考方式：普通会计只记录实际赚了多少钱；他的“概率会计”还要计算为了赚这些钱，承担了多少没有兑现的风险。

核心例子是“俄罗斯轮盘赌”：

一个人冒着死亡风险玩俄罗斯轮盘，侥幸活下来并赚得一千万；另一个人通过长期工作积累一千万。两人的账户余额相同，但财富的性质完全不同。前者只是某条幸运路径的幸存者，在大量其他可能的历史中，他已经失败甚至死亡。

因此，本章强调：

不能只评价结果，还要评价产生结果的过程。

利润相同，不代表策略质量相同。

隐藏的巨大风险在尚未发生时，很容易被误认为不存在。

我们只看见成功的幸存者，却看不见采用同样方法而失败的人。

好决策也可能暂时亏损，坏决策也可能偶然赚钱。

第二章与第一章的关系是：第一章指出“有钱不等于聪明”；第二章则提供判断方法，即把没有发生但原本可能发生的结果也纳入评价。

一句话概括：

真正应该核算的，不只是赚了多少钱，还有为了赚钱暴露在多少种失败可能之下。

从数学角度思考历史

第三章通常译作“从数学角度思考历史”。这一章把前两章的“可能发生但没有发生的历史”进一步变成一个概率分析框架。

塔勒布认为，我们看到的现实只是许多可能路径中的一条。例如，一项交易最终赚了钱，并不意味着它必然赚钱；如果让市场重新运行一千次，可能只有少数几次盈利，而现实恰好落在其中一次。

他由此引入几个重要观念：

现实只是一个样本：历史发生过一次，但一次结果不足以证明规律。

蒙特卡洛模拟：通过反复模拟不同情景，观察一项策略可能产生的全部结果，而不只研究实际发生的结果。

路径非常重要：即使两种策略的平均收益相同，其中一种也可能在中途遭遇无法承受的亏损。

时间会揭露风险：靠运气取得的成功可能维持几年，让人误以为那是能力；观察时间越长，隐藏风险越可能暴露。

不能把事后解释当成预测能力：事件发生后，人们很容易编出一套合理故事，但这种故事未必能事前预测结果。

第三章与前两章形成了递进关系：

1. 第一章：成功者不一定聪明，可能只是幸运。
2. 第二章：评价收益时，应把未发生的失败可能也算进去。
3. 第三章：可以用概率分布和模拟，系统地考察这些“另类历史”。

一句话概括：

历史不是唯一可能的剧本，只是随机性最终让我们看到的一个版本。

这章最实用的启示是：判断交易策略时，不要只回测“实际走过的行情”，还要问它在大量不同市场路径下是否都能存活。

随机性、无意义与科学知识分子

第四章通常译作“随机性、无意义与科学知识分子”。它主要讨论：面对随机数据，人为什么会强行寻找规律和因果解释。

塔勒布指出，人脑不喜欢承认“这件事可能没有明确原因”，所以常把噪声解释成信号。例如，市场一次普通波动发生后，评论者总能立即找到经济数据、政策消息或投资者情绪来解释它；但这些故事往往只是事后拼接，并没有预测能力。

本章的几个重点是：

随机结果也能形成漂亮的图案：数据中出现规律，不代表背后真的存在稳定机制。

解释能力不等于预测能力：一个故事听起来合理，只说明它能配合已知结果，不能证明它事前有效。

复杂语言可能掩盖无知：公式、术语和权威身份，会让没有证据的观点显得很科学。

真正的科学态度是怀疑：不仅寻找支持自己观点的证据，还要主动寻找什么情况能够证明自己错了。

承认“不知道”是一种理性能力：科学知识分子知道知识的边界，而伪专家倾向于对所有事情给出确定解释。

这里也能看到波普尔“证伪”思想的影响：理论不能因为几个成功案例就被证明正确，却可能被一个有力的反例推翻。因此，塔勒布更关心一个观点是否经得住反驳，而不是它能解释多少已经发生的事情。

与前三章的关系是：

1. 前三章说明现实只是众多可能结果中的一个。
2. 第四章说明，人们看到这个结果后，又会编出故事，把偶然包装成必然。
3. 于是，随机成功不仅被误认为能力，还会被理论化、专业化。

一句话概括：

随机性制造结果，人类再用故事把这个结果解释成规律。

对交易最直接的启示是：看到一套方法在过去赚钱时，先问它能否通过样本外检验、是否允许被证伪，而不是先接受那套听起来很完整的市场故事。

最不适者的生存：随机性能欺骗进化吗？

第五章通常译作“最不适者的生存：随机性能欺骗进化吗？”。这一章挑战了一个常见想法：能够长期留在市场中的人，一定是最优秀、最适应环境的人。

塔勒布认为，进化和市场竞争只能根据已经发生的历史进行筛选，却看不到其他可能路径。因此，一个策略即使长期来看很危险，只要灾难暂时没有发生，采用它的人就可能持续赚钱、扩大规模，并被当成成功典范。

典型情况是“多数时候小赚，偶尔一次巨亏”的策略：

平时不断赚钱，业绩看起来稳定；

失败概率很低，所以短期记录非常漂亮；

成功吸引更多资金，让风险敞口越来越大；

最终一次罕见事件就可能抹掉全部利润，甚至导致破产。

与此同时，谨慎的交易者因为保留现金、控制杠杆，在繁荣时期收益较低，反而可能被认为能力不足，提前遭到市场或公司的淘汰。于是，在灾难来临之前，市场看起来像是在奖励“最优秀者”，实际奖励的可能只是最适合当前这段幸运行情的人。

本章的核心包括：

幸存不等于适应性强：幸存者可能只是尚未遇到克制自己的环境。

市场会暂时奖励脆弱策略：承担隐藏风险的人，通常在风险兑现前表现最好。

选择机制也会被随机性欺骗：市场只能淘汰已经失败的人，无法提前淘汰本来很可能失败的幸运者。

短期优势可能是长期缺陷：过度适应当前环境，会让人在环境突变时更加脆弱。

评价策略必须包含极端情景：不能只看平均收益和连续盈利，还要看一次罕见事件会造成什么后果。

它承接了前几章的逻辑：不仅个人会把好运当成能力，市场本身也可能通过资金、职位和声誉，把幸运者误选为强者。

一句话概括：

被市场选中的不一定是最有能力的人，也可能只是风险尚未暴露的幸存者。

对交易最直接的启示是：看到长期稳定盈利的策略，尤其要检查它是否通过承担小概率巨亏来换取表面的稳定。

偏态与不对称

第六章通常译作“偏态与不对称”。它讨论一个交易中非常重要的区别：胜率高不等于赚钱，判断正确的次数也不等于决策质量。

关键在于，结果不仅有“发生概率”，还有“发生后的损益幅度”：

$$E[\text{期望收益}] = \sum (\text{结果发生的概率} \times \text{该结果的损益})$$

例如：

一项策略有99%的概率赚1元，1%的概率亏200元。它几乎每天都赚钱，但长期期望仍为负，而且可能一次破产。

另一项策略有90%的概率亏1元，10%的概率赚20元。它多数时候看起来都错，却可能长期盈利。

这就是收益分布的“不对称”：

负偏态策略：经常小赚，偶尔巨亏。胜率很高，账面平稳，但隐藏着灾难性风险。

正偏态策略：经常小亏，偶尔获得巨大收益。胜率很低，却可能具有正的期望收益。

因此，本章强调：

不要把“预测正确率”当成投资水平。

概率必须与结果大小一起考虑。

平均值、中位数和最常见结果可能完全不同。

一次罕见事件，可能决定几年甚至整个职业生涯的总收益。

高胜率策略往往更容易得到赞扬，因为隐藏风险尚未兑现。

低胜率策略则需要足够资金和耐心，才能等到少数关键收益出现。

这一章解释了第五章中那些“暂时最适者”为何显得如此优秀：他们可能采用的是频繁小赚、罕见巨亏的负偏态策略。灾难发生前，业绩记录几乎完美；灾难发生后，过去的成功才显出随机性。

一句话概括：

重要的不是你有多经常正确，而是正确时赚多少、错误时亏多少。

对交易最实用的检查方法是：不要只看胜率和连续盈利天数，还要同时看最大亏损、尾部风险、盈亏比以及在极端行情下能否存活。购买期权也不必然是好策略，出售期权也不必然是坏策略，最终仍取决于价格、概率和潜在损益是否匹配。

归纳法的问题

第七章通常译作“归纳法的问题”。它讨论的是：我们能否根据过去反复发生的事情，推断未来也一定如此？

所谓归纳，就是从有限观察中总结一般规律。例如：

过去一千天市场都很稳定；

因此判断市场以后仍会稳定；

过去从未出现某种极端事件；

因此认为这种事件不会发生。

塔勒布指出，这种推理在充满随机性和极端事件的领域非常危险。观察再多只白天鹅，也不能证明所有天鹅都是白色；但只要发现一只黑天鹅，就足以推翻这个结论。

本章主要受到休谟和波普尔的影响：

休谟的归纳问题：过去的规律会延续到未来，本身只是一个未经证明的假设。

波普尔的证伪思想：理论无法通过成功案例被最终证明，却可能被反例推翻。

没有失败记录，不等于不存在失败风险：它可能只说明观察时间不够长。

罕见事件难以从历史频率估计：如果灾难一百年才发生一次，十年的数据几乎不能说明问题。

经验越成功，有时反而越危险：连续盈利会增加信心，促使人扩大杠杆，使下一次失败更加致命。

这对金融市场尤其重要。一个策略回测十年都有效，可能是因为这十年恰好没有出现能够摧毁它的环境；它证明了策略曾经赚钱，却不能证明策略在未来所有情况下都安全。

这里还需要区分两句话：

“我没有看到风险存在的证据。”

“我看到了风险不存在的证据。”

前者只是缺少资料，后者才是一个更强的结论。人们却经常把前者误当成后者。

第七章也总结了全书第一部分的逻辑：

1. 成功可能来自运气。
2. 已发生的历史只是众多可能路径之一。
3. 市场会把幸运的幸存者误认为强者。
4. 高胜率可能隐藏小概率巨亏。
5. 过去没有发生灾难，无法证明未来不会发生。

一句话概括：

历史可以推翻一种错误认识，却很难保证一种成功经验永远正确。

对交易最直接的启示是：回测结果只能作为证据之一。还要研究策略依赖了哪些市场条件、在未出现过的极端情景中会怎样，以及一次反例是否足以导致永久性亏损。

太多邻家百万富翁

第八章通常译作“太多邻家百万富翁”。这一章进入全书第二部分，重点讨论：当参与者足够多时，纯粹靠运气也必然会产生一些看似不可思议的成功者。

塔勒布借用“猴子打字”的思想实验：如果让足够多的猴子随机敲击键盘，并持续足够长时间，其中某只可能偶然打出一部名著。但我们不能因此认为它具备文学才能。

投资市场也是如此。假设有一万人通过抛硬币决定交易：

第一轮大约有五千人赚钱；

连续五轮后，仍会有约三百人保持全胜；

连续十轮后，可能还有十人左右从未失败。

媒体和投资者最后看到的，正是这十位拥有完美记录的“投资天才”。他们本人也可能真诚地相信，成功来自自己的方法，而不是因为最初存在大量同样下注但已经失败的人。

本章的核心包括：

个体的小概率事件，在庞大群体中可能必然出现：彩票中奖对个人极罕见，但只要参与者足够多，出现中奖者并不奇怪。

不能只研究最终赢家：还要知道最初有多少参与者，以及失败者去了哪里。

业绩记录会受到筛选偏差影响：亏损的基金关闭，成功基金被宣传，于是现存基金的历史业绩显得格外优秀。

连续成功不能单独证明能力：必须判断这种记录靠随机过程产生的概率。

事后选择会制造规律：先查看全部结果，再挑出最漂亮的一组进行解释，很容易把偶然当成发现。

机会数量非常重要：尝试次数越多，出现极端赢家、巧合和“神奇预测”的概率越高。

这一章实际上把“幸存者偏差”量化了。第五章说市场会留下幸运者；第八章进一步说明，只要市场里有足够多的人，幸运者不仅会出现，而且一定会出现得像真正的高手一样。

评价一位连续十年盈利的基金经理时，不能只问“连续十年赚钱有多难”，还要问：

最初有多少基金经理参加竞争？

有多少失败者已经离场？

我们是在事前关注他，还是看到结果后才挑中他？

他的收益是否来自相同的隐藏风险？

换一段独立时期，表现能否继续？

一句话概括：

一个人的成功可能非常罕见，但在足够庞大的样本中，出现这样一个成功者却可能非常普通。

买卖比煎蛋还容易

第九章通常译作“买卖比煎蛋还容易”。它讨论的是：信息和交易过于便利，为什么反而可能损害投资表现。

现代市场中，点击一下就能买卖资产，比做一顿早餐还快。但人的情绪和判断能力并没有同步进化。交易成本和操作门槛降低后，人们更容易把焦虑转化为交易，最终出现过度反应和频繁买卖。

本章一个重要观点是：观察频率越高，看到的噪声越多。

假设某项投资长期具有正收益，但短期波动很大：

每年看一次，长期趋势比较明显；

每月看一次，会看到不少亏损月份；

每天看一次，涨跌可能接近随机；

每分钟查看，看到的几乎全是噪声。

投资本身没有变化，但高频查看会让人经历更多“亏损时刻”。由于人们对亏损的痛苦通常大于同等盈利的快乐，频繁查看账户会带来焦虑，并诱发不必要的操作。

本章还批评了金融新闻的事后解释：

市场每天都在随机波动；

媒体必须持续提供内容；

于是每次涨跌都会被配上一个听起来合理的原因；

这些解释往往无法事前预测，也很难被检验。

因此，本章强调：

更多信息不一定意味着更多知识：高频信息可能只是增加噪声。

能够交易不等于应该交易：操作便利会制造虚假的行动需求。

短期价格变化通常不需要宏大解释：很多波动只是正常随机性。

观察周期应与策略周期匹配：长期策略不应被分钟级波动支配。

减少信息输入也是风险管理：保护自己的情绪，能够防止在噪声中改变正确计划。

新闻故事容易制造控制感：知道市场今天为何上涨的故事，并不代表掌握了未来走势。

它与第八章的关系是：第八章说明大量参与者会随机产生成功者；第九章进一步说明，大量新闻和价格变化也会随机产生看似有意义的解释和信号。

一句话概括：

当观察和交易变得毫不费力时，真正困难的反而是什么都不做。

对交易最直接的启示是：预先确定查看账户、评估策略和调整仓位的频率，只在假设、风险或基本条件真正变化时行动，而不是对每次价格波动作出反应。

输家通吃：生活中的非线性

第十章通常译作“输家通吃：生活中的非线性”。它讨论的是：现实中的损益并不总按比例变化，一个小事件可能造成极大的结果。

在线性世界里，原因扩大一倍，结果也大致扩大一倍。但金融市场和现实生活中存在大量非线性关系：

小幅亏损可能没有影响，但再多亏一点就会触发追加保证金；

普通波动可以承受，超过某个阈值却会导致强制平仓；

连续犯很多小错可能尚可补救，一次重大错误却可能永久退出市场；

一粒沙通常无关紧要，但在临界状态下，最后一粒沙可能引发整个沙堆坍塌。

“输家通吃”是对“赢家通吃”的反转：在一些领域，一次失败可以吞掉此前所有成功。例如，一名交易者连续几年盈利，却因为一次高杠杆亏损而破产。从最终结果看，那一次失败“拿走了全部”。

本章的核心包括：

结果不与原因成正比：小变化越过临界点后，后果可能突然放大。

破产是一种不可逆状态：亏损以后还能恢复与被彻底淘汰，是性质不同的结果。

最终结果可能由少数极端事件决定：大量普通日子的贡献，可能不如一个异常日重要。

平均情况会掩盖临界风险：平均波动不大，不代表系统不会突然崩溃。

路径会影响结果：即使资产最后回到原价，中途被强制平仓的人也无法享受恢复。

生存比提高平均收益更重要：只要存在一次出局的可能，长期复利就可能中断。

它和第六章都在讲不对称，但侧重点不同：

第六章关注收益分布：高胜率可能对应罕见巨亏。

第十章关注反应机制：为什么一个看似不大的变化，会在杠杆、临界点和破产机制下被放大成灾难。

例如，资产下跌10%并不一定只造成10%的影响。对高杠杆账户来说，它可能触发保证金不足、强制卖出和永久损失；之后即使价格反弹，也已经没有资金继续参与。

一句话概括：

在非线性世界中，重要的不是平常能赢多少次，而是有没有一次失败会让你永远无法继续。

对交易最直接的启示是：风险管理不能只控制“平均亏损”，还要检查杠杆、强平线、流动性、相关性突变和极端行情下的最大后果。首要目标不是避免任何亏损，而是避免不可逆的出局。

我们是概率盲

第十一章通常译作“随机性与人脑：我们是概率盲”。这一章讨论：为什么人即使学过概率，面对真实风险时仍会系统性地判断错误。

塔勒布认为，人脑擅长识别具体危险、人物意图和因果故事，却不擅长理解抽象概率，特别是小概率、极端结果和长期重复事件。

我们可以在纸面上算对概率，情绪却仍会按另一套方式反应。

本章涉及的主要认知倾向包括：

偏爱因果故事：相比“这是随机波动”，人们更愿意接受一个明确、具体的解释。

忽视样本大小：从几次成功或失败中，迅速总结出稳定规律。

高估生动事件：刚发生、容易想象的风险会被高估；长期没有出现的风险则会被低估。

损失厌恶：同等金额的亏损带来的痛苦，通常大于盈利带来的快乐。

事后偏见：事情发生后，觉得结果早有征兆、原本就可以预测。

过度自信：把幸运结果归功于能力，把失败归咎于异常情况。

难以理解独立性：连续出现几次相同结果后，错误地认为下一次“应该反转”或趋势“必然继续”。

塔勒布还强调，知道这些偏差并不意味着能够摆脱它们。一个概率专家看到持仓连续亏损时，仍会感到痛苦；一个知道市场充满噪声的交易者，仍可能忍不住查看行情并频繁操作。理性知识和实际情绪反应之间存在距离。

因此，应对概率盲不能只依靠自制力，还需要改变环境和制度：

减少查看短期行情的频率；

事前设定仓位、止损和最大风险；

把决策规则写下来，避免临场修改；

用长期统计而不是最近几次结果评价策略；

确保单次错误不会导致永久出局；

对特别确定、特别容易讲成故事的判断保持警惕。

这一章把前面内容连接到了心理层面：

第九章说高频信息充满噪声；

第十章说一次极端事件可能造成非线性灾难；

第十一章解释为什么人脑恰好会沉迷噪声，同时低估极端风险。

一句话概括：

我们并非天生按照概率思考，而是按照故事、情绪和眼前经验作出判断。

最实用的启示是：不要把风险管理建立在“到时候我会保持理性”之上，而要提前设计规则，使自己即使情绪失控，也无法轻易犯下致命错误。

卡尔内阿德斯来到罗马：论概率与怀疑主义

第十三章通常译作“卡尔内阿德斯来到罗马：论概率与怀疑主义”。它讨论的是：既然知识不可靠、未来不可确定，我们应该怎样思考和行动？

卡尔内阿德斯是古希腊怀疑主义哲学家。相传他访问罗马时，第一天发表演说赞美正义，第二天又以同样强大的论证反驳正义。这件事说明：一套论证听起来有说服力，并不代表它必然为真；优秀的语言和逻辑可以为相反结论服务。

塔勒布借此主张一种“概率式怀疑主义”：

不轻易断言某个理论绝对正确；

根据现有证据，给不同结果分配不同可信度；

新证据出现后，随时调整判断；

即使无法确定真相，也仍然可以作出行动；

决策时不仅考虑“最可能发生什么”，还要考虑“如果判断错了会怎样”。

这不是“什么都不相信”，也不是拒绝行动。彻底怀疑会让人无法生活，而概率思维允许我们表达：

“这个观点目前比较可信，但并非确定。”

“我认为上涨概率较高，但仓位仍要控制。”

“证据支持这个解释，不过相反结果依然可能发生。”

本章的核心包括：

论证能力不等于真实性：漂亮故事和专业表达可能只是修辞。

信念不应只有真与假：现实中更合理的是不同程度的确信。

怀疑应当针对自己的观点：真正的怀疑主义不是只挑别人的错误。

行动不需要绝对确定性：我们可以在不确定中，根据概率和后果作出选择。

观点与风险敞口必须分开：你可以倾向于某个判断，但不应押上全部资产。

保持可修正性：当证据变化时，改变观点不是软弱，而是理性。

对交易而言，最重要的不是准确宣布市场方向，而是构造一种即使观点错误也能承受的仓位。例如，你可以认为某资产有较大概率上涨，但如果下跌会造成不可逆损失，就不应该重仓。反过来，一个发生概率较低但潜在收益极大的机会，也可能值得投入一小笔有限资金。

它承接前面章节的逻辑：

第十一章说人脑不善于处理概率；

第十二章说随机奖励会制造错误信念；

第十三章提出应对办法：对信念保持怀疑，把判断表达为概率，并通过风险控制允许自己犯错。

一句话概括：

理性不是确信自己正确，而是在承认自己可能错误的情况下，仍能作出不会被错误摧毁的决定。

\

巴克斯抛弃安东尼

这一章是全书第一部分的收束，借安东尼命运突然逆转的历史意象，说明：

一个人长期成功，不代表未来会继续成功；

历史结果往往由少数偶然事件决定；

事后人们会把失败解释成性格、战略或道德问题，却忽略当时存在的许多可能路径；

胜利者容易把运气当成能力，失败者也可能只是碰上了不利结果；

财富、地位和声誉都不是永久的，真正重要的是能否在运气转坏时继续生存。

塔勒布最终主张一种带有斯多葛主义色彩的态度：承认自己无法控制随机性，把精力放在控制风险、避免毁灭性损失，以及保持行动上能力上。

一句话概括：

好运离开任何人，所以不要把暂时的成功误认为命运已经站在自己这一边。

结语：梭伦早就告诉过你

如果你的版本把结语编为第十五章，它讲的是古希腊政治家梭伦对吕底亚国王克罗伊斯的提醒：一个人不能在事情尚未结束前，就断言自己是幸福或成功的。

财富和人生都要看完整过程，而不能只看中途的辉煌。真正值得追求的不是短期风光，而是经得住运气变化、能够长期存活的生活和决策方式。

08 实盘与数据（9 篇）

笔记 110: 今天学习了 Polymarket 的结构化套利：核心不是预测事件，而是利用市场之间严格成立的逻辑关系

作者：Wei Huang | 时间：2026-08-12 15:27 UTC | 字数：378 | 评分：49
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天学习了 Polymarket 的结构化套利：核心不是预测事件，而是利用市场之间严格成立的逻辑关系构造“最低兑付 $\geq$ \$1”的组合。设 $(YES(A)=A)$ 、 $(NO(A)=1-A)$ ，如果 $(A \rightarrow B)$ ，则买入 $(NO(A)+YES(B))$ ；如果 $(A \cap B = \emptyset)$ ，则买入 $(NO(A)+NO(B))$ ；如果 $(A \cup B = \Omega)$ ，则买入 $(YES(A)+YES(B))$ 。统一判断标准是：只要组合的保证最低兑付为 \$1，而实际买入成本 + 手续费 + 滑点 + 安全缓冲 $<$ \$1，就存在套利空间。实盘中最重要的是确认 Resolution Rules 下的逻辑关系真的成立，并用订单簿实际成交成本而不是表面价格计算利润。

笔记 111: 今天学习帖子：套利实操笔记 03 | 1200%年化，但最终我却亏损出场 如果你准备做资金费率套利，开

作者：jssyxd | 时间：2026-08-12 15:44 UTC | 字数：1349 | 评分：80
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天学习帖子：套利实操笔记 03 | 1200%年化，但最终我却亏损出场 如果你准备做资金费率套利，开单前先把这5个问题想清楚：[图片] 资费差还能维持多久？不要只看当前年化。先看每个结算周期能收多少bp、两边什么时候结算，以及这个差有没有快速下降的迹象。之前在笔记02中有提到，我是通过历史快照做回测，包括还有盘口买卖盘计算，去预测计算资费反转率。[图片] 价差怎么走，你会亏？不是只防价差扩大。先弄清楚自己多哪边、空哪边，价差扩大还是收窄对你不利，最多能承受多少bp。价差这一部分我监控中也有历史回测，看币种历史价差是否回归是否稳定。[图片] 双边盘口的真实深度够不够？当时直接看到的价格不等于整笔仓位的成交价格。开仓和平仓都要按照真实盘口计算平均成交价。一个简单技巧：在开单前先去看盘口挂单和实际成交情况，同时观察k线振荡情况，订单簿很薄，振幅很大，插针很多，都不太适合开仓，或者开大仓位，开进去后再后悔就晚了。[图片] 扣完全部成本还剩多少？另一边付出的资费、双边开平手续费和滑点，全部换算成bp，再和预计资费收入放在一起算。一般来说，年华1000%以上你至少保证吃个2-3轮再撤，这样风险较小。[图片] 两条腿不能同时成交或平仓怎么办？部分成交、接口延迟、盘口突然消失，都可能让套利单变成单边裸仓。开单前就要想好怎么补仓和退出。还有哦，套利切忌不玩合约，不扛单，扛单后悔一生。为什么先说这5点？因为昨天这笔1200%年化的套利，我至少踩中了前两个。当时两边每小时资费分别是：A：-0.0590% Hyper：-0.2033% 我在Hyper做多，在A做空。Hyper收0.2033%，A端付0.0590%，理论净资费差是0.1443%/小时，也就是14.43bp。直接乘 24×365 ，折合年化1264%。但后面几小时，资费差一直在缩。到了第四个结算周期，折合年化只剩40%左右，我选择平仓。这时候，价格上的亏损也出现了。开仓时价差约66.7bp。平仓时价差只剩40bp。因为我是多Hyper、空A。所以价差收窄的26.7bp全部是亏损。按照双边3.2万U计算，价差部分大约亏了84U。Hyper端虽然收到了80.98U资费，但A端同样需要支付资费，再扣掉价差亏损和双边手续费，最终还是亏损近30U出场。最后再强调一下：这类套利单笔利润很薄，仓位通常不小。多空对冲只能降低方向风险，资费反转、价差扩大、滑点和单腿成交仍可能造成亏损，所以开仓、平仓都要谨慎。

笔记 112: 今天做了什么：D6 数据收集日：①链上直查 Aave V3 以太坊主网 7 天清算事件——仅 3 笔

作者：zenkai | 时间：2026-08-12 15:57 UTC | 字数：476 | 评分：63
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天做了什么：D6 数据收集日：①链上直查 Aave V3 以太坊主网 7 天清算事件——仅 3 笔（平均 0.4 次/天，最大 ~\$6万），判定“日常清算”降级为事件监控项；②LI.FI 实测跨链出场成本（Base 0.25% vs Scroll 0.40%） $\times$ DefiLlama TVL（Base \$47亿 vs Scroll \$1000万 vs Mode \$200万）——验证退出流动性陷阱，新规则“新链 TVL<\$1亿 直接跳过”。

证据在哪：链上 get_logs 原始数据（会话记录 + /tmp/aave_liq.json）；LI.FI quote 实测；DefiLlama v2/chains TVL；卡片~/Desktop/链上套利共学/打卡/D6_20260812_数据收集日.md

卡点：清算数据只覆盖主网平静窗口，暴跌日爆发场景没实测；小链 token 地址查询失败（数据基建成本）。

明天任务（D7）：建成本模型框架（模块4）+ 把“清算事件监控”挂上监控清单，为 W2 回测做准备。

笔记 113: 今天做了dune的base链的视图。能够实时监控和某个DEX合约地址交互的钱包的交易量。以bn聚合器

作者：mOuseliuzi | 时间：2026-08-12 16:18 UTC | 字数：5290 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天做了dune的base链的视图。能够实时监控和某个DEX合约地址交互的钱包的交易量。以bn聚合器0xb300...7028d 为例如果有需要可以参考源码。对于链上交易可能会有帮助。

```
WITH candidates AS (
SELECT
block\_time,
hash AS tx\_hash,
"from" AS trader
FROM base.transactions
WHERE block\_date >= DATE '2026-08-12'
AND block\_date <= DATE '2026-08-19'
AND block\_time >= TIMESTAMP '2026-08-12 13:00:00 UTC'
AND block\_time < LEAST(current\_timestamp, TIMESTAMP '2026-08-19 13:00:00 UTC')
AND success = TRUE
AND "to" = 0xb300000b72DEAEb607a12d5f54773D1C19c7028d
),
raw\_quid\_buys AS (
SELECT
c.block\_time,
c.tx\_hash,
c.trader,
SUM(varbinary\_to\_uint256(l.data) / 1e18) AS quid\_bought
FROM candidates c
INNER JOIN base.logs l
ON l.block\_date = CAST(c.block\_time AS DATE)
AND l.tx\_hash = c.tx\_hash
AND l.contract\_address = 0x1a44233fae8d50f1aeb3a5d58dd426ff4814cb53
AND l.topic0 = 0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef
AND varbinary\_substring(l.topic2, 13, 20) = c.trader
GROUP BY 1, 2, 3
),
finalized\_txs AS (
SELECT
tx\_hash,
SUM(COALESCE(amount\_usd, 0)) AS finalized\_volume\_usd,
ARRAY\_JOIN(ARRAY\_SORT(ARRAY\_DISTINCT(ARRAY\_AGG(project))), ', ') AS dex\_projects
FROM dex.trades
WHERE blockchain = 'base'
```

```

AND block\_month = DATE '2026-08-01'
AND block\_time >= TIMESTAMP '2026-08-12 13:00:00 UTC'
AND block\_time < LEAST(current\_timestamp, TIMESTAMP '2026-08-19 13:00:00 UTC')
AND tx\_to = 0xb300000b72DEAEb607a12d5f54773D1C19c7028d
AND token\_bought\_address = 0x1a44233fae8d50f1aeb3a5d58dd426ff4814cb53
GROUP BY 1
),
price\_minutes AS (
SELECT timestamp, price
FROM prices.minute
WHERE blockchain = 'base'
AND contract\_address = 0x1a44233fae8d50f1aeb3a5d58dd426ff4814cb53
AND timestamp >= TIMESTAMP '2026-08-12 12:00:00 UTC'
AND timestamp < LEAST(current\_timestamp, TIMESTAMP '2026-08-19 13:00:00 UTC')
),
latest\_price AS (
SELECT MAX\_BY(price, timestamp) AS price, MAX(timestamp) AS price\_time
FROM price\_minutes
),
hybrid\_txs AS (
SELECT
r.block\_time,
r.tx\_hash,
r.trader,
r.quid\_bought,
COALESCE(f.finalized\_volume\_usd, r.quid\_bought \ COALESCE(pm.price, lp.price, 0)) AS volume\_usd,
CASE WHEN f.tx\_hash IS NOT NULL THEN 'Dune finalized' ELSE 'Realtime estimate' END AS tx\_data\_status,
f.dex\_projects,
lp.price AS fallback\_quid\_price\_usd,
lp.price\_time AS fallback\_price\_time,
CASE
WHEN r.block\_time < TIMESTAMP '2026-08-13 13:00:00 UTC' THEN 3.5
WHEN r.block\_time < TIMESTAMP '2026-08-14 13:00:00 UTC' THEN 3.0
WHEN r.block\_time < TIMESTAMP '2026-08-15 13:00:00 UTC' THEN 2.5
WHEN r.block\_time < TIMESTAMP '2026-08-16 13:00:00 UTC' THEN 2.0
WHEN r.block\_time < TIMESTAMP '2026-08-17 13:00:00 UTC' THEN 1.8
WHEN r.block\_time < TIMESTAMP '2026-08-18 13:00:00 UTC' THEN 1.3
ELSE 1.0
END AS multiplier,
DATE\_DIFF('day', DATE '2026-08-12', CAST(r.block\_time - INTERVAL '13' HOUR AS DATE)) + 1 AS trading\_day
FROM raw\_quid\_buys r
LEFT JOIN finalized\_txs f ON f.tx\_hash = r.tx\_hash
LEFT JOIN price\_minutes pm ON pm.timestamp = DATE\_TRUNC('minute', r.block\_time)
CROSS JOIN latest\_price lp
),
wallet\_scores AS (

```

```

SELECT
trader,
COUNT(\) AS successful\_buy\_transactions,
SUM(quid\_bought) AS total\_quid\_bought,
SUM(volume\_usd) AS purchase\_volume\_usd,
SUM(volume\_usd \ multiplier) AS points,
COUNT\_IF(tx\_data\_status = 'Dune finalized') AS finalized\_transactions,
COUNT\_IF(tx\_data\_status = 'Realtime estimate') AS realtime\_estimate\_transactions,
CASE
WHEN COUNT\_IF(tx\_data\_status = 'Realtime estimate') = 0 THEN 'Dune finalized'
WHEN COUNT\_IF(tx\_data\_status = 'Dune finalized') = 0 THEN 'Realtime estimate'
ELSE 'Partially finalized'
END AS data\_status,
COUNT(DISTINCT trading\_day) AS active\_competition\_days,
MIN(block\_time) AS first\_buy\_utc,
MAX(block\_time) AS last\_buy\_utc,
ARRAY\_JOIN(ARRAY\_SORT(ARRAY\_DISTINCT(ARRAY\_AGG(dex\_projects) FILTER (WHERE dex\_projects IS NOT NULL))), ' , ') AS
dex\_projects,
SUM(CASE WHEN trading\_day = 1 THEN volume\_usd ELSE 0 END) AS day1\_volume\_usd,
SUM(CASE WHEN trading\_day = 2 THEN volume\_usd ELSE 0 END) AS day2\_volume\_usd,
SUM(CASE WHEN trading\_day = 3 THEN volume\_usd ELSE 0 END) AS day3\_volume\_usd,
SUM(CASE WHEN trading\_day = 4 THEN volume\_usd ELSE 0 END) AS day4\_volume\_usd,
SUM(CASE WHEN trading\_day = 5 THEN volume\_usd ELSE 0 END) AS day5\_volume\_usd,
SUM(CASE WHEN trading\_day = 6 THEN volume\_usd ELSE 0 END) AS day6\_volume\_usd,
SUM(CASE WHEN trading\_day = 7 THEN volume\_usd ELSE 0 END) AS day7\_volume\_usd,
MAX(fallback\_quid\_price\_usd) AS fallback\_quid\_price\_usd,
MAX(fallback\_price\_time) AS fallback\_price\_time
FROM hybrid\_txs
GROUP BY 1
)
SELECT
ROW\_NUMBER() OVER (ORDER BY points DESC, trader) AS rank,
trader,
successful\_buy\_transactions,
total\_quid\_bought,
purchase\_volume\_usd,
points,
data\_status,
finalized\_transactions,
realtime\_estimate\_transactions,
active\_competition\_days,
first\_buy\_utc,
last\_buy\_utc,
dex\_projects,
day1\_volume\_usd,
day2\_volume\_usd,

```

```
day3\_volume\_usd,
day4\_volume\_usd,
day5\_volume\_usd,
day6\_volume\_usd,
day7\_volume\_usd,
fallback\_quid\_price\_usd,
fallback\_price\_time,
'Oxb300000b72deaeb607a12d5f54773d1c19c7028d' AS bn\_router
FROM wallet\_scores
ORDER BY rank
```

笔记 114: Day 9 | 两阶段预筛：用 28 条报价腿替代 48 条完整路径

作者：Courtney Snyder | 时间：2026-08-12 17:12 UTC | 字数：2242 | 评分：90
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 | 两阶段预筛：用 28 条报价腿替代 48 条完整路径

网站进展：截至 8 月 13 日，报名人数仍为 692；活动日历仍显示“暂无活动安排”，每日优秀学习笔记仍只有 8 月 7 日的三篇，今天没有新增官方议程或榜单。因此继续按第二阶段计划，把 Day8 暴露的匿名 API 配额问题转化为可验证的请求预算机制。

今天完成：

1. 实现配额感知的两阶段采样

新增 `quota\_aware\_sampler.py`。每个候选先请求买入链 USDC → WETH 和卖出链 WETH → USDC 两条现货报价，计算保守 `pre\_rebalance\_edge\_bps`；只有接近历史完整成本门槛时，才追加 WETH/USDC 两条库存再平衡报价。历史门槛来自 Day8 权威汇总：CHEAPEST 80.6312 bps、FASTEST 131.5281 bps，预筛额外留 20 bps 漂移余量，因此触发线分别为 60.6312 和 111.5281 bps。

2. 保留校准路径，避免历史门槛过期

每种排序固定保留一个 1,000 USDC、Base 买 → Arbitrum 卖的四腿校准候选，即使现货 Edge 很差也请求完整路径。校准会记录 `historical\_floor\_error\_bps`；这样可以在节省配额的同时检测再平衡成本是否漂移，而不是永远相信旧样本。

3. 增加 429 熔断

若任一候选出现 LI.FI HTTP 429，本次批次立即停止继续请求，后续任务记录为 `quota\_circuit\_breaker\_skipped`，不进行连续重试。输出明确区分 `prefilter\_rejected`、`calibration\_full\_path`、`quote\_failed` 和配额熔断跳过。

4. 补齐测试

新增 `test\_quota\_aware\_sampler.py`，覆盖预筛触发线、校准任务优先级，以及报价腿节省统计。当前全部测试 18/18 通过，所有脚本编译检查通过。仍使用合成地址，只读 LI.FI 报价，不签名、不发送交易。

真实采样时间：2026-08-13 01:07:28 至 01:07:58 (UTC+8)。

范围：CHEAPEST / FASTEST × 100 / 1,000 / 10,000 USDC × Base/Arbitrum 双方向，共 12 个候选。

本轮结果：

12/12 候选完成两腿预筛；10 条因现货 Edge 远低于触发线被提前淘汰，2 条校准候选完成四腿路径。

完整四腿跑法需要 48 条报价腿；今天实际请求 28 条，节省 20 条，即 41.67%。另有 4 次 Token 元数据请求，单独统计，不与报价腿混淆。

28 条报价腿全部成功；HTTP 429 为 0，报价失败为 0，熔断跳过为 0。

所有候选的保守现货 Edge 为 -151.8077 至 -124.9761 bps。它们距离 CHEAPEST 60.6312 bps / FASTEST 111.5281 bps 的预筛触发线非常远，因此没有候选因市场信号进入完整路径。

CHEAPEST 校准候选：现货 Edge -149.3002 bps，完整净 Edge -210.6915 bps，最新最低正 Edge 门槛 81.3913 bps，只比 Day8 历史下限高 0.7601 bps；报价窗口 4.043 秒，最长执行 7 秒。

FASTEST 校准候选：现货 Edge -146.3667 bps，完整净 Edge -258.6739 bps，最新最低正 Edge 门槛 132.3072 bps，只比 Day8 历史下限高 0.7791 bps；报价窗口 3.782 秒，最长执行 2 秒。

两个校准误差都小于预留的 20 bps，说明本轮历史门槛仍足以安全预筛，没有发现因节省请求而漏掉接近可交易的候选。

Paper Trading 判定：0/12 通过。10 条在低成本预筛阶段淘汰；2 条校准路径在完整模型中因净 Edge 低于 20 bps 淘汰。预筛淘汰不是完整模型成交结果，而是明确的“未进入高成本验证”状态。

权威输出：

data/day9/lifi_quota_aware_20260812T170758Z.jsonl

data/day9/lifi_quota_aware_20260812T170758Z.csv

data/day9/lifi_quota_aware_summary_20260812T170758Z.json

今天的核心认识：在当前样本中，现货 Edge 已经比完整门槛低 185 bps

以上，没有必要为每个候选重复支付两条再平衡报价的配额成本。两阶段流程把 API 配额当成工程预算：先用便宜信号淘汰明显不可能的候选，再用少量校准路径证明门槛没有显著漂移。这比限流后盲目重试更适合持续监控。

明日行动：把预筛触发线从“全局历史最低值”细化为 $order \times direction \times amount$ 的滚动分位数，并增加 margin 命中率、校准误差分布和疑似漏检计数；继续保持一次性运行、显式 429 熔断和只读 Paper Trading，不使用真实资金。

笔记 115: Day 9 · 不要把“看得见”误当成“能预知”

作者：99hansling | 时间：2026-08-13 05:21 UTC | 字数：1540 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 · 不要把“看得见”误当成“能预知”

这篇在讲什么

今天继续检查永续合约 funding（资金费）。Funding 是多空双方之间定期支付的费用：价格高于现货时，通常多头付给空头；反过来则空头付给多头。页面上能看到 funding、标记价格和预言机价格。

但今天要解决的不是“哪个币 funding 高”，而是一个更基础的问题：一个此刻看得到的数字，能不能真的在此刻支持下一小时的决策？

碰到的具体问题

此前探索用“未来 48 小时的平均 funding”定义好窗口。它能描述历史机会，却不能作为进场规则：进场时人不可能知道未来 48 小时。这就是 前瞻偏差——把当时还不存在的信息带回过去使用。

所以今天尝试把问题收紧为：能不能只用当下公开的 premium、funding 和结算时间，写出一个不依赖未来信息的下一小时命题？

做了什么

我只读核对了交易所公开规则和接口说明，没有请求市场行情、回测或交易。

| 核对项 | 确认到的事实 | 它不足以证明什么 |

|---|---|---|

| 结算节奏 | Funding 每小时支付 | 现在的快照已经锁定下一小时的实际 payment |

| 费率形成 | premium 每 5 秒采样，在一小时内取平均 | 期中看到的 premium 不会在结算前反转 |

| 公开字段 | 可以看到 funding、标记价、预言机价和未平仓量等上下文 | 字段里的 funding 一定是“下一小时最终费率” |

| 交易成本 | 永续交易有 maker/taker 费率 | 一小时 funding 必然覆盖开仓、平仓和对冲成本 |

关键发现是：下一小时最终 funding 依赖这一整小时的 premium 路径，而 premium 会随订单簿和现货预言机变化。公开文档列出了 funding 字段，却没有定义它对应上一期已实现值、本期暂态值，还是下一期已锁定的最终值。

今天的结论：先不把快照包装成信号

因此，今天没有把它升级成“可验证候选”，而是把它保留在研究待办里。原因不是 funding 永远没有机会，而是触发器缺少关键证据：结算前的字段语义是否足以指向下一小时最终 payment。

这一步看起来像是在放弃一个方向，其实是在保护研究质量。若把结算后的结果倒填成进场条件，历史图会很好看，但那不是可以复现的策略。即使未来确认了字段语义，也还必须扣除：开平仓费、滑点、现货或其他对冲腿的基差、保证金占用、清算风险和成交失败成本；单期 funding 很可能不足以覆盖这些摩擦。

为什么这件事重要

套利研究最危险的错误，不是得到负数，而是得到一个漂亮却不可能在当时做出的正数。每个触发条件都要回答“这个信息在入场那一刻是否真的存在”。不能回答，就不能进入采样，更不能拿来讲收益。

这也让策略真值表更诚实：不同的失败要分开记。旧的“未来 48 小时窗口”因为前瞻偏差不能用；今天的无前瞻版本则因为字段语义未被直接证明而暂缓。两者都不等于关闭整个 funding 方向。

验证

核对了 funding 的小时结算、小时内采样公式、公开上下文字段与交易费率规则

没有调用市场行情接口、没有新建试验、没有进行交易或模拟交易

项目验证通过：root 测试 92/92、EdgeProbe 测试 110/110，类型检查通过

当前重型数据采集仍为 0/1；借贷利差验证卡也仍是未冻结草案

今天的产出不是一个“买卖信号”，而是一条可复查的边界：看得见一个数字，不等于能在当时预知它结算后的含义。

笔记 116: 笔记 1: Polymarket 套利：真正的壁垒不是发现价差，而是数学与执行基础设施\

作者：ingram0529 | 时间：2026-08-13 11:01 UTC | 字数：6534 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

笔记 1: Polymarket 套利：真正的壁垒不是发现价差，而是数学与执行基础设施\

作者：tension-vip | 日期：2026-08-12 | 2134 字 · 评分 89.4 · 套利框架\

一句话核心：Polymarket 量化套利的天花板不是会不会算 $YES+NO=1$ \

的加法，而是能不能系统性发现大量市场之间违反概率与逻辑约束的定价，并活过执行层。\\

认知升级：从单市场加法到跨市场逻辑约束\\

过去理解套利停留在： $YES = \$0.62$ 、 $NO = \$0.33$ ，两边加起来 $\$0.95$ ，花 $\$0.95$ 买下两边最终兑付 $\$1$ ，理论利润 $\$0.05$ 。\\

真正的量化套利，核心根本不是会不会算这个加法，而是能不能系统性发现大量市场之间违反概率与逻辑约束的定价。套利\\ 机会不只存在于一个市场内部。\\

例：A 为特朗普赢得宾夕法尼亚，B 为共和党在宾夕法尼亚领先超过 5%。若 B 成立则 A 必然成立，所以理论上必须满足 $P(B) \leq P(A)$ 。一旦市场价格违反这个逻辑关系，就可能出现跨市场套利。\\

从暴力枚举到整数规划与 Frank-Wolfe\\

当市场数量扩大，可能结果会指数级爆炸：63 个二元事件就对应 $2^{63} \approx 9.22 \times 10^{18}$ 种结果，靠暴力枚举根本不可行。\\

解决方法进入真正的量化领域：Integer Programming（整数规划）。不是检查所有可能结果，而是把什么结果是合法的写成\\ 数学约束，让求解器直接寻找违反约束的价格。\\

找到错价只是第一步，还要回答买什么、买多少、理论上能赚多少。这里会进入 Marginal Polytope、Bregman/KL\\

Divergence、Bregman Projection、Frank-Wolfe Algorithm、Integer Programming Solver。\\

Frank-Wolfe 不枚举整个巨大结果空间，而是不断让整数规划求解器寻找新的有效方向，一轮一轮逼近最优解——本质是从发\\ 现一个价差，升级到求解整个概率系统中距离无套利状态最近的路径。\\

真正杀死策略的是执行层\\

数学上套利润 > 0 不等于实盘利润 > 0 。Polymarket 是 CLOB 订单簿交易：你可能先成交 YES，等准备成交 NO\\ 时价格已经变化。\\

原本 $\$0.30 + \$0.30 = \$0.60$ ，最后可能变成 $\$0.30 + \$0.78 = \$1.08$ ——所谓无风险套利瞬间变成裸露方向风险。

真正的系统必须继续处理：订单簿深度、VWAP、滑点、部分成交、延迟、流动性、仓位管理。Polymarket \\

甚至提供订单簿和基于深度估计成交价格的接口，说明看到的报价和真正能成交的价格完全是两个概念。

官方

API

AI 的正确位置：模糊世界与确定世界分工\\

先用 LLM 判断不同预测市场之间是否存在逻辑依赖，再交给确定性的数学系统验证。\\

链路：LLM 理解语义、发现候选关系 → 规则/数学模型验证逻辑 → 优化器寻找最优交易 → 订单簿模拟验证真实利润 → \\ 执行系统下单。\\

这比让 AI 自己看盘然后决定买什么靠谱得多：AI 负责处理模糊世界，数学负责处理确定世界。\\

关键数字与风险清单\\

相关研究估算，Polymarket 历史数据已被套利者实际提取的利润约为 4000 万美元；但 Alpha\\

一旦公开，真正的竞争优势会迅速从知道策略，迁移到数据、模型、工程、延迟、风控和执行质量。\\

风险一：跨市场逻辑套利的前提是两份市场规则严格蕴含，规则里一个 $\geq$ 与 $>$ 的差异就废掉整个约束。\\

风险二：数学解出正 edge 不等于实盘可执行，腿间价格漂移、部分成交、深度不足都会吃掉利润。\\

— 4 —\\

风险三：完整 Frank-Wolfe + Gurobi 级系统工程量极大，个人研究者不宜一上来硬啃，更适合先做只读 Arbitrage Scanner\\ 验证机会存在。\\

对工程实践的启示\\

与我们在 Polymarket 上做过的反转、中价位、趋势 bot 完全同构：读公开订单簿 → 找最简单的概率约束错误 → \\

加入订单簿深度和 VWAP → 记录理论套利与真实可执行套利 → Paper Trading → 长时间统计 → 再判断有没有真正的 edge。\\

先证明机会存在，再写交易系统；先验证统计优势，再投入真钱——这正是我们 paper → live 的验证链。\\

贰·证据与反证 Day 6：推翻完整集主线，建立证据型共学仓库\\

笔记 2: Day 6：推翻完整集主线，建立证据型共学仓库\\

作者：Helios | 日期：2026-08-10 | 3340 字 · 评分 89.3 · 实盘与数据\\

一句话核心：YES/NO 是同一物理挂单的互补镜像，完整集不是 alpha\\

来源，而是必须做数据质量控制的恒等式；真正的相对价值主线转向 PM-Deribit。\\

盘口镜像：完整集主线的反证\\

复核完整集盘口关系，发现 YES/NO 是同一物理挂单的互补镜像，而不是两个独立订单簿：YES_bid(P,S) ↔ NO_ask(1-P,S)，YES_ask(P,S) ↔ NO_bid(1-P,S)。

因此 YES_ask + NO_ask = 1 + spread ≥ 1，YES_bid + NO_bid = 1 - spread ≤ 1——页面上的完整集毛差在 ask 侧天然不可能出现。

实证：10,000 行快照的一至三档镜像检验，对齐率约 96.49% 至 98.27%，平均价格误差约 0.00003；主要异常来自两次 API 快照的时点 jitter。把 raw book 和 mirrored counterpart 相加会产生 2 倍虚假深度。

反推结论：昨天的 net_edge 公式本身描述了完整集支付（每份 1 pUSD 可 split 为 1 YES + 1 NO，等量 YES + NO 可 merge 回；50/50 结算两边各兑 0.5），但市场中会常规出现可扫描 alpha 的经验假设不成立——表面正 edge 必须先标记为数据质量异常。

新类型地图：按支付证明与真实属性分类

同 condition 完整集：支付恒等式 + CLOB 硬镜像，只作数据质量控制，不作主 alpha。

Negative Risk 完备篮子/转换：全腿成交后最接近原生结构锁定，作为第二主实验。

PM-Deribit/Binance 复制与参考对冲：模型和 basis 驱动的相对价值，成为新主线。

逻辑蕴含/时间嵌套/重复市场：依赖独立规则完全保持关系，作后期规则卡。

跨平台等价合约：规则、资金、结算、平台风险叠加，本期不接执行。

UMA/规则分歧是信息交易，做市返佣/流动性奖励是库存与逆向选择策略，collateral 只是资本效率——这些都不称套利。

return

新主线 S1/S2：PM-Deribit 相对价值

s1_exec_div = p_above_k_poly - pm_yes_ask；s2_exec_div = pm_yes_bid - p_above_k_poly。

S1 为买 PM YES、short K1/long K2；S2 为买 PM NO、long K1/short K2。Gross EV = PM EV + DR EV；Net EV = Gross EV - PM open fee - DR open fee - PM close cost。

p_above_k_poly 是风险中性模型概率，因此这不是无风险套利，而是模型和 basis 驱动的相对价值——PM 与 Deribit 的价格源和到期时点并不完全一致，S1/S2 有 basis/gap risk。

— 5 —

验证漏斗与净口径

验证顺序：mv_pm_dr_divergence（机会总体）→ evresponse（signal-time）→ v_positions entry/JIT → reconcile/executionfill → v_positions canonical net PnL。

主结果使用 Net EV Capture = $\Sigma(\text{eligible realized_pnl_net_canonical}) / \Sigma(\text{eligible entry ev_model_usd})$ 。不能平均单仓 ratio，也不能用遗留 gross/raw capture 代替净口径。

MV 是小时级历史分析面，不代表当前可成交；历史落仓存在选择偏差，必须报告机会总体到 CLOSED 的每层分母。

金句：今天最有价值的结果不是增加一个策略，而是推翻了错误主线。完整集只保留为不变量控制。

风险清单

完整集在 CLOB 上是硬镜像，任何 YES+NO 毛差先怀疑数据质量问题（快照 jitter、价格源回退），别当 alpha。

PM 与 Deribit 价格源、到期时点不完全一致，有 basis/gap risk。

MV 是小时级历史分析面，不代表当前可成交。

历史落仓存在选择偏差，必须报告机会总体到 CLOSED 的每层分母。

对工程实践的启示

与我们

Polymarket

实战高度相关：反转/趋势

bot

盯的是盘口谁在挂单，而这篇证明

YES/NO

本是同一物理订单的镜像，任何两边都挂的报价必须先镜像去重，否则 2 倍虚假深度会污染流动性统计。

把表面正 edge 先标记为数据质量异常这条纪律，对应我们在 gamma-api 结算、settle_window 上踩过的坑。

S1/S2

前先为阈值市场填写

Contract

Alignment

Card（核对阈值、≥/

\>、观察时点、时区、价格源、异常结算) 再看价格——正是先审后修的映射。\
叁·多结果盘口实证 Polymarket 套利验证: overround\
数学化简与手续费反推\
笔记 3: Polymarket 套利验证: overround 数学化简与手续费反推\
作者: youchen-design | 日期: 2026-08-08 | 1581 字 · 评分 85.8 · 做市与LP\
一句话核心: 多结果 NegRisk 盘口的套利利润 = 所有 K 个结果脱手 YES 价格总和 1, 与选哪个子集无关; taker 死于 1.8% 手续费均衡, maker 是唯一活路 (但缺腿即裸方向, 是做市不是套利)。\
数学化简: 利润与子集选择无关\
从 SDK 文件验证转换规则: 投入 n 张 NO → 拿回 (n1) 抵押品 + 每个剩余结果各一张 YES。代入后整条式子塌缩成: 利润 = \
(所有 K 个结果实际脱手 YES 的价格总和) 1。\
与选哪个子集无关。子集只影响本金占用和腿数, 真正在收的是多结果盘口的\
overround (贴水)。推文花大篇幅讲的寻找最赚钱的子集不创造利润——子集搜寻器是在优化一个不影响损益的东西。\
手续费公式: 用 21 万笔真实成交腿反推\
 $fee = 1.8\% \times \min(p, 1p) \times shares$ 。只有 taker 付, maker 恒为 0。拟合变异系数 0.044, 很干净; maker 的 110,050\
笔腿费用合计 \$0。\
踩坑记录: fee_type=SHARES 的字段单位是股不是美元, 没换算会把低价位费率灌到 66%, 修正后是平的 1.8%。\
taker 版实测为负: 这就是均衡\
— 6 —\
453 个 NegRisk 事件里只有 10 个 $\Sigma bid(YES) > 1$, 而这 10 个的手续费全部大于毛利。\
afc-west (K=4): 毛利 +0.0160 / 手续费 0.0183 / 净 0.0023; lol-worlds (K=6): +0.0090 / 0.0127 / 0.0037; fed-sept\
(K=5): +0.0070 / 0.0133 / 0.0063; fed-cuts (K=13): +0.0050 / 0.0059 / 0.0009。\
盘口刚好被套利到手续费边界——这不是巧合, 这就是均衡。论文研究期间 Polymarket 收 0 手续费、predict.fun 收\
200bps, 这个差异就是走不通的全部原因。\
maker 版还活着: 但那是做市不是套利\
 $\Sigma ask(YES)$ 1 约 0.055~0.081, 即每轮 0.5%~1.6%、免手续费。流量是真的 (fed-cuts 名目 \$933k、polymarket-migration\
\$535k, 成交都在数小时内) ——没人交易这个否认不成立。\
代价: 要在 K 条腿同时被动成交, 缺腿就是裸方向部位。这是做市, 不是套利。\
动手前必须先测的一个数字: 转换费\
SDK 原文: 转换的 fees deducted from both outputs——转换本身可能有费, 若存在会吃掉整个 0.5%~1.6%。这是唯一无法\
在不实际下单的情况下验证的变量, 必须用小额实单测: 在写任何监控工具之前先测, 因为它可能直接归零整个结论。\
补充: makerRebateBps 只有 Up/Down tag 有 2500bps, NegRisk 市场没有, 帮不上忙。\
风险清单与对工程实践的启示\
风险一: 手续费参数一变整个结论失效——费率必须按当前市场实时读取, 不能沿用历史固定值。\
风险二: 转换费未实测前, 0.5%~1.6% 的 maker edge 是纸面活还是真活未知。\
风险三: 多腿被动成交 = 裸方向风险, K 越大越难同时成交; 低概率腿的 $\min(p, 1p)$ 让 taker\
费率看似便宜, 但绝对毛利也小, 容易误判。\
启示: 这篇几乎是我们 Polymarket 经验的镜像——我们 paper_mm/LP 做市赚的就是 maker 0 费优势, taker\
全被手续费吃掉、maker 是唯一活路正是我们反转 bot 与做市 bot 的分水岭; 先测转换费再写监控, 对应我们先 dry-run\
验证逻辑再上实盘的部署习惯。 $fee = 1.8\% \times \min(p, 1p) \times shares$ 这条公式可以直接进我们的费率参数表做交叉验证。\
结·三篇串联 从发现到验证到执行的完整漏斗\
笔记 4: 从发现到验证到执行的完整漏斗\
作者: 综合整理 | 日期: 2026-08-13 | 三篇主线对照\
一句话核心: Polymarket 的套利没有免费午餐——数学能化简出 edge, 但 edge\
要过三关: 约束是否真的成立、盘口是否真能成交、手续费结构是否站在你这边。\
三篇串联主线\
层面一·数学与执行 — tension-vip: 套利 = 系统性发现概率/逻辑约束违例 (整数规划 + Frank-Wolfe), 但真正分生死的是\
CLOB 执行层。\
层面二·证据与反证 — Helios: YES/NO 是同一物理订单的镜像, 完整集无 alpha; 真正的 edge 在 PM-Deribit 相对价值与\
Net EV 漏斗。\
层面三·多结果实证 — youchen-design: NegRisk 利润 = Σ 脱手 YES 价 1; taker 死于 1.8% 手续费均衡, maker\
是唯一活路 (但缺腿 = 裸方向, 是做市不是套利)。\
贯穿一句话\
— 7 —\
三篇合起来恰好是从发现到验证到执行的完整漏斗, 每一层都在把看起来能赚变成算完才是: 数学负责化简与约束, 证据负\
责戳破幻觉, 手续费结构负责裁决谁活谁死。

笔记 117: 第九天学习记录：今天把 Robinhood Chain 的 Local-Lite 观察链路和自有执行

作者：yu | 时间：2026-08-13 12:09 UTC | 字数：473 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第九天学习记录：今天把 Robinhood Chain 的 Local-Lite 观察链路和自有执行器方案进一步收敛到可审计的代码基线。重点补齐单个已验签 Feed reader 对同一事件并发多 Lane、Feed 块号与哈希精确身份、锁内记录真实 B_next_feed_rx、state_ready 终态语义、包含 dropped/timeout/429 的全分母 capability，以及 readiness digest 不匹配时 fail-close。合并复验的全量测试为 270/270 通过，说明 P0 口径和离线代码基线已闭合；但真实短时 smoke 与 1000 事件 L1-L2 证据仍未补，不能宣称单块观察能力已达标。今天也明确公共 RPC 活窗的经济性继续 No-Go，A1 仅保留 BENCHMARK_ONLY，自有资金 readiness 仍为 NOT_READY；全程未连接钱包、未签名、未广播、未实盘。下一步只做更近状态源的独立只读方案和真实事件能力测量，先证明时效、完整成本与可重复净正收益，再讨论任何执行。

笔记 118: 根据自身能力选择验证层级——套利研究学习笔记

作者：OxPrint | 时间：2026-08-13 12:22 UTC | 字数：4350 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

根据自身能力选择验证层级——套利研究学习笔记

1. 三个层级

一个套利想法出现后，最重要的决策是：

当前证据和执行能力，允许我走到哪一步？

可以划分成三个层级：

继续观察

↓

模拟验证

↓

严格受控的小规模真实实施

每一级都对应不同的证据要求、技术要求和资金风险。

2. 第一层：继续观察

适合这些情况：

只看到过一两次机会

还不知道价差为什么产生

没有足够历史数据

无法确认机会是否重复

不清楚真实可成交规模

两端资产、链、合约映射还没有完全确认

执行路径还比较复杂

失败后不知道如何处理仓位

此时最好的动作是继续采样。

例如发现：

Binance HFT

比 DEX HFT

长期高 3%

不要马上交易。

先采集：

timestamp

Binance bid/ask

DEX buy/sell quote

spread

金额

DEX liquidity

CEX depth

Gas

funding

连续记录几天。

3. 观察阶段需要回答什么

至少回答六个问题：

机会多久出现一次？

例如：

每天 20 次

每周 2 次

一个月只有 1 次

频率直接决定策略价值。

每次持续多久？

100ms

5 秒

3 分钟

2 小时

如果平均只有 200ms，而你的执行系统需要 2 秒，这个 Edge 对你没有实际意义。

最大规模多少？

例如：

100 USDT：4%

1,000 USDT：3.8%

10,000 USDT：1.6%

100,000 USDT：0.2%

策略容量可能只有几千美元。

为什么存在？

要寻找：

流动性碎片化

新币上市

CEX/DEX资金隔离

跨链困难

提现关闭

做市商库存

市场参与者差异

为什么会消失？

例如：

套利机器人进入

做市商补库存

跨链完成

大户交易

交易所恢复提现

自己能不能执行？

别人能赚，不代表自己的：

API

服务器

资金

账户

钱包

RPC

延迟

也能做到。

4. 什么情况下进入模拟验证

当观察已经证明：

机会重复出现

+

逻辑基本解释得通

+

有足够交易规模

+

持续时间允许自己反应

就进入模拟。

这里开始把“研究”写成机器规则。

例如：

IF

Binance_bid / DEX_buy_price - 1 > 3%

AND

可成交金额 > 5000 USDT

AND

报价年龄 < 1 秒

THEN

生成套利信号

真正测试：

如果过去一个月严格按照这条规则执行，结果怎么样？

5. 模拟验证的重点

模拟阶段重点测试四件事。

信号质量

统计：

发现多少信号

多少是假信号

多少最终收敛

执行能力

模拟：

下单

部分成交

取消

对冲

平仓

风险

模拟最坏情况：

第一腿成交

第二腿失败

以及：

API 断线

RPC 超时

报价失效

价差突然扩大

系统稳定性

连续运行：

24 小时

72 小时

7 天

确认不会：

重复开仓

丢失订单

仓位记录错误

网络重连后重复发送交易

使用旧报价

6. Paper Trading 的合格标准

没有统一数字，但可以给自己建立准入门槛。

例如：

有效样本 ≥ 100

同时：

系统连续运行 ≥ 7 天

并且：

无严重状态机错误

无重复订单

无未知裸仓

策略结果至少表现出：

正期望值

可接受回撤

可接受失败率

这时候才值得考虑真实资金。

7. 第三层：严格受控的真实实施

进入真实资金阶段后，目标发生变化。

模拟已经回答：

理论上能不能赚钱？

小规模实盘回答：

真实世界是不是和模型一样？

主要验证：

真实滑点

真实成交速度

真实 Gas

真实 API 延迟

真实部分成交

真实交易失败率

8. 第一次真实实施应该非常小

假设以后正常策略规模：

10,000 USDT

首轮可以测试：

50 USDT

100 USDT

200 USDT

这里追求的是获取真实执行数据。

例如：

模型预测：

价差 3.2%

实际：

3.0%

模型预计：

500ms 完成对冲

实际：

1.7 秒

这些差异比首笔赚多少钱更重要。

9. 逐级放大仓位

不要从：

\$100

直接跳：

\$10,000

可以采用：

\$50

↓

\$100

↓

\$250

↓

\$500

↓

\$1,000

↓

\$2,500

↓

\$5,000

每一级都重新观察：

滑点

成交率

利润率

执行时间

风险敞口

因为策略规模增加后，市场环境也会改变。

10. 实盘必须设硬边界

至少包括：

单笔限制

MAX_TRADE_SIZE

总仓位限制

MAX_TOTAL_EXPOSURE

每日亏损限制

MAX_DAILY_LOSS

例如：

当天累计亏损达到 300 USDT

→ 自动停止交易

连续失败熔断

例如：

连续 3 次执行失败

→ PAUSED

最大裸仓时间

例如：

第一腿完成后

3 秒仍无法对冲

→ 紧急处理

紧急停止

任何时候：

KILL SWITCH

可以阻止新增订单。

11. 根据自身能力判断

可以把能力拆成五个维度。

| 能力 | 问题 |

| :--- | :----- |

| 研究能力 | 我能解释 Edge 为什么存在吗？ |

| 数据能力 | 我有足够数据证明它吗？ |

| 技术能力 | 我的系统能稳定执行吗？ |

| 风控能力 | 出问题后我知道怎么办吗？ |

| 资金能力 | 这次失败造成的损失能接受吗？ |

只要其中关键项明显不足，就降低一个层级。

例如：

研究清楚

+

策略有效

+

代码还不稳定

继续 Paper Trading。

或者：

代码稳定

+

真实成交数据很少

继续小金额实盘。

12. 建立“准入门”

以后每个策略都通过 Gate 决定升级。

Gate 1：观察 → 模拟

要求：

逻辑成立

有重复样本

有明确规则

Gate 2：模拟 → 小额实盘

要求：

历史验证通过

Paper Trading 通过

故障测试通过

风控完成

Gate 3：小额 → 正式实盘

要求：

真实执行数据足够

实际 Edge 为正

执行损耗可接受

系统稳定

Gate 4：增加仓位

要求：

容量允许

回撤允许

最近 Edge 没有明显衰减

这样“是否加仓”就变成一个数据决策。

13. 什么时候应该退回观察阶段

策略进入实盘以后，也可以降级。

例如：

实际利润连续下降
信号数量突然减少
失败率明显增加
滑点持续上升
平均价差快速缩小
市场规则发生变化
协议升级
交易所修改手续费

状态可以从：

LIVE

降到：

PAPER

甚至：

RESEARCH

策略生命周期应该允许升级和降级。

14. 一个实用的策略状态机

OBSERVE

↓

RESEARCH

↓

BACKTEST

↓

PAPER

↓

LIVE_SMALL

↓

LIVE

同时允许：

LIVE

↓

DEGRADED

↓

PAPER

或者：

LIVE

↓

PAUSED

↓

RESEARCH

最终可能：

RETIRED

这能避免一个已经失去 Edge 的策略继续机械运行。

15. 对套利最重要的一条原则

资金规模应该跟证据强度同步增加。

可以理解成：

证据少

→ 0 资金

初步证据

→ 模拟资金

较强证据

→ 极小真实资金

持续验证
→ 小规模资金
长期稳定
→ 正常策略仓位
这样每增加一级风险，都先获得一级新的证据。

09 风险与安全 （4 篇）

笔记 119: 链上套利残酷共学打卡 - 2026-08-12 (Day 8)

作者：KuTear | 时间：2026-08-12 15:01 UTC | 字数：1453 | 评分：72
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学打卡 - 2026-08-12 (Day 8)

今日主题：高性能链上 MEV 拓扑逆向、原子执行保护与交易取证方法论

一、跨池定价严重错配与多跳套利拓扑结构

1. 多跳流动性错配识别与路径闭环：

- 拓扑结构与路由生成：深入逆向高性能链（如 Solana）上的典型跨池套利路径。套利者以基准稳定资产为起点，通过第一跳高流动性成熟集中流动性池（CLMM/Whirlpool）以公允市价兑换目标中介资产；随后利用第二跳长尾/新兴 DEX 中因订单流滞后或流动性稀薄产生的严重错配池完成逆向兑换，实现超额资产提取并闭环回流至初始稳定资产。
- 流动性池状态失衡机理：分析了孤立池在缺乏活跃套利机器人套利或做市商提供深度时，易受单边大单冲击导致资产定价发生数量级偏离的微观市场结构。

二、原子化执行架构与 MEV Bundle 提取保护

1. 区块级原子保护机制：

- Jito Bundle 确定性打包：在非连续撮合或高性能公链环境下，采用 Bundle 机制将套利交易与小费（Tip）指令原子绑定，确保交易要么整体成功并在目标区块槽位精确执行，要么彻底回滚，杜绝因链上状态抢跑（Frontrun）导致的滑点亏损与 Gas/手续费摩擦损失。
- 动态分润与竞价博弈：研究了 MEV 提取过程中的 Tip 竞价模型。在面对高额确定性 Alpha 时，通过向验证节点动态让渡阶梯比例的利润（动态 Tip + 分润指令），在保证自身净利润最大化的同时，锁定极高的交易包含率（Inclusion Rate）。

2. 轻量化执行与底层聚合路由复用：

- 套利执行层可无需构建庞大复杂的定制化借贷/闪电贷合约，而是直接基于轻量级路由指令（如组合 DEX 聚合器底层 CPI 调用），大幅降低字节码开销与计算单元（CU）消耗，提升执行速度。

三、链上交易取证与反编译工程方法论

1. 资金流与现金流精细化锁定：

- 确立了标准化链上取证流程：优先通过账户变更快照（Pre/Post Token Balances）核算各参与实体的绝对净变化量，精准剥离中间多跳流转干扰，秒级锁定净盈利与协议扣费。

2. 底层 Instruction 调用栈还原：

- 深入解析 Inner Instructions 嵌套调用关系，精准重构涉及的多方 DEX 程序 ID、流动性池账户映射及授权转移路径。

3. 资产真伪与流动性陷阱鉴权：

- 针对长尾池套利，建立了代币 Mint 地址与元数据签名强校验机制，防范伪造同名代币（Fake Token）与貔貅池（Honeypot）流动性钓鱼陷阱。

四、生产级系统演进与风控思考

1. CU 计算单元与 Gas 消耗边界优化：

- 在高并发套利场景下，精确设定 Compute Budget limit 与 Unit Price，结合预估滑点设置严格的跨池最小获得量（Minimum Amount Out），防止状态滑移导致资产折损。

2. 私有交易流（Private RPC / Bundle Relay）通道建设：

- 依赖公共 RPC 广播容易在网络层被监听和反向复制，必须建立专线 RPC 节点直连验证者集群，确保套利信号从生成到上链的全链路低延迟与隐私安全。

笔记 120: 01.8 原始证据、聚合工具与交叉验证

作者：petree1 | 时间：2026-08-12 15:56 UTC | 字数：19234 | 评分：90
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

01.8 原始证据、聚合工具与交叉验证

1. 本节解决什么问题

扫链时，你会不断看到平台给出的现成结论：

聪明钱正在买入
某地址属于内部钱包
前十大地址持仓集中
某钱包胜率很高
这笔交易完成了买入
某代币风险较高

这些信息有些直接来自链上字段，有些来自计算，有些来自地址标签，还有一些只是平台信号。

本节帮助你回答一个核心问题：

平台给出一个结论后，怎样判断它属于哪一层证据，并回到可复现的链上数据完成验证。

1.1 本节主问题链

平台提出什么主张

- 主张包含哪些子结论
- 每个子结论来自原始数据、解码、计算、标签还是信号
- 需要回到哪些交易、日志、调用或状态
- 不同来源是否使用同一网络、区块、时间与口径
- 哪些结论已经确认
- 哪些仍是推断、线索或未知

1.2 第一遍必须建立的最低模型

第一次阅读先掌握五点：

1. 工具名称不能直接决定证据等级，要看当前结论具体依赖什么数据。
2. 越接近基础链上与执行证据，通常越容易复现，但仍要固定网络、区块和查询条件。
3. 解码、计算和标签都可能有价值，同时也会引入 ABI、公式、过滤条件和归因假设。
4. 一句话平台提示经常混合多个证据层，需要拆开验证。
5. 最终报告要分别记录事实、计算结果、身份推断、风险判断和未知项。

2. 从一条“聪明钱买入”提示开始

假设 GMGN 显示：

聪明钱地址 A 买入代币 T
该地址近三十天胜率为 80%

初学者容易把整句话当成一个已经确认的链上事实。实际需要拆成至少五个问题：

1. 地址 A 是否真的发起或参与了目标交易。
2. 目标交易是否真的让地址 A 获得代币 T。
3. “买入”依据交易路径、资产变化还是平台分类。
4. “聪明钱”标签依据什么规则产生。
5. 80% 胜率采用什么时间范围、成本口径和平仓定义。

前两项可以回到交易、调用、事件和余额状态。后三项需要继续核对平台算法、统计口径和标签来源。

2.1 人话模型

可以把平台结论理解成一份已经加工过的研究摘要：

链上发生的动作

- 工具识别动作
- 工具计算结果
- 工具添加地址身份
- 工具生成提示或排序

加工越多，阅读效率通常越高。每增加一层加工，也会增加新的假设和误差来源。

3. 五层信息加工链

3.1 第一层：基础链上与执行证据

这一层需要再分成两类。

第一类是链上原生对象：

1. 区块及区块哈希。
2. 交易原始字段。
3. 交易回执和日志。
4. 固定区块的余额、代码和 Storage。

第二类是节点重建的执行证据：

1. 调用级 Trace。
2. Opcode 级执行轨迹。

区块中没有原生保存一份调用树文件。节点使用 `debug_traceTransaction` 和 `callTracer` 等能力生成历史 Trace 时，通常需要根据历史状态重执行交易。调用级 Trace 已经对底层执行进行了聚合，Opcode Trace 更接近逐步执行细节。

Trace 对还原内部调用非常重要，同时需要记录客户端、Tracer、历史状态能力和查询条件。

3.2 第二层：解码结果

原始 Input 是一段字节数据。一个 Log 由合约地址、topics[] 和 data 组成，其中 Topics 和 Data 承载编码后的字节数据。工具结合 ABI 后，可以显示：

0xa9059cbb

→ transfer

原始参数字节

→ recipient 与 amount

Topic0

→ Transfer Event

解码让数据可读，也引入 ABI 是否正确、代理实现是否匹配和签名是否冲突等问题。

3.3 第三层：计算结果

这一层把多条原始事实组合成指标，例如：

1. 钱包盈亏。
2. 胜率。
3. 持仓集中度。
4. 代币价格和美元价值。
5. 净流入和净流出。
6. 地址之间的累计资金量。

计算结果需要同时记录公式、时间范围、数据覆盖和过滤规则。

3.4 第四层：身份与关系标签

常见标签包括：

聪明钱

狙击手

内部钱包

交易所

做市商

项目方

同一实体

链上地址本身通常不会声明这些身份。标签可能来自公开声明、充值地址模式、资金路径、行为特征、平台算法或人工研究。

3.5 第五层：信号与判断

这一层包括：

买入提醒

风险分数

热门排名

异常流入

建议关注

疑似关联

信号适合发现线索。交易决策仍要继续检查原始事实、计算口径、身份依据、流动性和风险边界。

3.6 五层关系图

基础链上与执行证据

↓ ABI 与页面解释

解码结果

↓ 公式与过滤条件

计算结果

↓ 公开资料、行为与归因规则

身份和关系标签

↓ 阈值、排名与策略条件

信号和判断

4. 证据等级属于主张

五层信息加工链描述信息经过了多少加工。A、B、C、D、U

描述某个具体主张具有什么证据基础。两套分类回答的问题不同，不能一一对应。

同一个平台页面可以同时包含多个等级的内容。因此，证据等级应绑定具体主张。

| 等级 | 当前课程中的使用方式 |

| :- | :----- |

| A 级 | 可以通过固定网络、区块和原始链上数据直接复现。 |

| B 级 | 由多条 A 级事实计算得到，并能检查公式、范围、过滤条件和原始样本。 |

| C 级 | 身份或关联推断获得多类独立证据支持。 |

| D 级 | 单个平台标签、社区说法或尚未复核的线索。 |

| U 级 | 数据不足、方法未知或当前无法复现。 |

4.1 同一页面可以混合多个等级

以 BscScan 交易页为例：

1. BscScan 页面显示本身属于工具展示。当交易字段、Receipt 和 Logs 能通过 RPC 或其他原始查询复现时，它们所支持的具体链上主张可以记为 A 级。
2. 方法名和参数名属于 ABI 解码结果，需要核对 ABI 与原始字节。
3. Transaction Action 属于页面摘要，需要回到 Input、Trace、Logs 和 State。
4. 地址名称标签属于身份线索，需要核对标签来源。

以 Dune 查询为例：

1. 表来源明确、SQL 与公式可以复现、时间范围与刷新状态明确，并且已经抽样回到原始交易、日志或状态核实时，相应计算主张可以记为 B 级。
2. 查询使用的底层表、过滤条件或口径不透明时，结论等级需要降低。

4.2 工具权威性不能替代主张复核

下面两句话需要分别处理：

Arkham 把地址 A 标为交易所

地址 A 确实由该交易所控制

第一句可以确认平台展示事实。第二句属于现实身份归因，需要公开声明、充值提现路径、地址复用和其他独立证据支持。

5. 重要工具在证据链中的位置

本节只建立工具与证据层的映射。完整操作保留在后续对应章节。

| 工具或工具类型 | 本节关注的输出与复核入口 |

| :----- | :----- |

| RPC | 读取交易、回执、日志和固定区块状态。记录网络、方法、区块参数和 Endpoint 能力。 |

| BscScan | 快速查看交易、地址、合约和事件。页面摘要与标签需要回到原始字段和日志。 |

| Phalcon | 展开调用树、调用输入、返回值、错误与状态变化。平台加工结果需要结合原始交易和回执。 |

| GMGN | 发现代币、钱包行为和聪明钱信号。标签、盈亏、胜率和买卖分类需要核对算法口径与链上交易。 |

| DEX Screener | 观察交易对、价格、成交量和流动性。聚合指标需要核对交易对、时间窗、池地址和数据刷新。 |

| GoPlus 与 Honeypot.is | 提供风险检查和模拟结果。输出属于风险线索，需回到合约权限、代码、真实交易和可重复模拟。 |

| Bubblemaps | 展示持仓地址和转账关系。连线只证明平台定义下的链上关系，不能单独证明共同控制。 |

| Arkham | 提供实体标签、地址画像、资金图和提醒。标签与实体归因需要外部证据和原始资金路径支持。 |

| Dune | 用 SQL 查询和计算链上数据。结果需要保存 SQL、表来源、时间范围、查询时间和刷新状态。 |

5.1 当前 BNB Chain 适用边界

1. GMGN 官方资料当前列出 BSC 支持，并明确提醒聪明钱信号只是一种信号。具体标签算法属于平台方法。
2. Arkham 官方资料列出 BNB Chain 支持。实体标签、默认过滤和盈亏计算仍属于平台加工层。
3. Dune 数据会经过 Raw、Decoded 和 Curated 等处理阶段，各阶段刷新时间可能不同。
4. DEX Screener、Bubblemaps、GoPlus、Honeypot.is 和 Phalcon 的具体 BNB Chain

覆盖与页面能力，在进入对应实操前按动态工具清单重新核查。

6. 微案例一：BscScan 的方法名从哪里来

假设交易页显示：

Method: Transfer

页面方法名本身属于解码层。验证步骤如下：

读取原始 tx.input

→ 取前四字节 0xa9059cbb

→ 签名数据库给出 transfer 候选

→ 核对与当前实现匹配的 ABI、参数结构和执行证据

→ 确认函数并解码 recipient 与 amount

→ 再与 Transfer Event 和余额状态交叉验证

可以确认的层次要分开：

1. 原始 Input 的前四字节是可以直接复现的交易事实。
2. 公共签名数据库只能给出候选，因为四字节 Selector 可能发生碰撞。
3. 与当前合约实现匹配的可信 ABI 比签名数据库候选提供更强解码依据。
4. 交易实际移动了目标代币，需要继续看执行结果、事件和状态。

7. 微案例二：GMGN 的聪明钱标签

假设 GMGN 把地址 A 标记为 Smart Money。

第一步先记录：

平台：GMGN

网络：BSC

地址：A

标签：Smart Money

观察时间：具体时间

此时可以确认 GMGN 在观察时刻展示了该标签。地址确实具有稳定盈利能力仍需复核：

1. 检查交易样本是否完整。
2. 检查盈亏和胜率的时间窗口。
3. 检查未实现盈亏、转入资产和空投如何处理。
4. 检查近期表现是否由少数极端交易贡献。
5. 回到代表性交易验证真实买入、卖出和资产变化。

因此，聪明钱标签第一次进入研究记录时通常记为 D 级线索。

后续复核要把不同子结论分开：

1. GMGN 显示地址 A 为 Smart Money，属于平台标签事实。
2. 自己使用明确公式和完整范围重算地址 A 过去三十天盈利 X，并抽样核对原始交易后，可以形成 B 级计算结果。
3. 盈利持续存在，且无法由空投或少数极端交易解释，可以支持自己的高盈利钱包判断。
4. 重算盈利不能证明 GMGN 内部采用了什么标签算法，也不能证明该地址未来会继续盈利。

8. 微案例三：Arkham 的实体标签和资金图

假设 Arkham 把多个地址合并为一个实体，并在 Visualizer 中画出资金连接。

需要拆成两个主张：

1. 地址之间存在平台展示的交易关系。
2. 多个地址由同一现实主体控制。

第一项可以沿图中的交易边回到交易哈希、代币、数量和区块。第二项需要继续寻找共同充值来源、提现归集、部署关系、重复行为和公开身份资料。

图上的连线也需要检查过滤条件。Arkham

Profiler

文档说明部分页面会使用默认价值过滤来减少垃圾交易。当前看不到某条小额交易时，先检查页面过滤，再判断数据是否缺失。

9. 微案例四：Dune 的一行统计结果

假设 Dune 查询显示：

某 Meme 币过去一小时净流入 500 BNB

复核时至少检查：

1. 查询针对哪个 Token 和哪些池。
2. BNB 指原生 BNB、WBNB 还是折算单位。

3. 净流入公式如何定义方向。
4. 是否排除添加和移除流动性。
5. 使用 Raw、Decoded 还是 Curated 表。
6. 数据刷新到哪个区块和时间。
7. SQL 是否公开并可重新运行。

只有表来源明确、SQL

与公式可以复现、时间范围与刷新状态明确，并且已经抽样回到原始交易、日志或状态核实时，这条统计才可以形成级计算证据。只看到截图时，它仍是等待复核的线索。

B

10. 平台主张拆解法

遇到任何平台结论，先改写成最小可验证句子。

10.1 示例主张

内部钱包 A 在开盘后十秒买入代币 T，并获利 100 BNB

10.2 拆解结果

1. 地址 A 的目标交易存在。
2. 目标交易所在区块和区块时间可以确认。
3. 地址 A 获得了代币 T。
4. 平台把该动作分类为买入。
5. 买入发生在平台定义的开盘后十秒内。
6. 地址 A 属于内部钱包。
7. 利润为 100 BNB。

这些子结论需要不同证据：

子结论类型	优先证据
交易存在与区块位置	交易、区块和回执。
获得代币	调用、Transfer Event、余额变化。
买入分类	资产输入输出、池交互和价格路径。
时间差	明确开盘定义，再比较区块时间或交易顺序。
内部钱包身份	资金来源、部署者关系、时序、行为和外部资料。
利润数值	成本基础、卖出收入、持仓估值、手续费和时间范围。

11. 完整交叉验证流程

11.1 第一步：写清楚待验证主张

避免只写“查一下这个钱包”。改写为：

验证地址 A 是否在区块 X 的交易中买入代币 T
 验证平台显示的三十日胜率能否按公开口径重算
 验证地址 A 与部署者 D 是否存在可确认资金关系

11.2 第二步：固定观察边界

普通历史交易至少保存：

1. 网络。
2. 交易哈希。
3. 区块高度。

涉及平台页面、动态标签、余额现状、索引延迟或严格复现时，再保存：

1. 观察时间。
2. 数据来源和页面链接。
3. 区块哈希。
4. RPC Endpoint 和查询参数。
5. 平台过滤条件和信息截止时间。

11.3 第三步：标记当前信息层

给每条信息注明：

原始对象
 解码结果
 计算结果

身份标签

信号判断

11.4 第四步：回到最近的原始证据

| 平台结论 | 优先返回路径 |

| :----- | :----- |

| 某地址转出代币 | 交易、调用、Transfer Event、固定区块余额。 |

| 某地址完成 Swap | Router 和池调用、两侧资产流、事件、余额变化。 |

| 某地址属于项目方 | 部署、初始资金、代币分配、权限和公开资料。 |

| 持仓高度集中 | 固定区块 balanceOf、总供应量、特殊地址清洗规则。 |

| 钱包盈利 | 逐笔成本、收入、手续费、转入转出和当前持仓估值。 |

11.5 第五步：复现解码或计算

1. 检查 ABI 来源与代理实现。
2. 保存查询或计算公式。
3. 固定时间窗口和区块范围。
4. 说明价格来源和计价单位。
5. 说明是否剔除 LP、销毁、交易所和合约地址。
6. 记录无法重算的环节。

11.6 第六步：解释来源差异

多来源结果不同，优先依次检查：

1. 网络或合约是否一致。
2. 区块高度和观察时间是否一致。
3. 索引器是否已经处理到目标区块。
4. ABI、代理实现和事件解码是否一致。
5. 价格、盈亏、胜率和持仓口径是否一致。
6. 是否存在缓存、默认过滤和特殊地址清洗。
7. 平台标签来源是否一致。
8. 当前证据是否仍不足。

11.7 第七步：分层写结论

推荐使用以下结构：

已确认事实

计算结果与口径

身份或关联推断

平台线索

未知项

下一步验证

12. 完整案例：从 BscScan 名称和可读结论回到合约地址与原始交易

本案例使用此前课程已经保存的 BNB Smart Chain 历史交易。

网络：BNB Smart Chain

交易哈希：0x278d43dea3775f4d98978fe0f15b0ef5b70e1c312b32d045564cee6b783e7fed

区块高度：113374190

Token 合约：0x55d398326f99059ff775485246999027b3197955

原始查询观察时间：2026-08-01 18:01 中国标准时间

原始来源：BSC 公共 RPC

本次复核时间：2026-08-12 21:31 中国标准时间

本次来源：BscScan Token 页面与 BNB Chain 公共 RPC

RPC Endpoint：https://bsc-dataseed.bnbchain.org

原始响应文件：01.8 案例证据 BSC-USD 交易与元数据.json

12.1 先固定对象，再记录平台名称

当前 BscScan Token 页面显示：

页面名称：Binance-Peg BSC-USD

页面符号：BSC-USD

合约地址：0x55d398326f99059ff775485246999027b3197955

页面精度：18

本课程此前把它简称为 USDT 或 BSC-USD 案例。跨平台研究应使用“网络加合约地址”固定对象，名称和标签作为附属元数据。

BscScan 的交易页具备显示 Token transfer、发送者、接收者和人类可读数量的能力，这些内容包含 ABI 解码和 Token 元数据加工。本案例没有归档目标交易当时的 BscScan 页面，因此不把特定页面摘要写成已经保存的平台事实。

12.2 回到原始交易字段

2026 年 8 月 12 日通过公共 RPC 重新取得的原始交易字段包括：

tx.from : 0xbd612a3f30dca67bf60a39fd0d35e39b7ab80774

tx.to : 0x55d398326f99059ff775485246999027b3197955

tx.input :

[illegible]

这里可以确认顶层交易调用了该合约，并保存了完整原始 Input。recipient 和 amount 还没有作为独立原始字段出现。

12.3 解码与显示数量

先把原始 Input 拆成字节位置：

前四字节：

0xa9059cbb

第一个 32-byte 参数字：

[illegible]

第二个 32-byte 参数字：

[illegible]

签名数据库可以把 0xa9059cbb 提示为 transfer 候选。结合与目标合约实现匹配的 ABI、两个静态参数字和后续 Transfer Event，可以确认本案例按 transfer(address,uint256) 解码。

解码结果为：

recipient : 0x2a0955ec4c97b951e2c6b624901b4fcfd23daab0

```
amount uint256 : 22890000000000000000
```

结合十八位精度，原始整数可以显示为 22.89 Token。

这里出现了两项加工：

1. 使用 ABI 把选择器和参数字节解码为函数与参数。
2. 使用 decimals 把原始整数换算为页面数量。

本次公共 RPC 在 latest 状态读取到十八位精度。对交易区块 113374190 查询 decimals 时返回 missing trie node，因此交易时点的历史状态仍未通过当前 Endpoint 复现。

12.4 与事件和执行结果交叉验证

本次重新取得的 Receipt 显示：

```
status : 0x1
```

Log address : 0x55d398326f99059ff775485246999027b3197955

Topic0 : 0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef

Topic1 尾部 : 0xbd612a3f30dca67bf60a39fd0d35e39b7ab80774

Topic2 尾部 : 0x2a0955ec4c97b951e2c6b624901b4fcfd23daab0

[illegible]

Event 的发送者、接收者和数值与 ABI 解码结果一致。严格调查仍可继续查询固定区块前后余额。

status	=	1	只能证明顶层执行没有回退。Input	只能证明提交了什么请求。Transfer	Event
--------	---	---	--------------------	----------------------	-------

可以证明成功执行后保留了相应日志。固定区块余额可以继续验证最终状态。

12.5 对照页面名称与合约元数据

本次通过 eth_call 在 latest 状态读取到：

name : Tether USD

symbol : USDT

decimals : 18

BscScan 当前页面名称是 `Binance-Peg BSC-USD`，合约方法当前返回 `Tether USD` 和 `USDT`。这说明页面名称、链上 Token 元数据和课程简称可以同时不同。

可以确认
当前如何展示该地址，也可以确认合约方法当前返回什么。页面为什么采用该名称仍属于平台元数据规则，不能从一次推导。

12.6 本案例的证据分层

结论	证据状态
交易存在于区块 113374190	已有交易和区块标识，属于可复现原始事实。
顶层 tx.to 是目标 Token 合约地址	来自原始交易字段。
Input 请求调用 transfer	来自选择器与 ABI 的交叉解码。
Token 接收者是 0x2a09...aab0	来自 ABI 解码结果，并与 Transfer Event 交叉验证。
显示数量为 22.89	依赖参数数值和十八位精度。当前精度已复核，交易区块历史状态查询失败。
BscScan 当前名称为 Binance-Peg BSC-USD	属于 2026 年 8 月 12 日的平台展示事实。
合约当前返回 Tether USD 和 USDT	来自 2026 年 8 月 12 日的 eth_call。
页面摘要完整等同资产最终变化	不能仅凭页面摘要确认，需要 Event 与 State。

12.7 案例结论

平台名称和 transfer 22.89 Token 可读结论可以被拆解并逐层复核。合约地址、交易字段、ABI 解码、Token 精度、事件和状态分别承担不同证据职责。

本案例已经覆盖真实平台名称、当前合约元数据、原始交易、ABI 解码和 Transfer Event。由于没有归档当前 BscScan HTML 页面，也没有取得交易区块的历史 Token 元数据和余额状态，它仍保留这些复现边界。

13. 常见差异类型

差异类型	优先检查
索引延迟	平台处理高度、刷新时间和回填状态。
观察时点不同	区块高度、区块哈希、观察时间和动态标签更新时间。
字段定义不同	交易发起者、Token 发送者、Spender、Recipient 等角色口径。
计算口径不同	成本基础、价格源、手续费、未实现盈亏和样本窗口。
默认过滤不同	垃圾交易、小额交易、特殊地址、失败交易和内部调用。
地址归因不同	公开来源、行为规则、人工标签和实体聚类方法。
网络或协议覆盖不同	链、协议版本、池类型和回填范围。
数据仍然不足	保留未知，不用平台数量弥补证据缺口。

14. 常见误判与纠正

误判	正确检查方向
平台显示 Smart Money，所以该地址未来会盈利	标签只提供历史或算法线索，继续检查样本、口径和当前交易。
BscScan 显示 Swap，所以资产结果已经确认	回到 Input、Trace、Event 和 State 检查真实资产输入输出。
Arkham 把地址标为交易所，所以身份已经链上证明	区分平台标签事实与现实控制者归因。
Bubblemaps 两个气泡有连线，所以一定同一控制人	连线提供关系线索，共同控制仍需多类证据。
Dune 查询有数字，所以一定属于原始事实	检查 SQL、底层表、计算公式、区块范围和刷新状态。
风险工具显示安全，所以代币可以放心交易	继续检查代码、权限、流动性、真实交易和动态状态。
多个平台给出相同标签，所以自动升级为高等级证据	检查平台是否共享同一标签源，独立性需要验证。
工具没有显示某条记录，所以链上没有发生	先检查原始交易，再检查覆盖、过滤、索引和缓存。

15. 保存一份最小交叉验证记录

每次研究可以保存以下内容：

研究主张：
网络与对象：
固定区块或时间范围：
平台输出和观察时间：
原始链上证据：
解码依据：
计算公式与过滤条件：
标签来源：
来源差异：
证据等级：
已确认结论：

未知项：
下一步：

简单历史交易无需机械填写全部字段。只有字段会影响当前结论时才记录。

16. 官方来源与时效边界

16.1 必要来源

| 关键主张 | 来源、核查日期和适用边界 |

| :----- | :----- |

| BscScan 交易页与事件日志包含原始字段、解码和页面摘要等不同层次，目标 Token 当前页面名称为 Binance-Peg BSC-USD | BscScan Transactions (<https://info.bscscan.com/txs/>)、BscScan Event Logs (<https://info.bscscan.com/what-is-event-logs/>) 与目标 Token 页面 (<https://bscscan.com/token/0x55d398326f99059ff775485246999027b3197955>)，2026-08-12 核查。页面结构、名称和标签会变化。 |

| 调用级 Trace 属于重执行后的调用帧聚合，Opcode Trace 属于更底层执行数据 | Geth Built-in Tracers (<https://geth.ethereum.org/docs/developers/evm-tracing/built-in-tracers>)，2026-08-12 核查。具体能力取决于客户端、Tracer 和历史状态。 |

| GMGN 提供聪明钱追踪、钱包指标和 BSC 支持，同时提醒将聪明钱视为单项信号 | GMGN Smart Money Tracking (<https://docs.gmgn.ai/index/track-smart-money>)、GMGN Wallet Detail (<https://docs.gmgn.ai/index/wallet-detail-page>) 与 GMGN Smart Money Guide (<https://gmgn.ai/blog/how-to-track-smart-money-with-ai-agents/>)，2026-08-12 核查。标签算法和界面属于动态平台能力。 |

| Arkham 提供 BNB Chain、实体标签、地址画像和交易图，部分页面含平台过滤和计算口径 | Arkham Multichain Support (<https://codex.arkm.com/the-intelligence-platform/multichain-support>)、Arkham Visualizer (<https://codex.arkm.com/the-intelligence-platform/visualizer>) 与 Arkham Profiler (<https://codex.arkm.com/the-intelligence-platform/profiler>)，2026-08-12 核查。身份归因仍需独立证据。 |

| Dune 数据分为 Raw、Decoded 和 Curated 等处理层，并具有不同刷新阶段 | Dune Data Catalog (<https://docs.dune.com/data-catalog/overview>) 与 Dune Data Freshness (<https://docs.dune.com/data-catalog/data-freshness>)，2026-08-12 核查。具体表覆盖和延迟需要现场确认。 |

16.2 时效边界

- 1. 平台网络覆盖、标签、算法、页面入口、价格和数据刷新属于动态事实。
- 2. 同一平台不同页面可能使用不同过滤和统计口径。
- 3. 工具标签可能共享第三方数据源，多个相同标签未必构成独立证据。
- 4. 本节只核查工具在证据链中的职责，完整功能和实操仍按后续章节现场复核。

17. 压缩总结与覆盖检查

17.1 最短判断规则

先拆平台主张

- 再标记原始、解码、计算、标签和信号层
- 固定网络、区块、时间与口径
- 回到最近的链上证据
- 复现必要解码和计算
- 解释来源差异
- 分开记录事实、推断、线索和未知

17.2 正文覆盖检查

| 能力目标 | 覆盖位置与当前状态 |

| :----- | :----- |

| 区分五层信息加工链 | 第 2 至 4 节已建立最低模型，并已说明它与证据等级属于不同维度。 |

| 理解不同工具的证据职责 | 第 5 至 9 节已覆盖。 |

| 拆解平台复合主张 | 第 10 节已覆盖。 |

| 执行完整交叉验证 | 第 11 节已给出通用流程。 |

| 完成真实平台主张与交易的证据升级 | 第 12 节已对照 BscScan 当前名称、合约元数据、原始交易和 Transfer Event，历史状态仍保留缺口。 |

| 识别常见差异和误判 | 第 13 至 14 节已覆盖。 |

| 解释、定位、执行和判断能力 | 正文学习中，题目层尚未启动。 |

18. 本节上下文交接卡

字段	内容
	本节核心结论
工具输出	需要按原始对象、解码结果、计算结果、身份标签和信号判断分层。证据等级绑定具体主张，平台名称不能替代复核。
正文生成状态	2026 年 8 月 12 日按第一遍可读性规则生成，当前进入正文学习。
已经掌握的动作	尚未通过本节互动评估。
仍然容易出错的点	等待学习反馈。重点观察平台标签与链上事实、调用流与资产流、原始字段与解码结果的混淆。
已保存的真实案例	BNB Smart Chain 目标 Token 历史转账的 BscScan 当前名称、合约元数据、原始交易、ABI 解码与 Transfer Event 对照。
教学示例	聪明钱买入提示、Arkham 实体标签、Dune 净流入统计和内部钱包复合主张。
案例证据缺口	案例已保存 RPC 原始响应和 BscScan 页面链接，但没有归档当前 BscScan HTML 页面。固定交易区块的 Token 元数据查询返回 missing trie node，固定区块前后余额尚未查询。GMGN、Arkham 与 Dune 微案例均为教学示例。
必须继承的术语	原始对象、解码、计算口径、标签、信号、证据等级、交叉验证、观察时间、过滤条件。
不要重复展开的内容	Input、Trace、Receipt、Logs 与 State 的职责，以及 RPC、节点和索引器机制已在 01.6 与 01.7 建立。
留给下一节的问题	怎样建立隔离钱包、只读连接、授权检查和签名安全边界。
跨章节接口	工具交叉验证交付第二章至第八章，策略计算证据交付第九章。
教学互动状态	正文学习中。
题目层状态	未启动。完成正文后只推荐下一项互动。
独立任务状态	未进入，属于选做。
案例内复习状态	尚未开始。后续案例自然复习原始字段与解码结果、调用流与资产流的分离。
四维能力状态	解释、定位、执行和判断均待后续案例评估。
下一小节建议	完成 01.8 学习后进入 01.9。

笔记 121: 今天完成了 P0 阶段的最后一个缺口：两跳 V3 → V4 路径的保守 PnL 评估。

作者：Quinn | 时间：2026-08-13 02:23 UTC | 字数：720 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天完成了 P0 阶段的最后一个缺口：两跳 V3 → V4 路径的保守 PnL 评估。

此前，系统中的两跳路径虽然能够完成 Quote，但因为原有模型只支持三跳循环，只能被标记为 UNSUPPORTED_HOP_COUNT。今天为它单独建立了两跳 PnL evaluator，让它能够和三跳路径一样，在同一套成本原则下进入 REJECT / PAPER_TRADE_CANDIDATE 判断。

实际 Fork 验证显示，两跳路径当前同样没有正收益：有限规模 Quote 后，终点 native ETH 的 WETH-equivalent 回收低于起始所需 WETH，加入保守 haircut、Gas 和风险缓冲后仍为 REJECT。这里的 Gas 仍只是明确记录的研究假设，不代表已经完成两跳真实执行路径的 Gas 测量。

随后重新运行了受控候选 batch。现在 manifest 中的 7 条候选都已经能够进入相应的保守 PnL 决策层：两跳路径和 6 条三跳路径全部得到 REJECT，不再存在因模型结构限制而无法判断的候选，也没有出现 PAPER_TRADE_CANDIDATE。

今天的收获是：候选路径能否 Quote、是否有适配的 PnL 模型、以及最终是否有利润，必须分开判断。补齐两跳模型后，当前 P0 阶段的“受控候选发现 → 同状态 Quote → 保守 PnL → 连续 Paper Trading → health 状态”闭环已经完整建立。

整个过程仍保持只读和本地 Fork 模拟，没有签名、授权、广播交易或使用真实资金。

笔记 122: 链上套利共学 Day 9 | LI.FI 跨链交易执行细节：五步调用法 + 授权 + 签名发送 + 状态

作者：Cz07cring | 时间：2026-08-13 11:07 UTC | 字数：2888 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 Day 9 | LI.FI 跨链交易执行细节：五步调用法 + 授权 + 签名发送 + 状态轮询

今天学什么

把 LI.FI 跨链交易的完整执行链路从头到尾过了一遍。重点看官方文档的三个页面：Five-Call Recipe（五步调用法）、Token Approvals（代币授权）、Transaction Execution（交易执行）。之前我们只做监控发现和报价，今天补上「真正把交易执行出去」的完整细节。

今天完成了什么

一，掌握了官方定义的「五步调用法」——agent 执行跨链转账的最小闭环：

- 第一步 取链列表：GET /chains，拿到可用链 ID
- 第二步 取代币：GET /tokens?chains=1,42161，拿到链上代币地址
- 第三步 拿报价：GET /quote，返回可直接执行的 transactionRequest
- 第四步 执行：签名并发送交易
- 第五步 轮询状态：GET /status，直到转账完成

二，transactionRequest 对象六个字段（执行层要签名消费的东西）：

- to：LI.FI 合约地址
- data：编码的函数调用
- value：原生代币金额，十六进制 wei
- gasLimit：预估 gas 上限
- gasPrice：建议 gas 价格，可选
- chainId：交易所在链 ID

三，quote 响应的 estimate 对象里的关键字段（比之前理解得更细）：

- toAmount：预期输出金额
- toAmountMin：滑点保护下的最小保证金额（这就是我们 verify 闸门用的最差口径）
- approvalAddress：代币授权的 spender 地址
- executionDuration：预估执行时间，秒
- feeCosts：协议费用数组
- gasCosts：gas 成本数组
- 还发现响应里带 transactionId，是跟踪这次转账的唯一标识

四，Token Approvals 授权细节（跨链转账最容易漏的一步）：

- 原生代币（地址是全零地址）不需要授权
- ERC20 代币必须先授权 LI.FI 合约才能花你的币
- 判断逻辑：fromToken 地址等于 0x0000...0000 就跳过授权，否则查 allowance
- spender 地址必须从 quote 响应的 estimate.approvalAddress 取，永远不要硬编码，因为不同桥的合约地址不一样
- 授权金额三种策略：精确授权（只授权本次 fromAmount，最安全，AI agent 推荐用这个）、无限授权（一次授权永久有效，但合约被黑风险高）、缓冲授权（授权两倍金额，安全性和体验折中）
- 常见错误五条：用错 spender 地址、不等待授权确认就发主交易、在错误的链上授权、授权 gas 不足、USDT 特殊情况（USDT 的 approve 实现要求先归零再授权）

五，交易执行五步细节：

- 第一步 验证链：确认钱包连接在 transactionRequest.chainId 对应的链上
- 第二步 估算 gas（可选）：quote 的 gasLimit 通常够准，只有要精确控制 gas 成本时才重新估算，估算时加 20% 缓冲
- 第三步 构建交易：把 transactionRequest 转成可发送的交易对象
- 第四步 签名发送：等源链确认，receipt.status 不是 success 就报错
- 第五步 处理结果：保存 txHash 用于状态轮询，同时存 fromChain、toChain、bridge（tool）可加速后续状态查询

六，执行错误处理与重试策略：

- insufficient funds（gas 加 value 不够）、nonce too low（有 pending 交易）、gas too low、execution reverted（合约拒绝，通常是 quote 过期，要重新取 quote）、replacement fee too low（替换 pending 交易时 gas 不够）
- 重试逻辑：insufficient funds、user rejected、denied 这三类不重试直接抛错；execution reverted 才重新取 quote 再试，最多两次

七，状态轮询细节（第五步调用）：

- 只有 txHash 是必填，fromChain、toChain、bridge 可选但能加速响应
- 轮询策略：每 10 秒查一次，最多 60 次（10 分钟），DONE 或 FAILED 才结束
- 状态响应有 sending 和 receiving 两个对象，各自带 txHash、amount、token、gas 信息，receiving 对象就是检测 PARTIAL 的依据（昨天学的）

八，gas 优化四条：用 EIP-1559、不过度估算 gas、检查 gas 价格、优先走 L2（Arbitrum、Optimism、Base gas 便宜很多）

踩过的坑

没有新代码坑，但有几个认知对照：approvalAddress 不能硬编码这个教训，跟我们项目之前踩的「spender 地址要动态取」是同一类问题——LI.FI 官方文档明确警告了，说明这是通用陷阱，不是我们特有的坑。

学习收获

一，对执行层最重要的认知升级：跨链转账不是「一笔交易」而是「授权一笔 + 转账一笔 + 轮询多次」。很多人以为 quote 里的 transactionRequest 直接签名发送就完了，实际上 ERC20 场景下必须先授权。漏掉授权，主交易会在链上直接 revert。

二，toAmountMin 的官方定位确认了我们 verify 闸门的正确性：它是「滑点保护下的最小保证到账」，拿它做净收益核算的最差口径是对的，比 toAmount（预期值）诚实。

三，executionDuration 字段值得利用：它给出桥的预估到账时间，结合我们之前实测的「快桥 3 秒、官方桥 19 分钟」，可以在选桥时做价差收敛风险的时间维度判断——到账越慢，价差被抹平的风险越大。

四，精确授权 vs 无限授权的权衡，对我们自动套利 agent 尤其关键：AI agent 官方推荐精确授权（每次只授权本次金额），因为 agent 自动执行时如果授权无限额度，合约一旦有漏洞就是灾难。这跟我们的安全边界设计（Guard 模式、白名单、金额上限）方向一致。

下一步

把 approvalAddress 动态解析和授权检查纳入执行层设计，作为签名发送前的强制前置步骤

把 executionDuration 纳入选桥逻辑，做价差收敛风险的时间维度判断

继续积累 shadow 数据

学习记录，非投资建议。